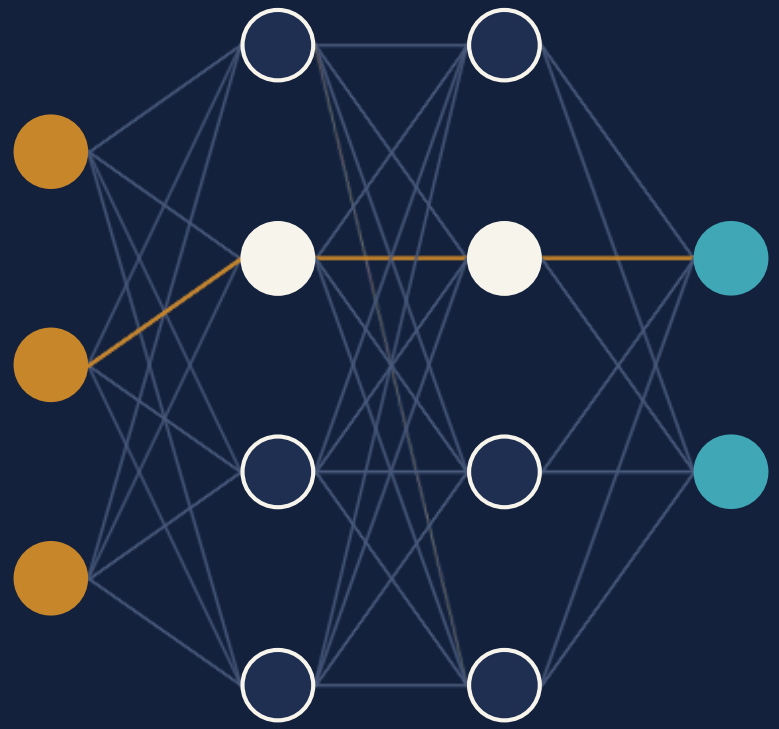
Deep Learning From Scratch

Mathematics, intuition and working NumPy code for every idea, from a single neuron all the way to a tiny GPT.

24 modules | Python 3 and NumPy only | Every output was really run



Input → hidden → hidden → output. The amber path is one signal travelling forward.

How to read this course

The visual language used on every slide

Concept card. A plain explanation in full sentences, written so you can follow it without a lecturer.

Code panel. Runnable Python with NumPy. # comments look like this

Output panel. The exact text printed when the code was run.

Key idea. The one sentence to remember from the slide.

Annotated figure. Charts and diagrams built from labelled boxes, arrows and callouts.

A study loop that works

1. **Read** the concept card and say the idea back in your own words.
2. **Work** the numeric example with pen and paper before looking at the answer.
3. **Type** the code yourself. Copying by hand builds memory far better than pasting.
4. **Run** it and compare your output with the output panel, number by number.
5. **Break it on purpose.** Change one value, predict the effect, then check.

Twenty four modules, three parts

Part I Foundations, modules 1 to 8

Part II Learning machinery, modules 9 to 17

Part III Architectures and beyond, modules 18 to 24

01 Foundations	**02** Math toolkit	**03** NumPy for deep learning	**04** Neuron and perceptron
05 Linear and logistic regression	**06** Gradient descent	**07** Activation functions	**08** Multilayer perceptrons
09 Backpropagation	**10** An autograd engine	**11** A neural network library	**12** Project: reading digits
13 Loss functions	**14** Weight initialisation	**15** Optimisers and schedules	**16** Regularisation
17 Training practice	**18** Convolutional networks	**19** Recurrent networks	**20** Attention and transformers
21 Embeddings	**22** Generative models	**23** Towards real frameworks	**24** Glossary and cheat sheets

Key idea. Each module depends only on the ones before it, so read in order the first time through.

Prerequisites and environment setup

What you need to know already

Python basics: variables, lists, loops, functions and classes. If you can write a class with a method, you are ready.

School algebra: rearranging equations and reading a graph. All calculus and linear algebra is taught from zero in Module 2.

What you will install

Package	Why we need it
numpy	Fast arrays and matrix maths. Our only real dependency.
matplotlib	Optional plotting of your own curves.
scikit learn	Only to load a small built in digits dataset.

CODE · setup_check.py

```
# Step 1, in a terminal:
pip install numpy matplotlib scikit_learn

# Step 2, in Python: confirm versions
import sys
import numpy as np

print("Python", sys.version.split()[0])
print("NumPy ", np.__version__)
```

OUTPUT

```
Python 3.13.16
NumPy 2.5.3
```

Tip. pip treats an underscore and a dash in a package name as the same, so scikit_learn installs the usual scikit learn package.

No equations: every idea is drawn with these symbols



Circle

one neuron, or one number



Tall block

a whole layer of neurons



Black arrow

data moving forward



Amber arrow

learning signal moving backward

(3, 4)

Grid and tag

a tensor and its shape



Operation nodes

multiply, add, square, squash



loss meter

Meter

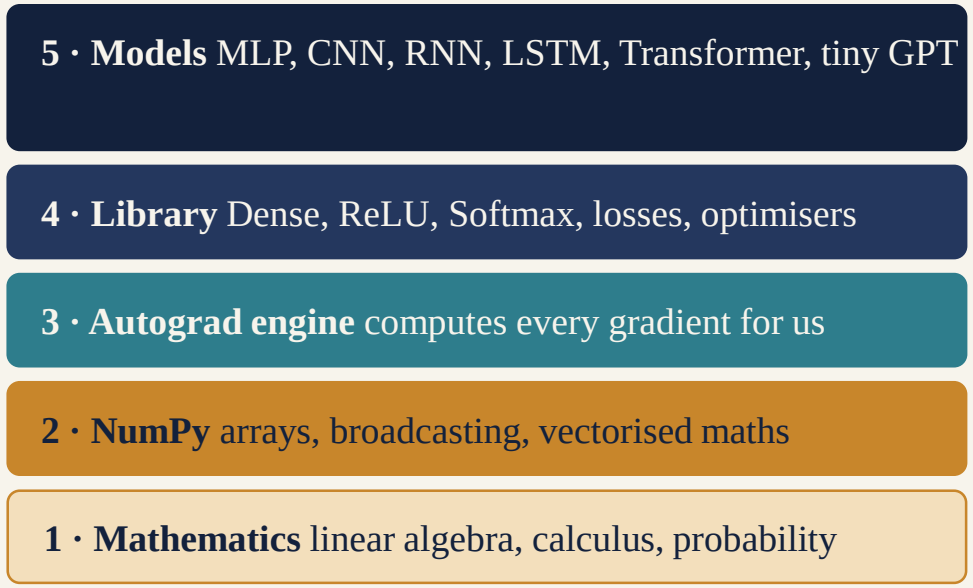
how wrong the model is

CODE STYLE

No hyphen characters, even in code: subtraction is np.subtract(a, b), negation is np.negative(x). In your own code use the normal minus sign.

How to read every diagram: follow black arrows left to right to see what the network computes, then amber arrows right to left to see how it learns.

What “from scratch” means: we build the whole stack



Most tutorials start at the top layer by importing a framework. You learn the syntax, but the machine stays a black box.

Here we start at the bottom. Every layer of the stack is derived on paper and then written by us in NumPy, so nothing is magic.

> **Key idea.** We will not import a deep learning framework until Module 23, and by then you will recognise every piece it offers.

By the final slide you will be able to

Explain
what a neural network computes, layer by layer, in plain words.

Derive
gradients by hand with the chain rule, including backpropagation.

Implement
layers, losses and optimisers in NumPy with no framework.

Build
your own autograd engine that differentiates any expression.

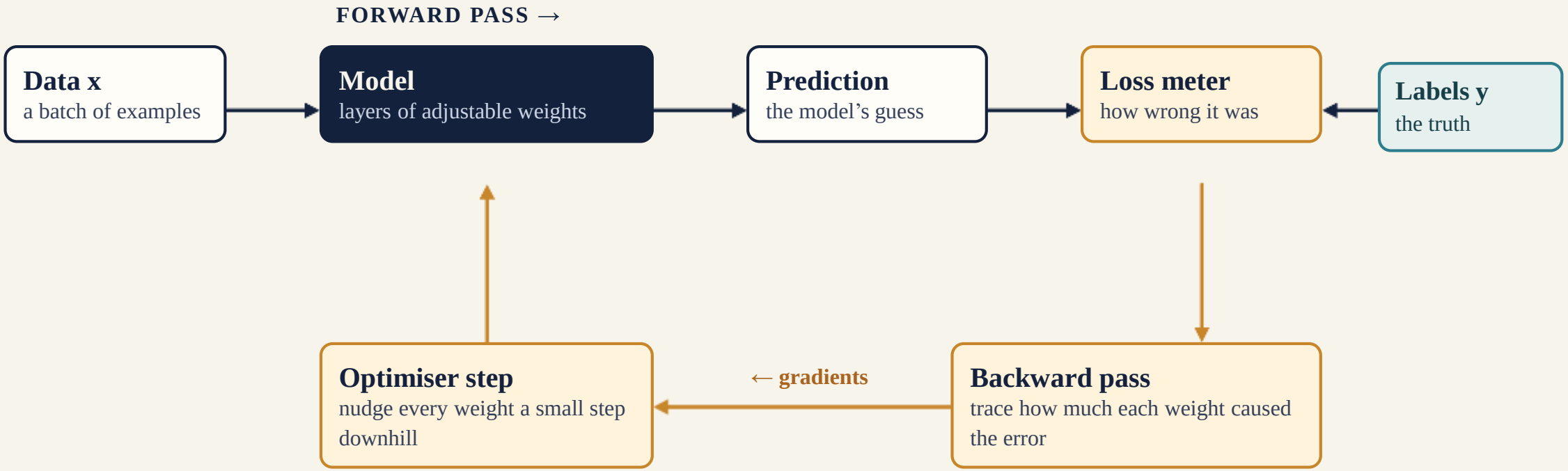
Train
a digit classifier end to end and read its learning curves.

Debug
training with sanity checks, gradient checks and diagnostics.

Dissect
convolutions, recurrent cells and attention down to each multiply.

Transfer
your understanding directly to PyTorch and modern tools.

How every neural network learns: one loop, repeated



One iteration: forward pass, loss, backward pass, update. Training repeats it thousands of times.

Key idea. Learning means nudging the weights in whatever direction makes the loss meter fall. Everything else is detail.

01

MODULE ONE

Foundations

What deep learning is, how it differs from classic programming, where it came from and why it works so well today.

IN THIS MODULE

- AI, machine learning and deep learning
- Rules versus learning from examples
- A short history in twelve dates
- Why it took off: data, compute, ideas
- Representation learning
- Kinds of learning and kinds of tasks
- The four ingredients of every model

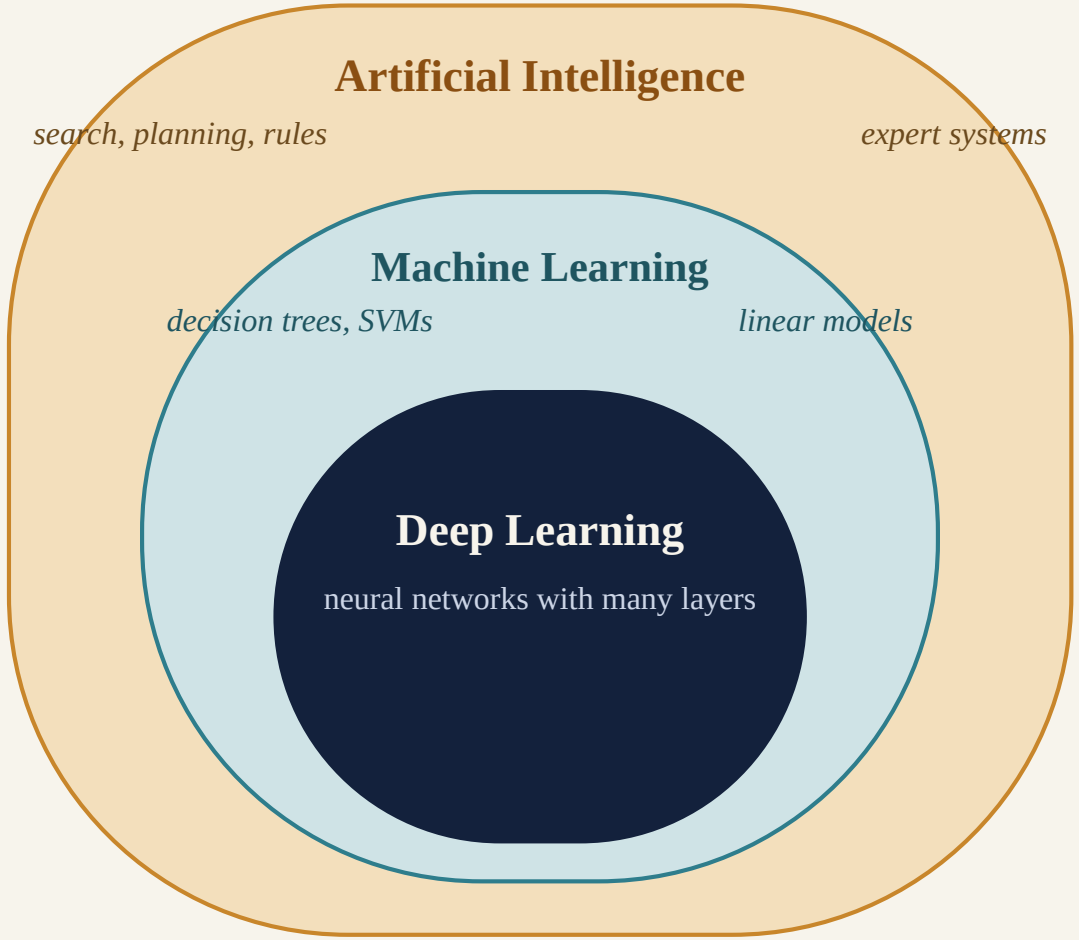
AI, machine learning and deep learning are nested ideas

Artificial intelligence is the broad goal: machines that perform tasks we associate with intelligence. It includes hand written rules and search.

Machine learning is the subset where behaviour is *learned from data* rather than written by a programmer.

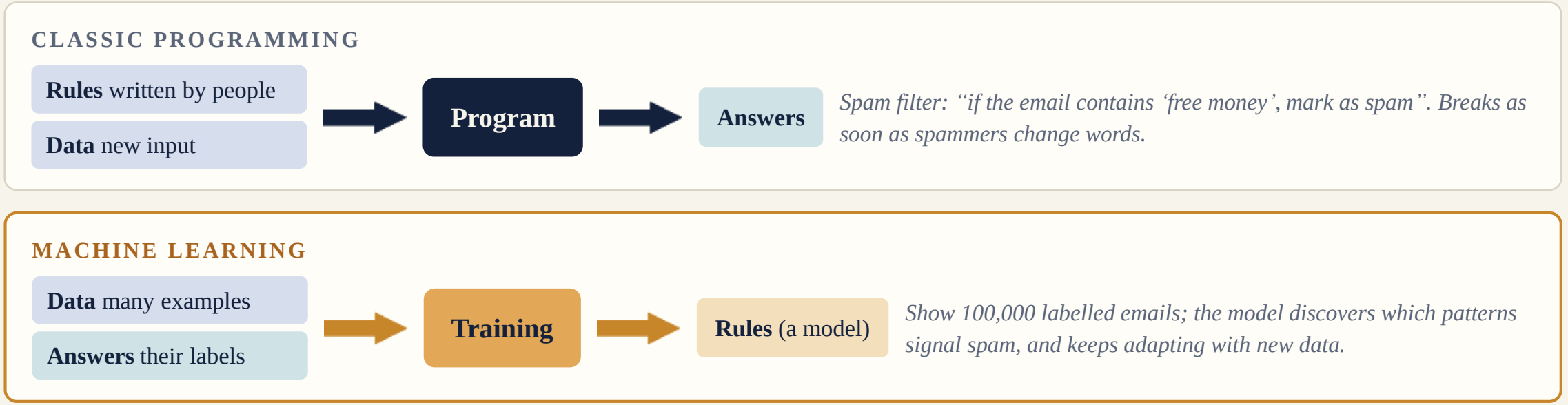
Deep learning is the subset of machine learning that uses neural networks with many stacked layers, each learning a richer representation of the input.

> **Key idea.** Every deep learning model is a machine learning model, but not the other way round.



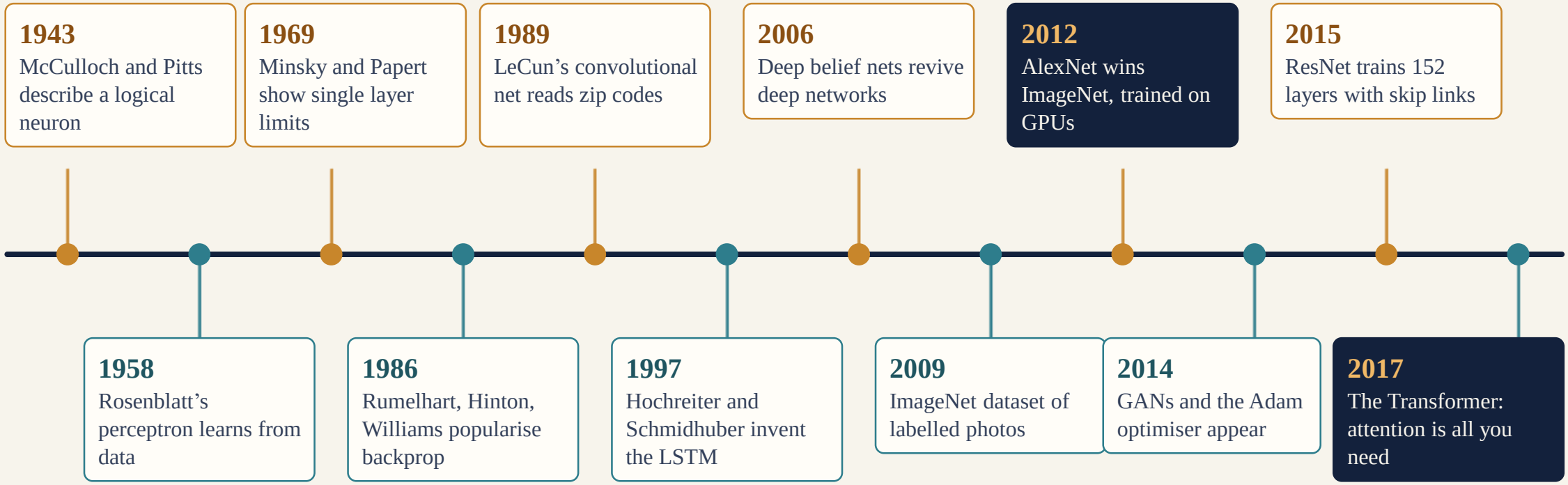
Writing rules versus learning rules

Machine learning flips the direction of classic programming. Instead of giving the computer rules, we give it examples and let it find the rules.



Key idea. A trained model is a program that was written by data. Once trained, it maps new inputs to answers exactly like the top row.

A short history of deep learning in twelve dates



Since 2018 the Transformer has been scaled into large pretrained models for language, images, audio and code. The ideas are old; scale and engineering are new.

Why deep learning took off: three pillars

MODERN DEEP LEARNING

Data

Phones, the web and sensors produce vast amounts of images, text and audio.

Large labelled sets such as ImageNet gave networks enough examples to learn millions of parameters without simply memorising.

Compute

Training is mostly matrix multiplication, and GPUs perform thousands of multiply and add operations in parallel.

Experiments that once took months now take hours.

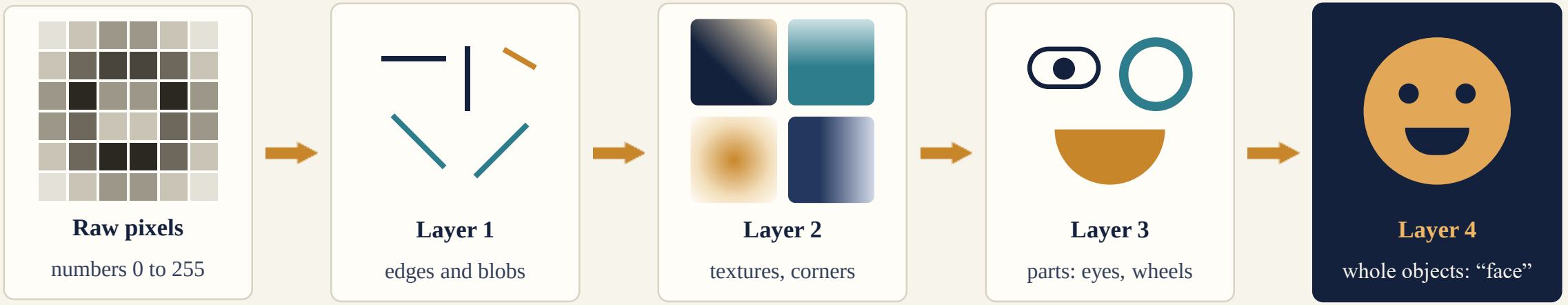
Algorithms

ReLU, careful initialisation, dropout, batch normalisation, Adam, residual connections and attention.

Each one made deeper networks trainable. You will build all of them.

Foundation: open source software, shared datasets and openly published research

Representation learning: each layer sees more abstract patterns

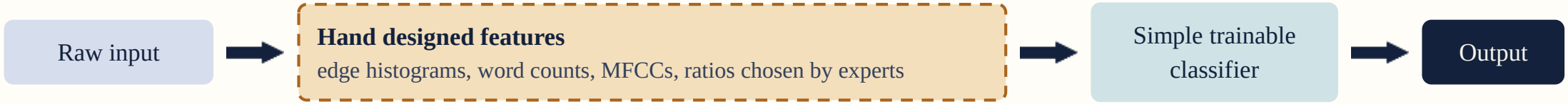


Nobody tells the network to look for edges or eyes. These detectors **emerge** because they help reduce the loss. Early layers learn simple, general patterns; deeper layers combine them into specific, meaningful ones.

Key idea. A deep network is a stack of learned feature detectors. Depth lets it build complex concepts out of simple ones.

Feature engineering versus feature learning

CLASSIC MACHINE LEARNING PIPELINE



Most of the human effort goes into the dashed box, and it must be redone for every new problem.

DEEP LEARNING PIPELINE



Every box is trainable. The whole chain is optimised together, end to end, by gradient descent.

Gain: no feature design, better accuracy on perceptual data like images, sound and text.

Cost: needs more data and compute, and the learned features are harder to interpret.

Four ways a model can learn

Supervised

Data: pairs (x, y)

Learn a mapping from input to a known correct label.

Examples: photo to “cat”, house features to price, email to spam or not.

Most of this course.

Unsupervised

Data: x only

Find structure such as clusters, compressions or densities.

Examples: grouping customers, autoencoders that compress images.

Module 22.

Self supervised

Data: x, labels made from x

Hide part of the data and predict it from the rest.

Examples: predict the next word; fill in a masked image patch.

How language models are trained. Module 20.

Reinforcement

Data: actions, rewards

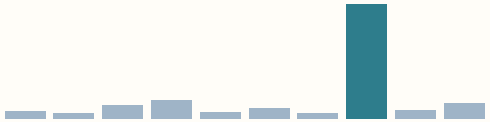
An agent acts in an environment and learns to maximise long term reward.

Examples: game playing agents, robot control.

Uses the same networks as a component.

Key idea. The network architecture can be identical in all four settings. What changes is where the training signal (the loss) comes from.

The main task types and what their outputs look like

Regression predict a quantity	One real number per example. Final layer: **linear** (no squashing). Loss: **mean squared error**. Example: a house price.	**£ 243,500**
Binary classification yes or no	One probability between 0 and 1. Final layer: **sigmoid**. Loss: **binary cross entropy**. Example: spam or not spam.	spam probability: 92%
Multiclass pick one of C classes	C probabilities that sum to 1. Final layer: **softmax**. Loss: **categorical cross entropy**. Example: which digit, 0 to 9.	
Sequence generation one token at a time	A softmax over the whole vocabulary at every step. Loss: **cross entropy per token**. Example: translation, chat.	The cat sat on ?

In the multiclass chart the tall teal bar is class 7, the model’s choice; grey bars are the other nine classes.

The four ingredients of every deep learning system

1 · DATA

What do we learn from?

Examples with their correct answers, split into training, validation and test sets. Quality and quantity set the ceiling on performance.

2 · MODEL

Which family of functions?

An architecture such as an MLP, CNN or Transformer: a wiring diagram full of adjustable weights.

Training

search for the weights that make the loss meter read lowest

3 · LOSS

What does “good” mean?

A single number, our loss meter, that reads low when predictions are right. It turns a vague goal into mathematics.

4 · OPTIMISER

How do we improve?

Uses the backward learning signal to adjust the weights: SGD, momentum, Adam. Decides how big each step is.

02

MODULE TWO

The Math Toolkit

Exactly the linear algebra, calculus and probability that neural networks use, taught from zero with pictures, numbers and code.

IN THIS MODULE

- Scalars, vectors, matrices, tensors
- Dot products and matrix multiplication
- Transpose, identity, norms
- Derivatives and numerical checks
- Gradients and the chain rule
- Jacobians and shapes
- Probability, expectation, Gaussians
- Entropy, cross entropy, KL divergence

Scalars, vectors, matrices and tensors

All data in deep learning is stored as **tensors**: arrays of numbers with any number of axes. The number of axes is called the **rank** (or number of dimensions, *ndim*).



Scalar

rank 0 · shape ()
a loss value, a learning rate

Vector

rank 1 · shape (4,)
one example's features

Matrix

rank 2 · shape (3, 4)
a batch, or a weight matrix

Tensor

rank 3 or more · shape (3, 3, 3)
colour images, video, text

Key idea. Shape is the most important property of a tensor. Most deep learning bugs are shape bugs, so we will annotate shapes on almost every slide.

Real data as tensors: the shapes you will meet

Data	Shape convention	Concrete example
Tabular rows	(N, D)	(32, 10): 32 customers, 10 features
Grey images	(N, H, W)	(64, 28, 28): 64 digit images
Colour images	(N, C, H, W)	(16, 3, 224, 224): RGB photos
Text token ids	(N, T)	(8, 128): 8 sentences, 128 tokens
Token vectors	(N, T, D)	(8, 128, 512): each token a 512 vector
Audio	(N, S)	(4, 16000): 1 second at 16 kHz
Video	(N, T, C, H, W)	(2, 16, 3, 112, 112): 16 frames

THE BATCH AXIS

N comes first. We process many examples at once because matrix hardware is far faster on batches than on single examples.

CHANNELS FIRST OR LAST

PyTorch stores images as (N, C, H, W). TensorFlow defaults to (N, H, W, C). Same data, different axis order.

Read a shape aloud like a sentence: “sixteen images, three channels, two hundred and twenty four by two hundred and twenty four pixels”.

A vector is both an arrow and a data point

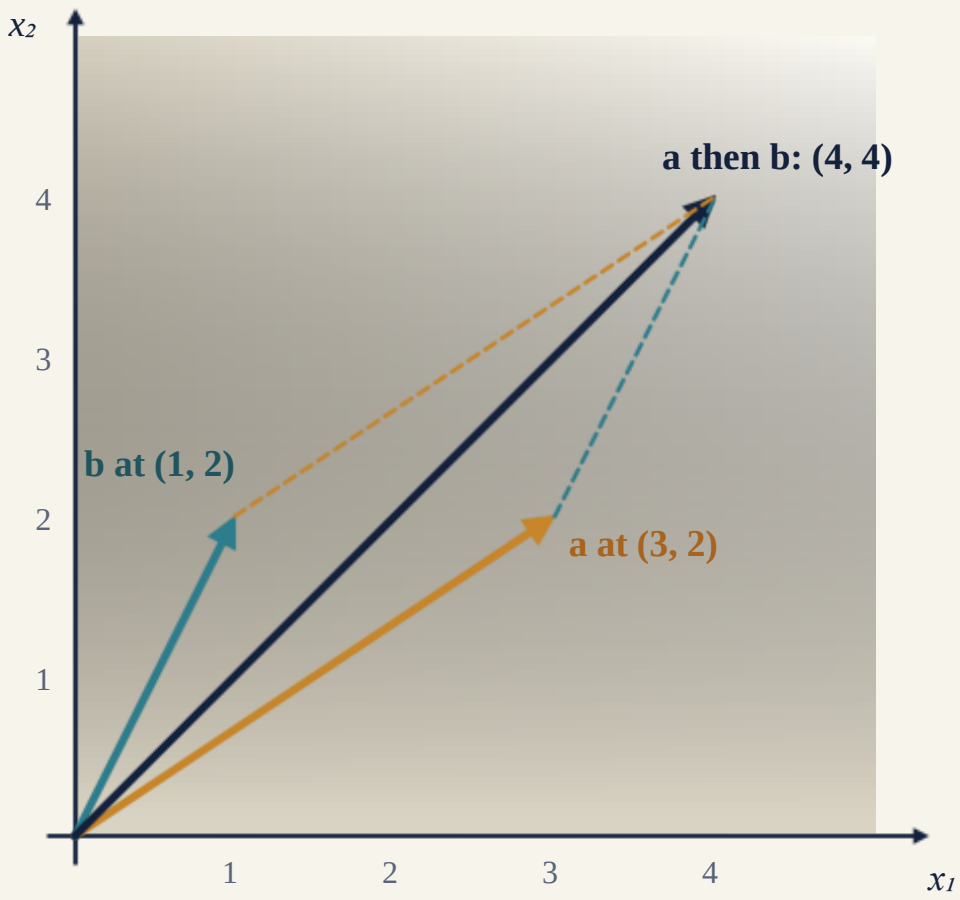
As a list of numbers: the amber vector a holds 3 and 2. In code it is np.array([3, 2]).

As an arrow: start at the origin, move 3 right and 2 up. Its length and direction carry meaning.

As a data point: a house with 3 bedrooms and 2 bathrooms is the point (3, 2) in “feature space”.

Adding two vectors means walking one arrow, then the other, tip to tail. The dashed lines show both orders land on the same spot, (4, 4).

Scaling stretches an arrow: doubling a keeps its direction and doubles its length, landing on (6, 4).



The dot product: multiply pairs, then add

Line two equal length vectors up, **multiply each matching pair, then add all the products**. The result is a single number.

WORKED EXAMPLE

a	1	2	3
b	4	5	6
×	4	10	18

→ **add: 32**

Key idea. A neuron's weighted sum of its inputs is exactly a dot product. It is the most executed operation in all of deep learning.

Geometric meaning: the dot product measures how much two arrows point the same way. Watch the three pictures below.



positive

similar direction



zero

perpendicular, unrelated



negative

opposite directions

If we first shrink both arrows to length one, the dot product becomes the **cosine similarity**: a score from −1 (opposite) through 0 (unrelated) to 1 (identical direction), used everywhere: search, recommendation and comparing embeddings.

The dot product and cosine similarity in NumPy

CODE · dot_product.py

```
import numpy as np

a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])

# three equivalent ways to compute a · b
print("a . b =", np.dot(a, b))
print("a @ b =", a @ b)
print("sum(a*b) =", np.sum(a * b))

# cosine similarity: divide by both lengths
cos = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("cos angle =", round(cos, 4))
print("angle deg =", round(np.degrees(np.arccos(cos)), 2))
```

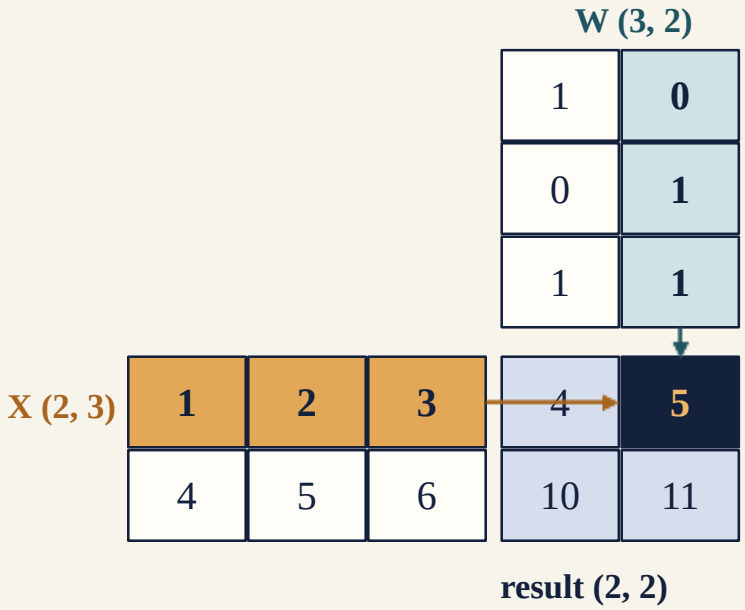
OUTPUT

```
a . b     = 32.0
a @ b     = 32.0
sum(a*b)  = 32.0
cos angle = 0.9746
angle deg = 12.93
```

Reading the output. All three methods agree on 32. The cosine 0.9746 means a and b point in almost the same direction: only 12.93 degrees apart.

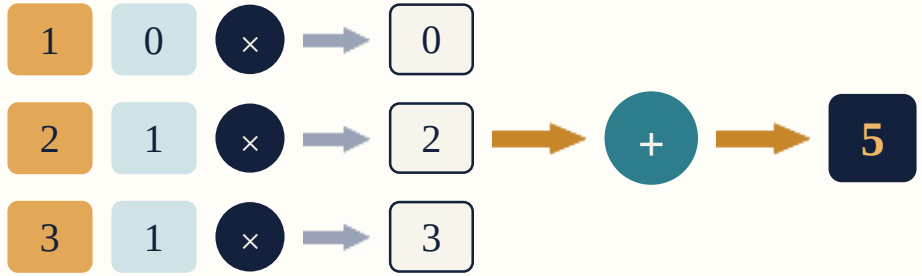
Habit. Prefer the @ operator. It is the same operator we will use for matrix multiplication, so one symbol covers both.

Matrix multiplication: row meets column



Slide the amber **row** of X across and the teal **column** of W down. Where they cross, the dark cell is born.

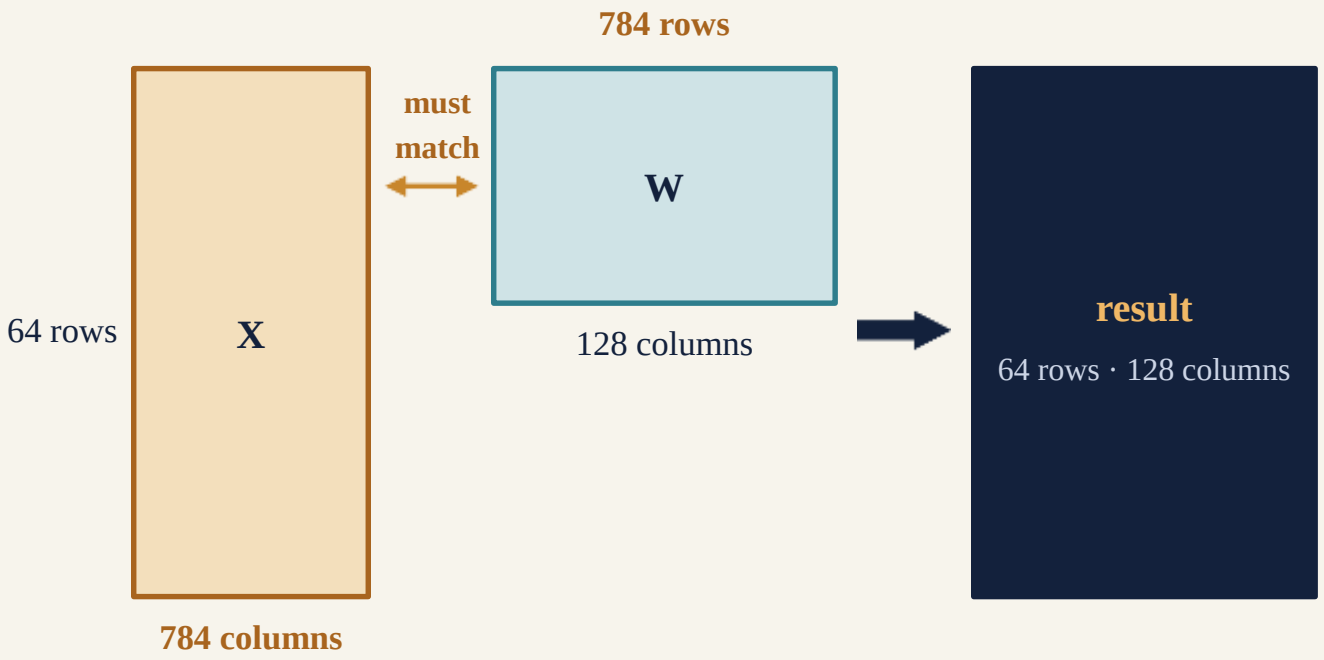
HOW THE DARK CELL IS BUILT



Pair the row with the column, multiply each pair, add the three products. It is exactly the dot product from slide 23.

Key idea. The other three cells are built the same way from their own row and column. Multiplying a batch by a weight matrix computes every neuron for every example in one go.

The shape rule, drawn as blocks that must fit together



The **inner edges** (784 and 784) must be equal and disappear. The **outer edges** (64 and 128) survive and give the result's shape.

Blocks	Verdict
(64, 784) with (784, 128)	fits: gives (64, 128)
(64, 128) with (128, 10)	fits: gives (64, 10)
(64, 784) with (128, 784)	does not fit: error
(64, 784) with (128, 784) flipped	fits after a transpose

A DENSE LAYER IS ONE OF THESE

64 examples with 784 features meet a weight block of 784 by 128, giving 128 neuron outputs for each of the 64 examples.

Order matters. Swapping the two blocks usually breaks the fit, so X with W is not the same as W with X.

Matrix multiplication in NumPy, checked by hand

CODE · matmul.py

```
import numpy as np

X = np.array([[1, 2, 3],
              [4, 5, 6]])      # shape (2, 3)
W = np.array([[1, 0],
              [0, 1],
              [1, 1]])         # shape (3, 2)

Z = X @ W
print(Z.shape)
print(Z)

# recompute Z[0, 1] as an explicit dot product
print(sum(X[0, k] * W[k, 1] for k in range(3)))
```

OUTPUT

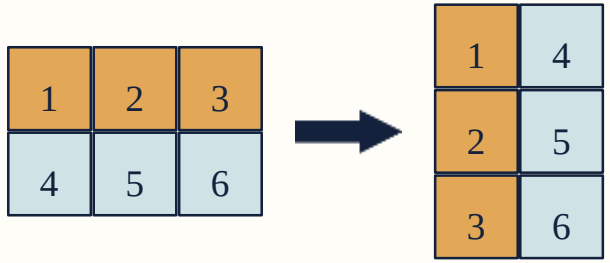
```
(2, 2)
[[ 4  5]
 [10 11]]
5
```

Matches the previous slide. The result shape (2, 2) follows the shape rule: the inner 3s cancel, the outer 2 and 2 remain.

Why vectorise? The explicit loop on the last line is fine for one entry, but X @ W calls optimised BLAS routines that are many times faster for real layer sizes.

Transpose and identity: two matrices you meet daily

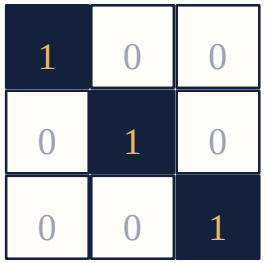
Transpose: swap rows and columns



Shape (2, 3) becomes (3, 2). The amber row becomes the amber column.

Used in backprop: flipping a product of two matrices flips each one and reverses their order. The code on the right checks it.

Identity: the “do nothing” matrix



Ones on the diagonal, zeros elsewhere. Multiply anything by it and you get **the same thing back**.

The **inverse** of a matrix undoes it, leading back to the identity. Deep learning rarely inverts matrices; we use gradients instead.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([[1, 0],
              [2, 1],
              [0, 3]])
print(A.shape, A.T.shape)
print(A.T)
print(np.array_equal((A @ B).T,
                     B.T @ A.T))
I = np.eye(2)
print(np.array_equal(I @ (A @ B), A @ B))
```

OUTPUT

```
(2, 3) (3, 2)
[[1 4]
 [2 5]
 [3 6]]
True
True
```

Norms: three ways to measure the size of a vector

L1 · grid walk
add up the steps along each axis. Here 3 across plus 4 up: **7**.

L2 · straight line
the length of the arrow itself, as a ruler measures it: **5**.

L∞ · biggest step
only the largest single coordinate counts: **4**.

```
v = np.array([3.0, 4.0])
print("L1 :", np.sum(np.abs(v)))
print("L2 :", np.sqrt(np.sum(v ** 2)))
print("Linf:", np.max(np.abs(v)))
print("check:", np.linalg.norm(v, 1),
      np.linalg.norm(v),
      np.linalg.norm(v, np.inf))
```

OUTPUT

```
L1  : 7.0
L2  : 5.0
Linf: 4.0
check: 7.0 5.0 4.0
```

Where they appear. L2 keeps weights small (weight decay) and caps exploding gradients. L1 pushes weights to exactly zero (sparsity).



Teal: straight line distance. Amber: walk along the grid like city blocks.

The derivative is the slope of the tangent line

Zoom in on any smooth curve and it looks like a straight line. The steepness of that line is the **derivative**. Watch the measured slope at position 1 settle as the nudge shrinks:

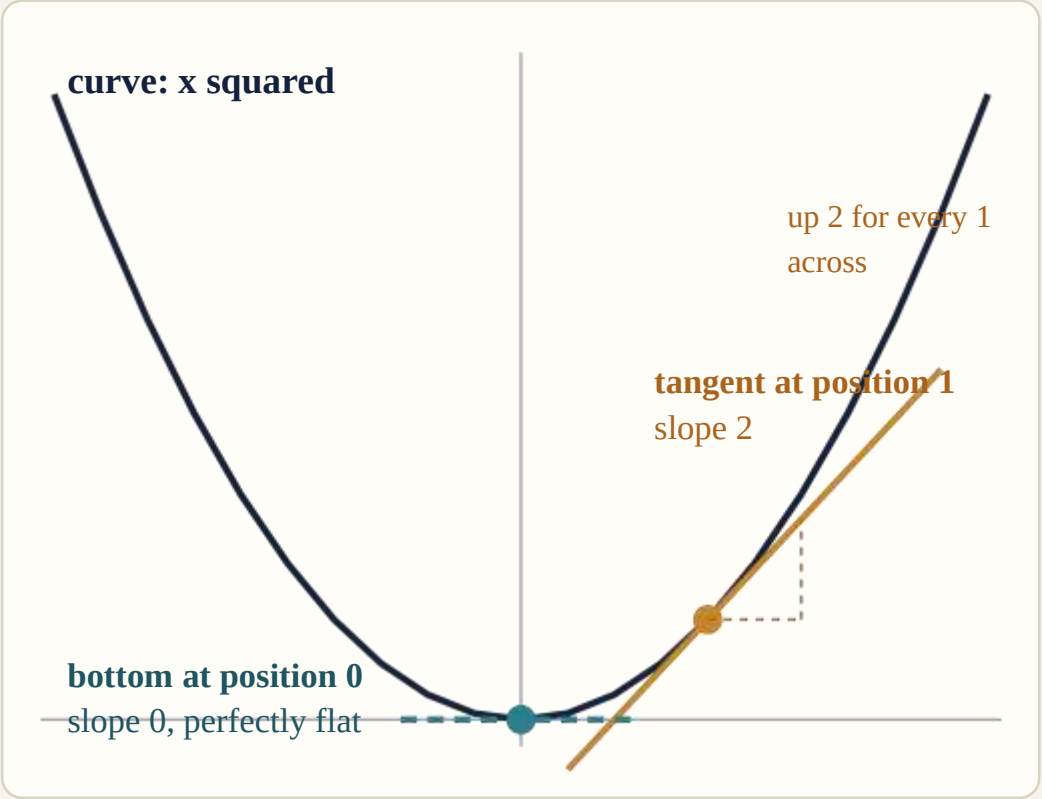
nudge 1	nudge 0.1	nudge 0.01	settles
rise 3	rise 0.21	rise 0.0201	2
slope 3	**slope 2.1**	**slope 2.01**	

Sign says up or down.

Size says how steep.

Zero marks a flat spot.

Key idea. To go downhill, step against the slope. At position 1 the slope points up to the right, so we move left. That one rule is gradient descent.



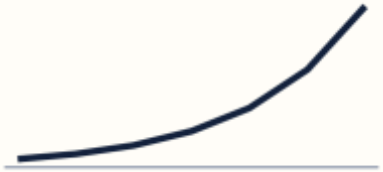
A gallery of slopes: how the shapes we use behave



Straight line. Same slope everywhere.



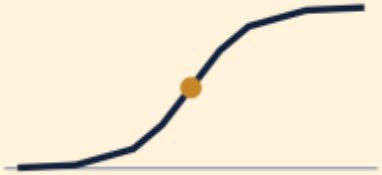
Square (bowl). Flat at the bottom, steeper the further out.



Exponential. Slope equals its own height, so it keeps accelerating.



Logarithm. Very steep at first, then flattens out.



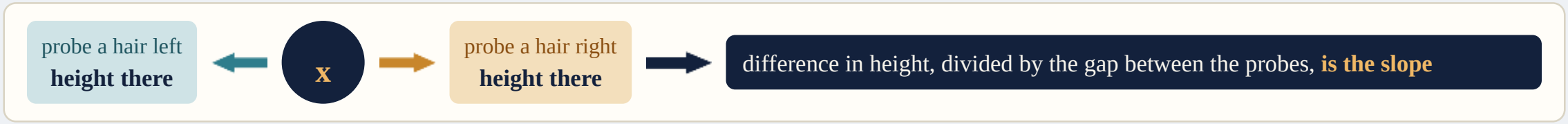
Sigmoid (S curve). Steepest in the middle, nearly flat at both ends.



ReLU (hinge). Slope 0 on the left, slope 1 on the right.

Watch the amber card. The flat ends of the S curve give almost zero slope. When learning signals pass through many such flat spots they fade away. Module 7 shows why ReLU fixed this.

Measuring a slope numerically with two probes



CODE · numerical_derivative.py

```
import numpy as np

def f(x):
    return x ** 2 + 3 * x

def numerical_derivative(f, x, h=0.00001):
    # right probe height minus left probe height,
    # divided by the gap 2h between the probes
    return np.subtract(f(x + h), f(np.subtract(x, h))) / (2 * h)

for x in [0.0, 1.0, 2.0]:
    exact = 2 * x + 3    # known true slope, for comparison
    approx = numerical_derivative(f, x)
    print(f"x={x}  exact={exact:.6f}  numeric={approx:.6f}")
```

OUTPUT

```
x=0.0  exact=3.000000  numeric=3.000000
x=1.0  exact=5.000000  numeric=5.000000
x=2.0  exact=7.000000  numeric=7.000000
```

Why two probes? One probe on a single side gives a lopsided estimate. Probing both sides cancels most of the error.

Use. Too slow for training, but the gold standard for **checking** hand built gradients in Module 9.

Partial derivatives: turn one knob at a time



EXPERIMENT 1
Hold y still. Turn x a hair. The meter moves **8 times** as much as the knob.

EXPERIMENT 2
Hold x still. Turn y a hair. The meter moves **3 times** as much as the knob.

Key idea. Each sensitivity is a **partial derivative**. Collect them all into one list and you have the **gradient**: one entry per knob, so it always has the same shape as the knobs.

THE GRADIENT AS A BAR CHART




```
def machine(x, y):
    return x ** 2 + 3 * x * y
h = 0.00001
dx = np.subtract(machine(1 + h, 2),
     machine(np.subtract(1, h), 2)) / (2 * h)
dy = np.subtract(machine(1, 2 + h),
     machine(1, np.subtract(2, h))) / (2 * h)
print("gradient:", np.round([dx, dy], 4))
```

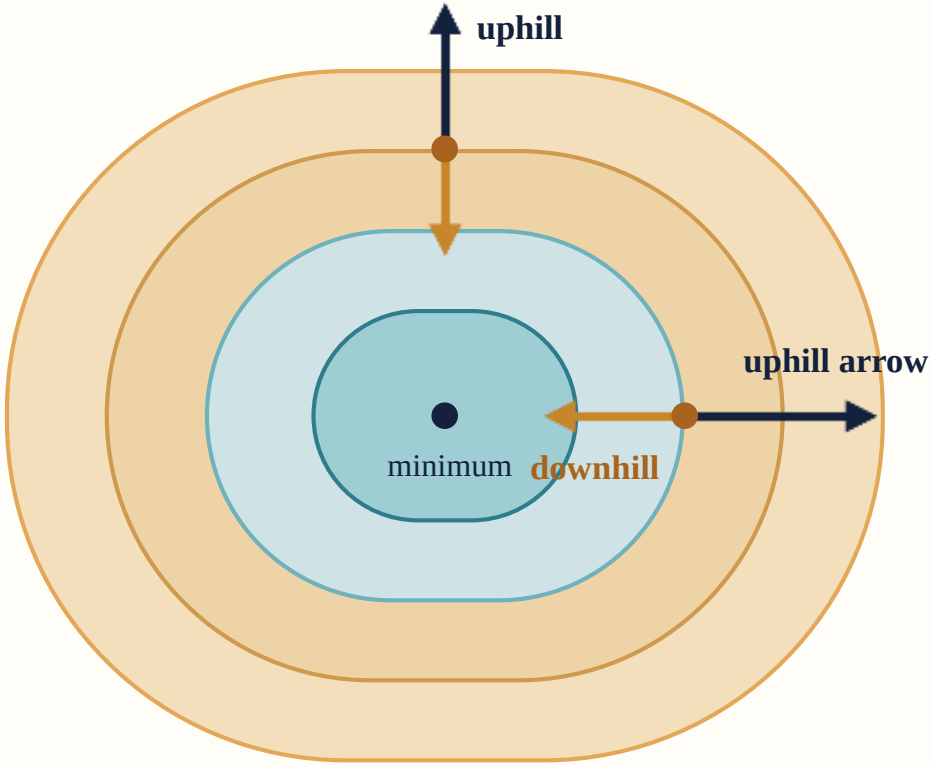
OUTPUT gradient: [8. 3.]

The gradient points straight uphill

Picture the loss as a landscape over the parameters. At any point the gradient has three remarkable properties:

1. It points in the direction of **steepest ascent**.
2. It is **perpendicular** to the contour line through that point.
3. Its **length** is how steep the slope is there.

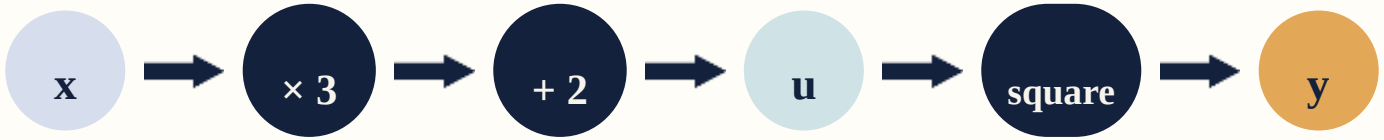
> **Key idea.** The amber arrows point steepest downhill toward the minimum. Following them, step after step, is how every network is trained.



Contour lines join points of equal height, like a map of a valley. Inner rings are lower.

The chain rule: speeds multiply along a chain

A network is a chain of simple stages. To know how the end responds to the start, find how fast each stage responds to its own input, then **multiply those speeds together**.



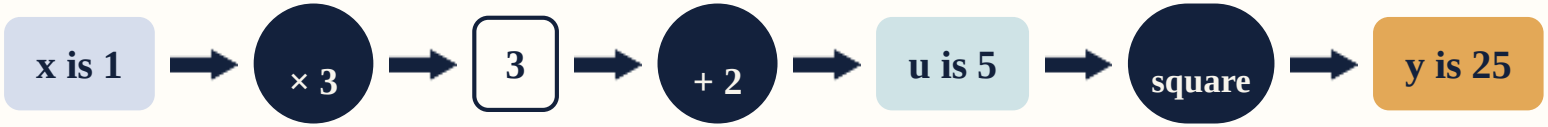
times 3 stage	plus 2 stage	square stage, when u is 5
3 times as fast	**same** speed	**10 times** as fast



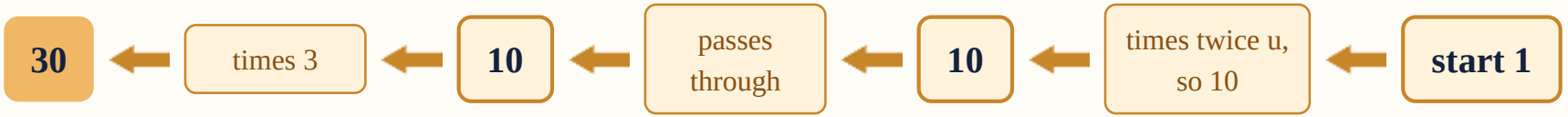
Chain rule: 3 times as fast, then 10 times as fast, makes 30 times as fast overall.

The chain rule with real numbers, forward then backward

FORWARD: values flow right



BACKWARD: the learning signal flows left, multiplied at each stage




```
g = lambda x: (3 * x + 2) ** 2
x = 1.0
analytic = 2 * (3 * x + 2) * 3  # 10 times 3
numeric = numerical_derivative(g, x)
print("analytic:", analytic)
print("numeric :", round(numeric, 6))
```

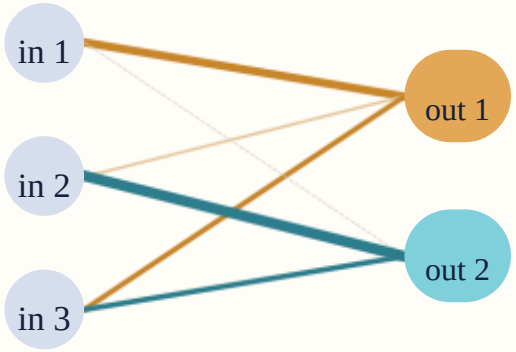
OUTPUT

```
analytic: 30.0
numeric : 30.0
```

Notice: the square stage needed u, which was 5, from the forward pass. Training stores forward values so the backward pass can reuse them.

Jacobians: a sensitivity table for many inputs and outputs

A linear layer: 3 inputs, 2 outputs

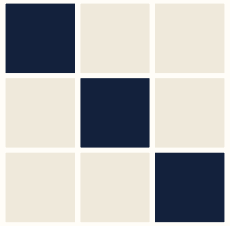


SENSITIVITY

2	0.5	1
0	3	1

row: one output
column: one input

Each cell says how strongly one output responds to one input. For a linear layer the table is **exactly its weight matrix**: thick lines, big numbers.



An elementwise squash

Each output depends only on its own input, so only the diagonal is filled. We skip the empty cells and simply multiply one by one.

WHY NETWORKS NEVER BUILD THE FULL TABLE

A network ends in a single number, the loss. So the backward pass only pushes **one list of signals** through each table, never storing the table itself. A layer of 1,000 by 1,000 would otherwise need a million cells per example.

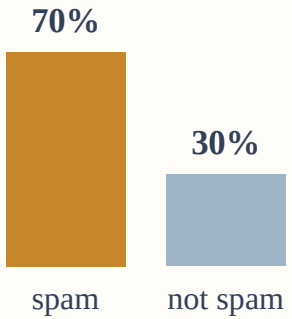
Key idea. The learning signal for any weight block always has that block's shape.

Probability distributions: how belief is spread over outcomes

A classifier does not output one answer; it outputs a **distribution**: a share of belief for every possible outcome. Shares are never negative and always fill exactly 100%.

Two outcomes

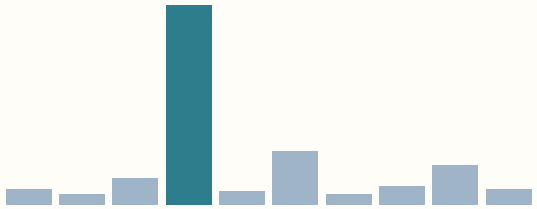
Bernoulli: spam or not



Produced by a **sigmoid** output.

Many classes

Categorical: which digit

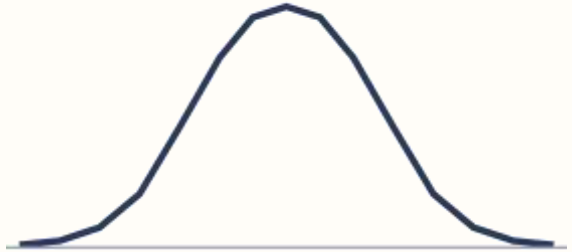


digits 0 to 9; tall bar is “3”

Produced by a **softmax** output.

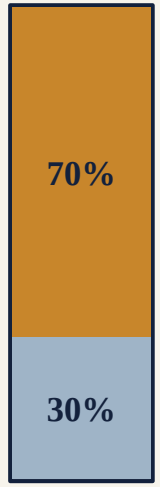
A continuous range

Gaussian: heights, noise



Used to **initialise weights** and model noise.

SHARES STACK TO 100%



Average, spread and the bell curve

The average is the balance point



Put each outcome on a beam with weight equal to its probability. The point where the beam balances is the **expected value**.

The spread is how far things wander



Same average, different spread. The **standard deviation** measures the typical distance from the average.



within 1 spread: about 68%

within 2 spreads: about 95%

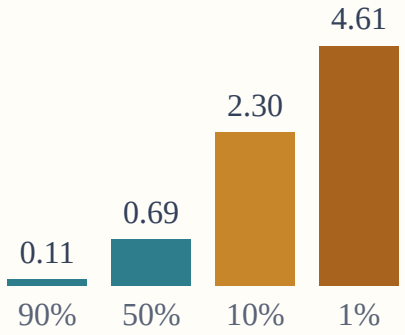
```
rng = np.random.default_rng(0)
heights = rng.normal(loc=170, scale=10,
                     size=100_000)
print("average :", round(heights.mean(), 2))
print("spread :", round(heights.std(), 2))
gap = np.abs(np.subtract(heights, 170))
print(round(np.mean(gap < 10), 3))
print(round(np.mean(gap < 20), 3))
```

OUTPUT

```
average : 169.99
spread  : 10.0
0.684
0.955
```

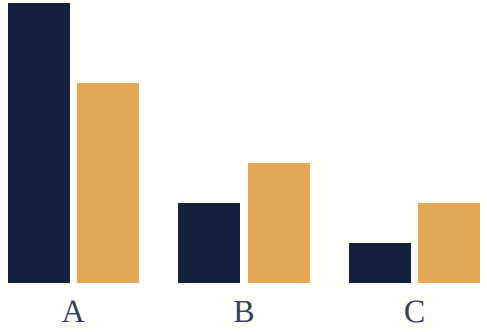
Surprise, entropy and cross entropy, the loss behind classifiers

1 · Rare means surprising



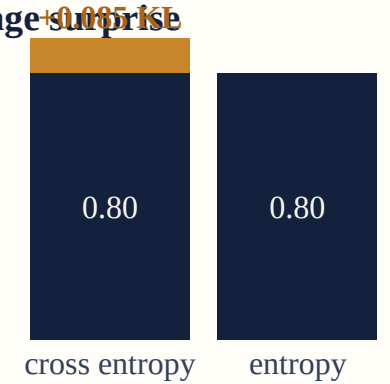
Bar height: surprise when an event of that probability happens.

2 · Truth versus belief



Dark: truth 70, 20, 10%. **Amber:** model 50, 30, 20%.

3 · Average surprise



Wrong beliefs add **extra** surprise: the amber cap.

Entropy: average surprise if you believe the truth. **Cross entropy**: average surprise if you believe the model while the world follows the truth. **KL divergence**: the gap between the two.

> **Key idea.** Training a classifier shrinks the amber cap: it pushes the model's bars toward the truth's bars.

```
p = np.array([0.7, 0.2, 0.1])  # truth
q = np.array([0.5, 0.3, 0.2])  # model
entropy = np.sum(p * np.log(1 / p))
cross = np.sum(p * np.log(1 / q))
kl = np.sum(p * np.log(p / q))
```

OUTPUT

```
entropy 0.8018
cross 0.8869
KL 0.0851
```

03

MODULE THREE

NumPy for Deep Learning

The handful of array skills that every neural network implementation relies on, each shown as a picture first and code second.

IN THIS MODULE

- Creating arrays and reading shapes
- Indexing, slicing and reshaping
- Broadcasting, drawn as stretching
- Reducing along an axis
- Why loops are slow: a timing race
- Random numbers and seeds

Creating arrays and reading their shapes

CODE · create_arrays.py

```
import numpy as np

a = np.array([1, 2, 3])       # from a Python list
M = np.zeros((2, 3))          # all zeros, given a shape
I = np.eye(3)                 # identity matrix
r = np.arange(0, 10, 2)       # start, stop, step
L = np.linspace(0, 1, 5)      # 5 evenly spaced points

for name, arr in [("a", a), ("M", M), ("I", I),
                  ("r", r), ("L", L)]:
    print(name, arr.shape, arr.dtype)
print(r)
print(L)
```

OUTPUT

```
a (3,) int64
M (2, 3) float64
I (3, 3) float64
r (5,) int64
L (5,) float64
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

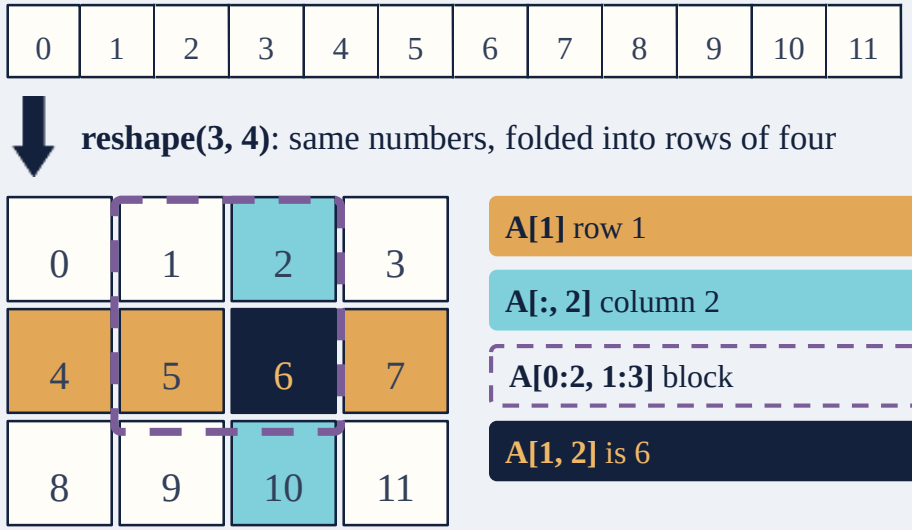
THE THREE THINGS EVERY ARRAY KNOWS

shape	dtype	ndim
its size per axis	number type	number of axes

Watch the dtype. Integers come out as int64; neural network maths needs float64 or float32, so weights are always created as floats.

Reshaping, indexing and slicing, seen on a grid

STEP 1 · TWELVE NUMBERS IN A LINE

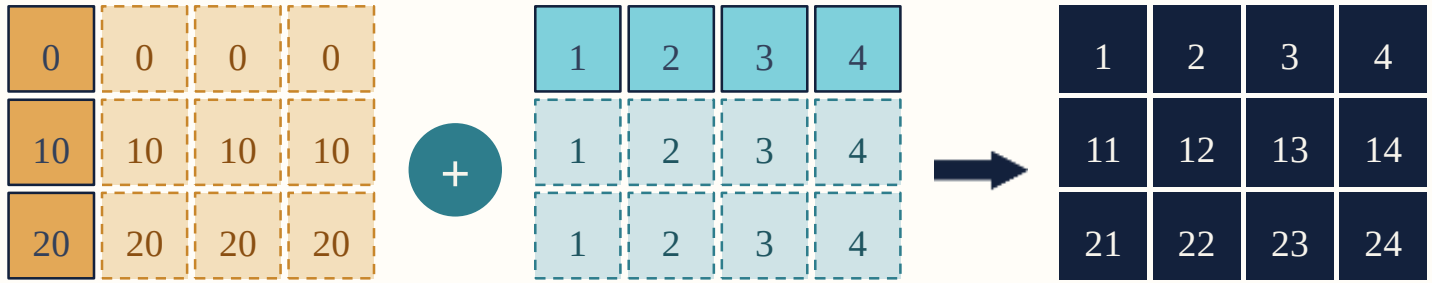



```
x = np.arange(12)
A = x.reshape(3, 4)
print(A)
print(A[1])        # second row
print(A[:, 2])     # third column
print(A[0:2, 1:3]) # a block
print(A.T.shape)
print(A.reshape(2, 6))
```

OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[4 5 6 7]
[ 2  6 10]
[[1 2]
 [5 6]]
(4, 3)
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

Broadcasting: small arrays stretch to fit big ones



column (3, 1) stretched right

row (4,) stretched down

result (3, 4)

THE RULE

Line the shapes up from the **right**. Each pair of sizes must be equal, or one of them must be 1. A size 1 axis is copied to fit.

WHERE YOU SEE IT

Adding a bias of shape (128,) to layer outputs of shape (64, 128): the bias stretches down over all 64 examples.

No copying happens. The dashed ghosts are virtual; NumPy reuses the same memory, so it is fast.

Broadcasting in code: the grid, then a real use

CODE · broadcasting.py

```
col = np.array([[0], [10], [20]])   # shape (3, 1)
row = np.array([1, 2, 3, 4])        # shape (4,)
print((col + row).shape)
print(col + row)

# real use: rescale each feature column
# by its own maximum so it lies in 0 to 1
X = np.array([[1.0, 20.0],
              [3.0, 40.0],
              [4.0, 80.0]])     # shape (3, 2)
print(X.max(axis=0))            # shape (2,)
print(X / X.max(axis=0))
```

OUTPUT

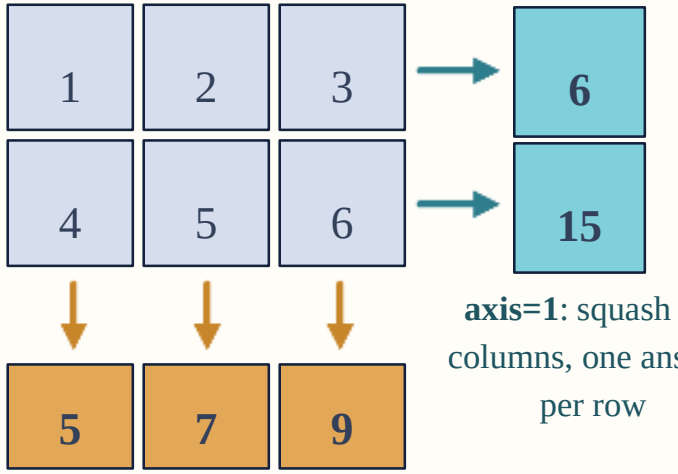
```
(3, 4)
[[ 1  2  3  4]
 [11 12 13 14]
 [21 22 23 24]]
[ 4. 80.]
[[0.25 0.25]
 [0.75 0.5 ]
 [1.   1.  ]]
```

(3, 2) ÷ (2,) → (3, 2)

Why rescale? Features on wildly different scales (1 to 4 versus 20 to 80) slow training. Rescaling is standard preprocessing.

Reducing along an axis: which direction gets squashed

A HAS SHAPE (2, 3)



axis=1: squash the columns, one answer per row

axis=0: squash the rows one answer per column

Rule of thumb: the axis you name is the one that **disappears**. Shape (2, 3) summed over axis 0 becomes (3,); over axis 1 becomes (2,).

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.sum())                # everything
print(A.sum(axis=0))          # per column
print(A.sum(axis=1))          # per row
print(A.mean(axis=1, keepdims=True))
print(A.argmax(axis=1))       # index of max
```

OUTPUT

```
21
[5 7 9]
[ 6 15]
[[2.]
 [5.]]
[2 2]
```

In networks: averaging the loss over a batch is a reduction over axis 0; argmax over axis 1 turns class scores into predicted labels.

The vectorisation race: a Python loop against NumPy

Task: add up the squares of **one million** random numbers. Time taken (shorter is better):




```
import time
x = np.random.default_rng(0).random(1_000_000)

t0 = time.perf_counter()
s = 0.0
for v in x:                    # one number at a time
    s += v * v
t1 = time.perf_counter()
s2 = np.dot(x, x)          # all at once, in compiled code
t2 = time.perf_counter()
loop_t, vec_t = np.subtract(t1, t0), np.subtract(t2, t1)
print(f"python loop : {loop_t:.4f} s")
print(f"np.dot      : {vec_t:.4f} s")
print(f"speedup     : {loop_t / vec_t:.0f}x")
print(f"same answer : {np.isclose(s, s2)}")
```

OUTPUT

```
python loop : 0.1042 s
np.dot      : 0.0012 s
speedup     : 88x
same answer : True
```

> **Rule.** Never loop over individual numbers in network code. Loop over batches or epochs only; let NumPy handle the elements.

Exact timings vary by machine; the gap of roughly two orders of magnitude does not.

Random numbers and seeds: reproducible experiments




```
rng = np.random.default_rng(seed=42)
print(rng.random(3))                # uniform in 0 to 1
print(rng.integers(0, 10, size=5))  # whole numbers

rng2 = np.random.default_rng(seed=42)
print(rng2.random(3))               # same as line one

# typical use: small random starting weights
W = rng.normal(0.0, 0.01, size=(784, 128))
print(W.shape, round(W.std(), 4))
```

OUTPUT

```
[0.77395605 0.43887844 0.85859792]
[0 6 2 0 5]
[0.77395605 0.43887844 0.85859792]
(784, 128) 0.01
```

Always set a seed when experimenting. Then a change in results comes from your change in code, not from luck.

04

MODULE FOUR

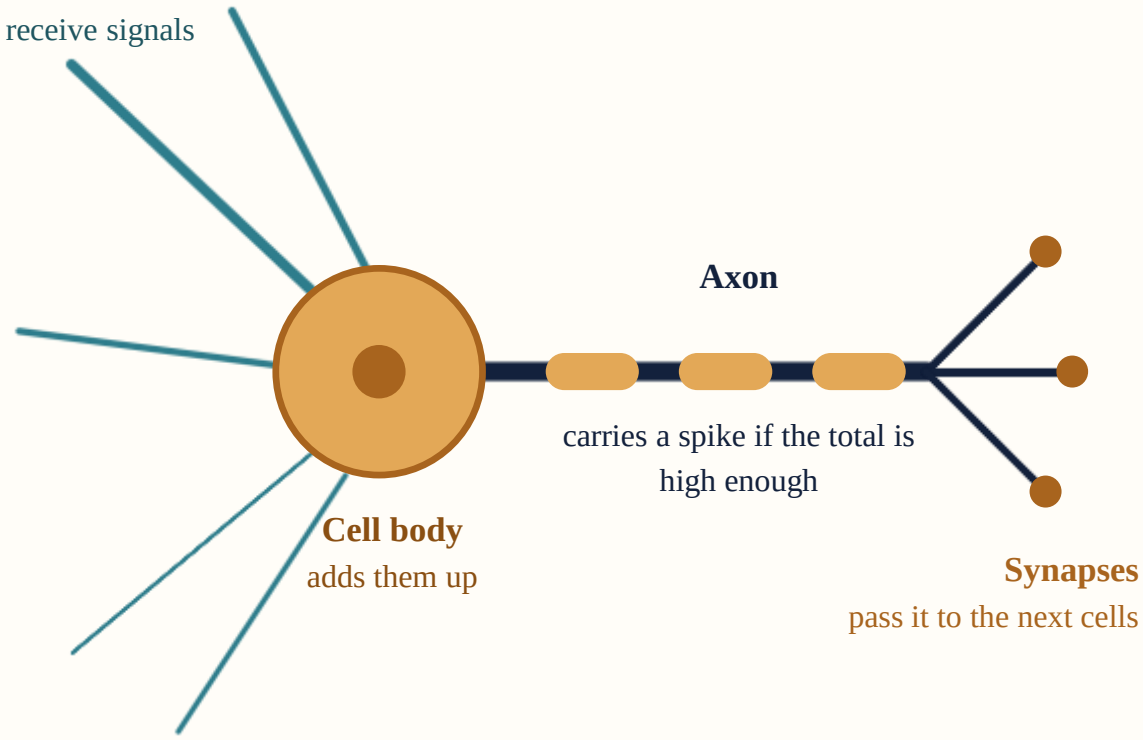
The Neuron and the Perceptron

The smallest learning machine: a single neuron that weighs evidence, makes a yes or no decision and corrects itself when wrong.

IN THIS MODULE

- The biological inspiration
- Anatomy of an artificial neuron
- Weighted sums with real numbers
- What the bias does
- The perceptron learning rule
- Learning logic gates in code
- The XOR wall

The biological inspiration

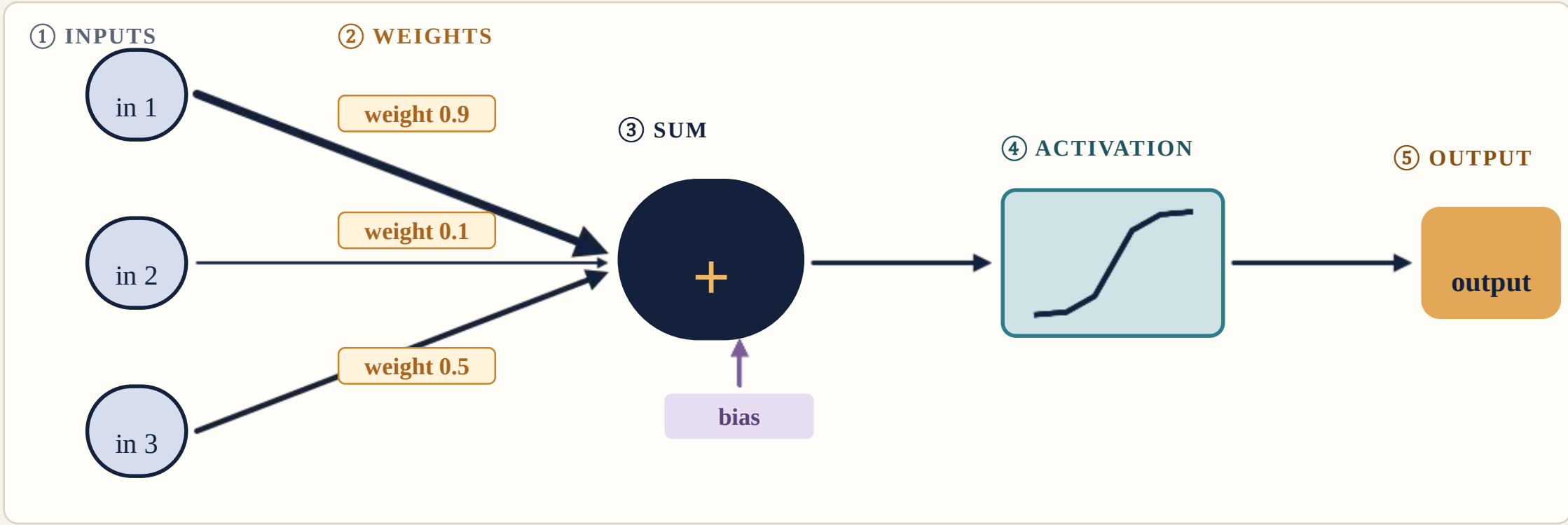


Thicker dendrite lines stand for stronger connections.

In the brain	In our model
dendrites	inputs: the numbers coming in
synapse strength	weights: how much each input counts
cell body	weighted sum of the inputs
firing threshold	bias plus an activation function
axon spike	output, fed to the next layer
learning	adjusting the weights

Honest caveat. Real neurons are vastly more complex. The artificial neuron borrows only the rough idea: weigh evidence, then decide.

Structure of an artificial neuron



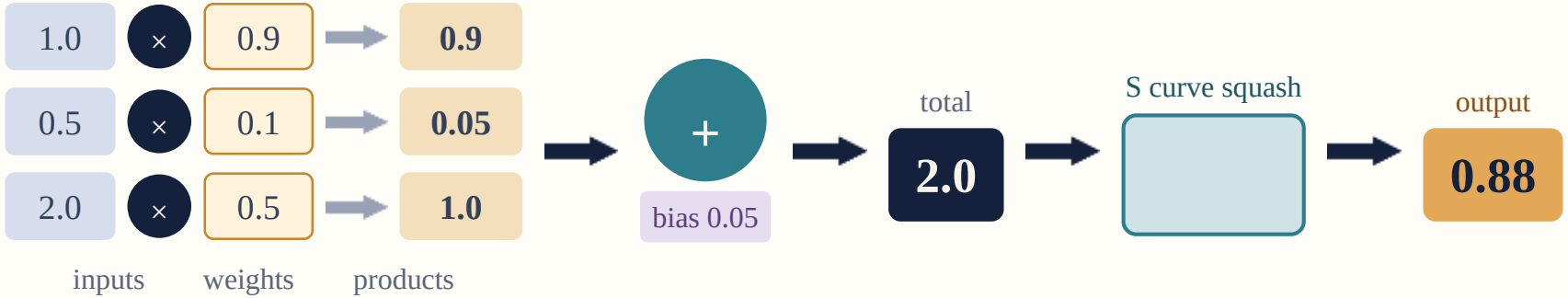
Weights set how much each input matters. Line thickness shows strength.

Sum multiplies each input by its weight and adds, plus a bias.

Activation squashes the total into a useful range.

Learning changes only the weights and the bias.

One neuron, real numbers, start to finish




```
x = np.array([1.0, 0.5, 2.0])   # inputs
w = np.array([0.9, 0.1, 0.5])   # weights
b = 0.05                        # bias
print("products:", w * x)
z = np.dot(w, x) + b
print("sum plus bias:", round(z, 4))
a = 1 / (1 + np.exp(np.negative(z)))  # S curve
print("activation:", round(a, 4))
```

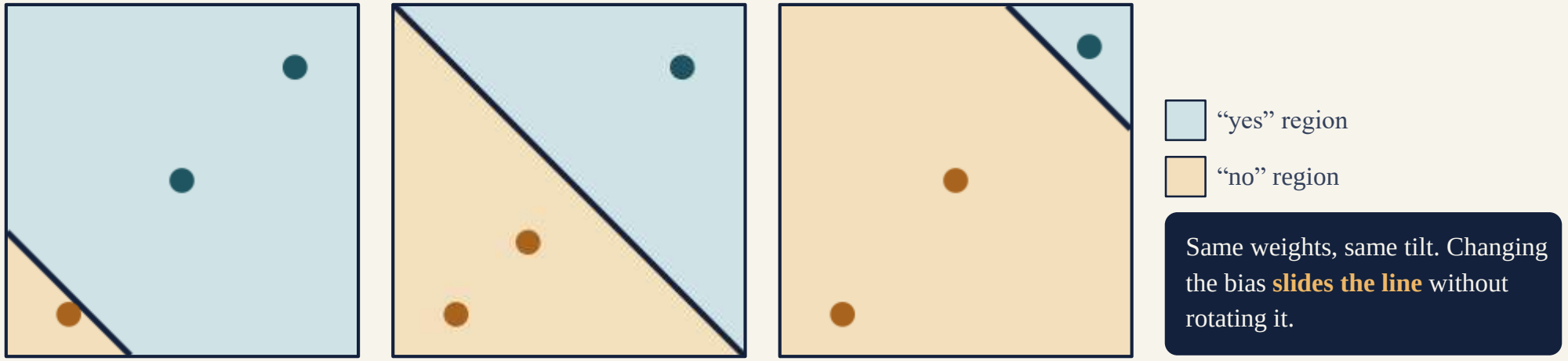
OUTPUT

```
products: [0.9  0.05 1.  ]
sum plus bias: 2.0
activation: 0.8808
```

Reading it: input 3 contributed most because it was large *and* strongly weighted. The neuron is 88% “on”.

Weights tilt the boundary, the bias slides it

A neuron with two inputs splits the plane with a straight line: on one side it says “yes”, on the other “no”. Here the weights stay fixed and only the bias changes.



generous bias

says yes to almost everything

balanced bias

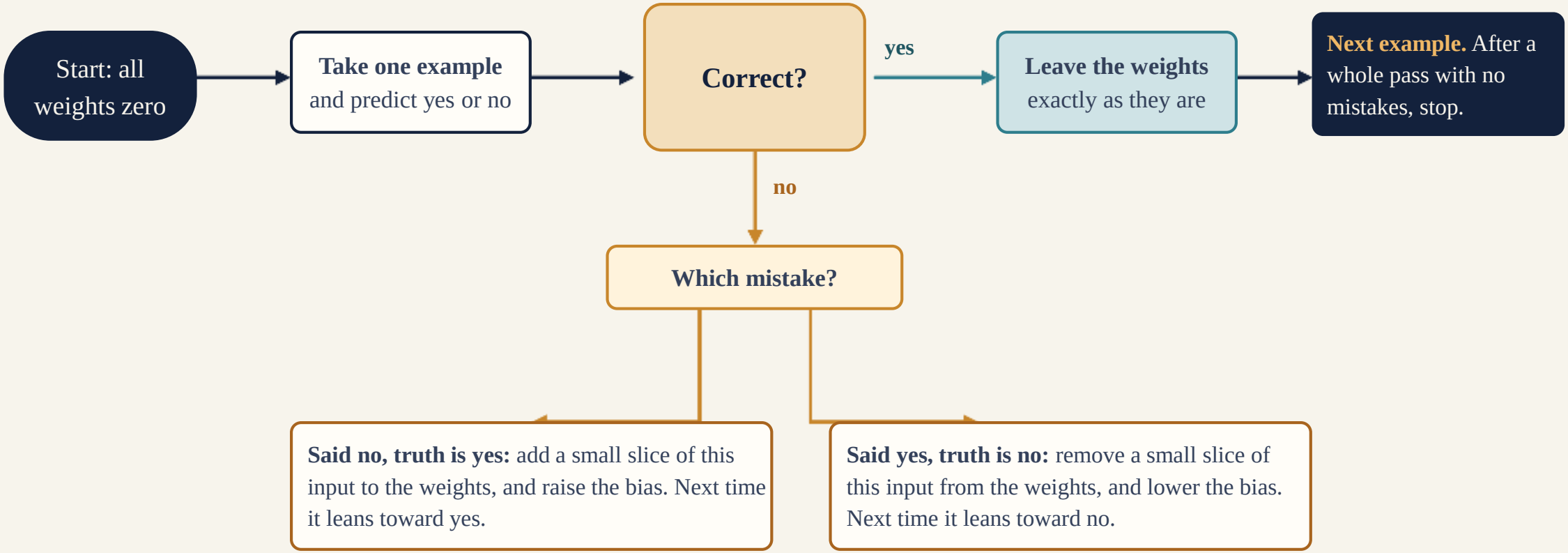
the line cuts the middle

strict bias

says yes only to extreme inputs

Key idea. Without a bias every boundary would be forced through the corner where all inputs are zero. The bias gives the neuron freedom to place its decision anywhere.

The perceptron learning rule as a flowchart



The size of the “small slice” is the **learning rate**. The perceptron only ever learns from its mistakes.

The perceptron from scratch in 20 lines

CODE · perceptron.py

```
import numpy as np

def predict(w, b, xi):
    return 1 if np.dot(w, xi) + b > 0 else 0

def train_perceptron(X, y, lr=0.1, epochs=10):
    w = np.zeros(X.shape[1])      # start with no opinion
    b = 0.0
    for epoch in range(epochs):
        errors = 0
        for xi, target in zip(X, y):
            update = lr * np.subtract(target, predict(w, b, xi))
            w = w + update * xi   # nudge toward the input
            b = b + update        # nudge the bias
            errors += int(update != 0)
        print(f"epoch {epoch}: {errors} mistakes")
        if errors == 0:
            break
    return w, b
```

HOW THE UPDATE LINE WORKS

Truth	Guess	update
1	1	0: no change
0	0	0: no change
1	0	+0.1: lean to yes
0	1	−0.1: lean to no

One line handles all four cases: the difference between truth and guess is either zero, or a push in the right direction.

Epoch means one full pass through every training example.

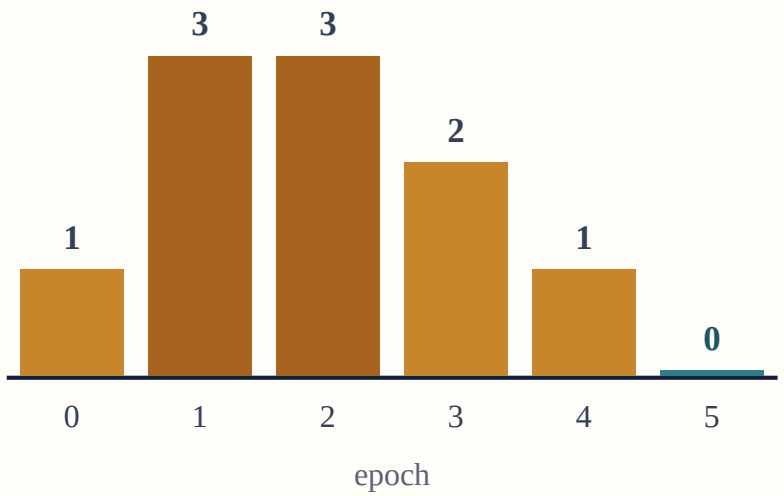
Training on the AND gate: watching mistakes vanish

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])
w, b = train_perceptron(X, y_and)
print("weights :", w)
print("threshold:", round(np.negative(b), 2))
print("AND preds:", [predict(w, b, xi) for xi in X])
```

OUTPUT

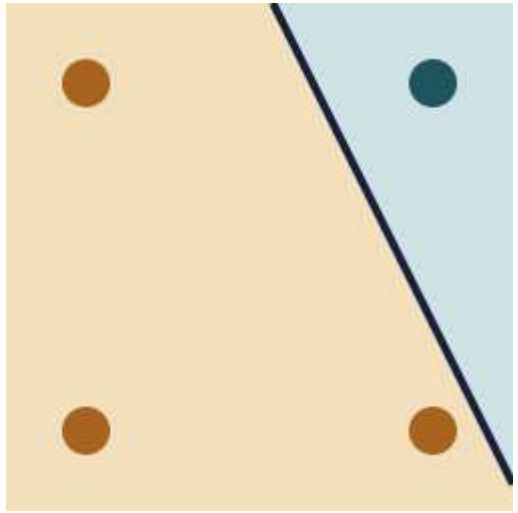
```
epoch 0: 1 mistakes
epoch 1: 3 mistakes
epoch 2: 3 mistakes
epoch 3: 2 mistakes
epoch 4: 1 mistakes
epoch 5: 0 mistakes
weights  : [0.2 0.1]
threshold: 0.2
AND preds: [0, 0, 0, 1]
```

Mistakes per epoch



Learning is not smooth: mistakes rise before they fall, as each fix can break another example.

The learned boundary



Teal dot: input (1, 1), the only “yes”. The line keeps it apart from the three amber “no” corners.

One neuron can be AND, OR or NOT, depending on its settings

AND

weights 1 and 1 | fire above 1.5

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

Only both inputs together reach a total of 2, clearing the bar.

OR

weights 1 and 1 | fire above 0.5

a	b	out
0	0	0
0	1	1
1	0	1
1	1	1

Same weights as AND; only the threshold dropped. Any one input is enough.

NOT

weight −1 | fire above −0.5

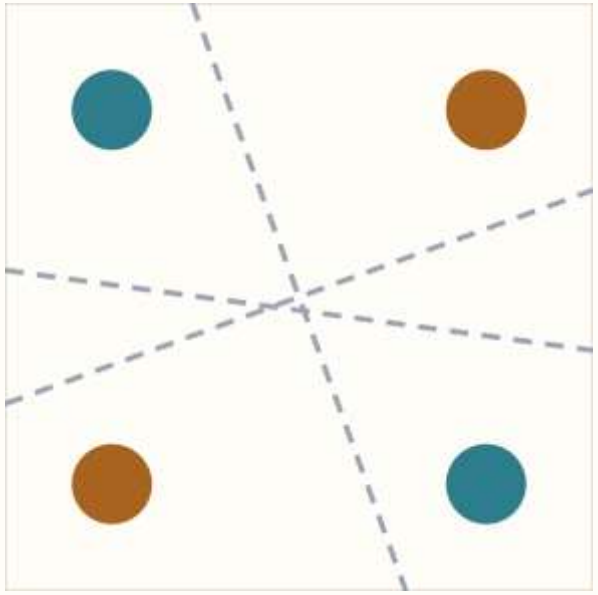
a	out
0	1
1	0

A negative weight is an **inhibitor**: the input argues against firing.

Key idea. The same neuron becomes different "programs" purely through its weights and threshold. Learning is the search for the right settings.

The XOR wall: no single straight line can do it

XOR: “one or the other, not both”



a	b	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The same colours sit on **opposite corners**. Every grey dashed line leaves at least one dot on the wrong side. Try it with a ruler: it is impossible.

THE CONSEQUENCE

In 1969 this simple fact cooled interest in neural networks for over a decade. The fix, a **hidden layer**, arrives in Module 8.

```
y_xor = np.array([0, 1, 1, 0])
w, b = train_perceptron(X, y_xor, epochs=8)
print("XOR preds:",
      [predict(w, b, xi) for xi in X])
```

OUTPUT

```
epoch 0: 2 mistakes
epoch 1: 3 mistakes
epoch 2: 4 mistakes
epoch 3: 4 mistakes
epoch 4: 4 mistakes
epoch 5: 4 mistakes
epoch 6: 4 mistakes
epoch 7: 4 mistakes
XOR preds: [1, 1, 0, 0]
```

It never settles. Half its answers stay wrong forever.

05

MODULE FIVE

Linear and Logistic Regression

Our first models trained with a smooth loss and gradient descent: predicting numbers, then predicting probabilities.

IN THIS MODULE

- Fitting a line through points
- Squared error drawn as real squares
- The tug of war that moves the line
- Linear regression in code
- The S curve and probabilities
- Cross entropy, the classifier loss
- Logistic regression and its metrics

Linear regression: draw the best straight line

We have pairs of numbers: an input and a target. We want a straight line that predicts the target from the input.

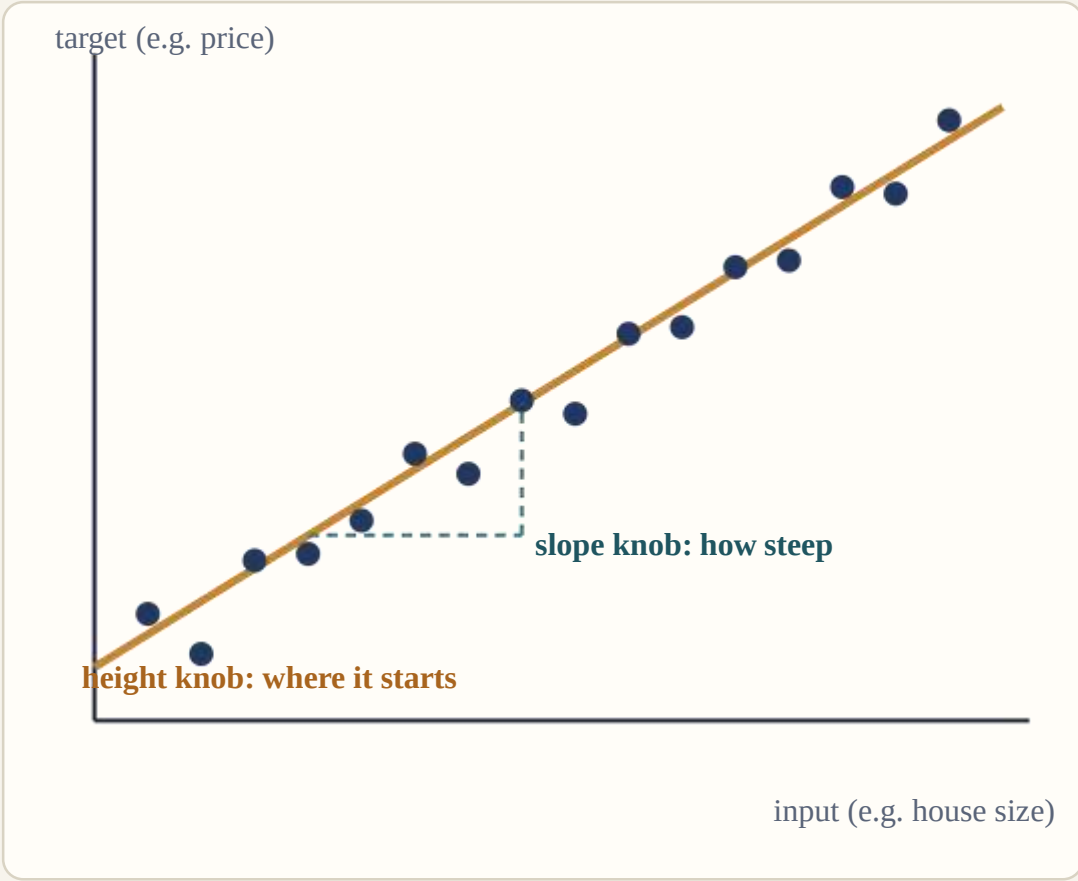
A line has exactly **two knobs**:

Slope (the weight)
rise per step of input

Height (the bias)
value when input is zero

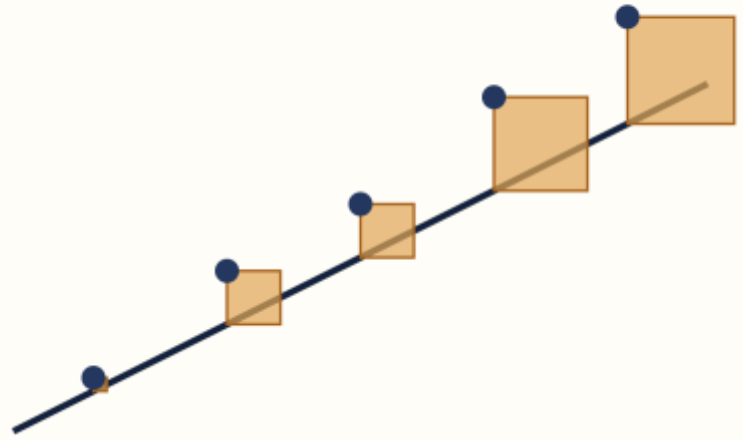
It is a **single neuron with no activation**: input times weight, plus bias.

> **The question:** which knob settings make the line "best"? We need a way to score a line. That score is the loss.

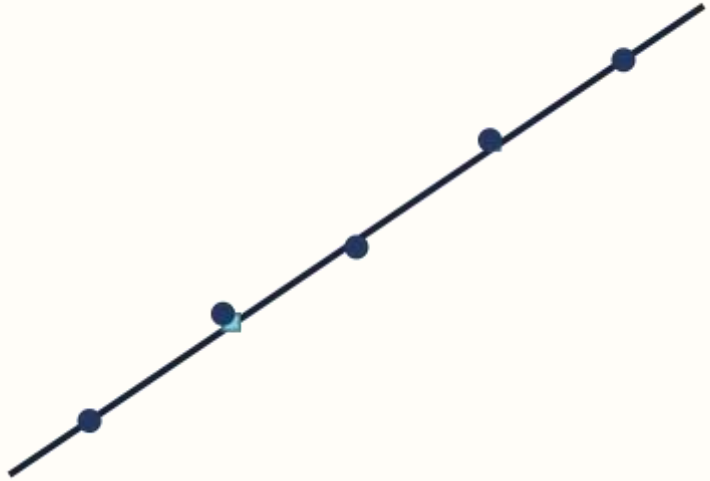


Mean squared error, drawn as actual squares

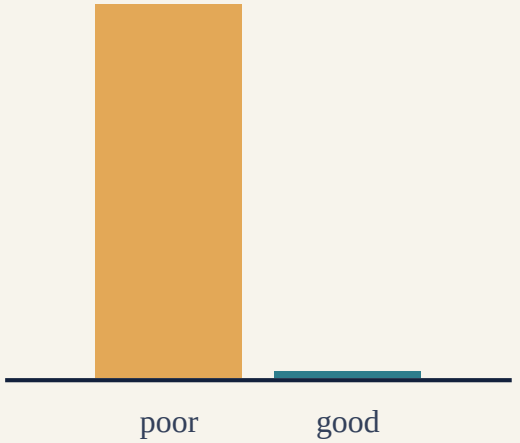
A poor line: big squares



A good line: tiny squares



AVERAGE SQUARE AREA



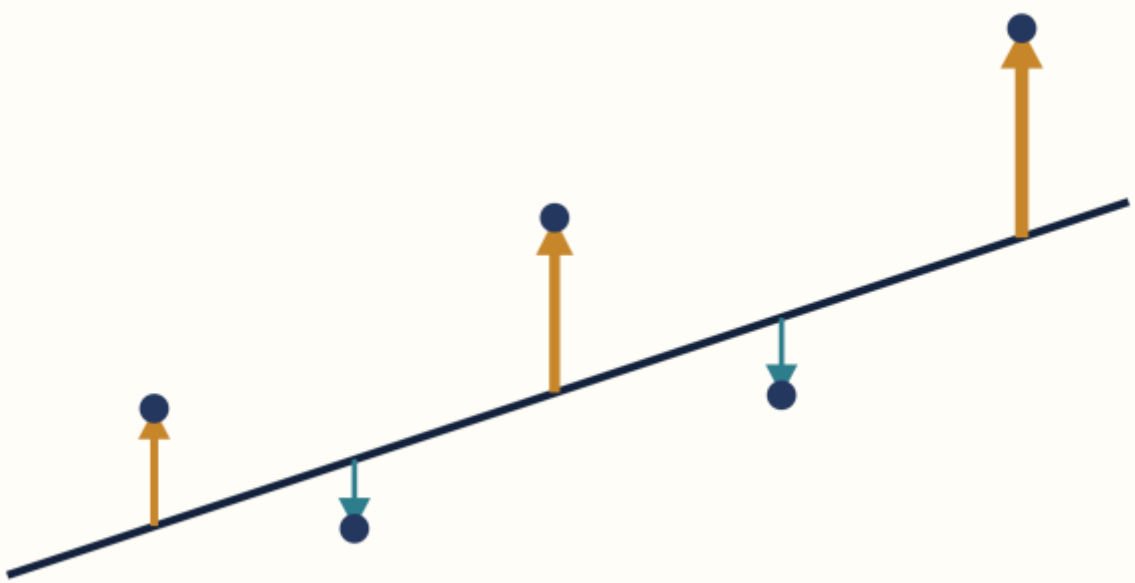
About **60 times** smaller for the good line.

The recipe: for each point, measure the vertical miss, build a square on it, then average the areas.

Why squares? Misses above and below cannot cancel out, and big misses are punished much more than small ones.

Training means turning the two knobs until the total square area is as small as possible.

A tug of war: every point pulls on the line



Amber arrows: points above the line pull it up. Teal arrows: points below pull it down. Arrow thickness shows the size of the miss.

PULL ON THE HEIGHT KNOB

Add up all the misses, ups as positive and downs as negative, and take the average. Net pull up here, so raise the line.

PULL ON THE SLOPE KNOB

Same misses, but each one is weighted by how far right its point sits, like a longer lever. The far right point pulls hardest, so steepen the line.

These two average pulls **are the gradient**. Gradient descent moves each knob a small step in the direction of its pull, then measures again.

> When all pulls balance, the line stops moving: that is the best fit.

Linear regression trained by gradient descent

```
import numpy as np
rng = np.random.default_rng(0)

# 1 · invent data: hidden slope 3, hidden height 2, plus noise
N = 100
x = rng.random(N) * 10
y = 3.0 * x + 2.0 + rng.normal(0.0, 1.0, N)

# 2 · start the two knobs at zero
w, b = 0.0, 0.0
lr = 0.01                  # learning rate: step size

for epoch in range(2001):
    y_hat = w * x + b                  # 3 · predict
    error = np.subtract(y_hat, y)      # 4 · the misses
    loss = np.mean(error ** 2)         # 5 · average square area
    dw = 2 * np.mean(error * x)        # 6 · pull on the slope
    db = 2 * np.mean(error)            #     pull on the height
    w = np.subtract(w, lr * dw)        # 7 · step against the pull
    b = np.subtract(b, lr * db)
    if epoch in (0, 1, 2, 5, 10, 50, 100, 500, 1000, 2000):
        print(f"epoch {epoch:4d}  loss {loss:9.4f}  w {w:.3f}  b {b:.3f}")
```

THE SEVEN STEPS

1 · make data with a known answer

2 · start with knobs at zero

3 · predict every point

4 · measure the misses

5 · score them: the loss

6 · compute the pulls: the gradient

7 · step each knob, then repeat

> Steps 3 to 7 are the training loop from slide 8. Every model in this course repeats them.

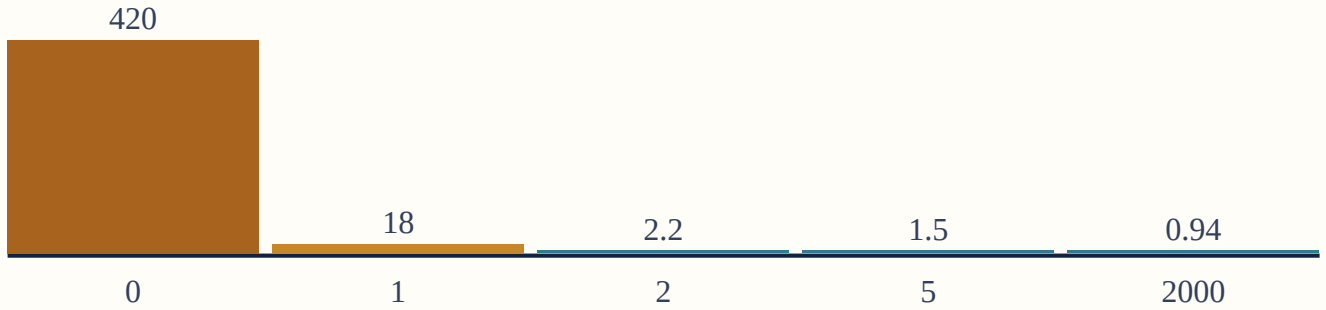
Watching the knobs find the hidden answer

OUTPUT

```
epoch    0  loss  420.3451  w 2.563  b 0.367
epoch    1  loss   18.1800  w 3.074  b 0.446
epoch    2  loss    2.1517  w 3.175  b 0.468
epoch    5  loss    1.4719  w 3.198  b 0.492
epoch   10  loss    1.4480  w 3.193  b 0.526
epoch   50  loss    1.2915  w 3.158  b 0.773
epoch  100  loss    1.1619  w 3.123  b 1.024
epoch  500  loss    0.9456  w 3.009  b 1.839
epoch 1000  loss    0.9401  w 2.989  b 1.977
epoch 2000  loss    0.9401  w 2.987  b 1.992
```

Why not exactly 3 and 2? The noise we added nudges the best possible line slightly. The loss cannot reach zero; it settles near the noise level.

The loss collapses in two epochs

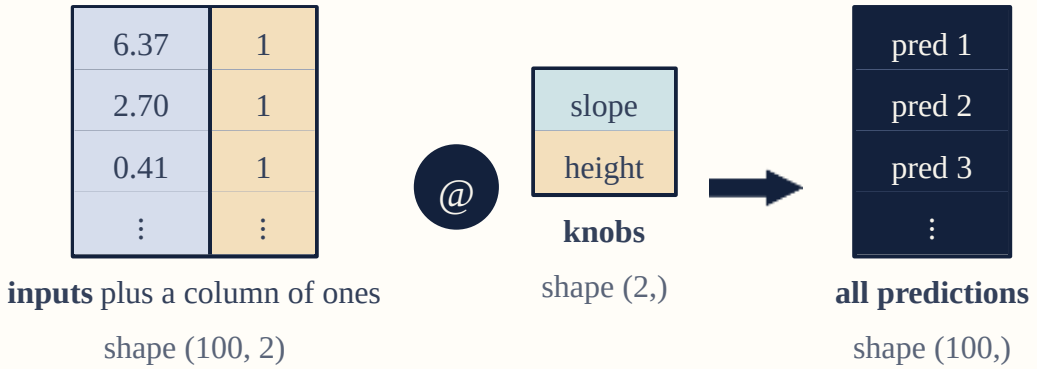


Knobs versus the hidden truth



The slope is found almost at once; the height creeps up slowly over hundreds of epochs.

The batch view, and a shortcut that confirms our answer



inputs plus a column of ones

shape (100, 2)

knobs

shape (2,)

all predictions

shape (100,)

> The column of ones turns the height knob into just another weight. One matrix product predicts all 100 points at once.

```
# build the batch matrix: inputs beside a column of ones
X = np.column_stack([x, np.ones(N)])

# NumPy can solve this small problem exactly, in one call
sol = np.linalg.lstsq(X, y, rcond=None)[0]
print("exact solution:", np.round(sol, 3))
```

OUTPUT

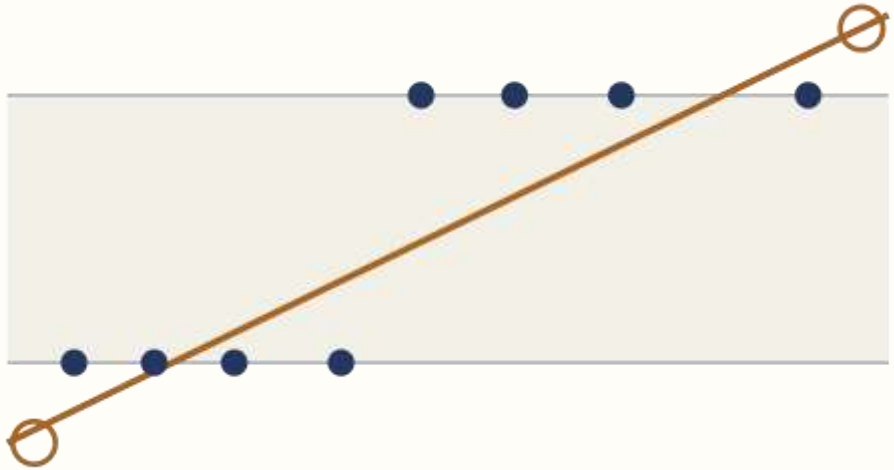
exact solution: [2.987 1.993]

Gradient descent reached 2.987 and 1.992. It matches. But exact shortcuts only exist for straight lines; deep networks **must** use gradient descent.

Why a straight line is wrong for yes or no answers

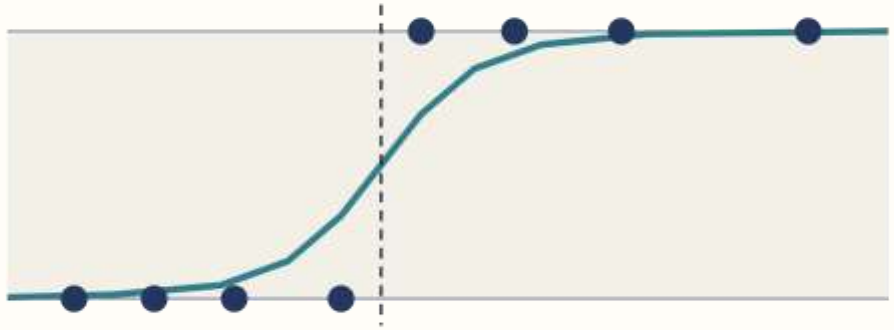
Task: predict whether a student passes (1) or fails (0) from hours studied. The answers are only ever 0 or 1.

Straight line: breaks the rules



Circled: predictions below 0 and above 1, which are meaningless as probabilities. One very studious outlier would also tilt the whole line.

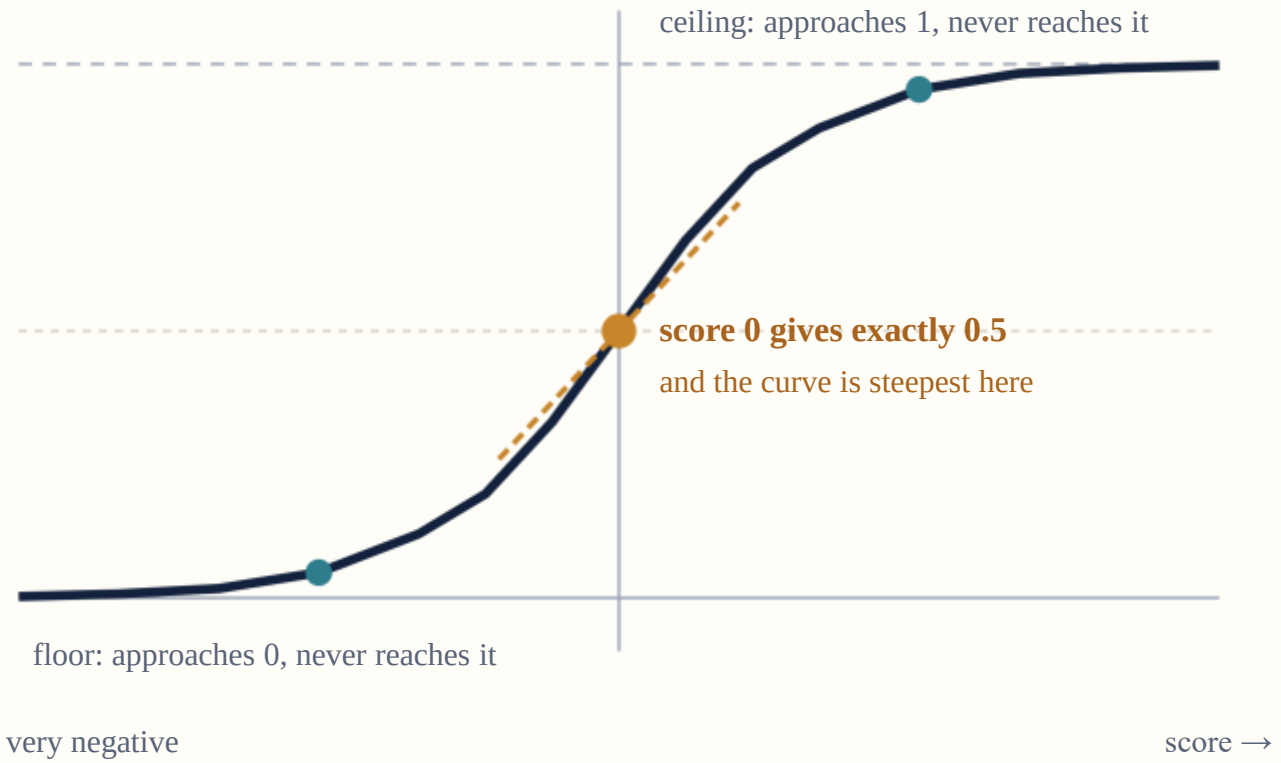
S curve: stays between 0 and 1



Every output is a valid probability. The dashed line marks the 50% point: the decision boundary.

The fix: keep the neuron’s weighted sum, but pass it through the S curve. That single change turns linear regression into **logistic regression**.

The sigmoid: an S curve that turns scores into probabilities



score in	probability out
−4	0.018
−2	0.119
0	0.5
2	0.881
4	0.982

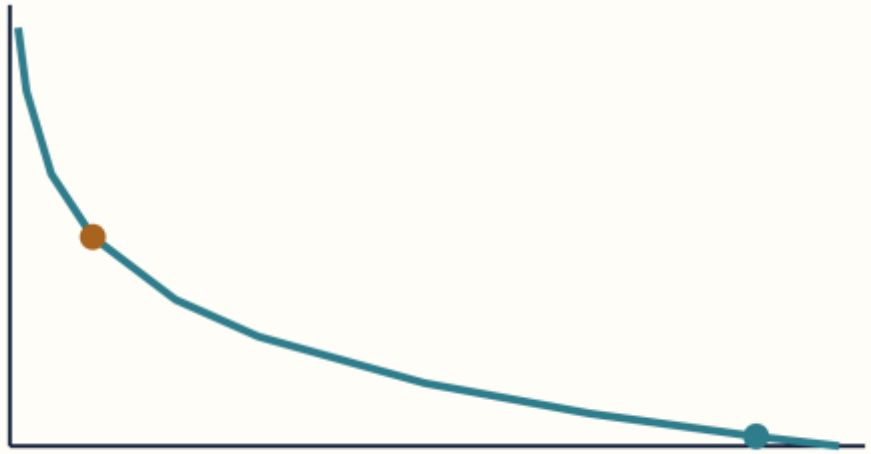
```
def sigmoid(z):
    return 1 / (1 + np.exp(np.negative(z)))
z = np.linspace(np.negative(4), 4, 5)
print(np.round(sigmoid(z), 3))
```

OUTPUT [0.018 0.119 0.5 0.881 0.982]

Symmetry: score 2 gives 0.881 and score −2 gives 0.119; together they make 1.

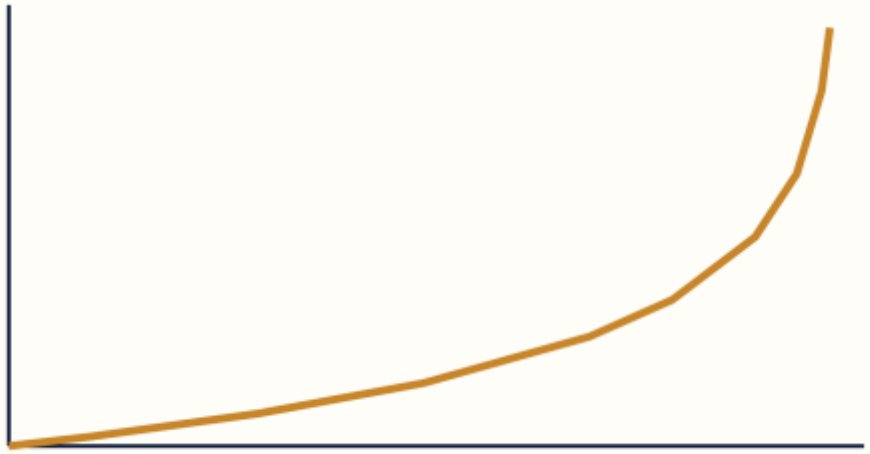
Binary cross entropy: confident mistakes cost the most

When the truth is “yes”



model says 0 predicted probability of yes model says 1

When the truth is “no”



model says 0 predicted probability of yes model says 1

probability given to the truth	loss
0.99 · nearly certain and right	0.01
0.5 · a coin flip	0.693
0.01 · nearly certain and wrong	4.605

Key idea. This is the surprise from slide 40: the loss is the surprise felt at the true answer. Being sure and wrong is punished almost without limit, which forces honest probabilities.

A beautiful shortcut: the learning signal is just the error

When the S curve and cross entropy work together, all the complicated slopes cancel, and the push on the score becomes simply **prediction minus truth**.

CASE A

truth **1** · predicted **0.9**



signal −0.1

almost right: a gentle nudge up

CASE B

truth **1** · predicted **0.2**



signal −0.8

badly wrong: a strong push up

CASE C

truth **0** · predicted **0.7**



signal +0.7

wrongly confident: push down

error × that weight's input → push on that weight averaged over the batch

Same shape as linear regression! Only the prediction changed. This pairing appears again with softmax in every classifier.

Logistic regression from scratch on two clouds of points

```
rng = np.random.default_rng(1)
N = 100
X0 = rng.normal([2.0, 2.0], 1.0, size=(N, 2))  # class 0 cloud
X1 = rng.normal([5.0, 5.0], 1.0, size=(N, 2))  # class 1 cloud
X = np.vstack([X0, X1])
y = np.concatenate([np.zeros(N), np.ones(N)])

def sigmoid(z):
    return 1 / (1 + np.exp(np.negative(z)))

w, b, lr = np.zeros(2), 0.0, 0.1
eps = 0.000000001            # keeps log away from zero
for epoch in range(1001):
    p = np.clip(sigmoid(X @ w + b), eps, np.subtract(1, eps))
    loss = np.mean(y * np.log(1 / p) +
           np.subtract(1, y) * np.log(1 / np.subtract(1, p)))
    error = np.subtract(p, y)        # prediction minus truth
    dw = X.T @ error / len(y)        # error times input, averaged
    db = np.mean(error)
    w = np.subtract(w, lr * dw)
    b = np.subtract(b, lr * db)
    if epoch in (0, 10, 50, 100, 200, 500, 1000):
        acc = np.mean((p > 0.5) == y)
        print(f"epoch {epoch:4d}  loss {loss:.4f}  accuracy {acc:.3f}")
```

COMPARE WITH LINEAR REGRESSION

Step	What changed
predict	add the S curve
loss	cross entropy, not squares
error	unchanged
pulls	unchanged, now as a matrix product
step	unchanged

Clipping the probabilities stops the logarithm from ever seeing an exact 0, which would give infinity.

X.T @ error computes the pull on **both** weights for **all 200** points in one product.

The clouds are separated, with honest probabilities

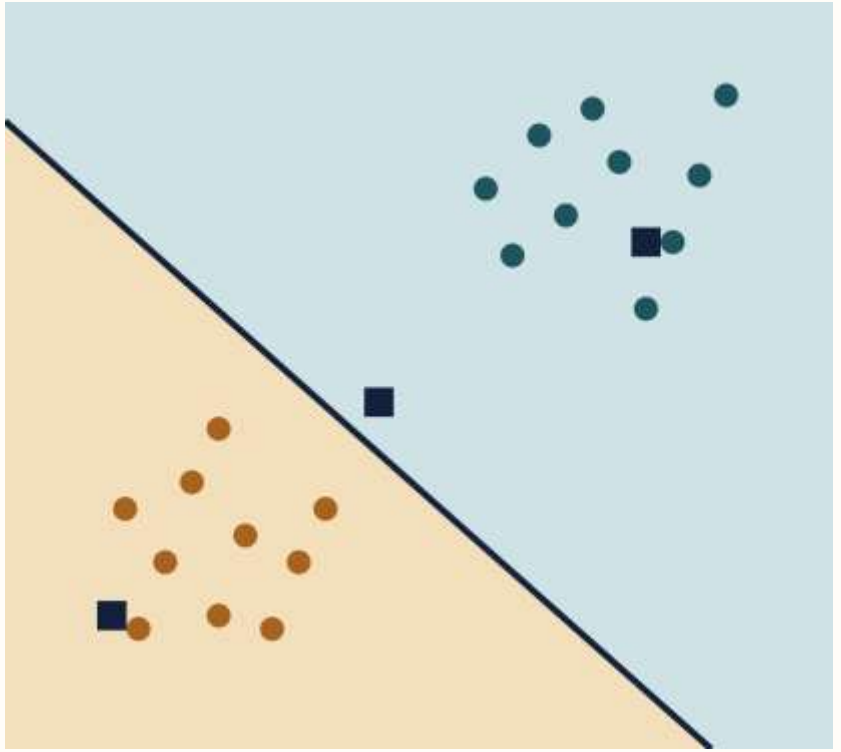
OUTPUT

```
epoch    0  loss 0.6931  accuracy 0.500
epoch   10  loss 0.5850  accuracy 0.505
epoch   50  loss 0.4827  accuracy 0.725
epoch  100  loss 0.3932  accuracy 0.890
epoch  200  loss 0.2860  accuracy 0.975
epoch  500  loss 0.1622  accuracy 0.995
epoch 1000  loss 0.1004  accuracy 1.000
```

0.6931 at the start is no accident: with all weights zero every prediction is 0.5, and the surprise of a coin flip is 0.693.

```
new = np.array([[1.0, 1.5],
                [3.5, 3.5],
                [6.0, 5.0]])
print(np.round(sigmoid(new @ w + b), 3))
```

OUTPUT [0.025 0.616 0.984]



Squares: the three new points. Far from the line: confident (0.025 and 0.984). Near the line: unsure (0.616).

Beyond accuracy: the confusion matrix, precision and recall

Worked example: a spam filter checks 100 emails, 50 of which really are spam.

	predicted spam	predicted not spam
really spam	**40** caught	**10** missed
really not spam	**5** false alarm	**45** correctly ignored

Accuracy: right overall · 85 of 100



Precision: of the flagged emails, how many were spam? · 40 of 45



Recall: of the real spam, how much was caught? · 40 of 50



F1 score blends precision and recall into one number, punishing whichever is weaker. Here about 0.84.

Why bother? If only 1% of emails were spam, a model saying “never spam” scores 99% accuracy and catches nothing.

06

MODULE SIX

Gradient Descent in Depth

The engine that trains every network: how big each step should be, how much data each step sees, and what the terrain looks like.

IN THIS MODULE

Walking downhill in fog

One update, every part labelled

Learning rate: too small, too big

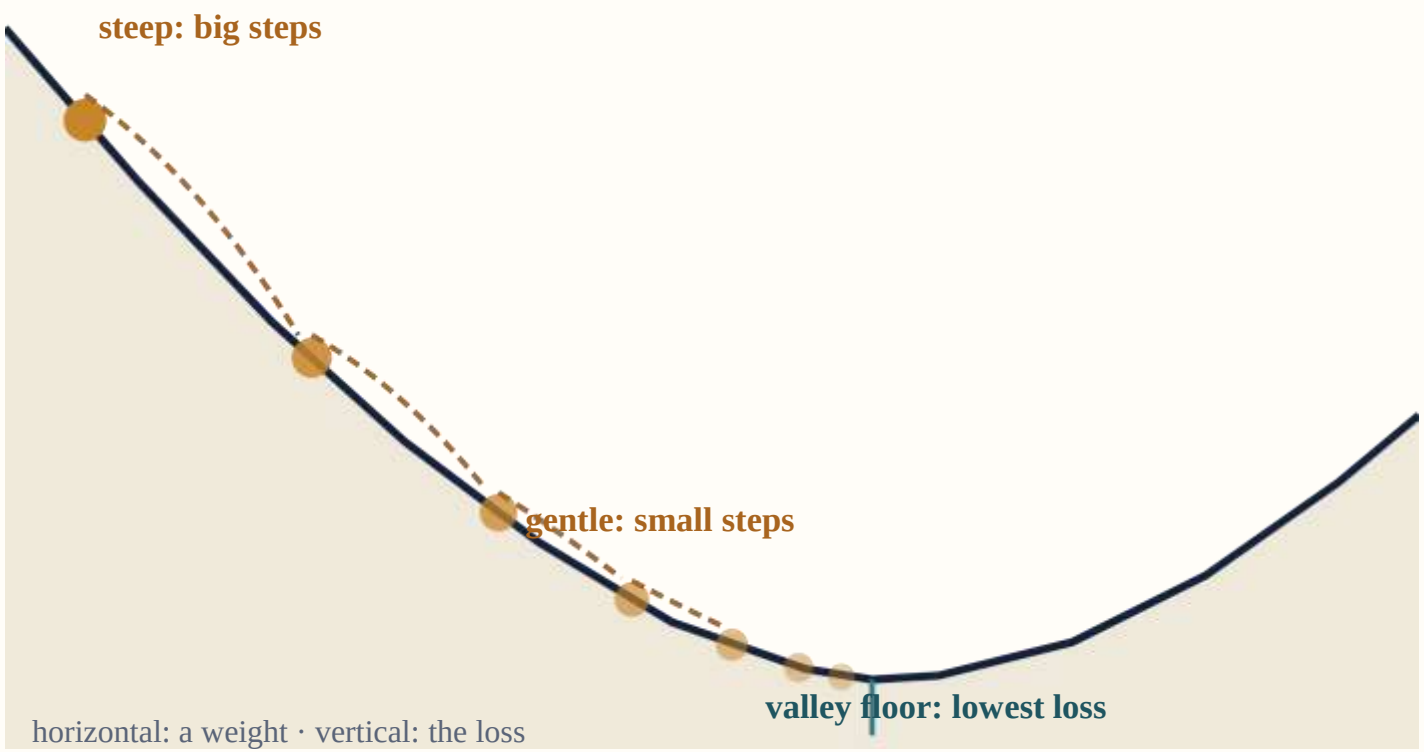
A learning rate experiment

Batch, stochastic and mini batch

Epochs and iterations

Valleys, saddles and plateaus

Walking downhill in thick fog



You cannot see the valley. You can only feel the slope under your feet. So you repeat:

1. Feel the slope here (the gradient).
2. Step downhill, against the slope.
3. Step size is slope times learning rate.
4. Repeat until the ground is flat.

> **Notice:** steps shrink automatically near the bottom because the slope shrinks. No one has to tell the walker to slow down.

One update, every part labelled



The slope

Computed fresh at every step from the current batch. It says which way is uphill and how steep. We go the other way.

The learning rate

A number **you** choose, typically 0.1 down to 0.0001. It is the single most important setting in deep learning.

Every weight at once

The same recipe is applied to all weights in parallel, each with its own slope, a million times per step if needed.

In code it is always one line per weight array: `w = np.subtract(w, lr * grad)`

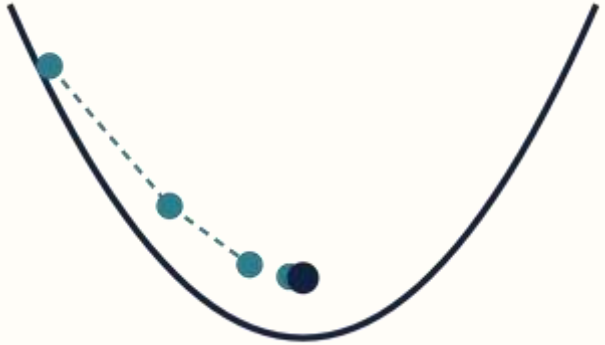
Too small, just right, too big

Too small



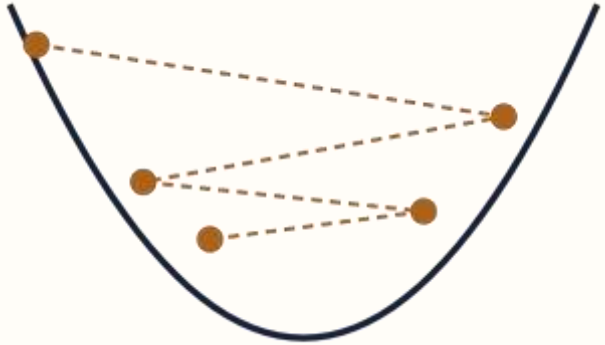
Safe but painfully slow; may run out of time far from the bottom.

Just right



Big strides early, small near the bottom. Arrives quickly and settles.

Too big



Overshoots the bottom each time, bouncing higher until the loss explodes.

Symptoms to recognise: loss barely moves, rate is too small. Loss jumps around or shows nan (not a number), rate is too big.

Practical habit: try rates spaced by factors of ten, such as 0.1, 0.01 and 0.001.

Five learning rates, twenty steps each

```
def loss(w):
    return w ** 2          # a bowl, lowest at w = 0
def slope(w):
    return 2 * w

for lr in [0.01, 0.1, 0.5, 0.9, 1.1]:
    w = 5.0                # start up the wall
    for step in range(20):
        w = np.subtract(w, lr * slope(w))
    print(f"lr {lr:<4}  distance from best {abs(w):12.4f}"
          f"  loss {loss(w):14.4f}")
```

OUTPUT

```
lr 0.01  distance from best       3.3380  loss       11.1425
lr 0.1   distance from best       0.0576  loss        0.0033
lr 0.5   distance from best       0.0000  loss        0.0000
lr 0.9   distance from best       0.0576  loss        0.0033
lr 1.1   distance from best     191.6880  loss    36744.2892
```

Distance left after 20 steps



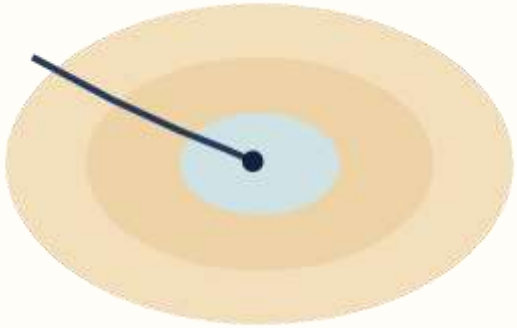
The 1.1 bar is cut off; it is 57 times the length of the 0.01 bar.

> A tiny change, 0.9 to 1.1, flips success into explosion. There is a cliff edge in learning rate.

Full batch, stochastic and mini batch descent

Full batch

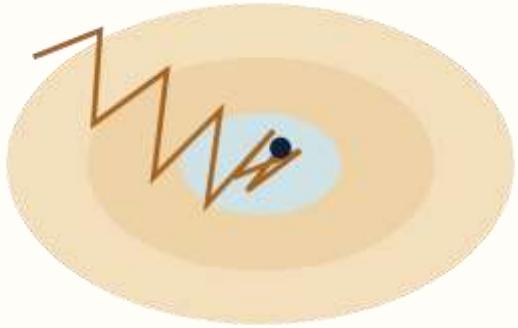
all examples per step



Smooth and exact, but every step reads the whole dataset: very slow.

Stochastic

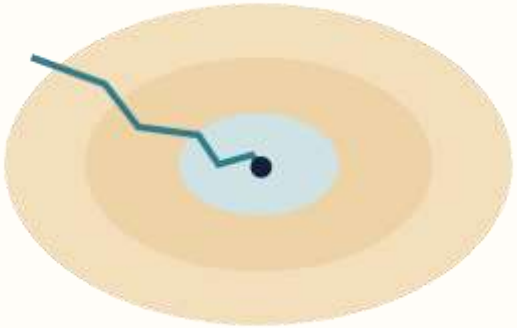
one example per step



Very cheap steps, but the direction is noisy and jittery.

Mini batch

32 to 256 examples per step



The standard. Cheap, fairly stable, and fast on matrix hardware.

Property	Full batch	Stochastic	Mini batch
cost of one step	huge	tiny	small
noise in direction	none	high	moderate
uses parallel hardware	yes, if it fits	poorly	very well

Batch, iteration and epoch, measured out

The dataset: 60,000 training images, cut into batches of 128



each teal block: one **batch** of 128 images, one **iteration**

last batch is partial: 96 images

one full sweep through every block is one **epoch**

batches per epoch	epochs chosen	weight updates in total
469	**10**	**4,690**
468 full plus 1 partial	passes over the data	469 per epoch, ten times

Shuffle every epoch. Mixing the order means each batch is a fair sample, and the model never learns the order of the file.

Shuffling and slicing a dataset into mini batches

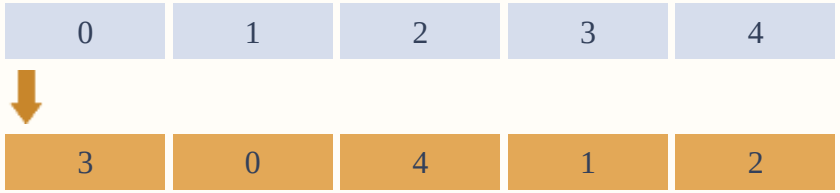
```
rng = np.random.default_rng(0)
X = rng.random((1000, 20))           # 1000 examples, 20 features
y = rng.integers(0, 10, size=1000)  # labels 0 to 9

batch_size = 128
order = rng.permutation(len(X))     # a shuffled list of row numbers
n_batches = 0
for start in range(0, len(X), batch_size):
    idx = order[start:start + batch_size]
    xb, yb = X[idx], y[idx]        # pick those rows
    n_batches += 1
    # ... forward, loss, backward, update go here ...
    if n_batches <= 2 or start + batch_size >= len(X):
        print(f"batch {n_batches}: X {xb.shape}  y {yb.shape}")
print("batches per epoch:", n_batches)
```

OUTPUT

```
batch 1: X (128, 20)  y (128,)
batch 2: X (128, 20)  y (128,)
batch 8: X (104, 20)  y (104,)
batches per epoch: 8
```

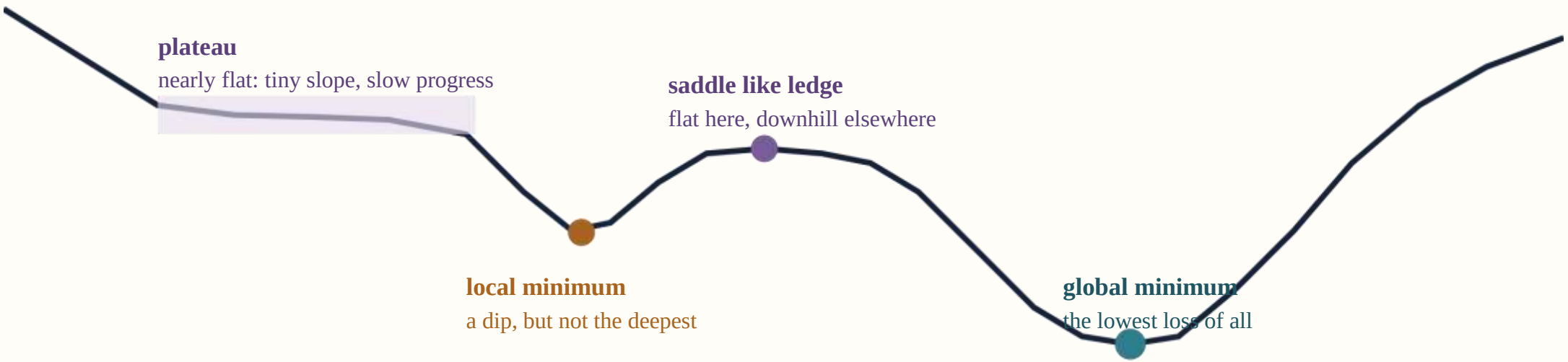
WHY SHUFFLE INDEXES, NOT DATA



Shuffling one list of row numbers keeps X and y perfectly paired.

1000 rows give seven full batches and one of 104.

Loss landscapes: valleys, plateaus, saddles and traps

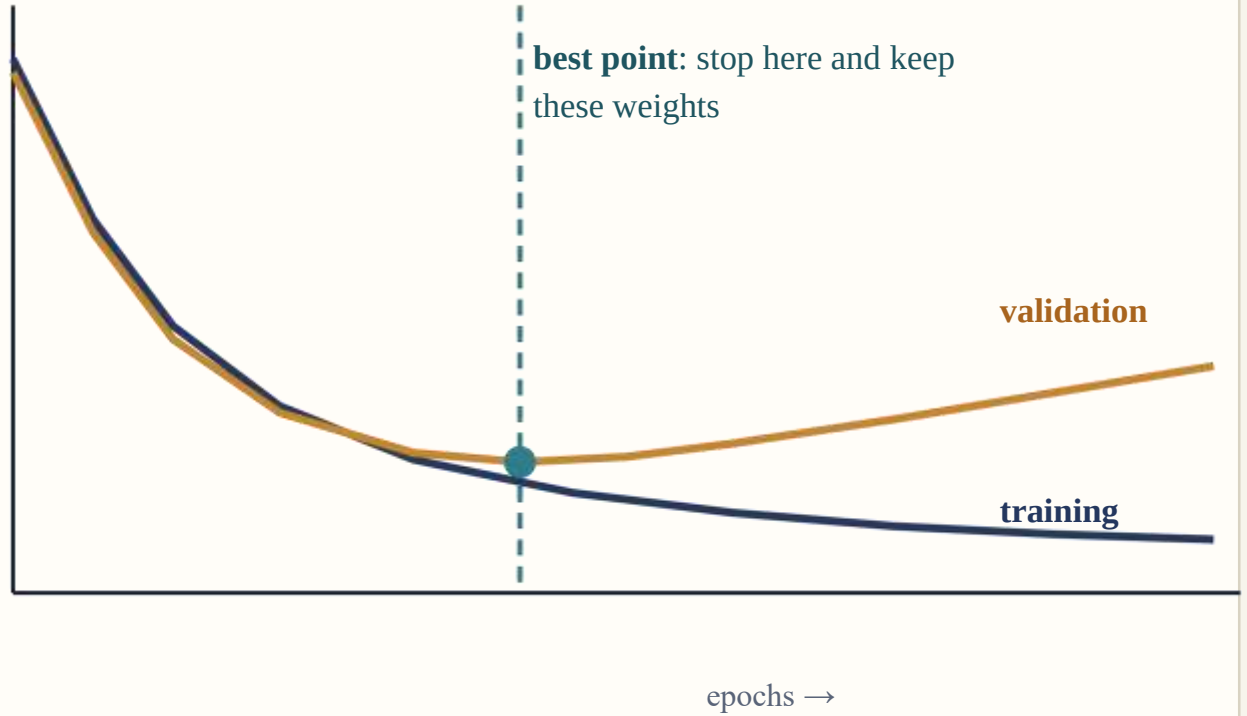


Good news 1. In networks with millions of weights, true dead end local minima are rare; most flat spots are saddles with an escape route.

Good news 2. Many different low valleys perform almost equally well; we rarely need the single best one.

The real enemies are plateaus and saddles, where slopes vanish. Momentum and Adam (Module 15) help push through them.

Convergence and four sensible stopping rules



1 · Budget. A fixed number of epochs. Simple, always used as a safety net.

2 · Flat loss. Training loss stops improving by more than a tiny amount.

3 · Flat ground. The slopes themselves become very small.

4 · Early stopping. Validation loss has not improved for several epochs: the model has started memorising. The favourite in practice.

Key idea. Lower training loss is not the goal. Good performance on **unseen** data is. The gap between the two curves is overfitting, the subject of Module 16.

07

MODULE SEVEN

Activation Functions

The small bends that give neural networks their power. Without them, a hundred layers are no smarter than one.

IN THIS MODULE

- Why straight layers collapse
- Sigmoid and its fading slope
- Tanh, centred on zero
- ReLU, the workhorse
- Leaky ReLU, ELU, GELU, SiLU
- Softmax for many classes
- All of them in code, and how to choose

Stacked straight layers collapse into one straight layer

Without activations

input → layer A → layer B → layer C

is exactly the same as

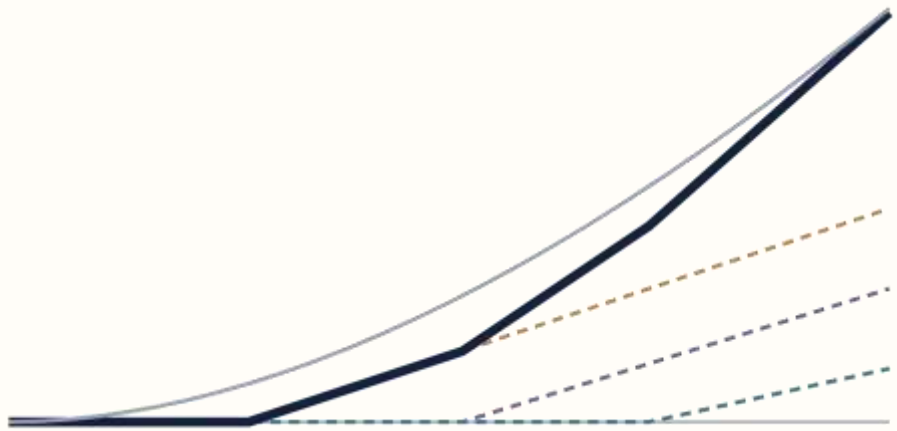
input → one single layer

```
two_layers = (X @ W1) @ W2
one_layer = X @ (W1 @ W2)
print(np.allclose(two_layers, one_layer))
print((W1 @ W2).shape)
```

OUTPUT

```
True
(3, 2)
```

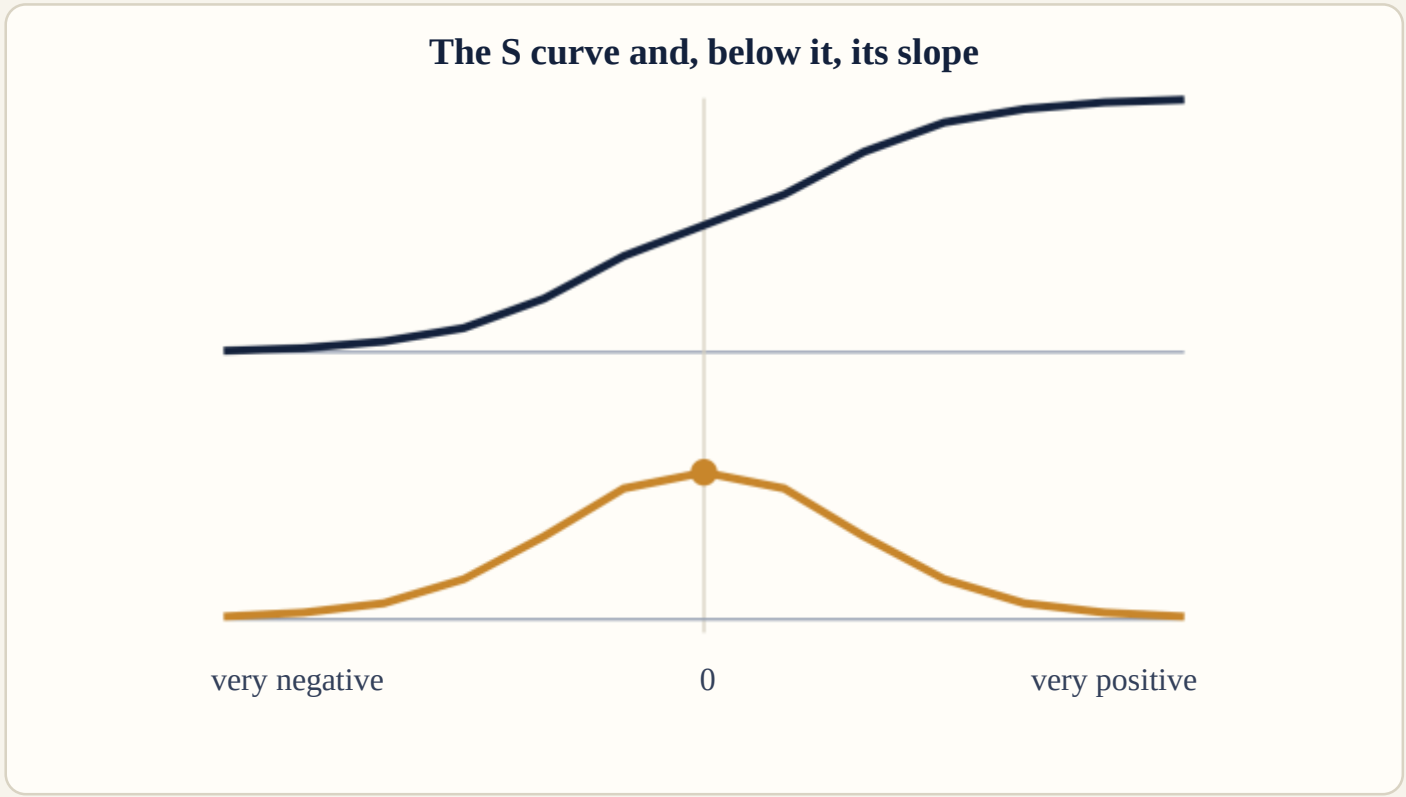
With a bend: hinges add up to any curve



Dashed: three ReLU hinges starting at different points. Thick: their sum, already close to the grey target curve. More hinges, closer fit.

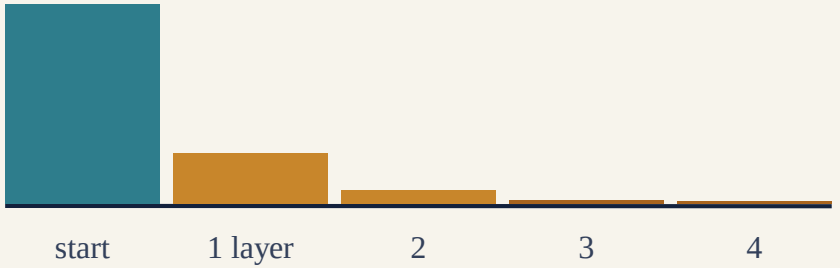
Key idea. A straight transformation of a straight transformation is still straight. Only a nonlinear activation between layers lets depth build curves, corners and complex decision boundaries.

Sigmoid and its fading slope



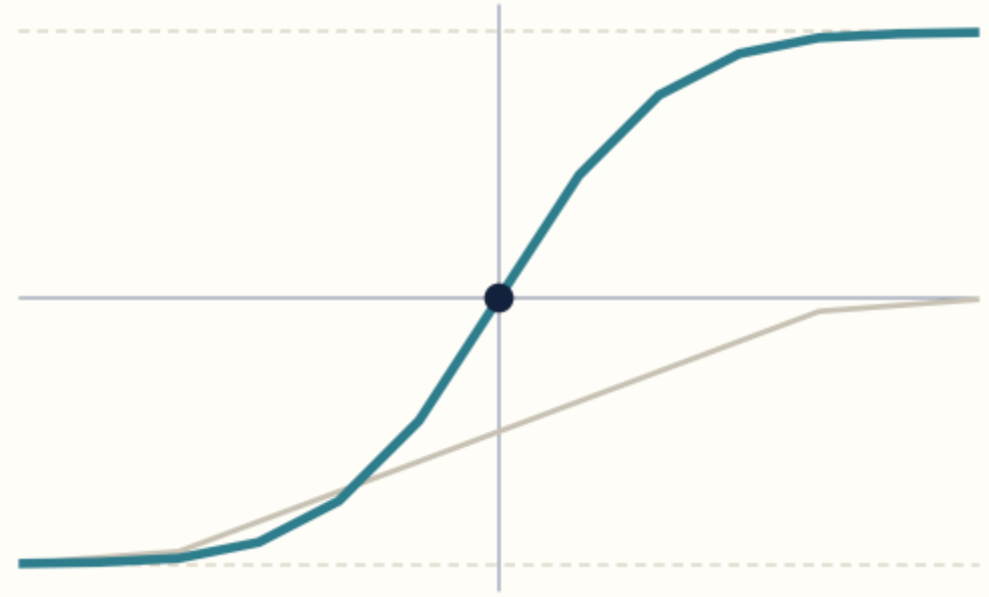
input	output	slope
−3	0.047	0.045
−1	0.269	0.197
0	0.5	0.25 · the maximum
3	0.953	0.045

LEARNING SIGNAL AFTER PASSING BACK THROUGH SIGMOIDS



The problem. Each sigmoid layer multiplies the backward signal by at most 0.25, so after four layers at most 1/256 survives; after ten, less than one millionth. Early layers stop learning. This is the **vanishing gradient**.

Tanh: the same S, stretched and centred on zero



Grey: the sigmoid squeezed into 0 to 1 for comparison.

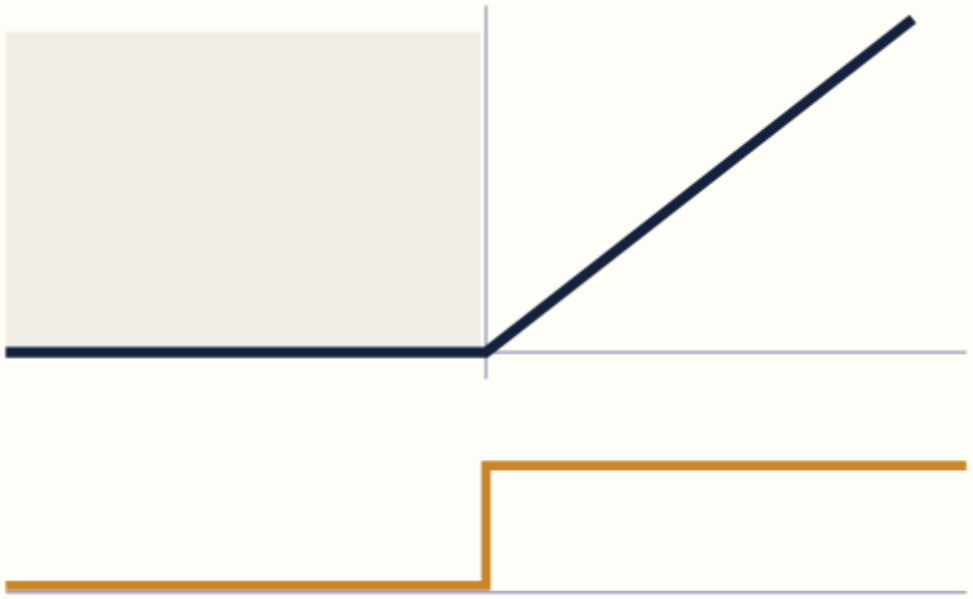
input	output	slope
−3	−0.995	0.01
−1	−0.762	0.42
0	0	1 · the maximum
3	0.995	0.01

Better than sigmoid: outputs are centred on zero, so the next layer sees balanced positive and negative values, and the peak slope is 1 instead of 0.25.

Still saturates: beyond about ±3 the curve is flat and the slope vanishes, so very deep tanh networks still struggle.

Where it lives today: inside LSTM and GRU cells (Module 19) and some generative models.

ReLU: keep the positives, zero the negatives



negative: output 0, slope 0 positive: output equals input, slope 1

Amber: the slope. Either fully on or fully off.

No fading. On the positive side the slope is exactly 1, so learning signals pass through any number of layers undiminished.

Very cheap. No exponentials, just “is it above zero?”.

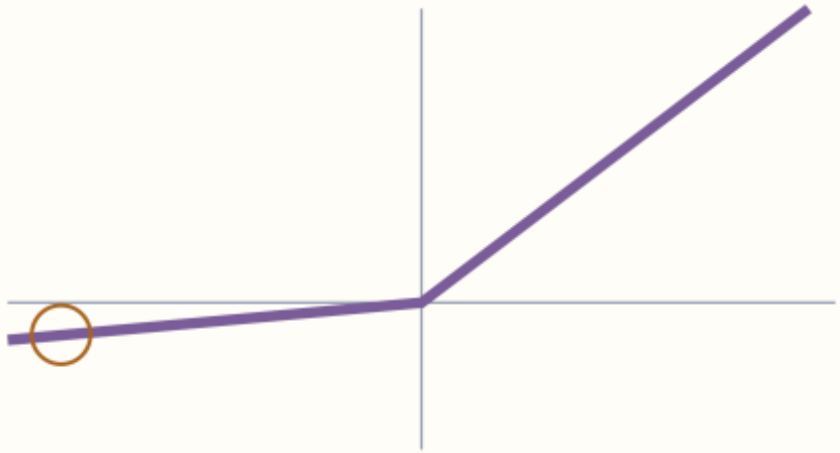
Sparse. Typically about half the neurons output exactly zero for a given input, a light efficient code.

Dying ReLU. A neuron pushed so far negative that it outputs 0 for every input gets slope 0 forever and never recovers.

Default choice for hidden layers of MLPs and CNNs since about 2011.

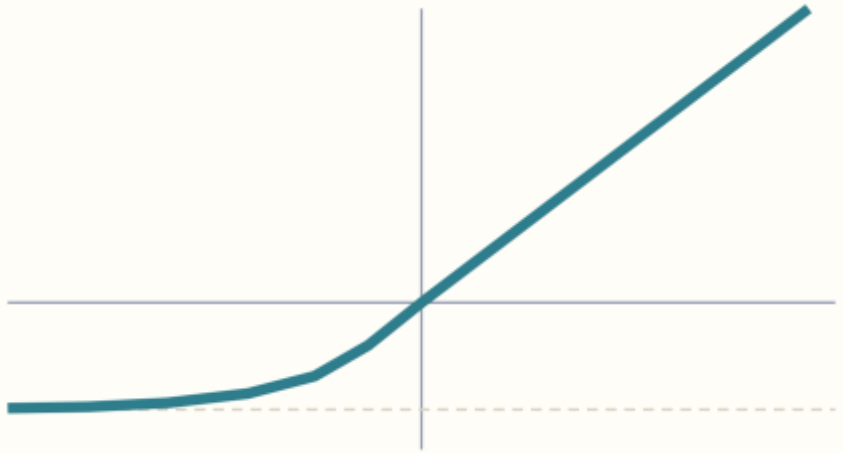
Leaky ReLU and ELU: give the negative side a small voice

Leaky ReLU



Negative inputs keep a **tiny slope**, usually 0.01. Input −3 gives −0.03 instead of 0, so neurons never die completely.

ELU

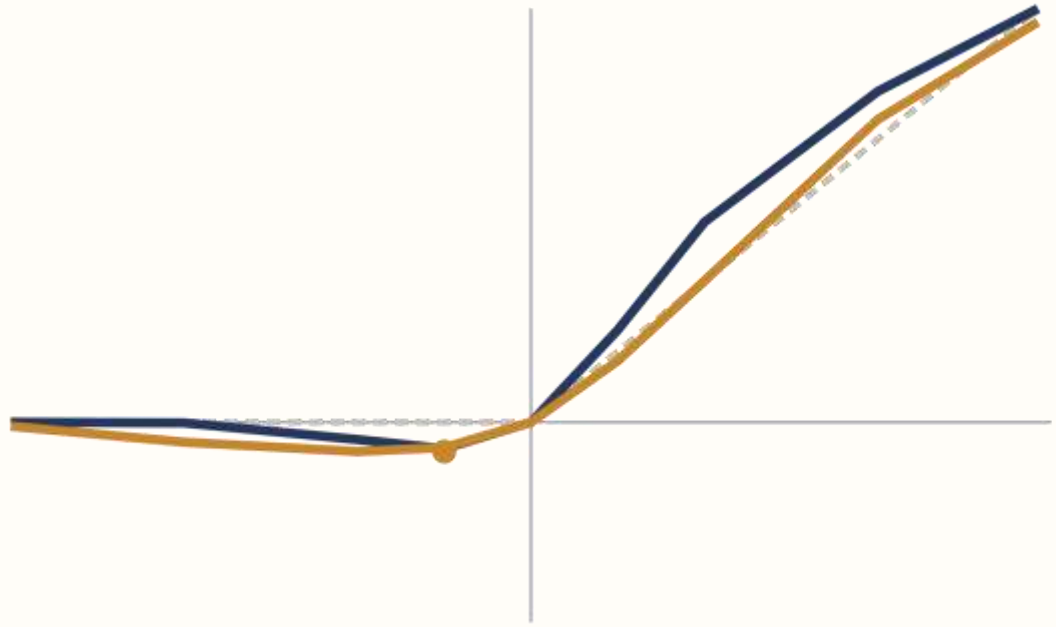


Negative side curves **smoothly** down to a floor of −1 (dashed). Input −1 gives −0.632; input −3 gives −0.95.

Shared idea: nonzero outputs and slopes for negative inputs, so every neuron keeps receiving a learning signal.

In practice they help a little in some networks; plain ReLU remains a strong default. Try them if many neurons die.

GELU and SiLU: ReLU with the corner sanded smooth



dashed: ReLU **navy:** GELU **amber:** SiLU, also called Swish

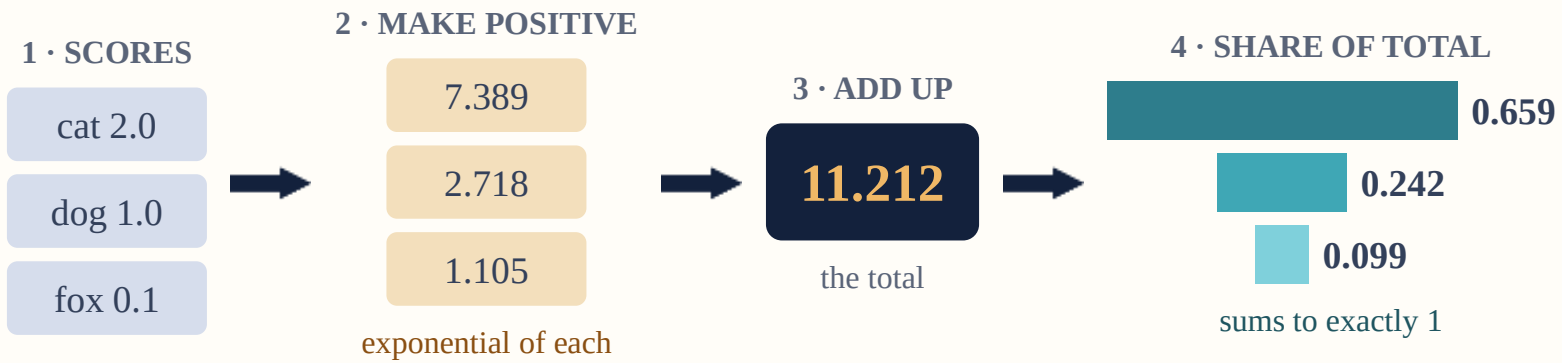
input	ReLU	GELU	SiLU
−3	0	−0.004	−0.142
−1	0	−0.159	−0.269
1	1	0.841	0.731
3	3	2.996	2.858

Idea: scale each input by how likely it is to be “on”. Large positives pass almost untouched; large negatives are switched off; near zero, partially.

The small dip below zero (dot) lets slightly negative values carry information.

Where: GELU in BERT and GPT style Transformers; SiLU in many modern vision and language models.

Softmax: turning a row of scores into shares of 100%



Why exponentials?

They make every score positive and **exaggerate gaps**: a lead of 1 point in score became almost three times the share.

A safety trick

Scores like 1000 would overflow. Subtract the largest score first: shares stay identical, numbers stay small. Scores 1000, 999, 998 give 0.665, 0.245, 0.090.

Where it goes

The final layer of every multiclass classifier, and inside attention (Module 20).

Every activation in a dozen lines

```
import numpy as np

def sigmoid(z):    return 1 / (1 + np.exp(np.negative(z)))
def tanh(z):       return np.tanh(z)
def relu(z):       return np.maximum(0, z)
def leaky_relu(z, a=0.01):
    return np.where(z > 0, z, a * z)
def elu(z, a=1.0):
    return np.where(z > 0, z, a * np.subtract(np.exp(z), 1))
def gelu(z):  # standard tanh approximation
    return 0.5 * z * (1 + np.tanh(np.sqrt(2 / np.pi)
            * (z + 0.044715 * z ** 3)))
def silu(z):       return z * sigmoid(z)

z = np.array([0.0, 0.5, 1.0, 2.0, 3.0])
for f in [sigmoid, tanh, relu, leaky_relu, elu, gelu, silu]:
    print(f"{f.__name__:<10}", np.round(f(z), 3))
```

OUTPUT

```
sigmoid    [0.5   0.622 0.731 0.881 0.953]
tanh       [0.    0.462 0.762 0.964 0.995]
relu       [0.  0.5 1.  2.  3. ]
leaky_relu [0.  0.5 1.  2.  3. ]
elu        [0.  0.5 1.  2.  3. ]
gelu       [0.    0.346 0.841 1.955 2.996]
silu       [0.    0.311 0.731 1.762 2.858]
```

Reading it: for positive inputs the ReLU family is identical; GELU and SiLU are slightly smaller near zero; sigmoid and tanh saturate toward 1.

np.where(condition, a, b) picks a where the condition holds and b elsewhere, element by element: the vectorised “if”.

Which activation where: a decision map

Name	Output range	Strength	Weakness	Typical home
Sigmoid	0 to 1	a probability	fading slope	yes or no output
Tanh	−1 to 1	zero centred	saturates	LSTM, GRU gates
ReLU	0 upward	fast, no fading	dead neurons	MLP, CNN hidden
Leaky	all values	never dies	extra setting	ReLU alternative
ELU	above −1	smooth, centred	slower to compute	some CNNs
GELU	about −0.17 up	smooth	more compute	Transformers
SiLU	about −0.28 up	smooth	more compute	modern vision, LLMs
Softmax	shares of 1	class probabilities	whole row at once	multiclass output

QUICK DECISION FLOW

Is it the **output** layer?

Yes: number → none; yes/no → sigmoid; one of many → softmax

No, hidden: start with ReLU

Many dead neurons? Try Leaky ReLU. Building a Transformer? Use GELU or SiLU.

> **Rule of thumb:** the activation choice matters far less than the learning rate. Do not over think it.

08

MODULE EIGHT

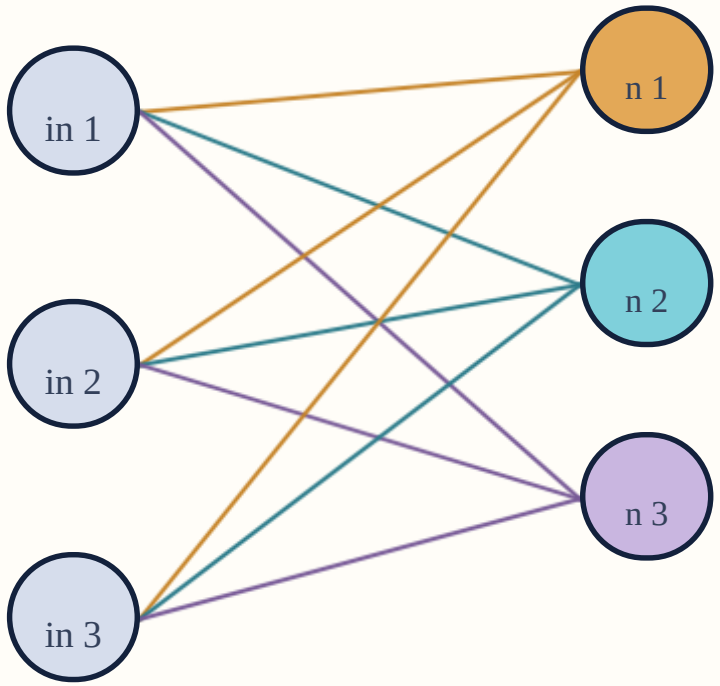
Multilayer Perceptrons

Neurons organised into layers, layers stacked into networks, and the forward pass that carries an input all the way to a prediction.

IN THIS MODULE

- From one neuron to a layer
- Anatomy of a full network
- Shapes through every layer
- A forward pass with real numbers
- XOR finally solved
- Forward pass code and counting weights
- Why depth helps

From one neuron to a layer: many neurons, same inputs



w	w	w
w	w	w
w	w	w

The weight matrix: column 1 holds neuron 1's weights, column 2 neuron 2's, and so on.

shape: 3 inputs by 3 neurons

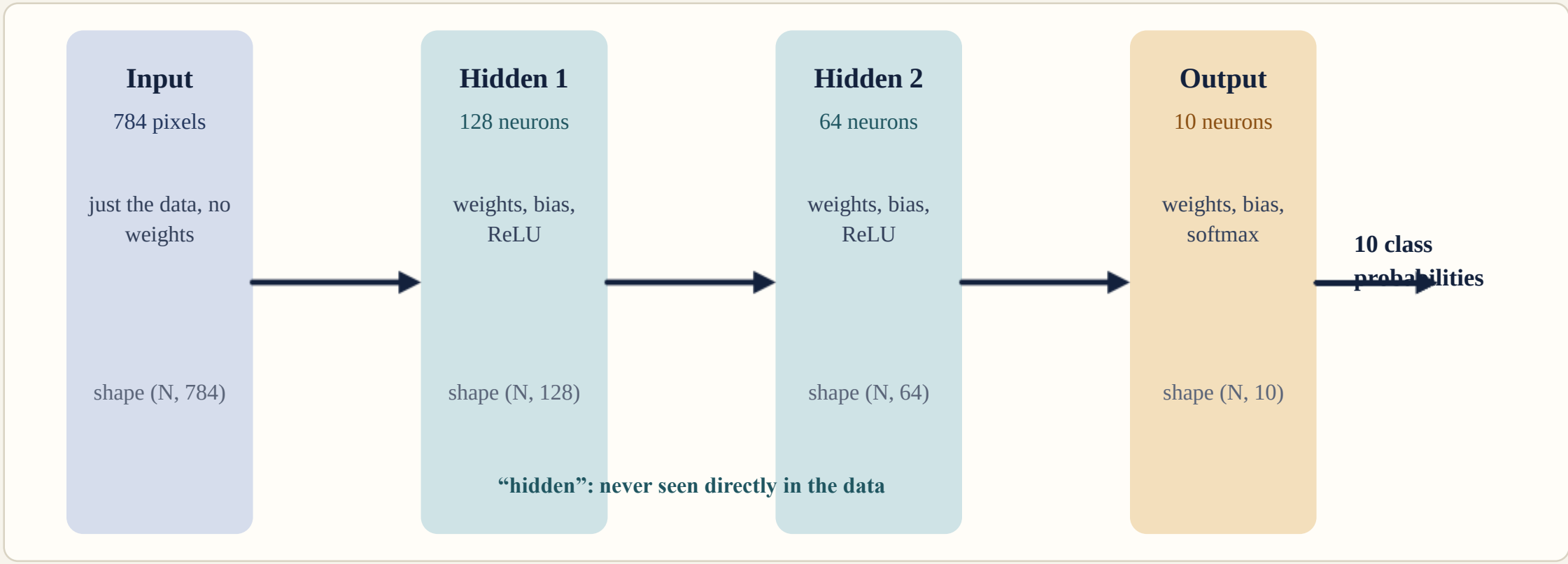
A **layer** is a row of neurons that all look at the same inputs, each with its own weights and bias.

Colour coding: each neuron's three incoming lines become **one column** of the weight matrix.

So the whole layer is one matrix product: inputs times weight matrix, plus a bias for each neuron, then the activation applied to every result.

> **Fully connected** (or **dense**): every input reaches every neuron. With 3 inputs and 3 neurons, that is 9 weights and 3 biases.

Structure of a multilayer perceptron

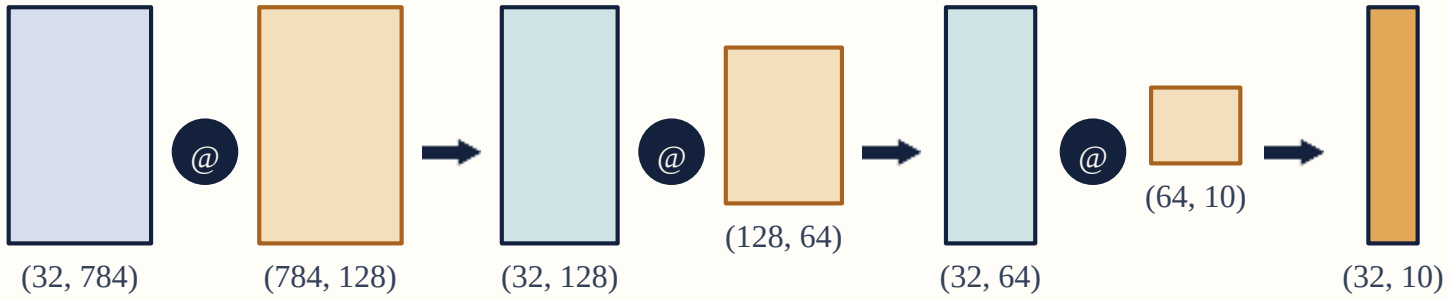


Depth: the number of layers with weights: here 3.

Width: neurons per layer: 128, then 64.

Module 12 trains this same design, sized for 8 by 8 digit images.

Following the shapes through every layer

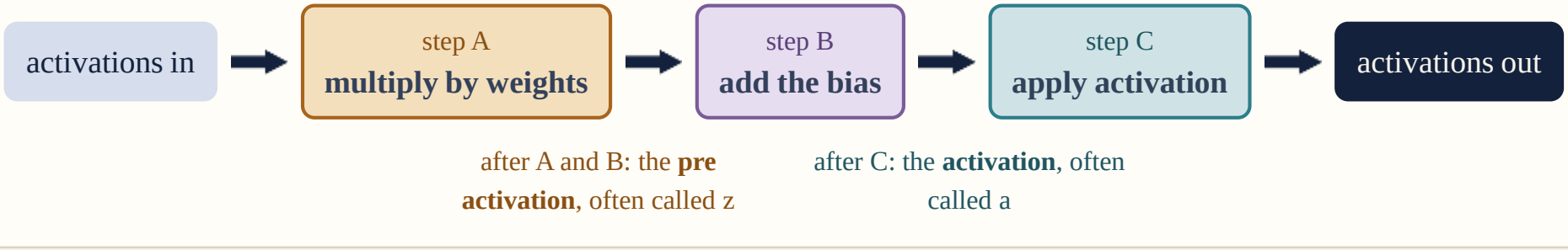


Layer	Weights	Bias	Output	Parameters
Hidden 1	(784, 128)	(128,)	(32, 128)	100,480
Hidden 2	(128, 64)	(64,)	(32, 64)	8,256
Output	(64, 10)	(10,)	(32, 10)	650
Total				109,386

The batch size, 32, rides along untouched in the first axis. The second axis changes at each layer to the layer's width.

92% of all weights live in the first layer, because it touches all 784 pixels.

The forward pass: the same three steps, layer after layer

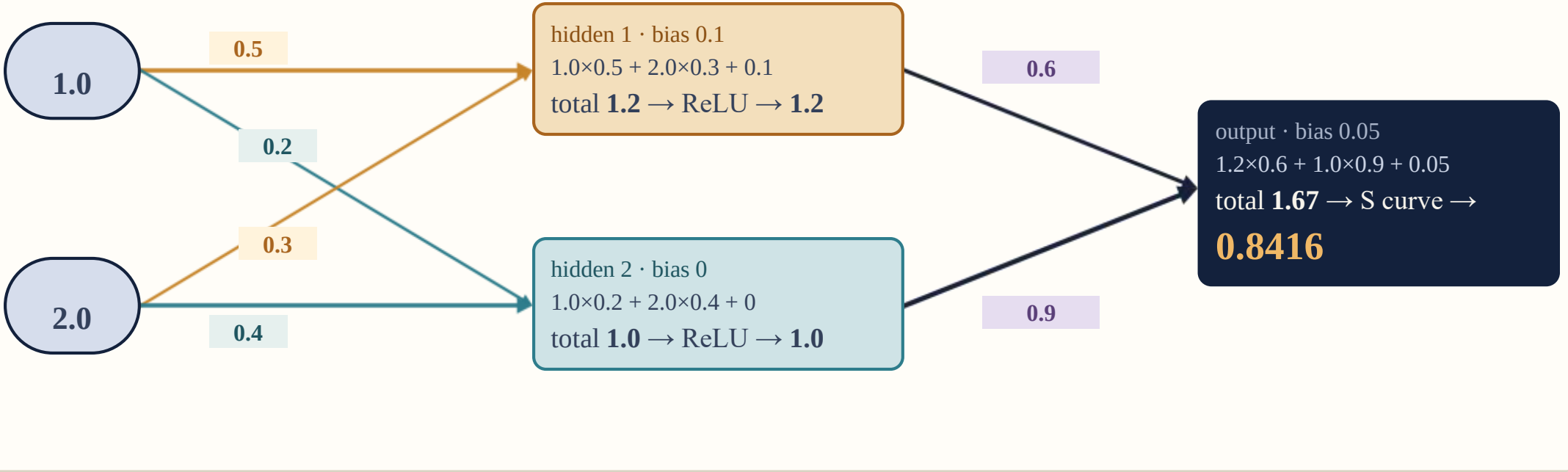




Keep a copy. During training each layer stores its input and its pre activation. The backward pass will need them, as in the chain rule example on slide 36.

Key idea. However large a network is, the forward pass is just this loop: multiply, add, bend, and hand the result to the next layer.

A complete forward pass with real numbers




```
x = np.array([1.0, 2.0])
W1 = np.array([[0.5, 0.2], [0.3, 0.4]]); b1 = np.array([0.1, 0.0])
W2 = np.array([0.6, 0.9]); b2 = 0.05
h = relu(x @ W1 + b1); out = sigmoid(h @ W2 + b2)
```

OUTPUT

```
h  [1.2 1. ]
out 0.8416
```

Two hidden neurons break the XOR wall

The hand built network

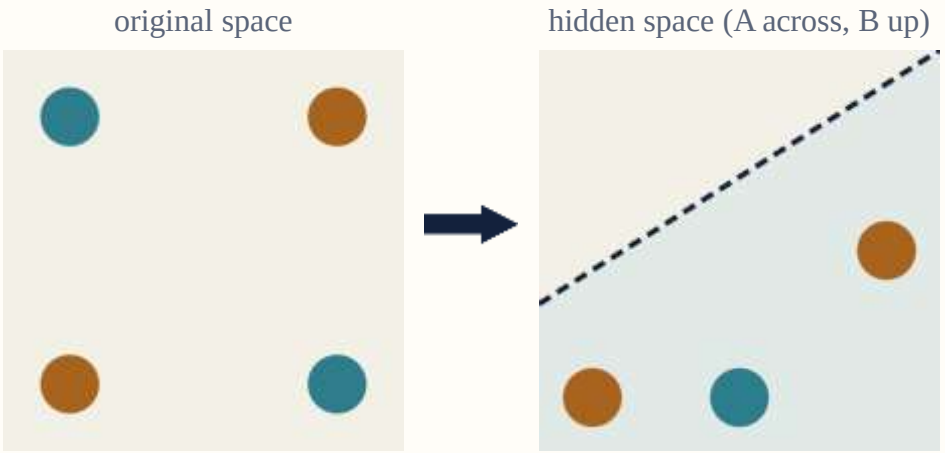
Hidden A counts the inputs that are on: a plus b, through ReLU.

Hidden B fires only if *both* are on: a plus b, minus 1, through ReLU.

Output takes A and subtracts B twice: “at least one, but punish both”.

a b	A	B	output
0 0	0	0	0
0 1	1	0	1
1 0	1	0	1
1 1	2	1	0

The hidden layer moves the points



Inputs 0 1 and 1 0 both land on the same hidden point, and a single straight line now separates the colours.

```
H = relu(X @ W1 + b1); print(H @ W2)  # output: [0. 1. 1. 0.]
```

A full MLP forward pass for a batch of 32 images

```
rng = np.random.default_rng(0)
sizes = [784, 128, 64, 10]

params = []                       # one (W, b) pair per layer
for n_in, n_out in zip(sizes[:3], sizes[1:]):
    W = rng.normal(0.0, np.sqrt(2 / n_in), size=(n_in, n_out))
    b = np.zeros(n_out)
    params.append((W, b))

def forward(X, params):
    a = X
    for i, (W, b) in enumerate(params):
        z = a @ W + b             # steps A and B
        a = relu(z) if i + 1 < len(params) else z
        print(f"layer {i + 1}: {a.shape}")
    return a                      # raw scores (logits)

X = rng.random((32, 784))         # stand in for 32 images
logits = forward(X, params)
e = np.exp(np.subtract(logits, logits.max(axis=1, keepdims=True)))
probs = e / e.sum(axis=1, keepdims=True)  # softmax per row
print("probs shape:", probs.shape,
      "row sums:", np.round(probs.sum(axis=1)[:3], 3))
print("parameters:", sum(W.size + b.size for W, b in params))
```

OUTPUT

```
layer 1: (32, 128)
layer 2: (32, 64)
layer 3: (32, 10)
probs shape: (32, 10)
row sums: [1. 1. 1.]
parameters: 109386
```

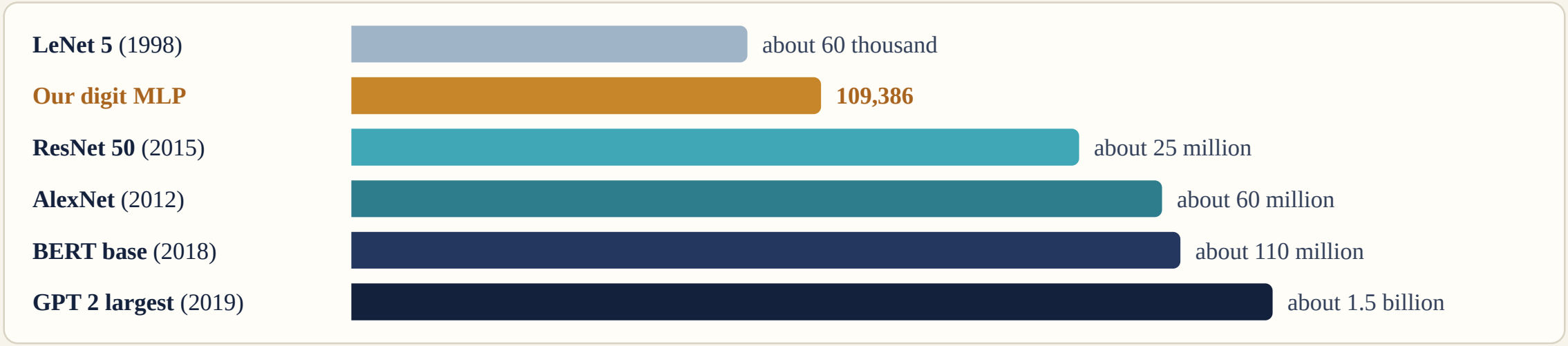
Every row sums to 1: each of the 32 images gets its own 10 way probability split.

Line 13: “i + 1 < len(params)” is true for every layer except the last, so ReLU is applied everywhere but the output.

The random weight spread, square root of 2 over the inputs, is He initialisation (Module 14).

How big is our network compared with famous ones?

Each bar step to the right is ten times more parameters, so a short gap means a huge difference. Approximate published sizes:

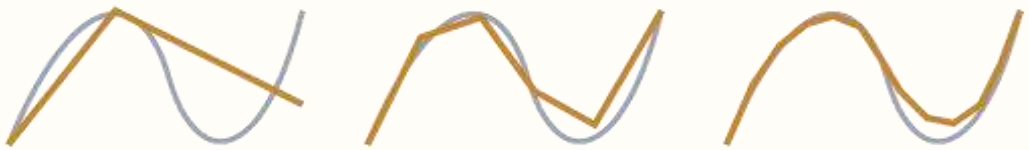


Today’s largest language models are larger still, by further factors of ten to a hundred. The building blocks are the same ones you are learning.

Insight: AlexNet has more parameters than ResNet 50 yet is weaker. Architecture matters more than raw size.

Universal approximation, and why depth beats width

One hidden layer can fit any smooth curve…



2 neurons 5 neurons 20 neurons

Each ReLU neuron adds one bend. With enough bends, the orange polyline can hug any grey target as closely as you like. This is the **universal approximation theorem**.

…but "enough" may mean an astronomically wide layer.

Depth multiplies: think of folding paper



1 fold: 2 2 folds: 4 3 folds: 8 10 folds

1,024

Each layer can fold the space produced by the layer before, so the number of distinct regions can **double per layer**. Width only adds regions one at a time.

Key idea. Shallow networks can, in principle, represent anything; deep networks can represent the same things with far fewer neurons, by reusing features built in earlier layers.

09

MODULE NINE

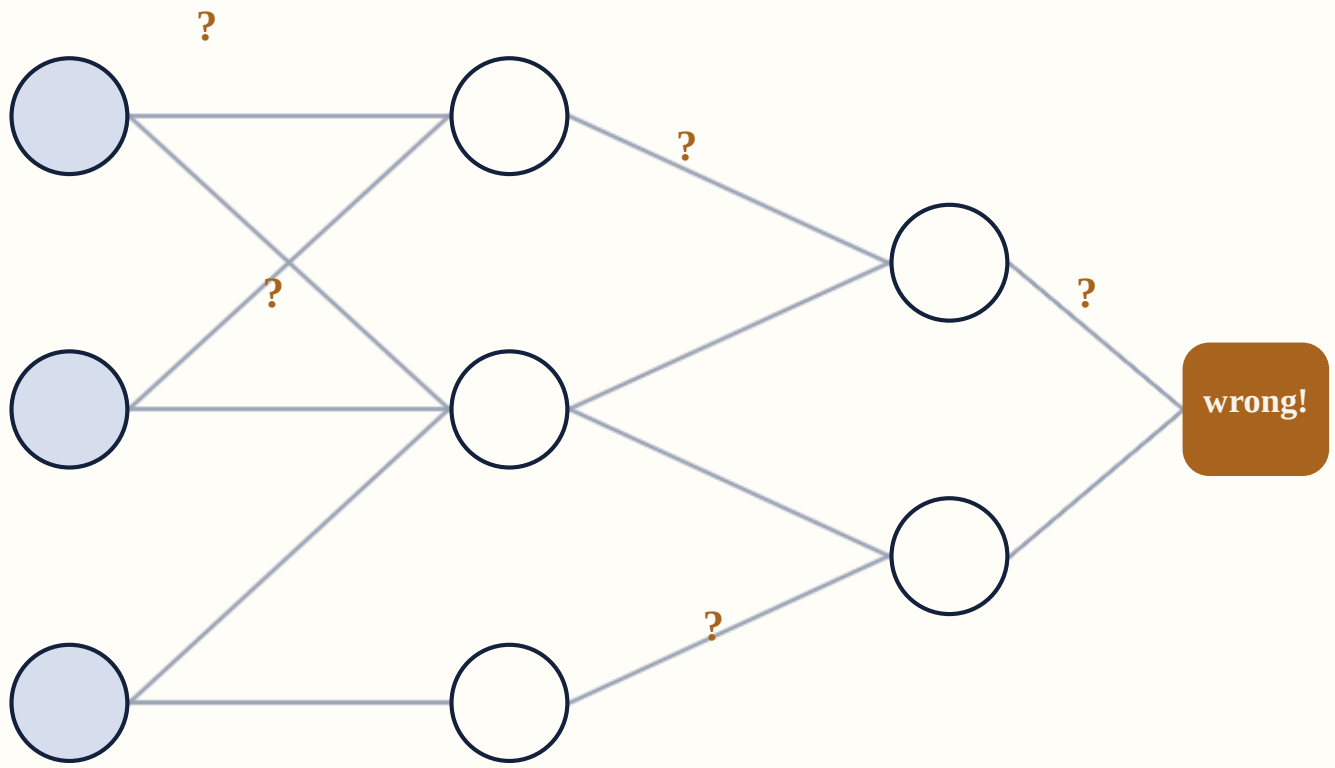
Backpropagation

How a network works out, for every single weight, how much it was to blame for the error. Shown entirely as flowing signals on graphs.

IN THIS MODULE

- The blame assignment problem
- Computational graphs
- Signals flowing backward
- How add, multiply and max gates behave
- Through a neuron, a layer, ReLU, softmax
- A full two layer backward pass
- Gradient checking
- Vanishing signals, measured

The blame assignment problem



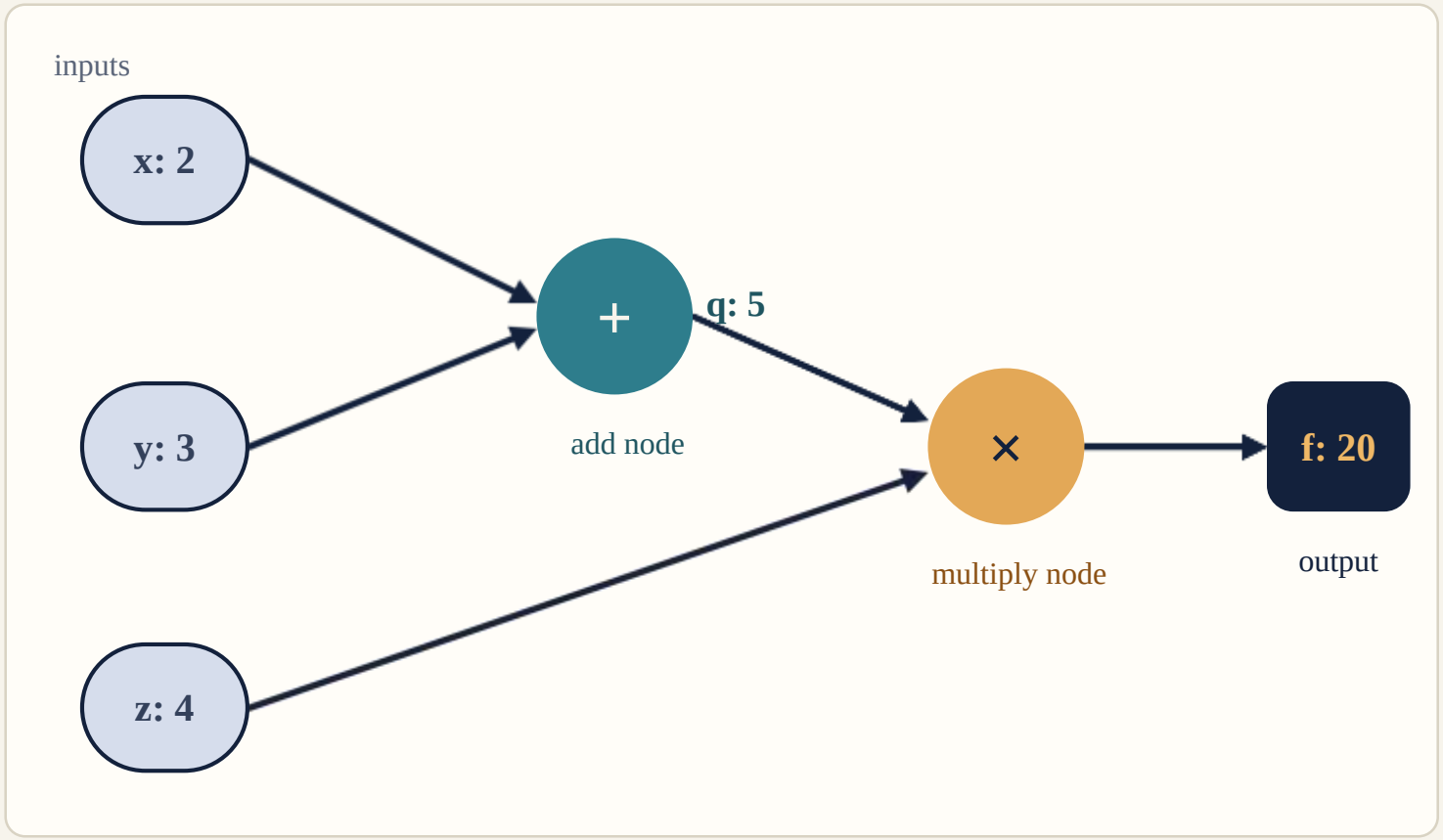
Twelve connections, one wrong answer. Which weights should change, in which direction, and by how much?

Naive approach: nudge each weight one at a time and re run the network to see what happens. For a million weights, that is a million forward passes per step.

Backpropagation: one forward pass plus one backward pass gives the slope for **every** weight at once, at roughly the cost of two forward passes.

How? Send the error backward through the graph, letting each node pass blame to its inputs in proportion to their influence.

Any calculation can be drawn as a graph of tiny operations



The calculation: **add x and y, then multiply the result by z**.

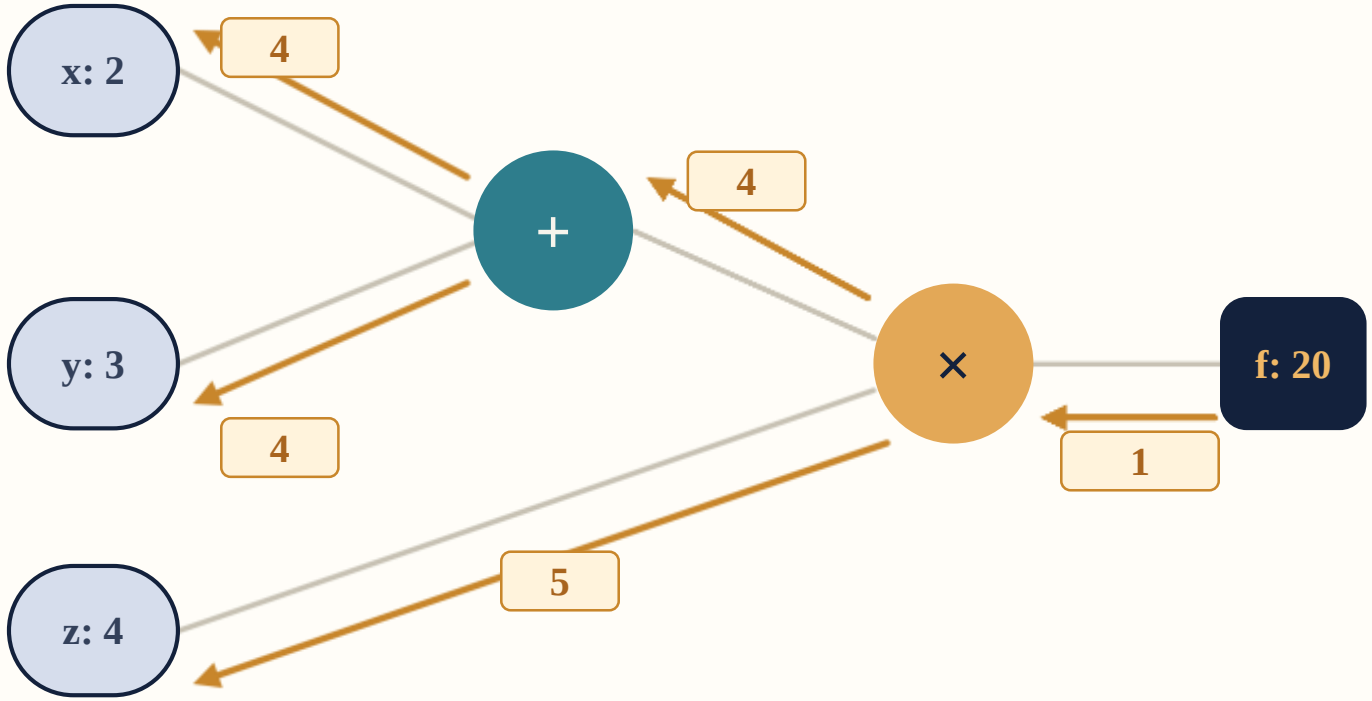
Circles are operations that each do one tiny job.

Arrows carry values forward. Each arrow's value is written beside the node it leaves.

Forward pass: 2 and 3 make 5; 5 and 4 make 20.

Key idea. A neural network is just a very large graph of these same tiny nodes: multiply, add, ReLU, exp, log.

Now run the graph backward: blame flows to every input

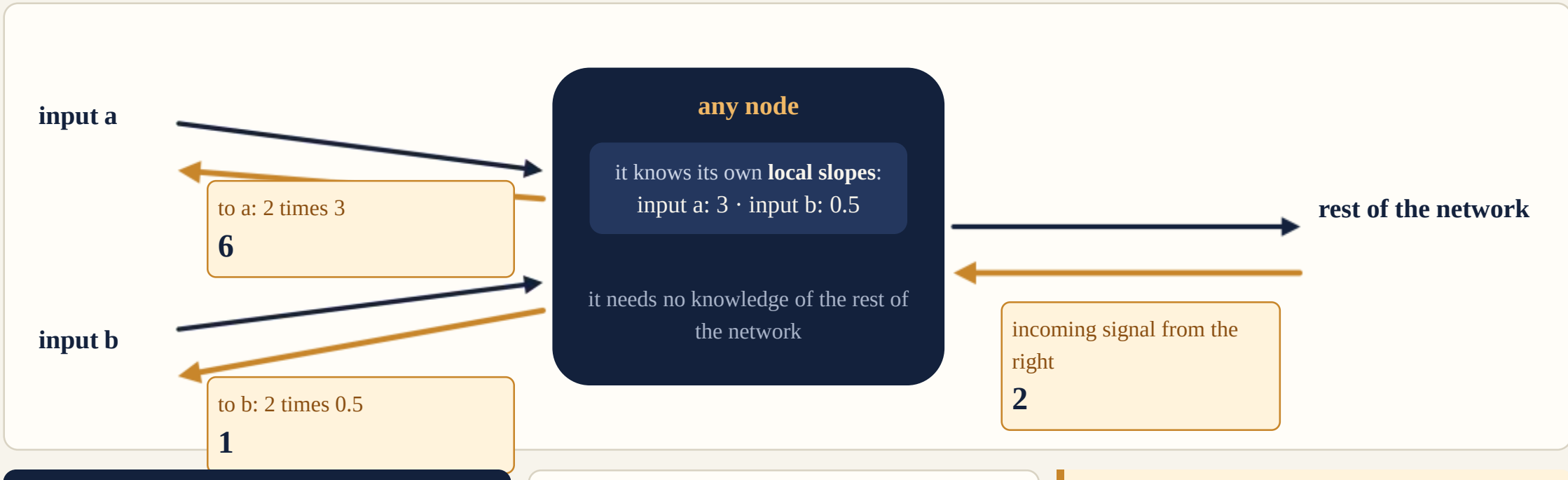


Amber numbers: how much f would change per unit nudge of each value. Read them right to left.

1. Start at the output with **1**: f moves one for one with itself.
2. Through the **multiply** node: q receives z's value, **4**; z receives q's value, **5**. Each input gets the *other* input.
3. Through the **add** node: the 4 is copied unchanged to both x and y.

Check: raise x from 2 to 2.01; q becomes 5.01; f becomes 20.04. A rise of 0.04, which is 4 times the nudge.

Every node follows one rule: incoming signal times local slope



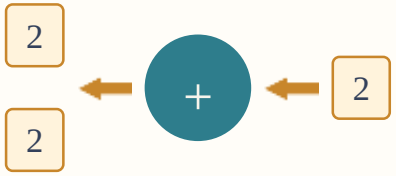
The rule: the signal sent back to each input is the signal received from the output, multiplied by that input's local slope.

If a value feeds several nodes, the signals coming back from all of them are **added** together.

Local work, global result: this is the chain rule organised as message passing.

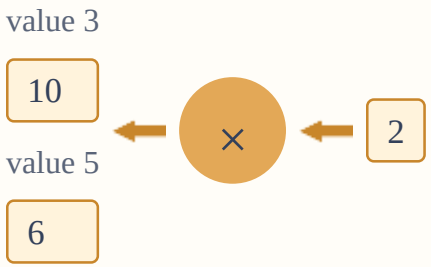
Four node types, four ways of passing the signal back

Add: distributor



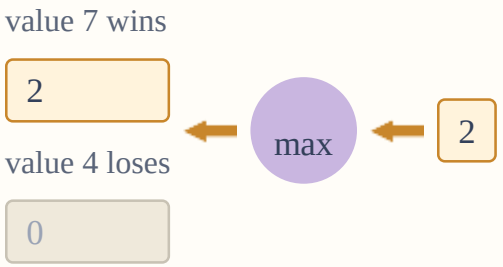
Copies the incoming signal, unchanged, to every input.

Multiply: swapper



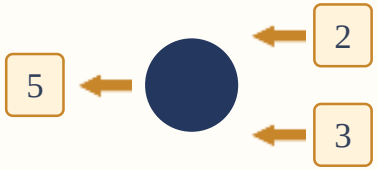
Each input gets the signal times the **other** input's value: 3 gets 2×5, 5 gets 2×3.

Max and ReLU: router



All signal goes to the winner. ReLU is max with zero: open gate or closed gate.

Branch: collector

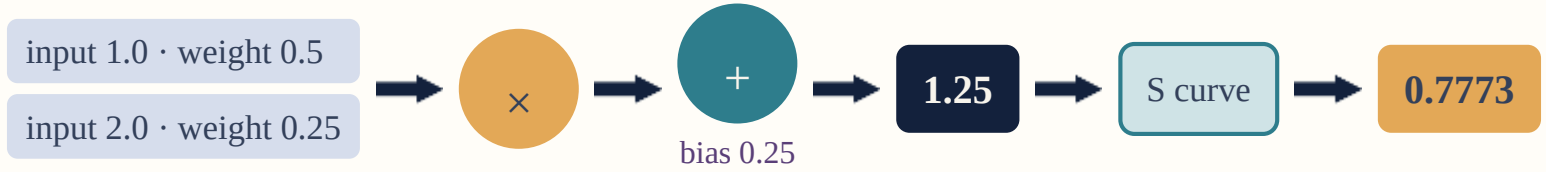


A value used in two places **adds up** the signals from both uses.

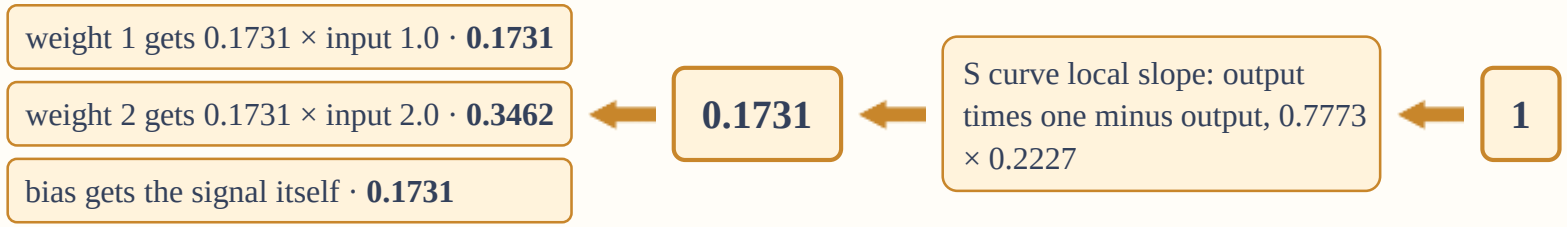
Key idea. Memorise these four personalities and you can backpropagate through any graph by hand, including whole networks, without writing a single equation.

Backpropagating through one sigmoid neuron

FORWARD



BACKWARD, starting from a signal of 1 at the output




```
a = sigmoid(np.dot(w, x) + b)
local = a * np.subtract(1, a)
print("dw", np.round(x * local, 4), "db", round(local, 4))
print("dx", np.round(w * local, 4))
```

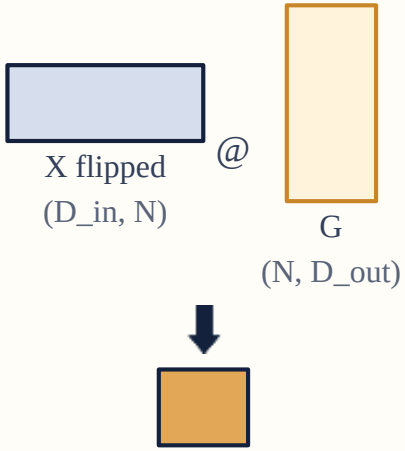
OUTPUT

dw [0.1731 0.3462] db 0.1731
dx [0.0866 0.0433]

Backprop through a dense layer: three outputs, three block puzzles

A layer took inputs X of shape (N, D_in) and weights W of shape (D_in, D_out). A signal G of shape (N, D_out) arrives from above. We must send back three things:

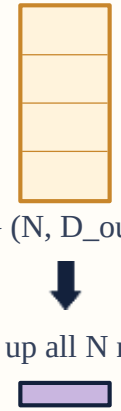
Signal for W



(D_in, D_out), the shape of W

Each weight: its input times the signal at its output, summed over the batch.

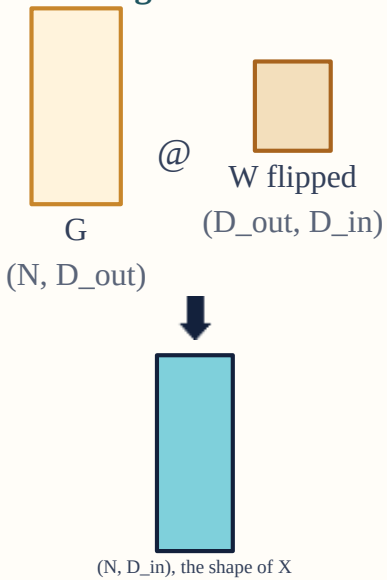
Signal for the bias



(D_out,), the shape of b

The bias was added to every example, so it collects every example's signal.

Signal for X



Passed down to the layer below, which repeats the process.

```
dW = X.T @ G      db = G.sum(axis=0)      dX = G @ W.T      # the whole layer backward pass in three lines
```

MODULE 9 · A SURVIVAL TRICK

Lost? Let the shapes tell you the answer

You need the signal for W. You only have two pieces, X and G. There is exactly **one** way to combine them that produces W’s shape.

TRY 1 · X with G

(32, 784) with (32, 128)

inner sizes 784 and 32 do not match



TRY 2 · G flipped with X

(128, 32) with (32, 784)

fits, but gives (128, 784): backwards



TRY 3 · X flipped with G

(784, 32) with (32, 128)

gives (784, 128): exactly W’s shape



THE THREE STEP METHOD

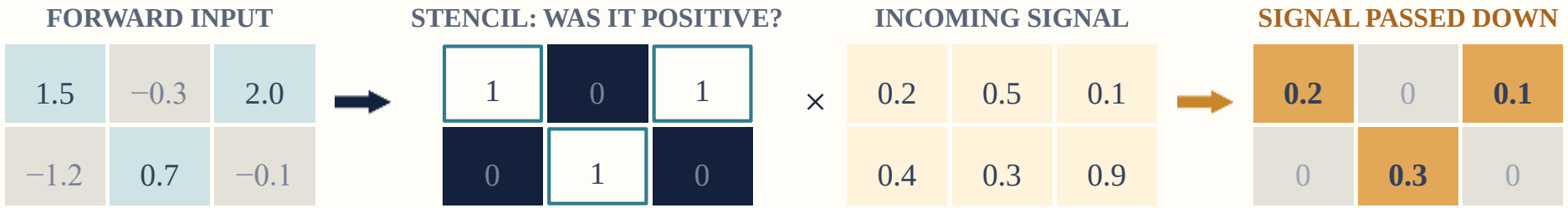
1 · Write the shape the answer must have (the shape of the thing you differentiate).

2 · List the arrays available and their shapes.

3 · Combine them, flipping as needed, until the shapes fit.

Why it works. The batch axis N must disappear, because a weight’s signal is summed over all examples. The only matrix product that eats the N axis is the one where both N’s sit on the inside.

Backprop through ReLU: a stencil that blocks some signals

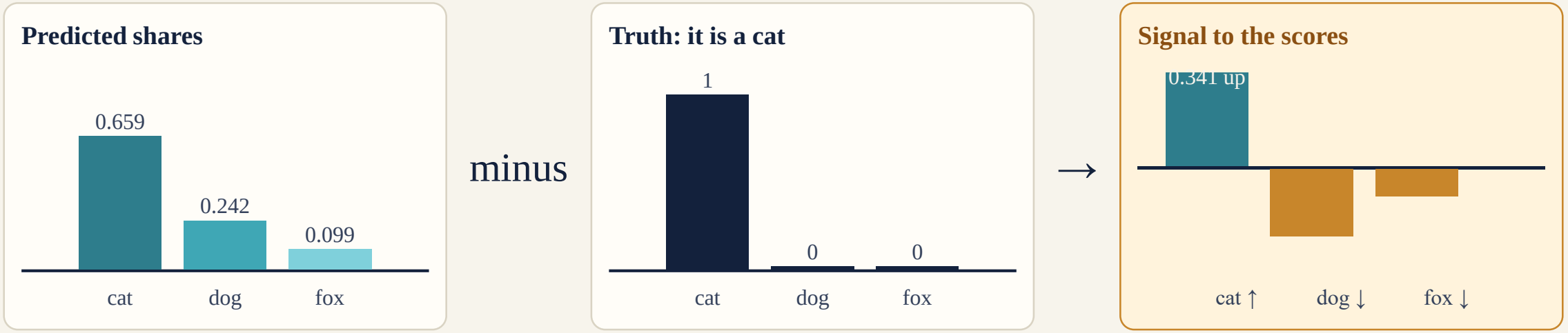



```
# build the forward input shown above
Z = np.subtract([[2.0, 0.0, 2.5], [0.0, 1.0, 0.0]],
                [[0.5, 0.3, 0.5], [1.2, 0.3, 0.1]])
G = np.array([[0.2, 0.5, 0.1], [0.4, 0.3, 0.9]])
mask = (Z > 0)
print(mask.astype(int))
print(G * mask)      # ReLU backward: one line
```

OUTPUT

```
[[1 0 1]
 [0 1 0]]
[[0.2 0.  0.1]
 [0.  0.3 0. ]]
```

The classifier’s starting signal: prediction minus truth, again



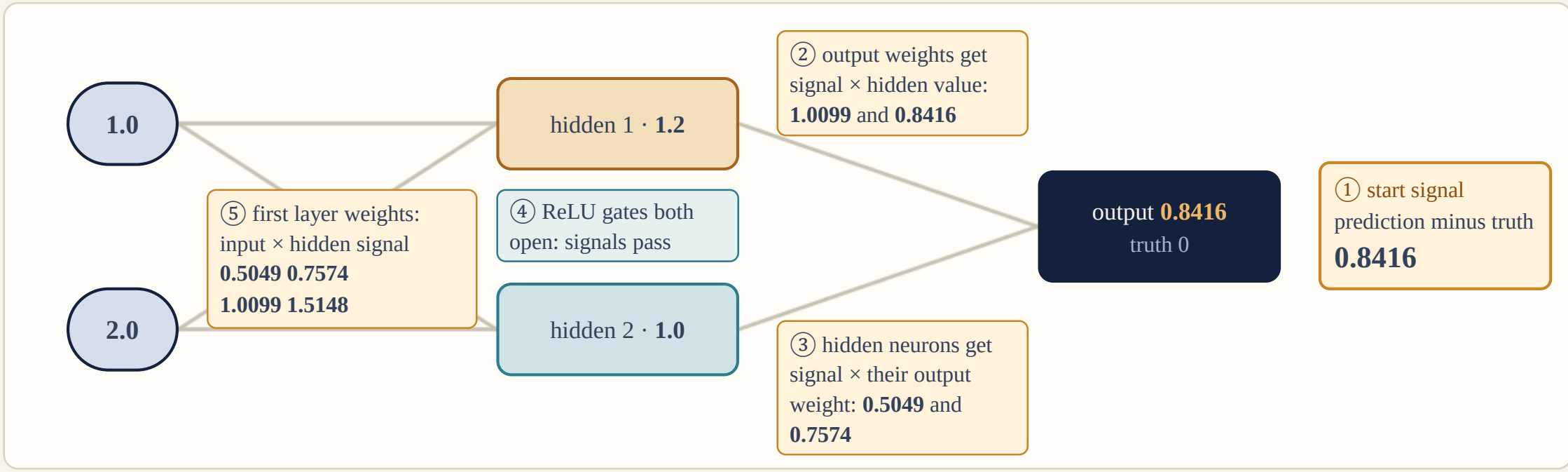

```
p = np.array([0.659, 0.242, 0.099])  # softmax output
onehot = np.array([1.0, 0, 0])       # the truth
g = np.subtract(p, onehot)
print("push up on true class by:", round(np.negative(g[0]), 3))
print("push down on wrong classes:", g[1:])
```

OUTPUT

```
push up on true class by: 0.341
push down on wrong classes: [0.242 0.099]
```

The full backward pass through our two layer network

Same network as slide 98, output 0.8416. Suppose the truth is **0**: the model was wrongly confident. Loss 1.8425. Follow the amber signal right to left.



Reading it: every number is positive, so every weight will be **lowered** by gradient descent, pulling the output down toward the truth of 0. The biggest push lands on the weight from input 2.0 to hidden 2.

The backward pass in code, matching the diagram line by line

```
x = np.array([1.0, 2.0])
W1 = np.array([[0.5, 0.2], [0.3, 0.4]]); b1 = np.array([0.1, 0.0])
W2 = np.array([0.6, 0.9]); b2 = 0.05
y = 0.0                                  # the truth

# forward, keeping what backward will need
z1 = x @ W1 + b1; h = relu(z1)
z2 = h @ W2 + b2; out = sigmoid(z2)

# backward
dz2 = np.subtract(out, y)          # ① prediction minus truth
dW2 = h * dz2; db2 = dz2           # ② swap at the multiplies
dh = W2 * dz2                      # ③ signal to hidden outputs
dz1 = dh * (z1 > 0)                # ④ ReLU stencil
dW1 = np.outer(x, dz1); db1 = dz1  # ⑤ first layer weights

print("dz2", round(dz2, 4)); print("dW2", np.round(dW2, 4))
print("dh ", np.round(dh, 4)); print("dW1", np.round(dW1, 4))
```

OUTPUT

```
dz2 0.8416
dW2 [1.0099 0.8416]
dh  [0.5049 0.7574]
dW1 [[0.5049 0.7574]
     [1.0099 1.5148]]
```

np.outer pairs every input with every hidden signal, building the 2 by 2 grid of first layer weight signals.

Five lines of backward code mirror five forward lines in reverse order. Every network’s backward pass has this mirror structure.

Gradient checking: two independent routes to the same answer

Route A · backprop

One forward and one backward pass. Fast, exact in principle, but our code might contain a bug.

forward → backward → answer A

Route B · two probes

For each weight: nudge it up a hair, measure the loss; nudge it down, measure again (slide 32). Slow but almost impossible to get wrong.

loss up loss down → answer B

answer A answer B → **relative error**: size of the difference, divided by the sizes of both

less than one in ten million	around one in ten thousand	one in a hundred or worse
correct	**suspicious**	**bug**

Practical tips: check on a tiny network and a few examples, use float64, and switch off randomness such as dropout while checking. ReLU kinks can cause rare false alarms.

Checking our first layer weight signals numerically

```
def loss_fn(W1):                # full forward pass, returns the loss
    h = relu(x @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    return np.log(1 / np.subtract(1, out))  # truth is 0

eps = 0.00001
num = np.zeros_like(W1)
for i in range(2):
    for j in range(2):
        Wp = W1.copy(); Wp[i, j] += eps                  # probe up
        Wm = W1.copy(); Wm[i, j] = np.subtract(Wm[i, j], eps)  # probe down
        num[i, j] = np.subtract(loss_fn(Wp), loss_fn(Wm)) / (2 * eps)

rel = (np.linalg.norm(np.subtract(num, dW1))
       / (np.linalg.norm(num) + np.linalg.norm(dW1)))
print("numeric", np.round(num, 4))
print(f"relative error {rel:.12f}")
```

OUTPUT

```
numeric [[0.5049 0.7574]
         [1.0099 1.5148]]
relative error 0.000000000013
```

VERDICT

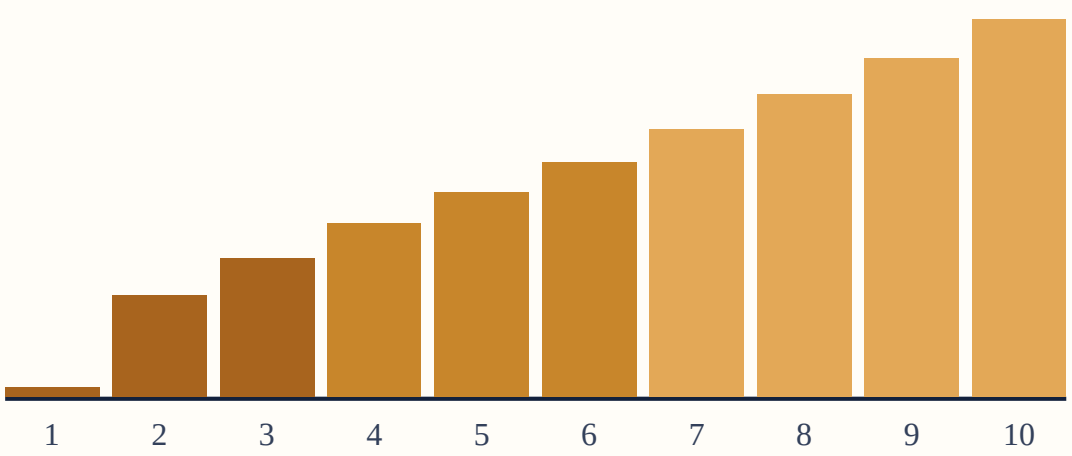


▲ **here** suspicious bug

About one part in a hundred billion: our hand derived backward pass is correct.

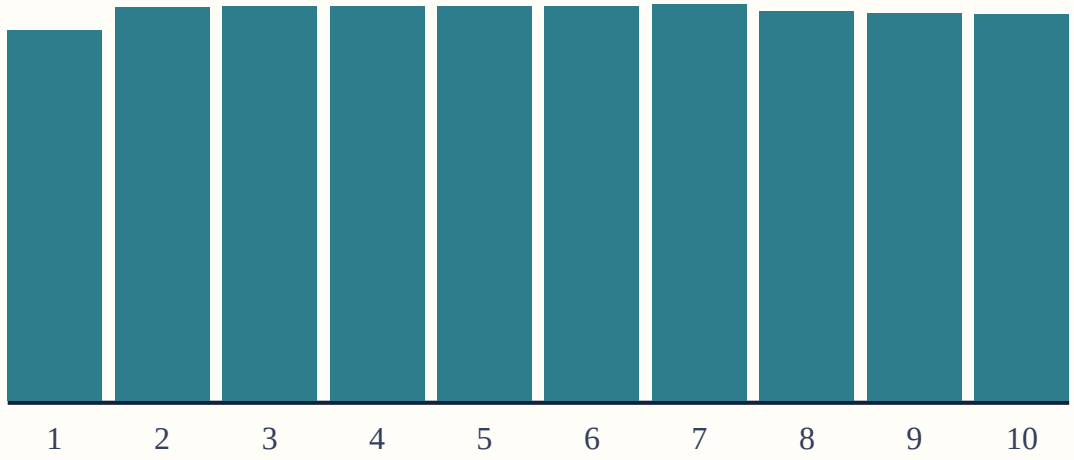
Vanishing signals, measured in a ten layer network

Sigmoid hidden layers



layer 10: 0.116 · layer 1: 0.000000015

ReLU hidden layers, He start



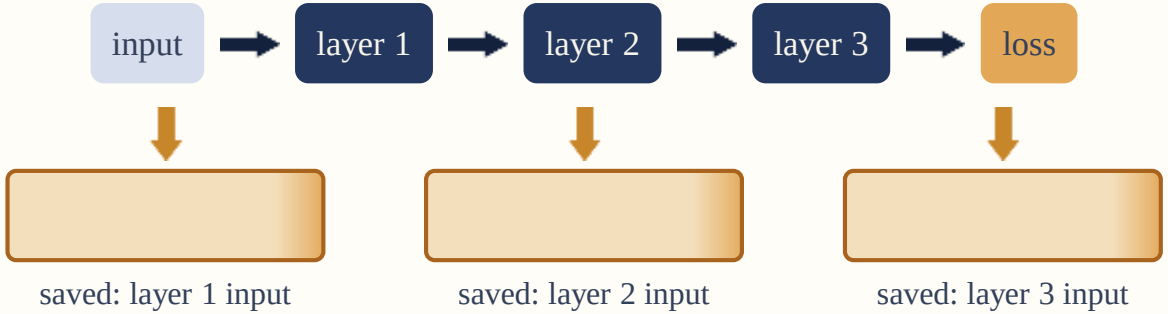
every layer between 0.087 and 0.260

Bar height: average size of each layer’s weight signal on a “ten times per step” scale, so every 40 pixels is a factor of ten. Layer 1 is nearest the input.

With sigmoids, the first layer’s signal is about **8 million times** weaker than the last layer’s. Early layers barely learn.

ReLU plus a well scaled start keeps the signal healthy through all ten layers. Module 14 explains the starting scale.

The price of backprop: remembering the forward pass



The backward pass reads these saved activations in reverse order, then frees them.

Training
Memory grows with batch size × depth × width. This, not the weights, is usually what fills a GPU.

Inference only
No backward pass means nothing needs saving; each layer's output can be discarded once the next layer has used it.

The trade
"Gradient checkpointing" saves only some activations and recomputes the rest: less memory, more compute.

MODULE 9 · SUMMARY

Backpropagation on one page

THE LOOP

1 · Forward: compute and save every intermediate value.

2 · Start the signal at the loss: prediction minus truth.

3 · Walk the graph backward, node by node.

4 · Each node multiplies the incoming signal by its local slope.

GATE PERSONALITIES

+ copies the signal to all inputs

× swaps: each input gets the other’s value

max routes everything to the winner

branches add their signals

LAYER RECIPES

Dense: dW = X.T @ G
db = G.sum(axis=0)
dX = G @ W.T

ReLU: dZ = G * (Z > 0)

Sigmoid: dZ = G * a * np.subtract(1, a)

Softmax + cross entropy: probabilities minus one hot truth

Always: gradient shape equals parameter shape; verify with a gradient check.

Next: make the computer build and walk the graph for us. That is autograd.

10

MODULE TEN

Building an Autograd Engine

A hundred lines of Python that record every calculation and then run it backward automatically. The heart of every modern framework.

IN THIS MODULE

- Three ways to get slopes
- The Value object, drawn
- Operations that remember their inputs
- Ordering the graph and walking it back
- Neurons, layers and an MLP on top
- Training it: XOR, solved automatically
- From scalars to tensors

Three ways to get slopes, and why we automate the third

By hand

What we did in Module 9. Exact and fast to run, but slow to write and easy to get wrong for big models.

Numerical probes

Nudge each weight both ways. Trivial to write, but two forward passes per weight: hopeless for millions.

Automatic

Record every operation as it runs, then replay the record backward with the gate rules. Exact, fast, and zero hand derivation.

Forward mode

1 input → all outputs

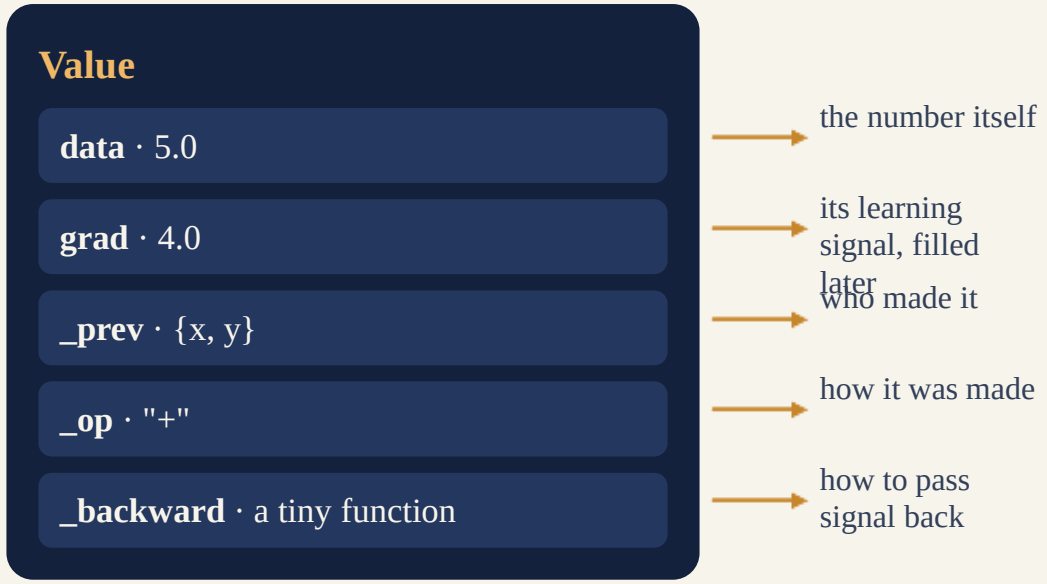
One pass per **input**. With a million weights: a million passes.

Reverse mode (backprop)

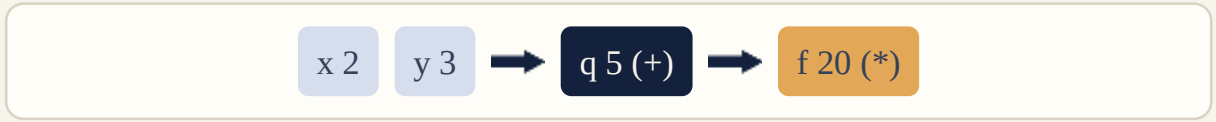
all weights ← 1 loss

One pass per **output**. A network has one loss, so a single backward pass gives every slope.

The Value object: a number with a memory



Every time two Values are combined, the result is a **new Value** that remembers its parents and carries a little function saying how to push signal back to them.



Doing ordinary arithmetic therefore **builds the graph** silently. Calling backward on the final Value walks it in reverse.

> **This is exactly how PyTorch tensors work**, except each node holds a whole array instead of one number.

The Value class: storage, addition and multiplication

```
import math
import numpy as np
NEG_ONE = np.negative(1.0)

class Value:
    def __init__(self, data, children=(), op=""):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None    # leaves do nothing
        self._prev = set(children)
        self._op = op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += out.grad          # add: copy the signal
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")
        def _backward():
            self.grad += other.data * out.grad  # multiply: swap
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    __radd__ = __add__       # so 2 + Value(3) also works
    __rmul__ = __mul__
```

Each operation does two jobs: compute the result now, and leave behind a closure, _backward, that knows exactly how to pass signal to its own inputs later.

Why "+="? A Value used in several places must collect signal from every use: the branch rule from slide 108.

Wrapping numbers: plain Python numbers are turned into Values so Value(2) * 3 also works.

The gate personalities appear here as two tiny functions. Nothing else is needed for the core.

Subtraction and the three activations

```
def minus(self, other):            # built from add and multiply
    return self + other * NEG_ONE

def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), "tanh")
    def _backward():
        self.grad += np.subtract(1, t * t) * out.grad
    out._backward = _backward
    return out

def sigmoid(self):
    s = 1 / (1 + math.exp(np.negative(self.data)))
    out = Value(s, (self,), "sigmoid")
    def _backward():
        self.grad += s * np.subtract(1, s) * out.grad
    out._backward = _backward
    return out

def relu(self):
    out = Value(max(0.0, self.data), (self,), "relu")
    def _backward():
        self.grad += (out.data > 0) * out.grad  # router
    out._backward = _backward
    return out
```

EACH ACTIVATION'S LOCAL SLOPE

tanh	one minus output squared
sigmoid	output times one minus output
relu	1 if it fired, else 0

minus needs no backward of its own: it is assembled from add and multiply, which already know their rules.

Every slope reuses the **forward output** captured inside the closure, exactly as slide 31 promised.

Topological order: never visit a node before its users

STEP 1 · LINE UP THE NODES, INPUTS FIRST



STEP 2 · CALL _backward IN REVERSE



Why the order matters: q may only pass its signal on once it has received *all* of its own signal from f. Reverse topological order guarantees that.

The build function is a depth first search: a node joins the list only after all of its parents have joined.

```
def backward(self):
    topo, visited = [], set()

    def build(v):                # depth first search
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build(child)
            topo.append(v)       # after its parents

    build(self)
    self.grad = 1.0              # start signal
    for v in reversed(topo):
        v._backward()

def __repr__(self):
    return (f"Value(data={self.data:.4f}, "
            f"grad={self.grad:.4f})")
```

The engine reproduces every hand calculation

```
from engine import Value

# test 1: the graph from slide 106
x, y, z = Value(2.0), Value(3.0), Value(4.0)
f = (x + y) * z
f.backward()
print("f:", f.data)
print("x:", x); print("y:", y); print("z:", z)

# test 2: the sigmoid neuron from slide 109
x1, x2 = Value(1.0), Value(2.0)
w1, w2, b = Value(0.5), Value(0.25), Value(0.25)
a = (x1 * w1 + x2 * w2 + b).sigmoid()
a.backward()
print("a:", round(a.data, 4), " w1.grad:", round(w1.grad, 4),
      " w2.grad:", round(w2.grad, 4), " b.grad:", round(b.grad, 4))
```

OUTPUT

```
f: 20.0
x: Value(data=2.0000, grad=4.0000)
y: Value(data=3.0000, grad=4.0000)
z: Value(data=4.0000, grad=5.0000)
a: 0.7773  w1.grad: 0.1731
w2.grad: 0.3462  b.grad: 0.1731
```

Quantity	by hand	engine
x, y, z signals	4, 4, 5	4, 4, 5 [icon: Check]
neuron output	0.7773	0.7773
weight signals	0.1731, 0.3462	same

We wrote no derivative for these expressions. The engine worked them out from the recorded operations.

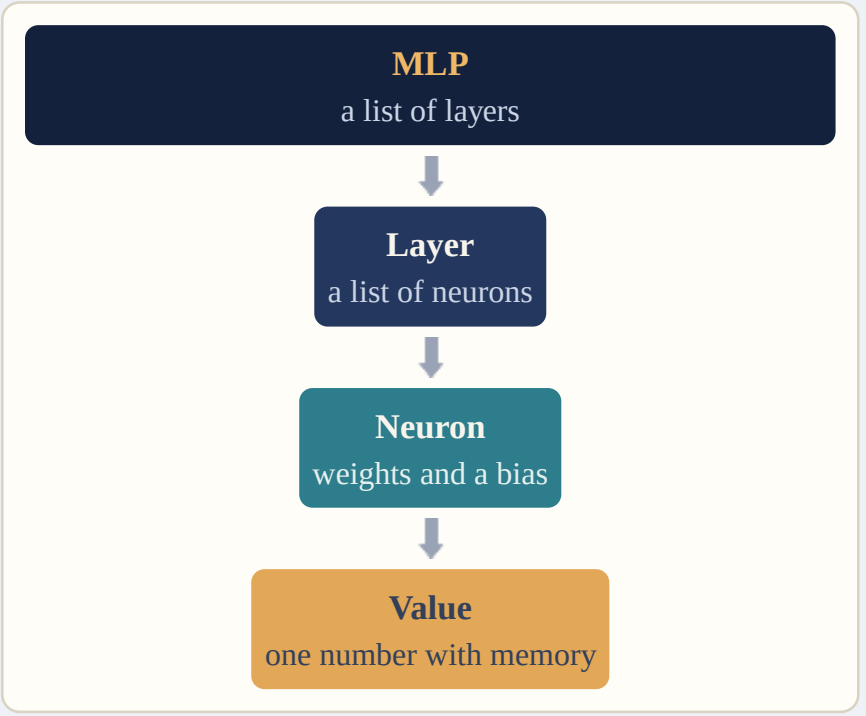
Neuron, Layer and MLP: three small classes

```
rng = np.random.default_rng(1)

class Neuron:
    def __init__(self, n_in, act="tanh"):
        self.w = [Value(float(rng.normal(0.0, 1.0))) for _ in range(n_in)]
        self.b = Value(0.0)
        self.act = act
    def __call__(self, x):
        total = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return total.tanh() if self.act == "tanh" else total.sigmoid()
    def parameters(self):
        return self.w + [self.b]

class Layer:
    def __init__(self, n_in, n_out, act="tanh"):
        self.neurons = [Neuron(n_in, act) for _ in range(n_out)]
    def __call__(self, x):
        out = [n(x) for n in self.neurons]
        return out[0] if len(out) == 1 else out
    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, n_in, sizes):
        dims = [n_in] + sizes
        self.layers = [Layer(dims[i], dims[i + 1],
            "tanh" if i + 1 < len(sizes) else "sigmoid") for i in range(len(sizes))]
    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]
```



Each class only needs **parameters()** to list its Values; autograd handles all the gradients.

Training the engine on XOR, the problem that stopped the perceptron

```
xs = [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]]
ys = [0.0, 1.0, 1.0, 0.0]              # XOR, the old enemy
model = MLP(2, [4, 4, 1])
print("parameters:", len(model.parameters()))

lr = 0.5
for step in range(501):
    # 1 · forward and loss (sum of squared misses)
    preds = [model(x) for x in xs]
    loss = sum((p.minus(y) * p.minus(y)
                for p, y in zip(preds, ys)), Value(0.0))
    # 2 · clear old signals, then backward
    for p in model.parameters():
        p.grad = 0.0
    loss.backward()
    # 3 · step every parameter downhill
    for p in model.parameters():
        p.data = np.subtract(p.data, lr * p.grad)
    if step in (0, 50, 100, 200, 300, 500):
        print(f"step {step:3d}  loss {loss.data:.4f}")

print("predictions:", [round(float(model(x).data), 3) for x in xs])
```

THE CLASSIC BUG

Forgetting to reset grad to zero. Because signals are **added** (+=), old steps would pile onto new ones and training would go haywire.

The network is 2 inputs, two hidden layers of 4, and 1 output: **37** parameters in total.

> This is the exact pattern PyTorch uses: zero_grad, backward, step.

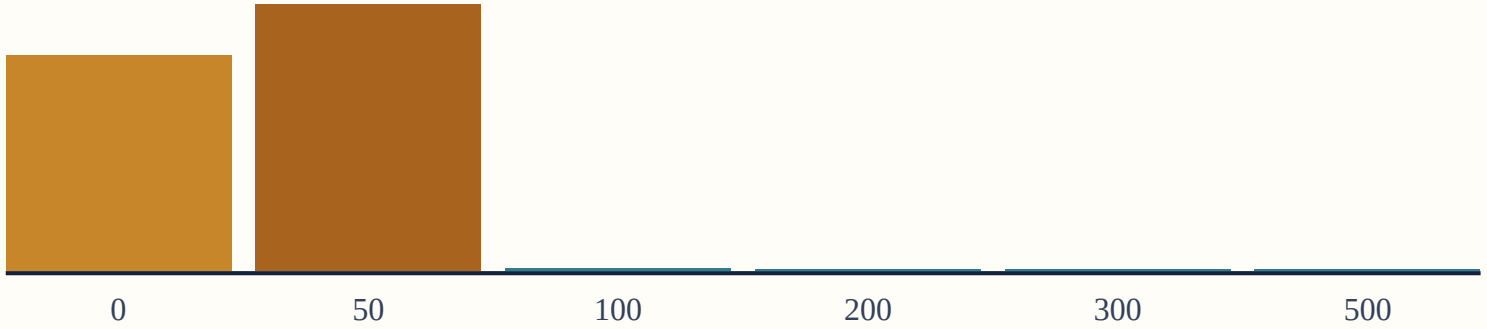
XOR solved, with every gradient computed automatically

OUTPUT

```
parameters: 37
step   0  loss 1.1611
step  50  loss 1.4345
step 100  loss 0.0126
step 200  loss 0.0038
step 300  loss 0.0022
step 500  loss 0.0012
predictions: [0.009, 0.979, 0.99, 0.023]
```

Notice step 50: the loss went *up* before collapsing. Early training is often bumpy while the hidden units find their roles.

Loss at each checkpoint



Predictions versus truth



From scalar nodes to tensor nodes

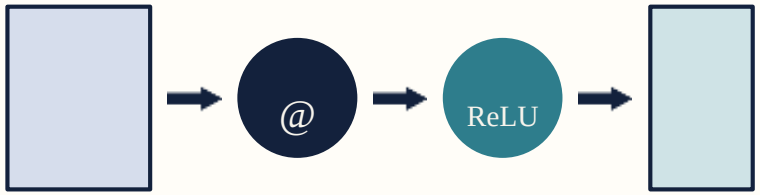
Our engine: one node per number



…and on, and on

Our digit network would need millions of Value objects per batch, each a slow Python object. Perfect for learning, far too slow for real work.

Frameworks: one node per array



Each node holds an entire matrix. The backward rule for the whole block is one vectorised line, exactly the layer recipes of slide 110.

Same idea, same gate rules, a handful of nodes instead of millions.

Next module: we build exactly that, layer objects holding NumPy arrays, each with a forward and a backward method.

MODULE 10 · RECAP

What autograd really is, in five pictures

1



Wrap numbers so each one remembers its parents and its operation.

2



Every operation leaves behind a tiny backward rule: copy, swap, route or scale.

3



Normal code silently builds the graph during the forward pass.

4



backward() orders the graph and runs each rule in reverse.

5



Zero the grads, backward, step: the universal training loop.

You now understand what loss.backward() does in PyTorch. It is this engine, scaled to tensors and written in fast compiled code.

11

MODULE ELEVEN

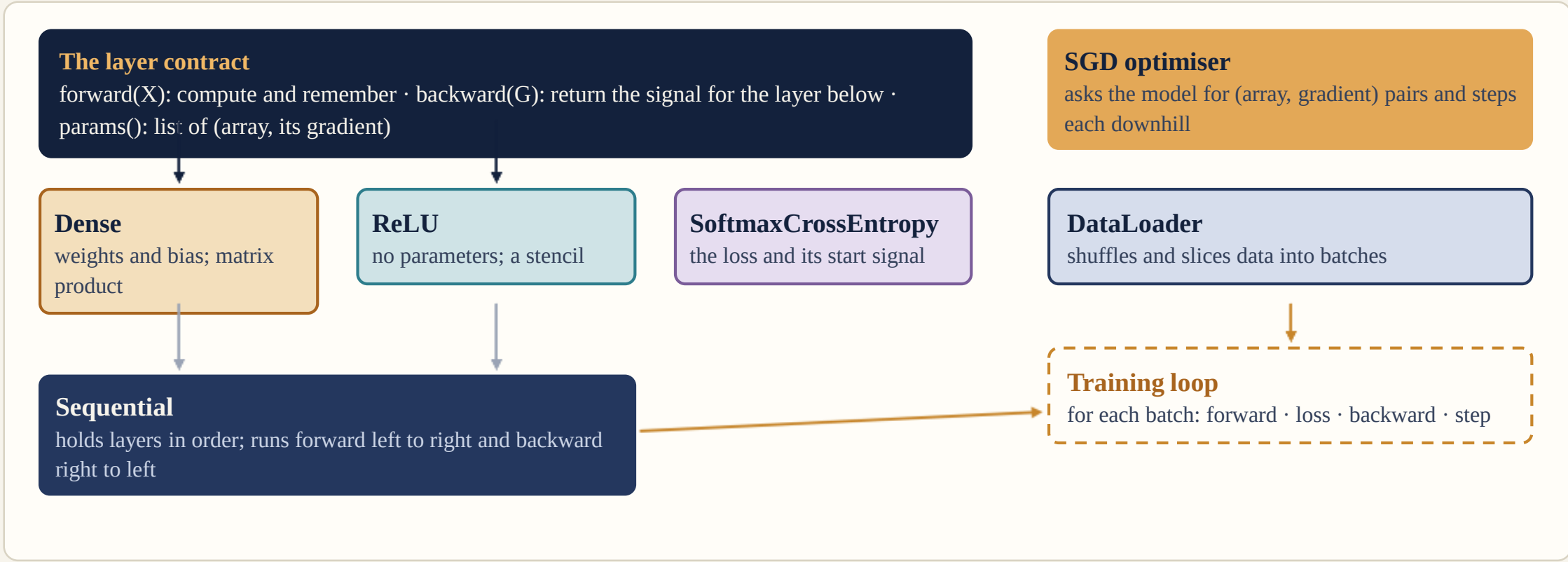
A Neural Network Library

Fast, array based building blocks you can snap together: layers, a loss, a container, an optimiser and a data loader. About one hundred lines.

IN THIS MODULE

- The library blueprint
- Dense layer
- ReLU layer
- Softmax cross entropy loss
- Sequential container
- SGD optimiser and data loader
- The training loop and evaluation

The library blueprint: six parts, one contract



Key idea. Because every layer honours the same three method contract, adding a new layer type (convolution, dropout, attention) never requires changing anything else.

The Dense layer: slide 110 turned into a class

```
import numpy as np

class Dense:
    def __init__(self, n_in, n_out, rng):
        self.W = rng.normal(0.0, np.sqrt(2 / n_in), size=(n_in, n_out))
        self.b = np.zeros(n_out)
        self.dW = np.zeros_like(self.W)
        self.db = np.zeros_like(self.b)

    def forward(self, X):
        self.X = X                      # remember for backward
        return X @ self.W + self.b

    def backward(self, G):
        self.dW = self.X.T @ G          # signal for W
        self.db = G.sum(axis=0)         # signal for b
        return G @ self.W.T             # signal for the layer below

    def params(self):
        return [(self.W, self.dW), (self.b, self.db)]
```

QUICK SHAPE TEST

```
layer = Dense(64, 32, rng)
Z = layer.forward(rng.random((5, 64)))
dX = layer.backward(np.ones_like(Z))
print("forward :", Z.shape)
print("dW, db  :", layer.dW.shape, layer.db.shape)
print("dX      :", dX.shape)
```

OUTPUT

```
forward : (5, 32)
dW, db  : (64, 32) (32,)
dX      : (5, 64)
```

> Every gradient matches its parameter’s shape, and dX matches the input: the layer below can take over.

The ReLU layer: a stencil with no parameters

```
class ReLU:
    def forward(self, X):
        self.mask = X > 0        # the stencil
        return X * self.mask

    def backward(self, G):
        return G * self.mask     # block where off

    def params(self):
        return []                # nothing to learn
```

```
r = ReLU()
A = r.forward(np.subtract(rng.random((2, 4)), 0.5))
print("fraction kept by ReLU:", r.mask.mean())
print(r.backward(np.ones((2, 4))))
```

OUTPUT

```
fraction kept by ReLU: 0.375
[[1. 1. 1. 0.]
 [0. 0. 0. 0.]]
```

THE STENCIL FROM THE TEST

1	1	1	0
0	0	0	0

3 of 8 cells were positive: the 0.375 printed above. A signal of ones passes only through the open cells.

Design note: activations are layers too. Treating them uniformly keeps the Sequential container simple.

Softmax cross entropy: loss and starting signal together

```
class SoftmaxCrossEntropy:
    def forward(self, logits, y):
        # softmax with the safety shift from slide 90
        shifted = np.subtract(logits, logits.max(axis=1, keepdims=True))
        e = np.exp(shifted)
        self.probs = e / e.sum(axis=1, keepdims=True)
        self.y = y
        rows = np.arange(len(y))
        p_true = np.clip(self.probs[rows, y], 0.000000000001, 1.0)
        return np.mean(np.log(1 / p_true))   # average surprise

    def backward(self):
        rows = np.arange(len(self.y))
        G = self.probs.copy()
        G[rows, self.y] = np.subtract(G[rows, self.y], 1.0)
        return G / len(self.y)     # probs minus one hot, averaged
```

probs[rows, y] picks, from each row, the probability given to that row's true class: one number per example.

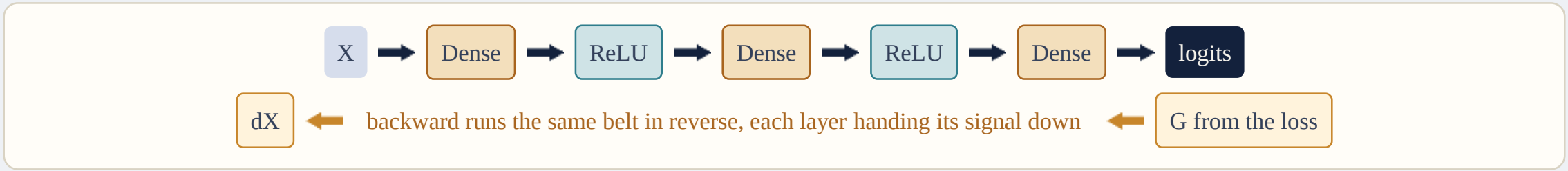
```
loss = SoftmaxCrossEntropy()
logits = np.zeros((4, 10))  # no opinion
print(round(loss.forward(logits,
      np.array([0, 3, 5, 9])), 4),
      round(np.log(10), 4))
```

OUTPUT

2.3026 2.3026

Sanity anchor: a model with no opinion over 10 classes scores the log of 10, about 2.30. Remember this number.

Sequential: a conveyor belt of layers




```
class Sequential:
    def __init__(self, *layers):
        self.layers = layers

    def forward(self, X):
        for layer in self.layers:
            X = layer.forward(X)
        return X

    def backward(self, G):
        for layer in reversed(self.layers):
            G = layer.backward(G)

    def params(self):
        return [p for layer in self.layers for p in layer.params()]
```

Twelve lines turn a list of layers into a whole network that can run forward, run backward and list all of its learnable arrays.

reversed is the whole secret of backprop's ordering for a simple chain: the topological sort of slide 126 reduces to "go backwards".

The SGD optimiser and the data loader

OPTIMISER

```
class SGD:
    def __init__(self, model, lr=0.1):
        self.model = model
        self.lr = lr

    def step(self):
        for p, g in self.model.params():
            np.subtract(p, self.lr * g, out=p)
```



out=p writes the result back into the very same array the layer holds. No copies, so the layer sees its new weights immediately.

DATA LOADER

```
class DataLoader:
    def __init__(self, X, y, batch_size, rng, shuffle=True):
        self.X, self.y = X, y
        self.batch_size = batch_size
        self.rng = rng
        self.shuffle = shuffle

    def __iter__(self):
        n = len(self.X)
        idx = self.rng.permutation(n) if self.shuffle else np.arange(n)
        for start in range(0, n, self.batch_size):
            b = idx[start:start + self.batch_size]
            yield self.X[b], self.y[b]
```

yield makes the loader a generator: write “for xb, yb in loader” and a freshly shuffled set of batches arrives every epoch.

The training loop: all the parts working together

```
model = Sequential(Dense(64, 64, rng), ReLU(),
                   Dense(64, 32, rng), ReLU(),
                   Dense(32, 10, rng))
loss_fn = SoftmaxCrossEntropy()
opt = SGD(model, lr=0.1)
loader = DataLoader(X_train, y_train, batch_size=32, rng=rng)

for epoch in range(1, 31):
    total, count = 0.0, 0
    for xb, yb in loader:
        logits = model.forward(xb)              # 1 · forward
        loss = loss_fn.forward(logits, yb)      # 2 · loss
        model.backward(loss_fn.backward())      # 3 · backward
        opt.step()                              # 4 · step
        total += loss * len(yb); count += len(yb)
    tr_acc = accuracy(model, X_train, y_train)
    va_acc = accuracy(model, X_val, y_val)
    print(f"epoch {epoch:2d}  loss {total / count:.4f}"
          f"  train acc {tr_acc:.3f}  val acc {va_acc:.3f}")
```



four strokes per batch, like an engine

No zero_grad needed: our layers overwrite their gradients on every backward rather than adding to them.

The loss is weighted by batch size so the last, smaller batch counts fairly in the epoch average.

Evaluating, predicting, and two checks before training

```
def accuracy(model, X, y):
    # index of the largest score per row, compared with the truth
    return np.mean(model.forward(X).argmax(axis=1) == y)

def predict_proba(model, X):
    logits = model.forward(X)
    e = np.exp(np.subtract(logits, logits.max(axis=1, keepdims=True)))
    return e / e.sum(axis=1, keepdims=True)

# two checks before the first update
print("parameters:", sum(p.size for p, _ in model.params()))
print("loss before training:",
      round(loss_fn.forward(model.forward(X_train), y_train), 4))
```

OUTPUT

parameters: 6570
loss before training: 2.4522

Check 1 · size

64×64 + 64, plus 64×32 + 32, plus 32×10 + 10 makes 6,570. Matches.

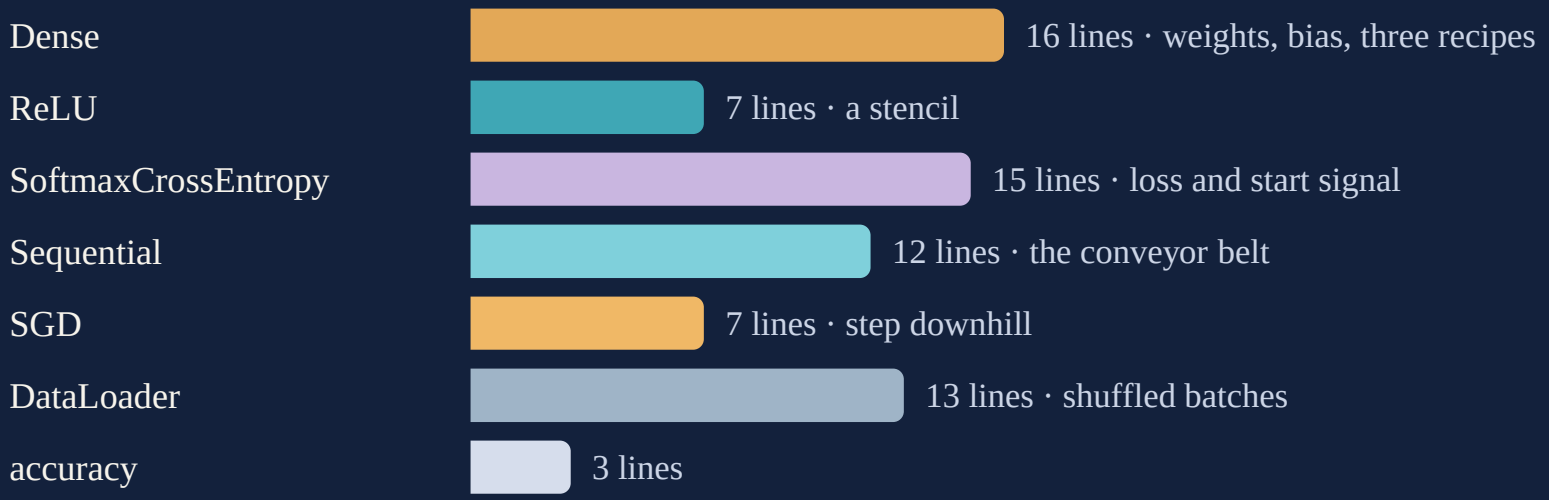
Check 2 · starting loss



Close to the no opinion value of 2.30 (grey). A random network should be near it; a value like 15 would signal a bug or bad initialisation.

MODULE 11 · RECAP

Our whole library, measured in lines of code



About 75 lines give a trainable, extensible deep learning library.

Next: point it at real handwriting and see what it learns.

12

MODULE TWELVE · PROJECT

Reading Handwritten Digits

An end to end project with our own library: real images, a real split, real learning curves, and an honest look at the mistakes.

IN THIS MODULE

- The dataset: 1,797 tiny images
- Loading, scaling and splitting
- Labels as one hot rows
- The model and its training log
- Learning curves
- Confusion matrix and error analysis
- Results and what to try next

What a handwritten digit looks like to a computer

IMAGE 0 · LABEL “0” · INTENSITY 0 TO 16

0	0	5	13	9	1	0	0
0	0	13	15	10	15	5	0
0	3	15	2	0	11	8	0
0	4	12	0	0	8	8	0
0	5	8	0	0	9	8	0
0	4	11	0	1	12	7	0
0	2	14	5	10	12	0	0
0	0	6	13	10	0	0	0

An **8 by 8 grid: 64 numbers**. Dark cells are ink; the ring of ink makes a zero. The network never sees the shape, only these 64 values in a row.

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.data / 16.0   # rescale to 0 to 1
y = digits.target
print(digits.images.shape, X.shape, y.shape)
print("class counts:", np.bincount(y))
```

OUTPUT

```
(1797, 8, 8) (1797, 64) (1797,)
class counts: [178 182 177 183 181
               182 181 179 174 180]
```

Balanced classes: about 180 of each digit, so plain accuracy is a fair score.

Splitting the data three ways before touching the model



the model **learns** from these

we **tune** choices on these

opened **once**, at the end

```
rng = np.random.default_rng(42)
order = rng.permutation(len(X))        # shuffle before splitting
train, val, test = order[:1200], order[1200:1497], order[1497:]
X_train, y_train = X[train], y[train]
X_val, y_val = X[val], y[val]
X_test, y_test = X[test], y[test]
print("train", X_train.shape, " val", X_val.shape, " test", X_test.shape)
print("pixel range:", X.min(), "to", X.max())
```

OUTPUT

```
train (1200, 64)  val (297, 64)  test (300, 64)
pixel range: 0.0 to 1.0
```

THE GOLDEN RULE

If you make decisions by looking at the test set, its score stops being an honest estimate of real world performance.

Scaling to 0 to 1 keeps the first layer’s weighted sums in a sensible range for the starting weights.

One hot encoding: a label as a row of switches

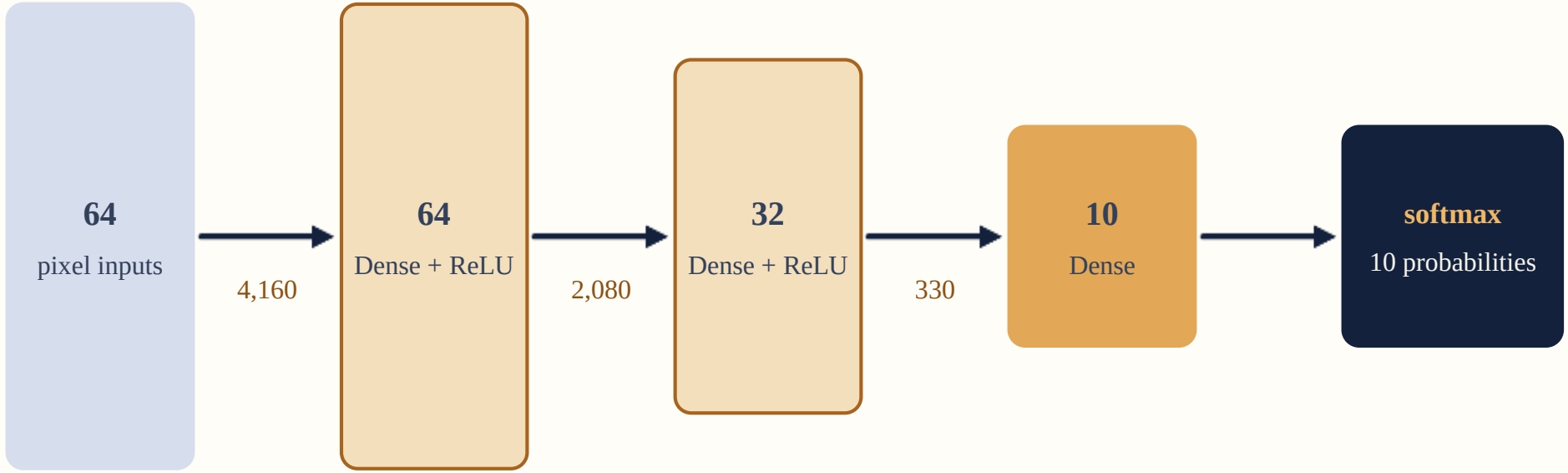
	0	1	2	3	4	5	6	7	8	9
label 3	0	0	0	1	0	0	0	0	0	0
label 7	0	0	0	0	0	0	0	1	0	0
model output	0.01	0.02	0.04	**0.81**	0.01	0.06	0.01	0.01	0.02	0.01

Why switches? A label like 7 is not “more” than 3. One hot rows treat every class as an equal, separate category, matching the 10 softmax outputs one for one.

```
onehot = np.eye(10)[y]  # row y of the identity
# our loss indexes probs[rows, y] directly,
# which is the same as using these rows
```

The learning signal is the output row minus the one hot row: push up the switch that should be on, push down all others.

The network: 64 pixels in, 10 digit scores out




```
model = Sequential(Dense(64, 64, rng), ReLU(),
                   Dense(64, 32, rng), ReLU(),
                   Dense(32, 10, rng))   # 6,570 parameters
```

Small on purpose: 1,200 training images cannot support a giant model without heavy memorising.

The training log: 30 epochs in about a second

OUTPUT

```
parameters: 6570
loss before training: 2.4522
epoch  1  loss 1.7318  train acc 0.581  val acc 0.532
epoch  2  loss 0.7729  train acc 0.900  val acc 0.886
epoch  3  loss 0.4184  train acc 0.932  val acc 0.896
epoch  5  loss 0.2331  train acc 0.908  val acc 0.892
epoch 10  loss 0.1192  train acc 0.978  val acc 0.949
epoch 15  loss 0.0727  train acc 0.986  val acc 0.966
epoch 20  loss 0.0507  train acc 0.994  val acc 0.973
epoch 25  loss 0.0370  train acc 0.997  val acc 0.966
epoch 30  loss 0.0255  train acc 0.997  val acc 0.970
test accuracy: 0.9467
```

Epoch 1 to 2

From near guessing to 90% in two passes: the easy patterns are learned first.

Epoch 5 dip

Accuracy wobbles even as loss falls. Mini batch noise is normal; judge the trend.

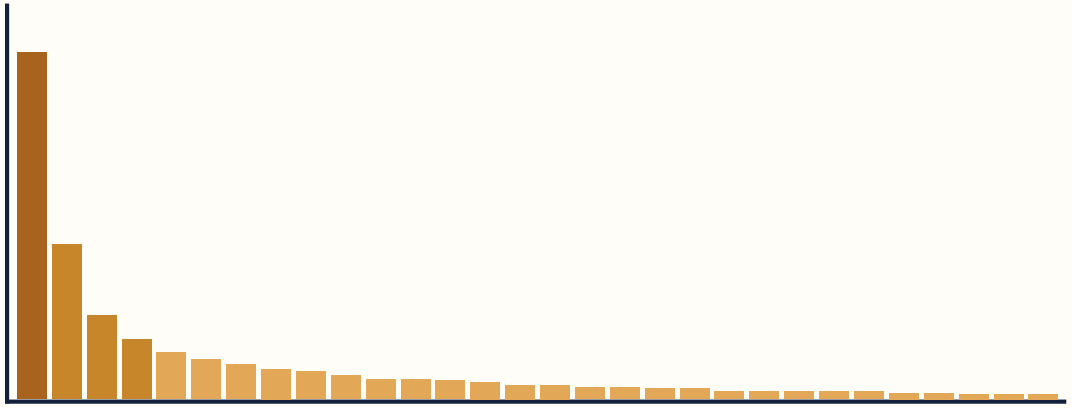
Epoch 20 onward

Training accuracy nears 100% but validation plateaus around 97%: the start of overfitting.

Test accuracy 94.7%: 284 of 300 unseen digits read correctly by code we wrote entirely ourselves.

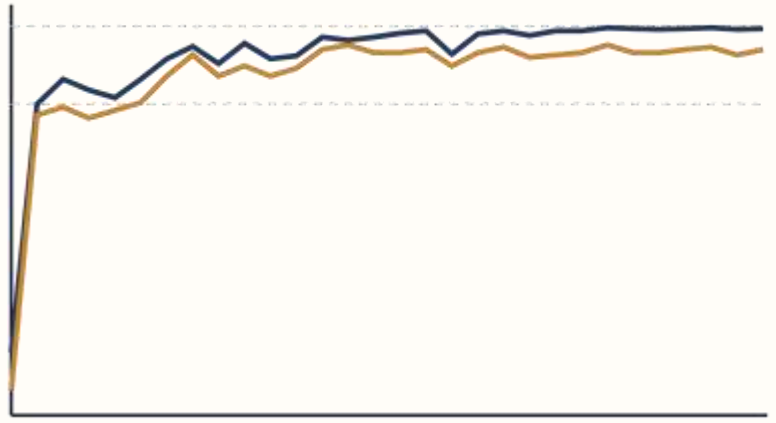
Learning curves: every epoch of the run, drawn to scale

Training loss per epoch



epoch 1 · 1.73 epoch 30 · 0.026

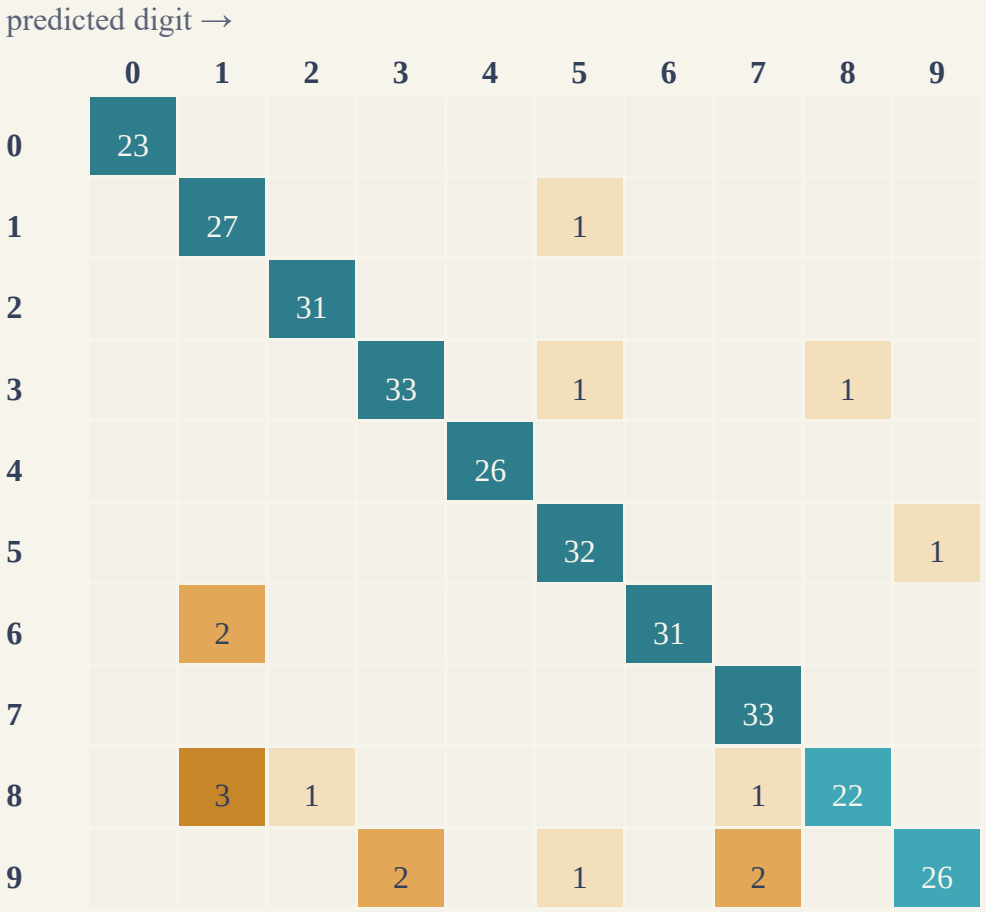
Accuracy per epoch



navy: training **amber**: validation dashed lines: 100% and 90%

Reading the gap: the navy line ends near 100% while amber settles around 97%. A small, stable gap like this is healthy; a widening gap would call for regularisation (Module 16).

Where the 300 test digits landed



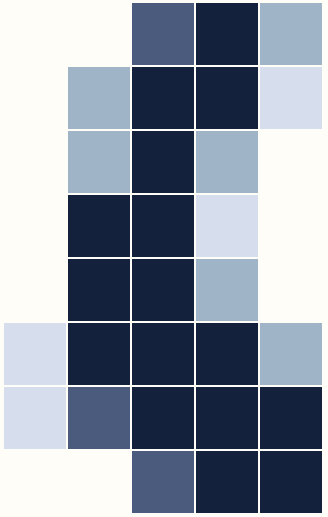
How to read it: row is the true digit, column is the model's guess. Teal diagonal: correct. Amber cells: mistakes, darker for more.

Weakest rows: 8 (22 of 28 right) and 9 (26 of 32). Perfect rows: 0, 2, 4 and 7.

Column 1 attracts errors: 8s and 6s drift toward 1. A thin, faint stroke at 8 by 8 resolution can look like a 1.

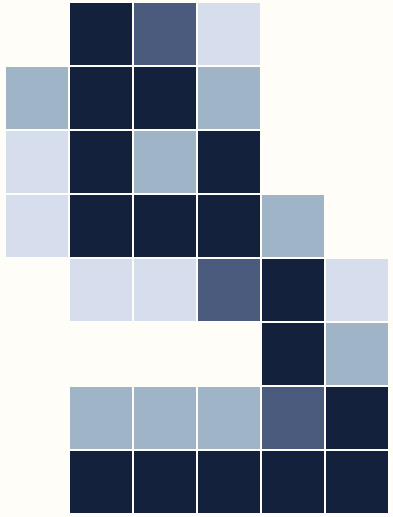
Looking at the mistakes themselves

truth 6 · guessed 1 · 84% sure



The loop of the 6 is squashed; the strong vertical stroke dominates.

truth 9 · guessed 5 · 88% sure



A heavy bottom bar makes this 9 resemble a 5. Honestly, a human might hesitate too.

MOST COMMON CONFUSIONS

8 → 1
6 → 1
9 → 7
9 → 3

Lesson: mistakes are often confidently wrong (84 and 88%). High confidence is not the same as correctness.

MODULE 12 · RESULTS

Project scorecard, and where to go from here

test accuracy	parameters	library code	random guessing
94.7%	6,570	~75	10%
284 of 300	three dense layers	lines, all our own	the baseline

IDEAS TO PUSH FURTHER, AND THE MODULE THAT COVERS EACH

Better optimiser. Adam reaches the same accuracy in fewer epochs. Module 15.

Regularise. Dropout and weight decay narrow the train and validation gap. Module 16.

Augment. Shift digits by a pixel to create new training examples. Module 16.

Use the layout. Convolutions know that neighbouring pixels belong together. Module 18.

Tune the schedule. Lower the learning rate as training settles. Module 15.

Scale up. Swap in 28 by 28 MNIST: change 64 to 784 and nothing else. Module 23.

13

MODULE THIRTEEN

Loss Functions

The loss is how you tell a network what you want. Choose it badly and a perfect optimiser will faithfully learn the wrong thing.

IN THIS MODULE

- What every loss must do
- Squared, absolute and Huber
- Keeping cross entropy finite
- Multiclass cross entropy, worked
- Taming huge scores
- Hinge and focal losses
- A selection guide

Four jobs every loss function must do



1 · Zero when perfect

A flawless prediction should score the minimum, so there is a clear target.

2 · Grows with error

Worse predictions must always score worse, otherwise the optimiser is misled.

3 · Useful slopes

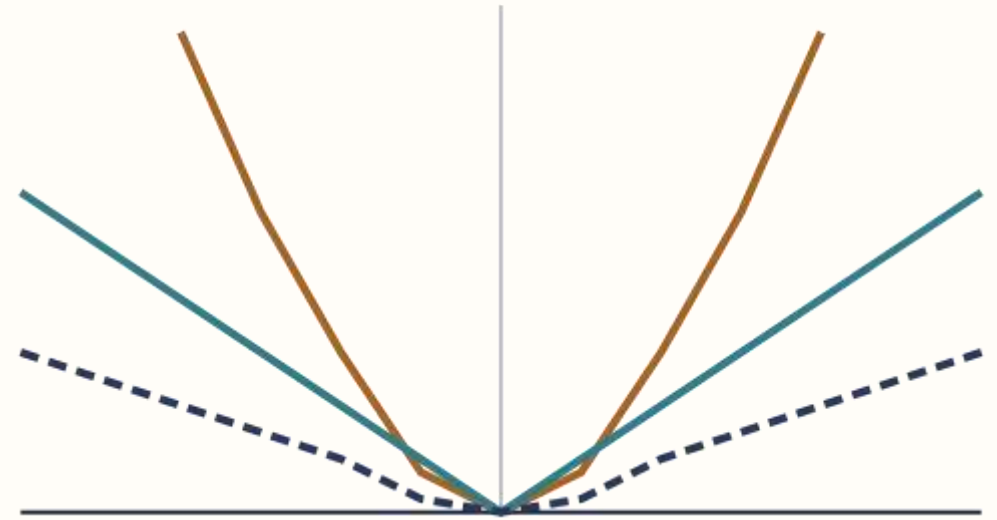
It must be smooth enough that the backward pass gives meaningful directions everywhere.

4 · Matches the goal

It should punish the mistakes you actually care about, in proportion to how much you care.

Why not train on accuracy directly? Accuracy jumps in steps: nudging a weight slightly usually changes nothing, so its slope is zero almost everywhere. Smooth losses give a signal on every step.

Squared, absolute and Huber: how hard to punish a big miss



squared (MSE) **absolute** (MAE) **Huber**, dashed

Squared: doubles the miss, quadruples the penalty. Smooth, but one wild outlier dominates.

Absolute: penalty grows evenly. Robust to outliers, but a sharp corner at zero.

Huber: squared for small misses, absolute for large ones. The best of both.

OUTLIER TEST · data 2, 3, 3, 4, 20

Best single guess under squared loss: **6.4**, the mean, dragged by the 20.

Under absolute loss: **3.0**, the median, unbothered.

Keeping cross entropy finite: the log of zero problem



One impossible example poisons the whole batch and every weight.

```
p = np.array([0.0, 0.5, 1.0])      # probability given to the truth
print(np.log(1 / p))               # naive surprise

eps = 0.000000001                  # fix 1: clip away from zero
print(np.round(np.log(1 / np.clip(p, eps, 1.0)), 3))
```

OUTPUT

```
[      inf 0.69314718 0.       ]
[20.723  0.693  0.   ]
```

Fix 1 · clip

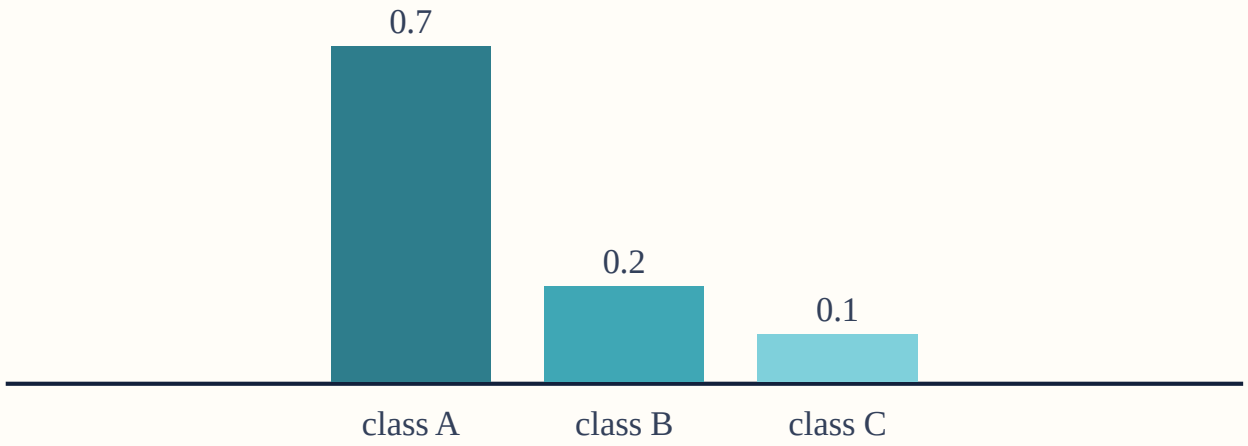
Replace exact zeros with a tiny number. The loss becomes large (20.7) but finite.

Fix 2 · work with scores

Compute the loss straight from the raw scores (logits), never forming a probability that can underflow to zero. Frameworks call this “with logits”.

Multiclass cross entropy only looks at the true class’s bar

The model’s belief, the same in all three cases



truth is A	bar 0.7	**0.357**
truth is B	bar 0.2	**1.609**
truth is C	bar 0.1	**2.303**

Loss for each case, the surprise at the true class, drawn as amber length.

Key idea. The loss reads only one bar, the true class. Yet its signal reaches every bar, because softmax ties them: lifting the true bar must lower the others.

Taming huge scores: the shift before softmax

Naive route

1000, 999, 998 → exp: inf, inf, inf → inf ÷ inf: nan

Computers cannot store the exponential of 1000; it overflows to infinity.

Shifted route

1000, 999, 998 → take away 1000: 0, −1, −2 → 0.665, 0.245, 0.090

Softmax only cares about **gaps** between scores, so sliding them all down changes nothing except safety.

```
z = np.array([1000.0, 999.0, 998.0])
print(np.exp(z))
naive = np.exp(z) / np.exp(z).sum()
print(naive)

s = np.exp(np.subtract(z, z.max()))    # largest becomes 0
print(np.round(s / s.sum(), 3))
```

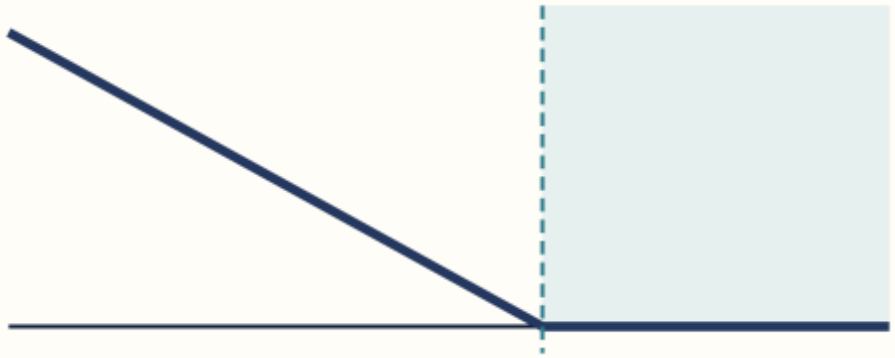
OUTPUT

```
[inf inf inf]
[nan nan nan]
[0.665 0.245 0.09 ]
```

The same trick, called log sum exp, keeps cross entropy stable too.

Hinge and focal losses: two different philosophies

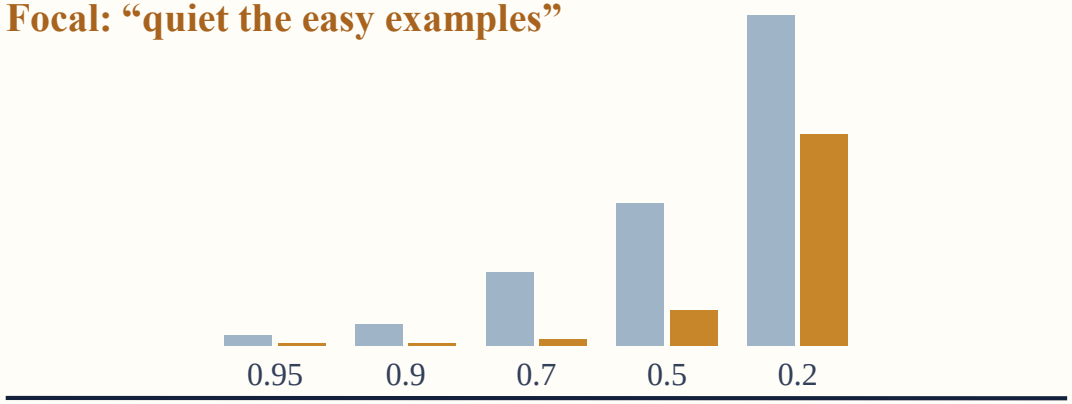
Hinge: “win by a margin, then stop caring”



wrong side margin safely right

Correct and confident enough examples (teal zone) contribute **nothing**. Used by support vector machines and some ranking models.

Focal: “quiet the easy examples”



probability given to the truth · **grey** cross entropy · **amber** focal

An easy example at 0.95 is quietened **400 times**; a hard one at 0.2 only 1.6 times. Training attention shifts to the hard cases. Popular in object detection, where easy background dominates.

Choosing a loss: match the output layer to the task

Task	Final layer	Loss	Watch out for
predict a number	none (linear)	squared error	outliers dominate
number, noisy targets	none (linear)	Huber or absolute	slower near zero
yes or no	sigmoid	binary cross entropy	use the logits form
one of many classes	softmax	cross entropy	shift before exp
several labels at once	sigmoid per label	binary cross entropy each	not softmax
rare positives	sigmoid or softmax	focal or weighted CE	tune the weighting
next word or token	softmax over vocabulary	cross entropy per token	huge output layer



Always decide in this order. The three choices come as a matched set; mixing them (softmax with squared error, say) trains slowly or badly.

14

MODULE FOURTEEN

Weight Initialisation

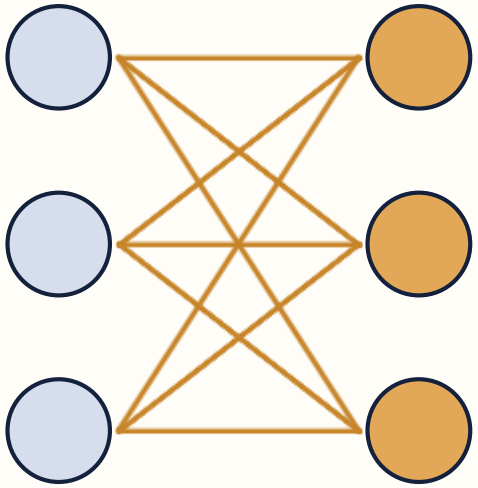
How a network starts decides whether it can learn at all. Too small and signals fade; too large and they explode.

IN THIS MODULE

- Why not start at zero
- Signals shrinking and growing through depth
- The spread that keeps signals steady
- Xavier start for tanh
- He start for ReLU
- Experiment and cheat sheet

Why weights cannot all start the same

Identical start: clones forever



Same weights in, same outputs out, same learning signal back, same update. The three hidden neurons stay **exact copies**: effectively one neuron.

All zeros is even worse: with zero weights the hidden outputs are all identical and the signals that should separate them never appear.

EXPERIMENT · 3 HIDDEN NEURONS, ALL WEIGHTS START AT 0.5

OUTPUT AFTER 100 STEPS

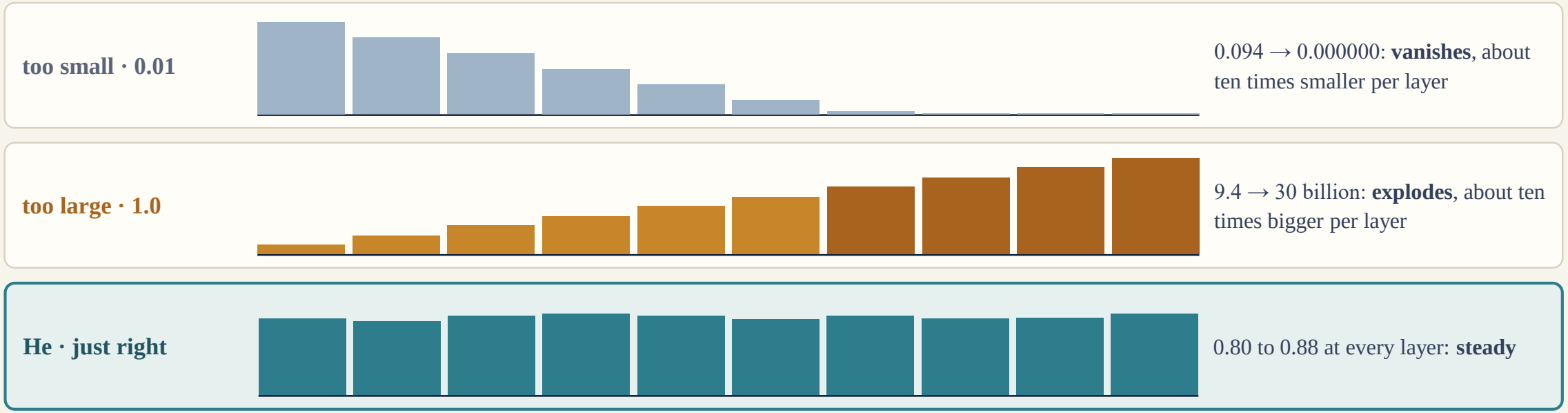
```
[[0.5569 0.5569 0.5569]
 [0.4437 0.4437 0.4437]
 [0.408  0.408  0.408 ]
 [0.5089 0.5089 0.5089]]
distinct columns: 1
```

The weights moved, but every column (one per neuron) moved identically.

The cure: start every weight at a different small **random** value. Randomness breaks the symmetry so neurons can specialise.

Random, but how big? Signals through ten ReLU layers

Typical size (spread) of the activations after each layer of a 10 layer, 256 wide ReLU network, for three choices of starting weight spread:



The first two rows use a “ten times per step” scale (bar height grows by a fixed step per factor of ten); the He row uses a plain scale.

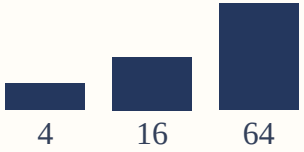
Why the right spread depends on how many inputs a neuron has

A neuron adds up random pushes



Each input times its weight is a small random push, up (grey) or down (amber).

More inputs, bigger total



Random pushes partly cancel: four times more inputs gives a total only **twice** as big (the square root).

So shrink each weight to match



Dividing the weight spread by the square root of the input count keeps the output size **constant**.

THE RECIPES, IN WORDS

Xavier (tanh, sigmoid): spread is the square root of 1 over the number of inputs.

He (ReLU): ReLU zeroes about half its inputs, so double the variance: the square root of 2 over the number of inputs.

```
# Xavier
W = rng.normal(0.0, np.sqrt(1 / n_in), size=(n_in, n_out))
# He, used in our Dense layer
W = rng.normal(0.0, np.sqrt(2 / n_in), size=(n_in, n_out))
```

The same test with tanh: a trap hidden in the large start

too small · 0.01



0.157 → 0.000000: vanishes

too large · 1.0

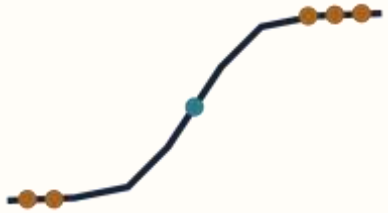


0.974 everywhere, looks fine, but **87%** of outputs are pinned near ±1

Xavier



0.63 → 0.24: a gentle decline, still alive and learnable



Saturated neurons sit on the flat ends of the S curve, where the slope is almost zero: **forward looks healthy, backward is dead**.

Lesson: check the slope as well as the size. For squashing activations, Xavier keeps neurons in the steep middle of the curve where they can learn.

The measurement behind the last three slides

```
def act_stats(scale_fn, act, depth=10, width=256, seed=0):
    rng = np.random.default_rng(seed)
    a = rng.normal(0, 1, size=(1000, width))   # 1000 inputs
    stds = []
    for _ in range(depth):
        W = rng.normal(0, scale_fn(width), size=(width, width))
        z = a @ W
        a = np.maximum(0, z) if act == "relu" else np.tanh(z)
        stds.append(a.std())           # typical size
    return stds

he = lambda n: np.sqrt(2 / n)
xavier = lambda n: np.sqrt(1 / n)
print("relu He     ", np.round(act_stats(he, "relu"), 3))
print("tanh Xavier ", np.round(act_stats(xavier, "tanh"), 3))
```

OUTPUT (rounded)

relu He

0.831 0.801 0.862 0.880 0.860
0.816 0.860 0.829 0.842 0.883

tanh Xavier

0.629 0.485 0.407 0.358 0.320
0.295 0.270 0.256 0.245 0.235

Swap in **lambda n: 0.01** or **lambda n: 1.0** to reproduce the vanishing and exploding rows.

A cheap habit for any new architecture: print activation sizes per layer before training.

Practical initialisation habits

Biases at zero

Random weights already break symmetry, so biases can safely start at exactly zero.

Match the activation

He for the ReLU family; Xavier for tanh and sigmoid. Frameworks often default to a sensible choice per layer type.

Seed it

Fix the random seed so that runs are repeatable and comparisons are fair.

Quiet final layer

Starting the output layer smaller gives near uniform first predictions, so the starting loss is close to the no opinion value.

Normalisation helps

Batch and layer normalisation (Module 16) rescale activations each step, making networks far less fragile to the starting scale.

Best start of all

Weights already trained on a related task: transfer learning (Module 18) skips the random start entirely.

Initialisation cheat sheet

Start	What happens	Use it?
All zeros	Every neuron identical forever: symmetry never breaks	Never for weights
Tiny random	Signal shrinks about ten times per layer, then vanishes	No
Large random	ReLU explodes to billions; tanh saturates at ±1	No
Xavier	Spread shrinks with the square root of the input count	Yes: tanh, sigmoid
He	Xavier with doubled variance to make up for ReLU zeroing half	Yes: ReLU family
Pretrained	Starts from useful features learned elsewhere	Best when available

One line memory aid: random to break symmetry, scaled by the square root of the inputs to keep every layer the same size.

15

MODULE FIFTEEN

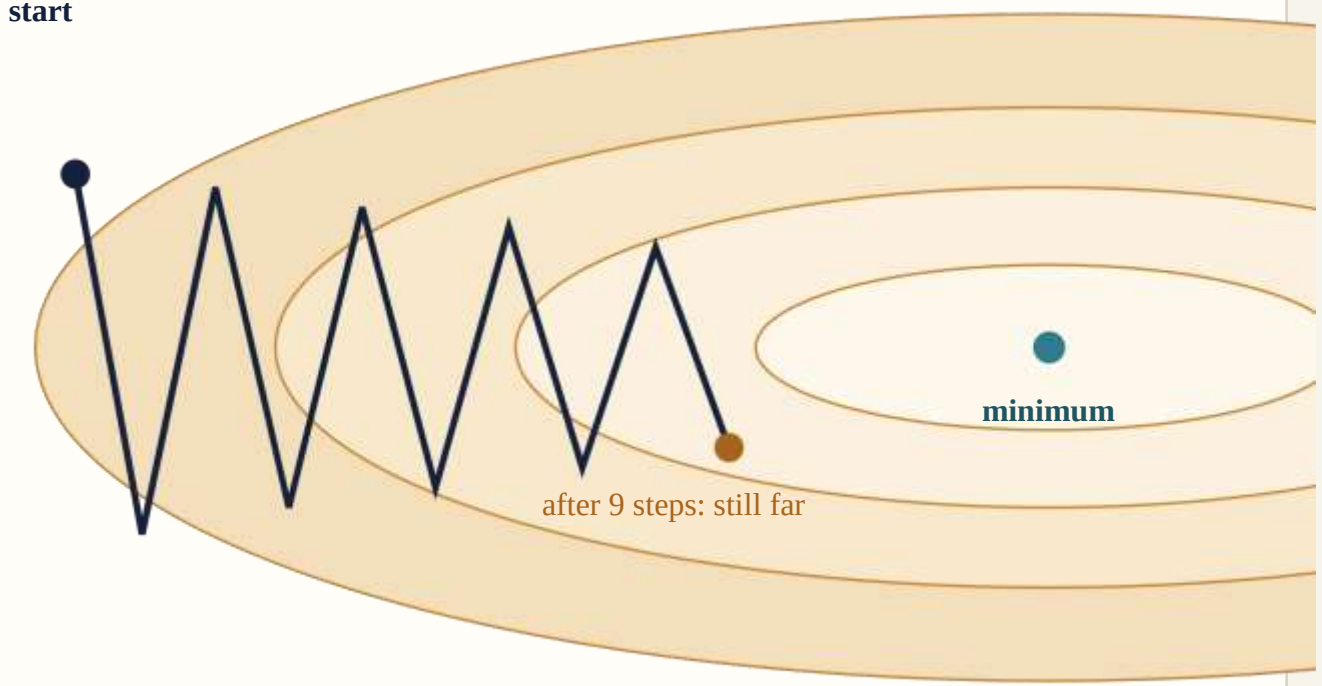
Optimisers

Plain gradient descent knows only the slope under its feet. Better optimisers add memory, so they roll through ravines and adapt their stride.

IN THIS MODULE

- Why plain SGD zigzags
- Momentum and Nesterov
- AdaGrad and RMSProp
- Adam, step by step
- Code and a head to head race
- AdamW and learning rate schedules

Plain SGD in a ravine: bouncing walls, crawling floor



Across the ravine the walls are steep, so the gradient is big and each step overshoots to the opposite wall.

Along the floor the slope is gentle, so progress toward the minimum is tiny.

The dilemma: a smaller learning rate stops the bouncing but makes the crawl even slower.

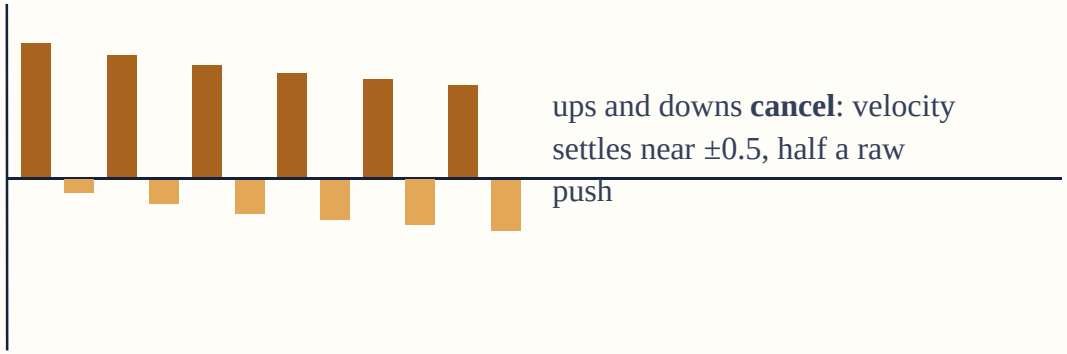
Real loss surfaces are full of such ravines, in millions of directions at once.

Momentum: a heavy ball that remembers where it was going

Keep a running velocity. Each step, keep 90% of the old velocity and add the new push. Watch what happens to two kinds of push over 20 steps:

Across the ravine: pushes flip

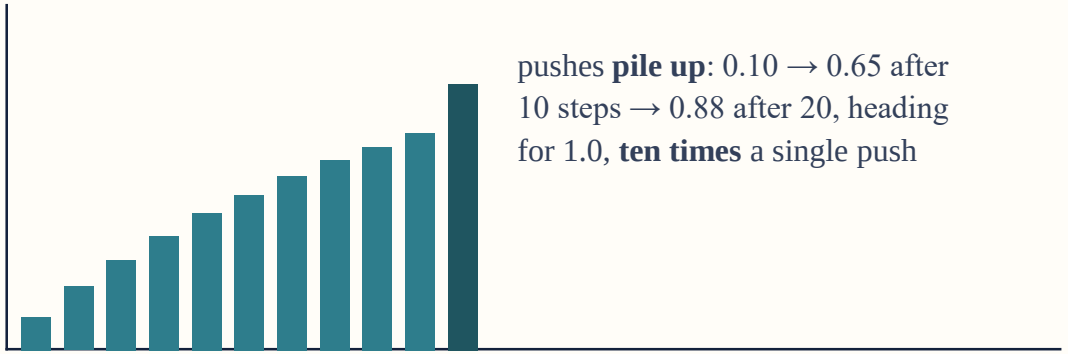
push: up 1, down 1, up 1, down 1 …



ups and downs **cancel**: velocity settles near ±0.5, half a raw push

Along the floor: pushes agree

push: 0.1, 0.1, 0.1, 0.1 …



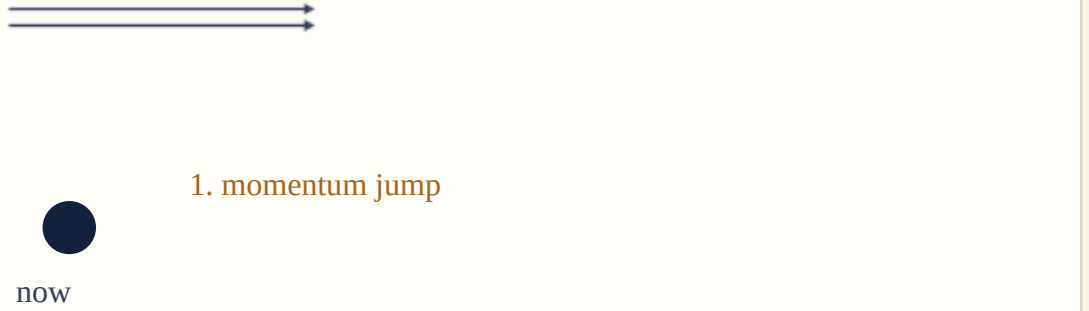
pushes **pile up**: 0.10 → 0.65 after 10 steps → 0.88 after 20, heading for 1.0, **ten times** a single push

Result: the zigzag is damped and the slow, consistent direction is amplified, exactly what the ravine needed.

```
v *= 0.9;  v += g   # remember, then add
np.subtract(p, lr * v, out=p)
```

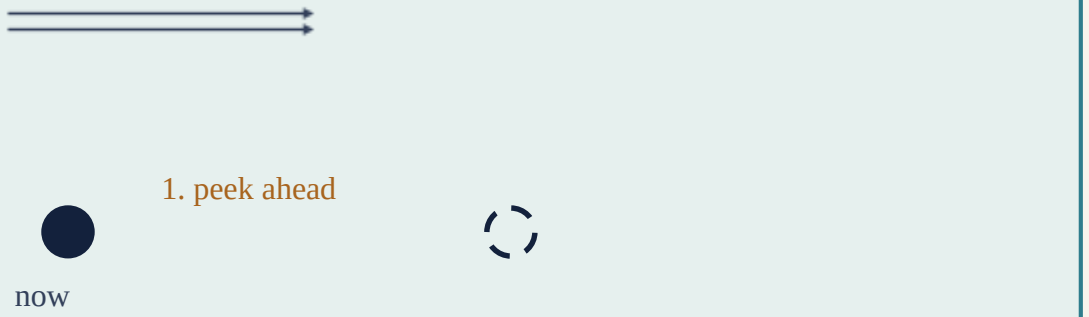
Nesterov: look where you are about to land, then correct

Classic momentum



Both arrows are decided before moving, so the ball can overshoot a valley before it notices.

Nesterov momentum



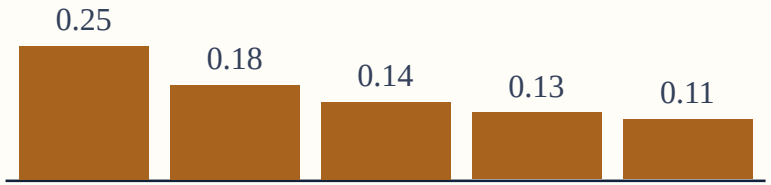
If the landing spot is already uphill, the correction pulls back early: less overshoot, faster settling.

Analogy: classic momentum steers by the rear view mirror; Nesterov glances through the windscreen first. In frameworks it is usually a single switch on SGD, such as **nesterov=True**.

AdaGrad: every weight gets its own step size

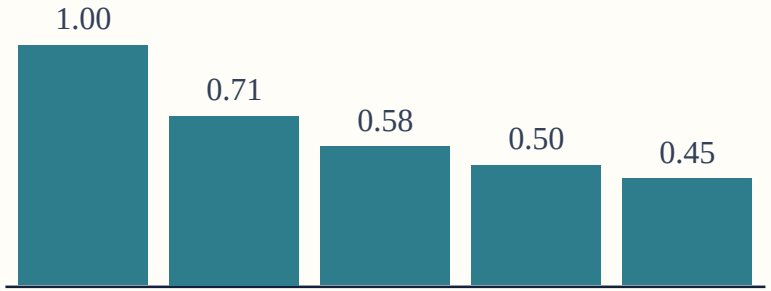
Each weight keeps a tally of how big its pushes have been. A weight that is pushed hard gets its steps **shrunk**; a weight that is rarely pushed gets **relatively larger** steps.

Steep weight: pushes of size 4



Learning rate multiplier per step: the big history tames the steep direction.

Gentle weight: pushes of size 0.1



Actual step size per step: despite a push 40 times smaller, it moves **4 times further** than the steep weight.

Great for sparse features, such as rare words, which deserve big updates on the few occasions they appear.

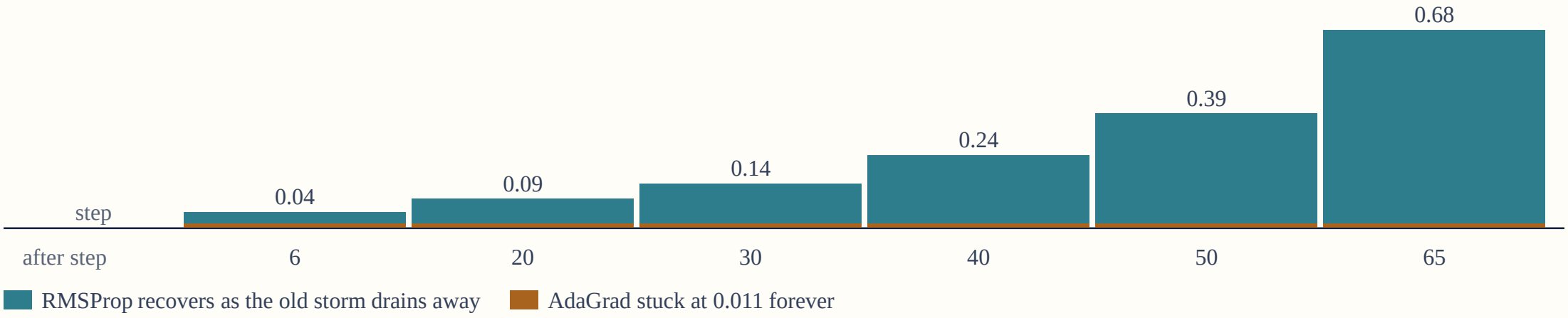
The flaw: the tally only ever grows, so every step size keeps shrinking until learning **stalls**.

RMSProp: a tally that forgets, so learning never stalls

[icon: Archive] **AdaGrad** keeps a box that only fills: every old push is remembered forever.

[icon: Droplets] **RMSProp** uses a leaky bucket: each step 10% of the old tally drains away, so recent pushes dominate.

Scenario: a weight is pushed hard (size 4) for 5 steps, then the landscape flattens and pushes drop to 0.1. Bars show the step it actually takes afterwards.



Adam: momentum and RMSProp working as a team



INPUT
this step's gradient for one weight

MEMORY 1 · DIRECTION
Smoothed average of recent gradients, keeping 90% each step. **Which way** have we been pushed lately? (momentum)

MEMORY 2 · SIZE
Smoothed average of squared gradients, keeping 99.9%. **How big** are pushes here usually? (RMSProp)

COMBINE
direction divided by typical size

step, times the learning rate

Steep, noisy weight: big typical size, so steps are scaled **down**.

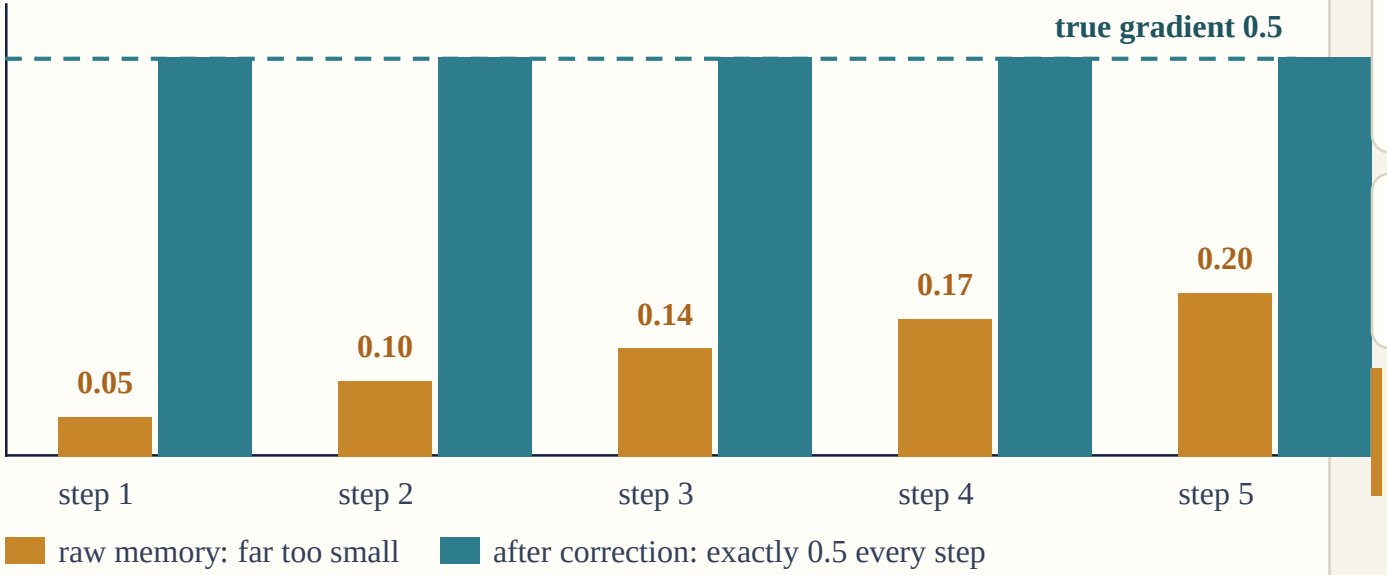
Flat, steady weight: small typical size, so steps are scaled **up**.

Net effect: every weight moves about one learning rate per step, whatever its gradient scale.

Defaults that work almost everywhere: learning rate **0.001**, memories **0.9** and **0.999**.

Memories that start empty are biased toward zero, so Adam rescales them

Feed a constant gradient of **0.5** for five steps. The memory starts at zero and keeps 90% of itself each step, so early on it badly underestimates the truth:



The fix in words: at step 1 the memory holds only 10% of the full weight, at step 2 only 19%, and so on. Divide by that fraction to undo the shortfall.

Why it matters: the size memory (99.9%) fills ten times slower. Uncorrected, the first steps would divide by almost nothing and lurch wildly.

After a few hundred steps the correction fades to nothing; it only matters at the start.

Momentum and Adam as drop in replacements for our SGD

```
class Momentum:
    def __init__(self, model, lr=0.05, beta=0.9):
        self.model, self.lr, self.beta = model, lr, beta
        self.v = [np.zeros_like(p) for p, _ in model.params()]
    def step(self):
        for (p, g), v in zip(self.model.params(), self.v):
            v *= self.beta;  v += g        # remember, add push
            np.subtract(p, self.lr * v, out=p)

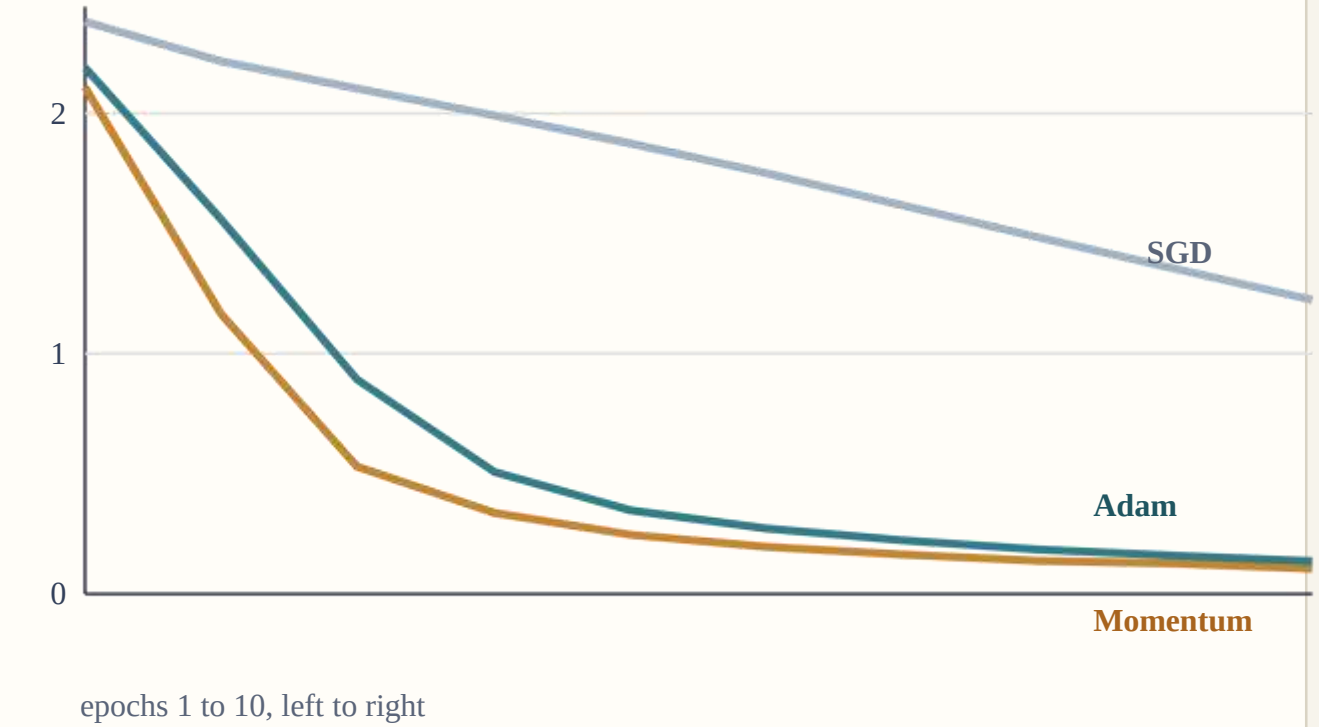
class Adam:
    def __init__(self, model, lr=0.001, b1=0.9, b2=0.999, eps=0.00000001):
        ...  # store settings; m and s start as zeros; t = 0
    def step(self):
        self.t += 1
        for (p, g), m, s in zip(self.model.params(), self.m, self.s):
            m *= self.b1;  m += np.subtract(1, self.b1) * g      # direction
            s *= self.b2;  s += np.subtract(1, self.b2) * g * g  # size
            m_hat = m / np.subtract(1, self.b1 ** self.t)   # warm up fix
            s_hat = s / np.subtract(1, self.b2 ** self.t)
            step = self.lr * m_hat / (np.sqrt(s_hat) + self.eps)
            np.subtract(p, step, out=p)
```

Same interface: both expose **step()** and read **model.params()**, so the training loop does not change at all.

State per weight: Momentum stores one extra array per parameter; Adam stores two. Adam triples parameter memory.

In place updates (*=, +=, out=p) change the arrays the layers own, so no copying is needed.

Head to head on the digits: same network, same seed, ten epochs



OUTPUT (epochs 1, 5, 10)

```
SGD lr 0.01       2.382  1.875  1.224
   val acc 0.825
Momentum lr 0.01 2.109  0.246  0.105
   val acc 0.953
Adam lr 0.001     2.191  0.348  0.136
   val acc 0.956
```

At the same small learning rate, momentum reaches a loss **twelve times lower** than plain SGD.

Adam matched momentum using its **default** settings, with no tuning. That robustness is why it is the usual first choice.

AdamW: shrink the weights directly, outside Adam's rescaling

Weight decay gently pulls every weight toward zero each step, discouraging any weight from growing huge (more in Module 16). The question is **where** to apply the pull.

Adam with the pull mixed in



The pull passes through Adam's divide by typical size. Weights with big gradients get their pull **shrunk**, so decay ends up uneven and weak exactly where it is needed.

AdamW: pull applied separately



Every weight shrinks by the **same small fraction** each step, regardless of its gradient history. Cleaner, and it generalises better.

```
# after the usual Adam update
p *= np.subtract(1, lr * weight_decay)
```

In practice: AdamW with weight decay around 0.01 to 0.1 is the default optimiser for training Transformers and most modern models.

Big steps early, small steps late: shaping the learning rate over time

Constant



Simple, but ends up bouncing around the minimum.

Step decay



Divide by ten at set epochs. The classic for image models.

Warm up, then cosine



Ramp up gently, then glide down smoothly. The modern default.

```
def lr_at(step, total=1000, warmup=100, lr_max=0.001, lr_min=0.00001):
    if step < warmup:                  # straight ramp up
        return lr_max * (step + 1) / warmup
    progress = np.subtract(step, warmup) / np.subtract(total, warmup)
    span = np.subtract(lr_max, lr_min)
    return lr_min + 0.5 * span * (1 + np.cos(np.pi * progress))

for s in [0, 49, 99, 100, 325, 550, 775, 999]:
    print(f"step {s:4d}   lr {lr_at(s):.6f}")
```

OUTPUT

```
step    0   lr 0.000010
step   49   lr 0.000500
step   99   lr 0.001000
step  325   lr 0.000855
step  550   lr 0.000505
step  775   lr 0.000155
step  999   lr 0.000010
```

Why warm up? Adam's memories are unreliable at first; small early steps stop an early lurch.

16

MODULE SIXTEEN

Regularisation and Normalisation

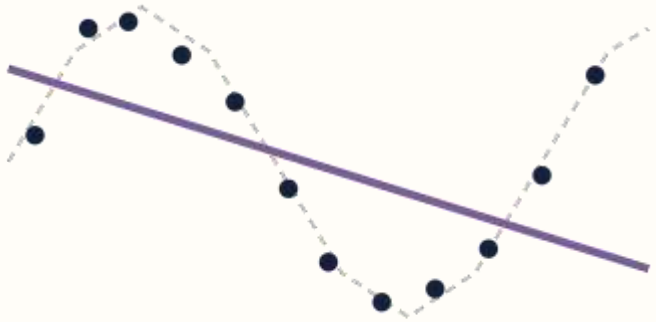
A network big enough to learn is big enough to memorise. These tools make it learn the pattern instead of the noise.

IN THIS MODULE

- Underfitting and overfitting
- Train, validation and test roles
- A network memorising noise
- Weight decay, L1 and L2
- Dropout and early stopping
- Augmentation, batch and layer norm

Too simple, just right, too flexible: the same 20 noisy points

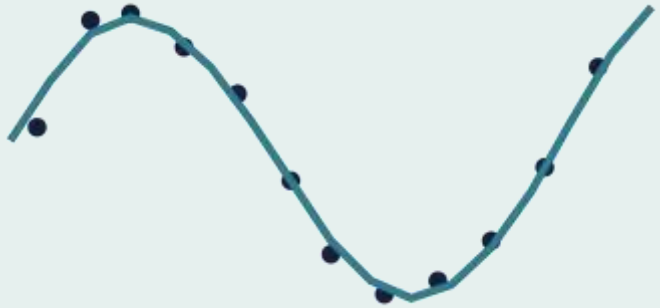
Underfit · straight line



train 0.180	val 0.247

Misses the wave entirely. Bad on **both** sets: high **bias**.

Just right · gentle curve



train 0.048	**val 0.039**

Captures the wave, ignores the jitter. **Lowest** validation error.

Overfit · wild curve

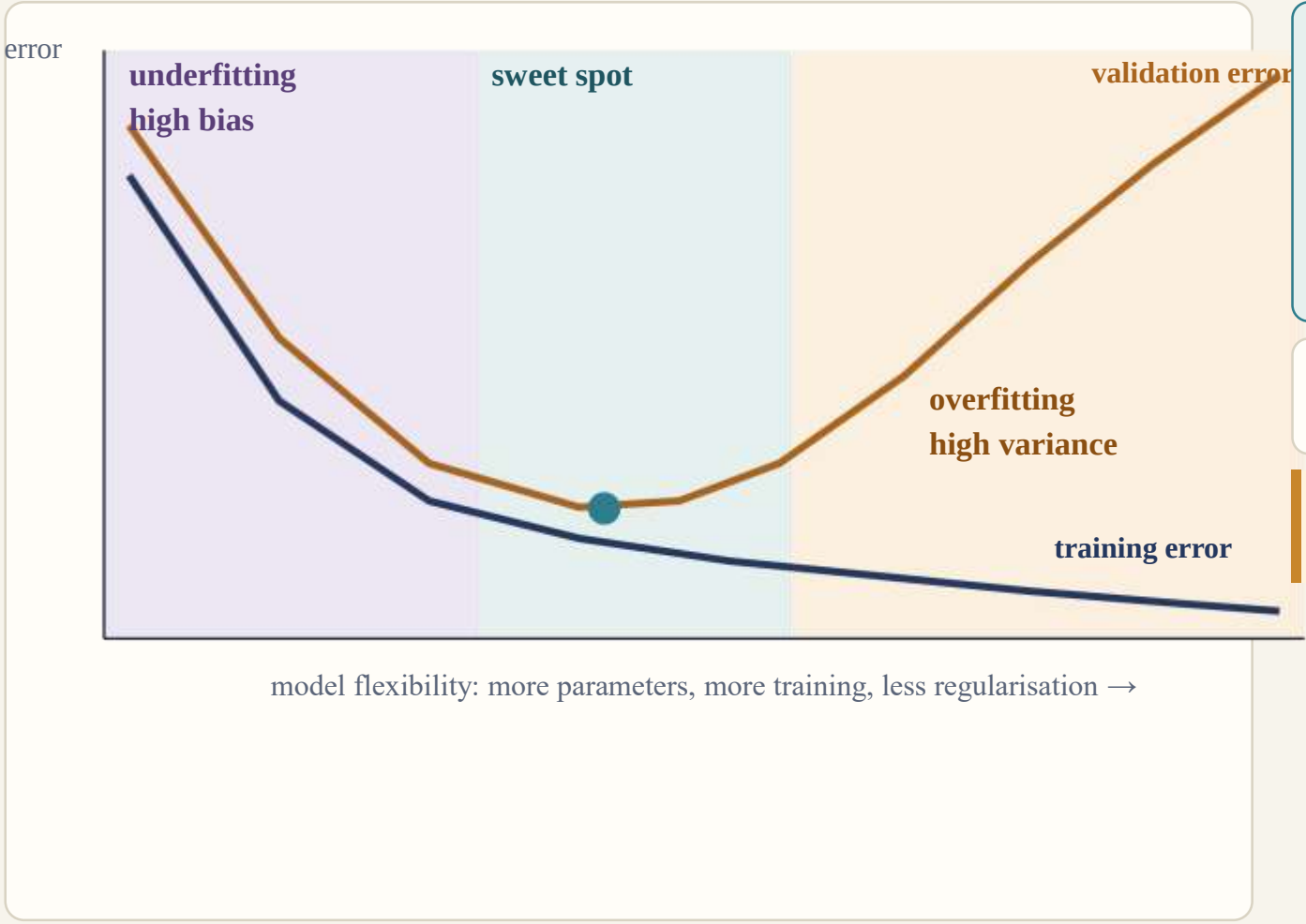


train 0.010	**val 251,485**

Threads every point, noise included, then swings wildly between them: high **variance**.

The tell: training error keeps improving with flexibility, but validation error is what you actually care about.

The U shaped curve every practitioner watches



MEASURED, POLYNOMIAL FITS

degree	train	validation
1	0.1797	0.2470
3	0.0480	**0.0387**
5	0.0387	0.0654
9	0.0273	4.7909
15	0.0102	251484.5

Bias: errors from a model too rigid to express the truth.
Variance: errors from a model so flexible it chases noise.

Regularisation pushes a big model back toward the sweet spot without shrinking it.

Three piles of data, three different jobs



The textbook

Gradients come only from here. The weights are fitted to it directly.

Seen: every epoch.

[icon: ClipboardCheck]

The mock exams

Used to pick settings: learning rate, size, dropout, when to stop. It shapes **your** choices, not the weights.

Seen: after each epoch, for scoring only.

The sealed final exam

Opened **once**, at the very end, to report how the finished model will do on new data.

Seen: one time only.

Leakage warning: if you tune settings while peeking at the test set, it quietly becomes a second validation set and your reported score turns optimistic.

Give a big network scrambled labels and it learns them anyway

THE SETUP

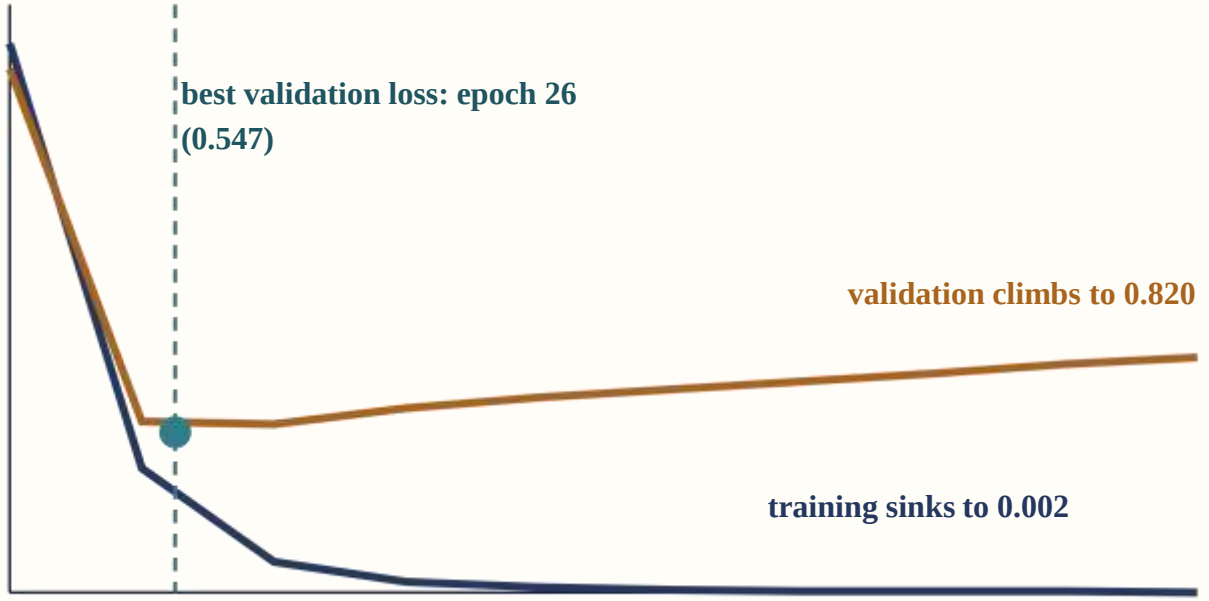
Only **300** training digits.
87 of them (30%) given a random wrong label.
Network 64, 256, 256, 10: **85,002** weights.
Adam, 200 epochs. Validation labels are clean.

3	7?	1	8	2?	0	5	9	4?	6

Roughly three in every ten training labels are lies.

> Final training accuracy against the scrambled labels: **1.000**. It memorised every lie.

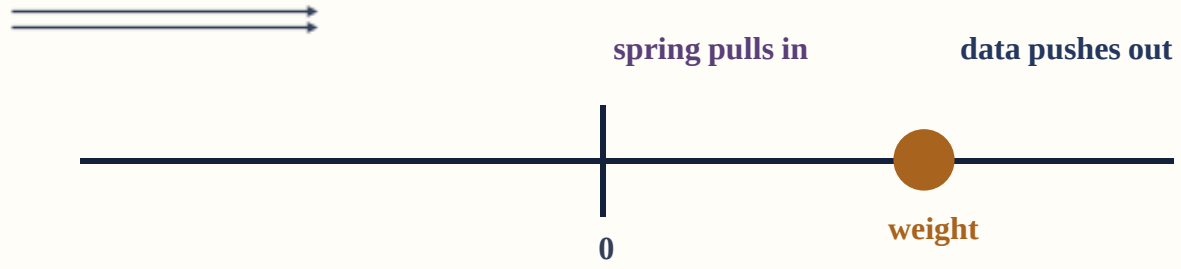
LOSS, MEASURED EVERY 20 EPOCHS



epochs 1 to 200 → final validation accuracy **0.771**; at epoch 26 it was **0.832**

Weight decay: a spring pulling every weight back toward zero

TUG OF WAR ON ONE WEIGHT



Weights the data **really needs** win the tug of war and stay large.

Weights only used to **memorise** quirks lose and drift back to zero.

Small weights make **smoother** functions that cannot thread every noisy point.

```
# after each optimiser step, shrink weight matrices (not biases) a little
for p, g in model.params():
    if p.ndim == 2:  p *= np.subtract(1, lr * 3.0)
```

OUTPUT · SAME NOISY SETUP

plain
train acc 1.000 val acc 0.771
typical weight size 0.1398

weight decay
train acc 0.800 val acc 0.838
typical weight size 0.0461

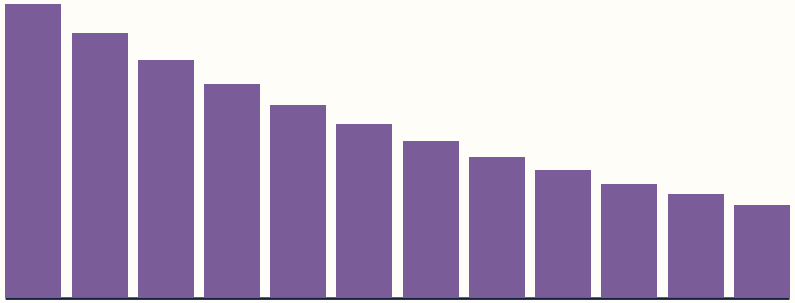
Weights three times smaller. Training accuracy drops to 0.80 because it **refused** to memorise most of the lies, and validation accuracy rises.

Too strong a spring underfits: a decay of 5.0 gave 0.798.

Two kinds of pull: proportional shrink or constant nudge

Start one unused weight at 1.0 and apply only the penalty for 12 steps:

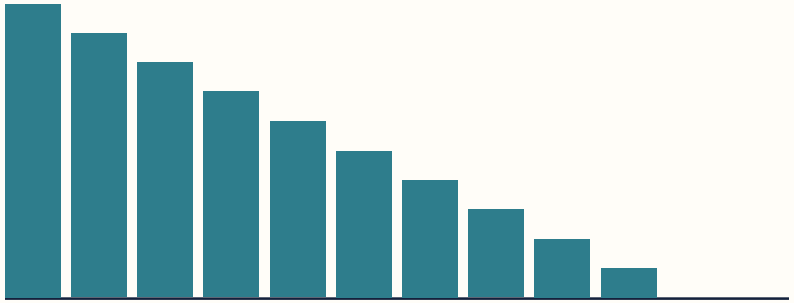
L2: lose 10% of what is left



1.00, 0.90, 0.81, 0.73 … 0.31. The pull weakens as the weight shrinks, so it gets **small but never zero**.

Effect: many small weights, all sharing the work.

L1: lose a fixed 0.1 each step



1.0, 0.9, 0.8 … 0.1, then **exactly zero** and it stays there. The nudge never weakens.

Effect: sparse weights; useless inputs are switched off completely.

WHEN TO USE

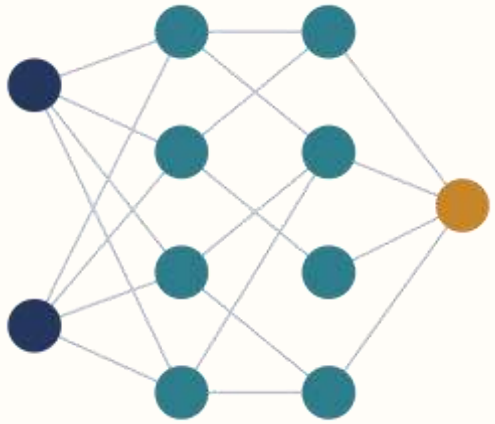
L2 is the deep learning default (as weight decay).

L1 when you want feature selection or a compact model with many zeros.

Both trade a little training fit for a simpler, more general model.

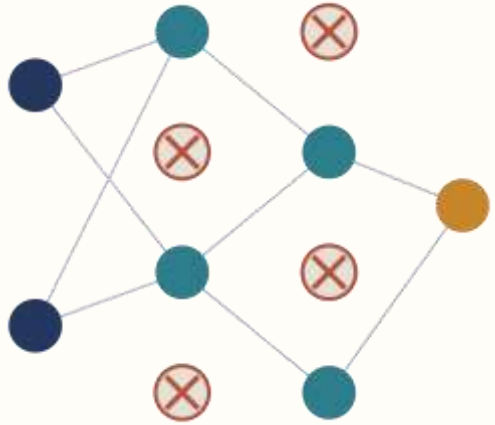
Dropout: randomly silence neurons so none can be relied on alone

Full network



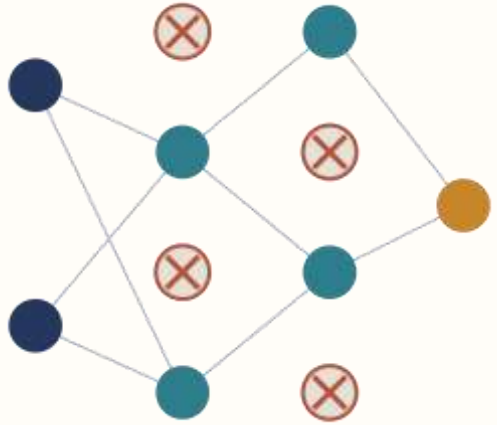
Used at **test time**: everyone present.

Training batch 1



Each neuron is a coin flip: half are switched off.

Training batch 2



A fresh random team every batch.

Team analogy: if any colleague might be absent, everyone must understand the job. No fragile partnerships.

Ensemble view: training samples a huge family of thinned networks; test time averages them all at once.

A dropout layer in twelve lines, and what it does to the noisy run

```
class Dropout:
    def __init__(self, p_drop, rng):
        self.keep = np.subtract(1.0, p_drop)
        self.rng = rng
        self.training = True
    def forward(self, X):
        if not self.training:
            return X                    # test time: do nothing
        self.mask = (self.rng.random(X.shape) < self.keep) / self.keep
        return X * self.mask            # drop and rescale
    def backward(self, G):
        return G * self.mask            # same mask going back
    def params(self):
        return []
```

Why rescale? 4 inputs of 1.0 → 2 survive → each becomes 2.0 → **total still 4**

OUTPUT · NOISY SETUP

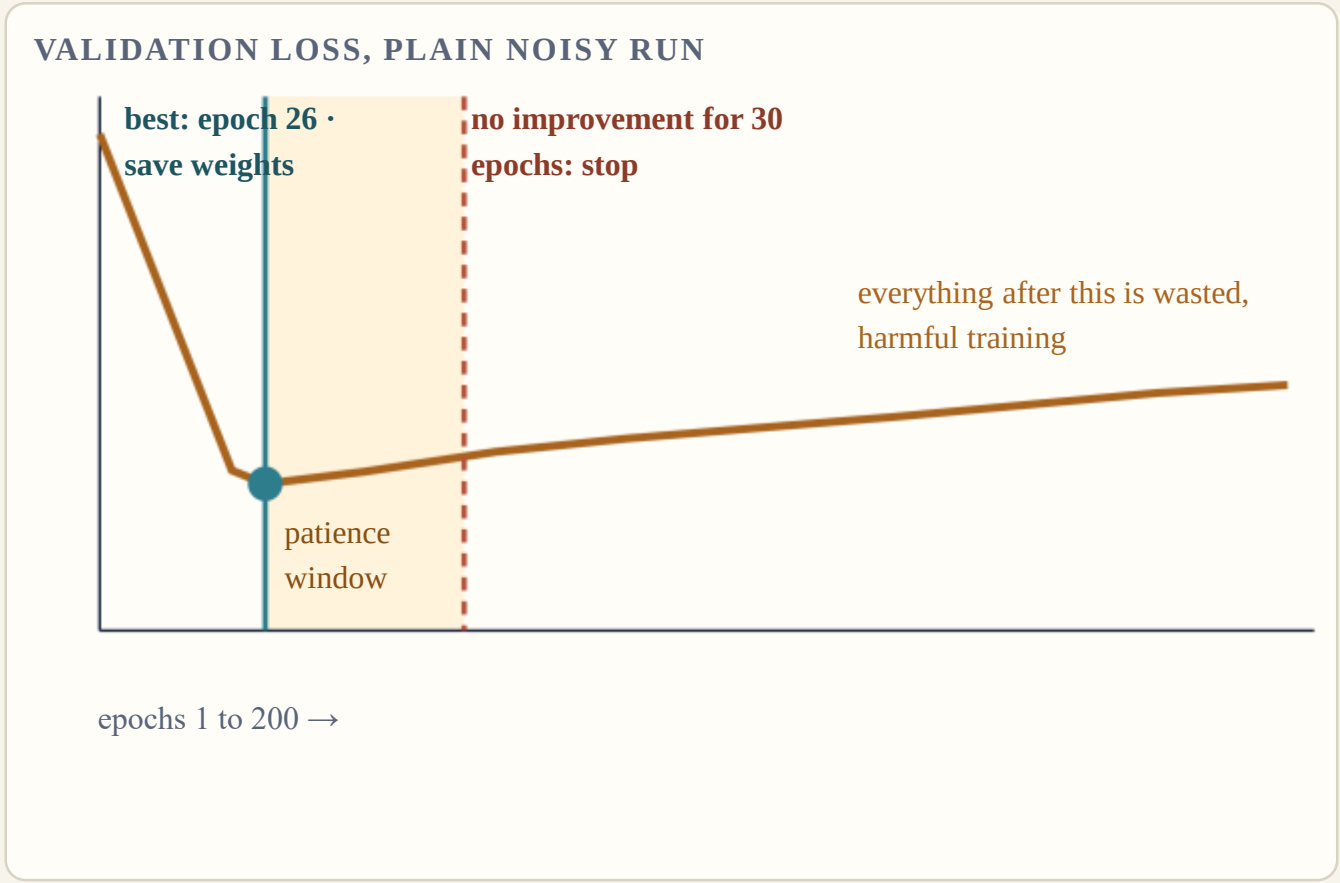
plain
- best val loss 0.547 (epoch 26)
- final val acc 0.771

dropout 0.5
- best val loss 0.433 (epoch 98)
- final val acc 0.818
- val acc at best epoch 0.872

Memorising slows sharply: training loss after 200 epochs is **0.045** with dropout versus **0.002** without.

Classic bug: forgetting to switch **training** off before evaluating gives noisy, worse scores.

Early stopping: keep the best checkpoint, not the last one




```
best, wait = float("inf"), 0
for epoch in range(max_epochs):
    train_one_epoch()
    v = validation_loss()
    if v < best:
        best, wait = v, 0
        save_checkpoint()
    else:
        wait += 1
        if wait == 30: break
load_checkpoint()
```

Measured: val accuracy **0.832** at the checkpoint versus **0.771** at epoch 200, and 72% fewer epochs.

The cheapest regulariser there is: it costs nothing and should be on by default whenever you have a validation set.

More data for free: change the input, keep the label



Data	Typical safe changes	Careful with
Images	crops, flips, rotations, colour jitter, blur, cutout patches	flipping text or digits: a mirrored 7 is not a 7
Audio	background noise, speed and pitch shifts, time masking	pitch shifts when pitch is the label
Text	synonym swaps, back translation, random word dropout	changes that flip the meaning

The rule: only apply changes that a human would agree leave the answer unchanged. Augment training data only, never validation or test.

Batch norm and layer norm: same recipe, different direction

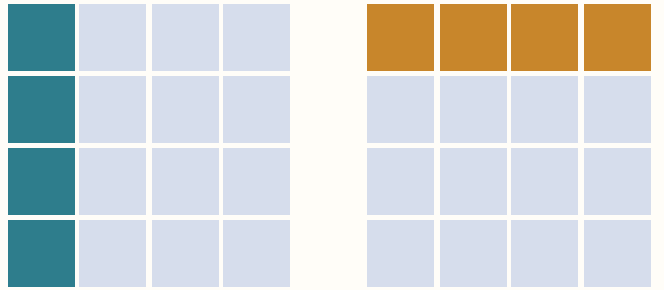
The recipe for one group of numbers

1. Find the average; shift it to **zero**
2. Find the spread; rescale it to **one**
3. Apply a **learned** scale and shift

Worked: one feature over a batch of four

2	average 5 spread 2.24 →	−1.34
4		−0.45
6		0.45
8		1.34

Which numbers form a group?



Batch norm

one feature, down all examples

Layer norm

one example, across all features

rows: examples in the batch · columns: features

BN: CNNs. Needs decent batch sizes; keeps running averages for test time.

LN: Transformers and RNNs. Works with any batch size, same at train and test.

17

MODULE SEVENTEEN

Training in Practice

Neural networks fail silently: buggy code still produces a number. A disciplined workflow catches problems early.

IN THIS MODULE

- A step by step recipe
- Sanity checks before real training
- Reading loss curves
- Gradient clipping
- Hyperparameter search
- Debugging and reproducibility

Six stages, each one verified before the next

1
Know the data

Look at hundreds of examples. Check labels, balance, duplicates, odd values.

2
Set a baseline

Simplest model first, even logistic regression. Now you know what "good" means.

3
Sanity check

Starting loss correct? Can it overfit one batch? (next slide)

4
Overfit on purpose

Grow the model until training loss is low. Capacity first.

5
Regularise

Now trade training fit for validation score: more data, augmentation, decay, dropout.

6
Tune and squeeze

Search hyperparameters, train longer, ensemble. Test set once, at the end.

Golden rule: change one thing at a time, and keep a written log of what each change did.

Stages 4 and 5 mirror the U curve: first reach the overfitting side, then pull back to the sweet spot.

Two checks that take a minute and catch most bugs

Check 1 · The starting loss

An untrained 10 class model should be about equally unsure of every class, one in ten each. Its surprise is then a known value:



$$\approx 2.30$$

OUTPUT

```
expected start loss for 10 classes: 2.3026
actual start loss: 2.5196
```

Close enough. A start of 15 or 50 would mean wildly confident random guesses: check initialisation and input scaling.

Check 2 · Overfit one batch

Train on the **same 32 examples** over and over. A correct pipeline must drive the loss to almost zero.

```
xb, yb = X[:32], y[:32]
for step in range(1, 201):
    loss = loss_fn.forward(model.forward(xb), yb)
    model.backward(loss_fn.backward()); opt.step()
```

OUTPUT

```
step   1  loss 2.5196
step  25  loss 0.0042
step 200  loss 0.0001
accuracy on the single batch: 1.0
```

If it **cannot** memorise 32 examples, something is broken: labels misaligned, gradients wrong, learning rate off.

Six loss curve shapes and what each one is telling you



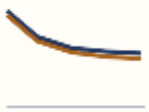
Rising or nan
Learning rate far too high, or a bug. Lower it tenfold.



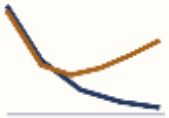
Flat at the start value
Not learning: rate too low, dead ReLUs, or gradients not connected.



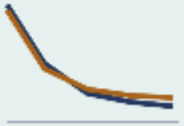
Quick drop, high plateau
Rate a bit high, bouncing in the valley. Add a decay schedule.



Both high, close together
Underfitting. Bigger model, longer training, less regularisation.



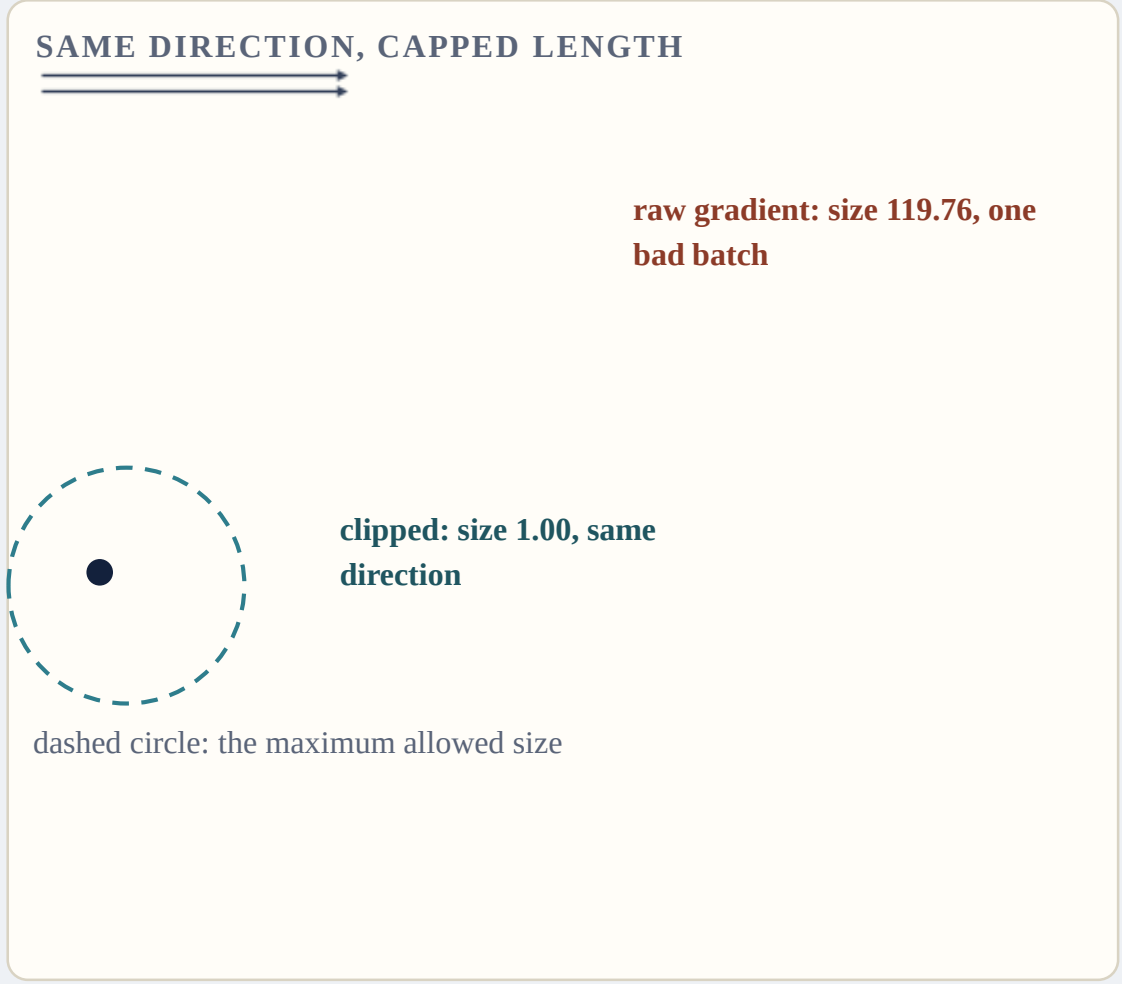
Gap opens, validation rises
Overfitting. Regularise, augment, stop early, get data.



Low, small steady gap
Healthy. Consider training longer or scaling up.

training loss validation loss Tip: plot loss on a log scale to see late progress clearly.

Gradient clipping: a speed limit on every step




```
def clip_gradients(params, max_norm):
    total = np.sqrt(sum(np.sum(g * g) for _, g in params))
    if total > max_norm:
        for _, g in params:
            g *= max_norm / total  # shrink all, keep direction
    return total

# in the loop: backward, then clip, then step
model.backward(loss_fn.backward())
clip_gradients(model.params(), 1.0)
opt.step()
```

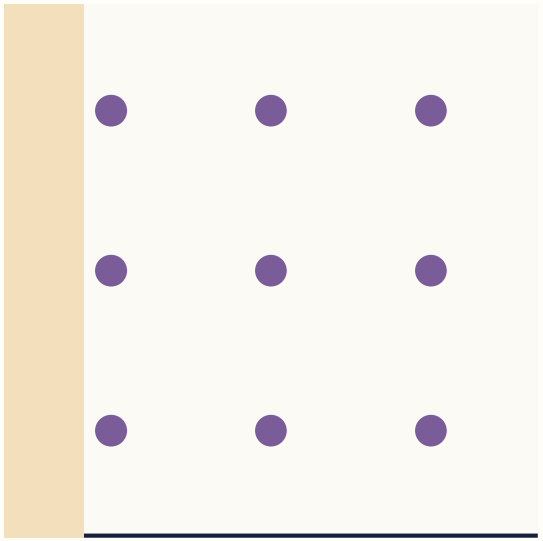
OUTPUT

```
gradient size before 119.76  after 1.00
```

Standard for RNNs and Transformers, typically with a maximum of **1.0**. Normal steps pass through untouched.

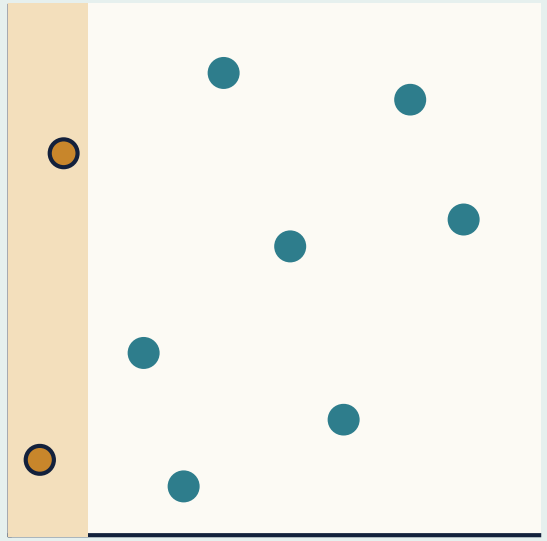
Random search beats grid search for the same budget

Grid: 9 trials



Only **3 distinct** learning rates tried; all miss the good band.

Random: 9 trials



9 distinct learning rates; two land in the good band.

Reading the squares: across is learning rate (the knob that matters); up is a knob that barely matters. Amber band: where good runs live.

PRIORITY ORDER

1 · learning rate
2 · batch size, schedule
3 · model size, weight decay
4 · dropout, everything else

Sample learning rates on a **ten times** scale: 0.0001, 0.001, 0.01, not evenly spaced.

The bugs that bite most often, and how to spot them

Symptom	Likely cause	Quick test
Accuracy stuck at chance (10%)	Labels shuffled separately from inputs	Print a few images with their labels
Loss becomes nan	Rate too high, log of zero, divide by zero	Lower the rate; check clipping inside the loss
Great training, poor test score	Overfitting, or test data processed differently	Same scaling code path for every split?
Suspiciously perfect validation	Leakage: duplicates across splits	Search for overlapping examples
Evaluation scores jump around	Dropout or batch norm left in training mode	Evaluate twice: results must match
Learns, but slower than expected	Inputs not scaled, gradients not reset	Print input range and gradient sizes
Shape error, or silent wrong shape	Broadcasting turned (N,) and (N, 1) into (N, N)	Print shapes after every layer once

Mindset: assume there is a bug until the sanity checks pass. Visualise inputs right before they enter the model, not as stored on disk.

Reproducibility and the training checklist

MAKE EVERY RUN REPEATABLE

```
rng = np.random.default_rng(42)
# one seed drives init, shuffling, dropout
config = {"lr": 0.001, "batch": 32,
          "epochs": 30, "seed": 42}
log.append({**config, "val_acc": 0.953})
```

Record: code version, data version, every setting, seed, and the resulting scores. Tools such as TensorBoard, Weights and Biases or MLflow automate this.

Repeat: run important comparisons with three seeds. A gain smaller than the seed to seed spread is noise.

BEFORE YOU TRUST A RESULT

- Looked at the raw data and labels
- Starting loss near the expected value
- One batch overfits to near zero
- Beats a simple baseline
- Training and validation curves both plotted
- Best checkpoint kept, evaluation mode on
- Repeated with another seed
- Test set opened exactly once

18

MODULE EIGHTEEN

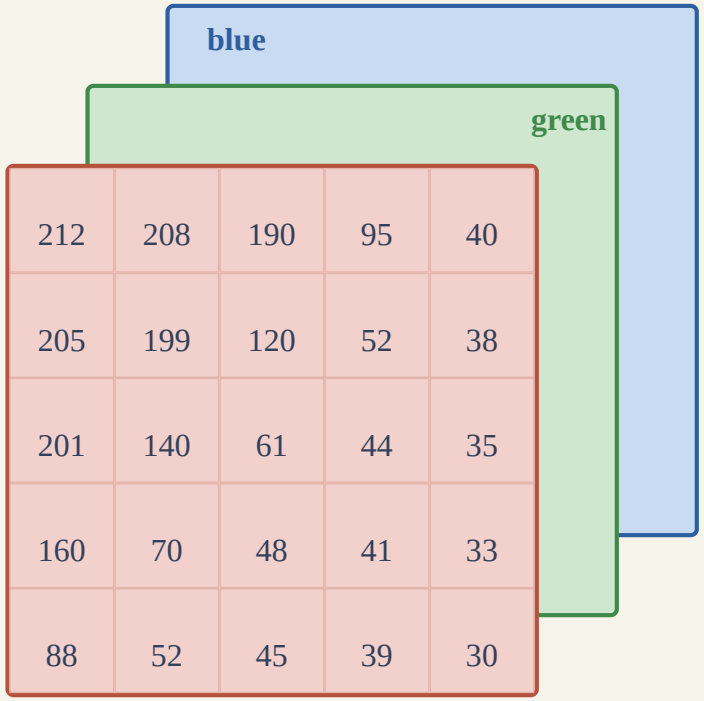
Convolutional Neural Networks

Small pattern detectors that slide across an image, reusing the same weights everywhere. The architecture that made computers see.

IN THIS MODULE

- Why plain networks struggle with images
- Convolution by hand, step by step
- Stride, padding, channels, pooling
- Convolution in NumPy, fast and slow
- From LeNet to ResNet
- Transfer learning

A colour image is a stack of three grids



red channel: 0 is none, 255 is full

SHAPE OF A BATCH

(N, C, H, W)

images, channels, height, width

32 photos of 224 by 224: (32, 3, 224, 224)

Channels are separate measurements at the same spot. Grey images have 1; colour has 3; inside a CNN there may be hundreds.

Our digits are (1797, 1, 8, 8): one grey channel on an 8 by 8 grid, scaled to 0 to 1.

Crucial: neighbouring numbers are related. A pixel is usually similar to the pixels beside it. CNNs exploit this.

Why a plain fully connected network struggles with photos

[icon: Weight]

1 · Far too many weights

224 by 224 by 3 inputs: **150,528**

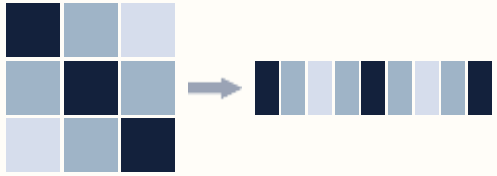
times 1,000 hidden neurons

150.5 million weights in layer one

Huge memory, and plenty of room to memorise.

[icon: Shuffle]

2 · Layout is thrown away



Flattening forgets which pixels were neighbours. Shuffle the pixel order consistently and a plain network scores exactly the same.

[icon: Move]

3 · Must relearn everywhere

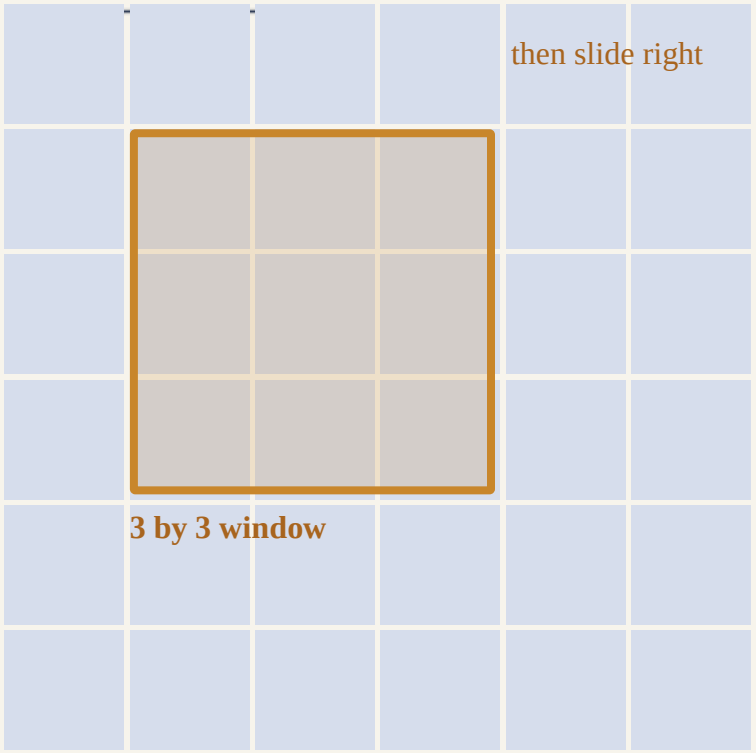
[icon: Ca

[icon: Cat]

A cat detector for the top left uses different weights from one for the bottom right. Each position must be learned separately.

The CNN answer: look at **small local patches** (keeps neighbours together) with **one shared detector** slid everywhere (few weights, works at any position).

Convolution: slide a small stencil, multiply, add, write one number

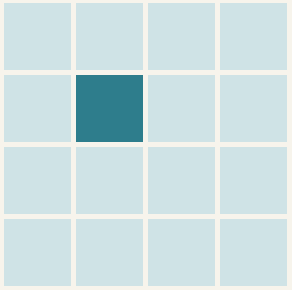


kernel (learned)

w1	w2	w3
w4	w5	w6
w7	w8	w9

9 products, added, plus a bias

feature map



Local: each output looks at only 9 neighbouring pixels.

Shared: the same 9 weights are reused at all 16 positions. A 6 by 6 image needs only 10 parameters, not 36 by 16 plus 16.

Map: the output grid shows **where** the kernel's pattern appears. Bright cell: strong match.

It is exactly a neuron (Module 4), applied to every patch.

By hand: an edge detector finds where dark meets bright

image 5 by 5

0	0	9	9	9
0	0	9	9	9
0	0	9	9	9
0	0	9	9	9
0	0	9	9	9

kernel

−1	0	1
−1	0	1
−1	0	1

"right side minus left side"

Window 1 (teal box, columns 1 to 3)
Each row: 0 times −1, 0 times 0, 9 times 1 → **9**
Three rows → **27**: strong edge

Window 2 (columns 2 to 4)
0 times −1, 9 times 0, 9 times 1 → 9 per row → **27**

Window 3 (columns 3 to 5)
9 times −1, 9 times 0, 9 times 1 → 0 per row → **0**: flat, no edge

```
def conv2d_naive(image, kernel):
    H, W = image.shape;  k = kernel.shape[0]
    out = np.zeros((positions(H, k), positions(W, k)))
    for i in range(positions(H, k)):         # slide down
        for j in range(positions(W, k)):     # slide across
            out[i, j] = np.sum(image[i:i + k, j:j + k] * kernel)
    return out
```

OUTPUT

```
[[27. 27.  0.]
 [27. 27.  0.]
 [27. 27.  0.]]
```

Hand designed kernels, and why CNNs learn their own

Vertical edges

−1	0	1
−2	0	2
−1	0	1



Lights up on left to right brightness changes

Horizontal edges

−1	−2	−1
0	0	0
1	2	1



Lights up on top to bottom changes

Blur

1	1	1
1	1	1
1	1	1

all divided by 9

Averages each neighbourhood: softens noise

Sharpen

0	−1	0
−1	**5**	−1
0	−1	0

centre boosted

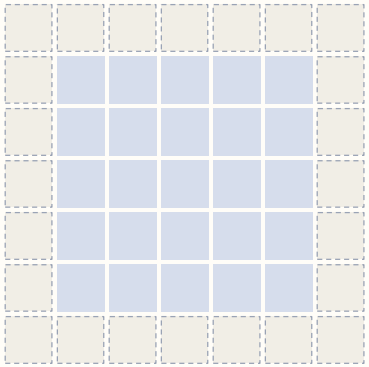
Exaggerates a pixel's difference from its neighbours

The deep learning twist: we never write these numbers. The 9 kernel values are **weights**, learned by backpropagation like any other.

Remarkably, trained first layers rediscover edge and colour blob detectors much like these, on their own.

Two dials that control the size of the feature map

Padding: a border of zeros



A 3 by 3 kernel with a 1 pixel border keeps a 5 by 5 image at **5 by 5** ("same" padding), and lets edge pixels be centres too.

Stride: how far the window jumps



stride 1 overlapping, full size map

stride 2 skips every other spot: map **halves**

Stride 2 is a cheap way to shrink the map and widen what each later neuron sees.

```
def positions(n, k, pad=0, stride=1):
    # the window's far edge runs from k up to the padded size
    return len(range(k, n + 2 * pad + 1, stride))

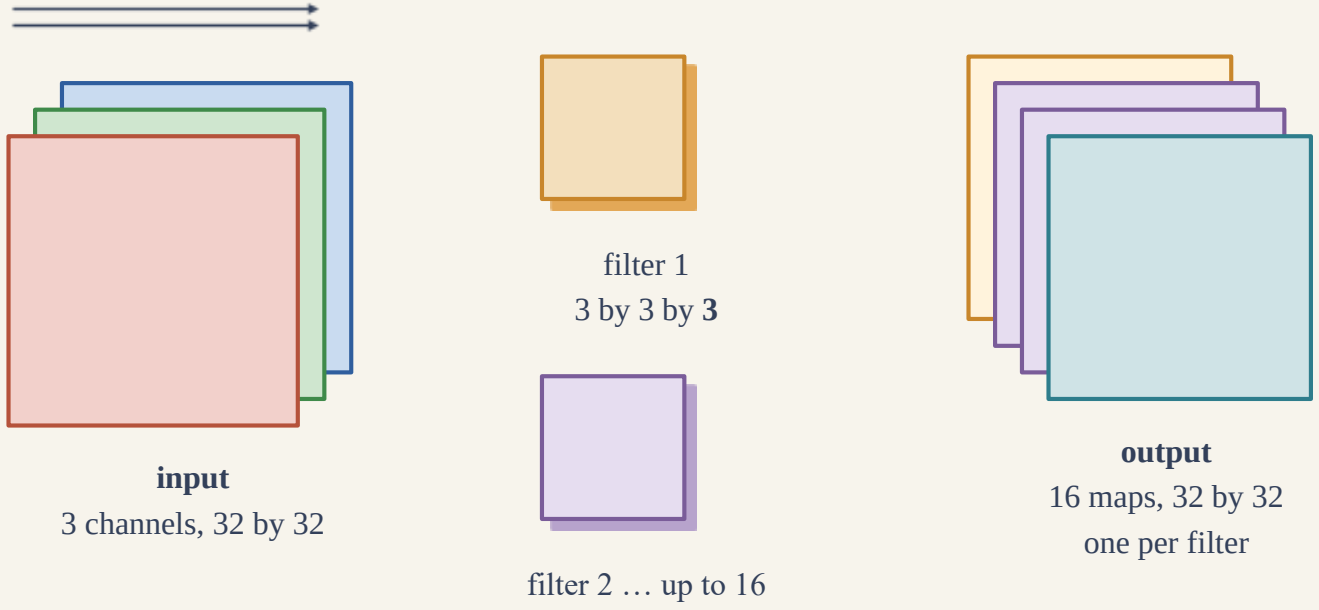
print(positions(5, 3), positions(32, 5), positions(32, 3, pad=1),
      positions(224, 7, pad=3, stride=2), positions(7, 3, stride=2))
```

OUTPUT

```
3 28 32 112 3
```

5 with 3 gives 3 · 32 with 5 gives 28 · padding keeps 32 · ResNet's first layer halves 224 to 112

Many filters in, many maps out: a convolution layer in full



Each filter is as deep as the input. It looks at all 3 colours of a 3 by 3 patch at once: 27 weights plus 1 bias.

Each filter makes one map. 16 filters, 16 output channels. The next layer's filters are then 16 deep.

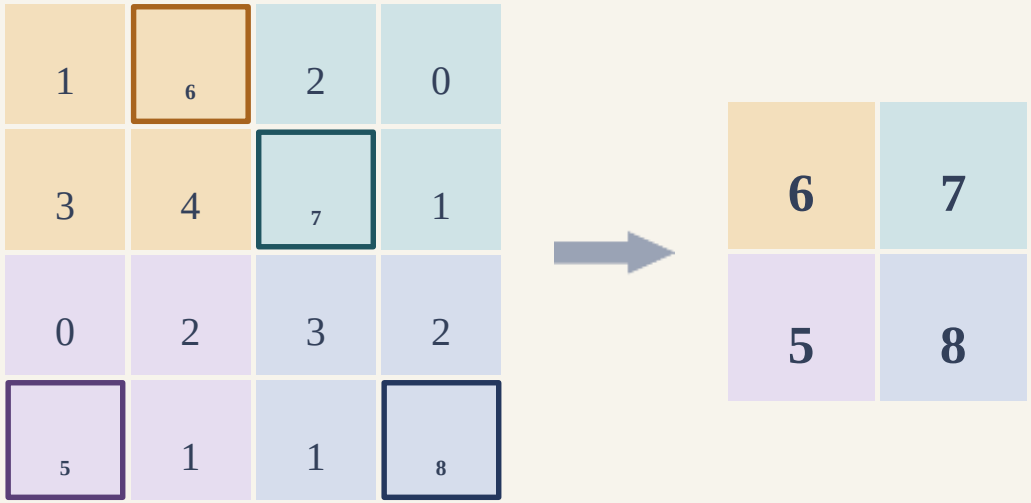
PARAMETER COUNT

3 by 3 by 3 weights per filter: 27
plus 1 bias: 28
times 16 filters: **448 parameters**

The count does not depend on image size at all. A 32 by 32 or a 1,000 by 1,000 image uses the same 448.

Shapes: input (N, 3, 32, 32) → weights (16, 3, 3, 3) → output (N, 16, 32, 32) with padding 1

Max pooling: keep the strongest signal in each 2 by 2 block



Shrinks each map to a quarter of the area: 4 by 4 becomes 2 by 2. Less computation downstream.

Tolerates small shifts: move the 7 one cell within its block and the output is unchanged.

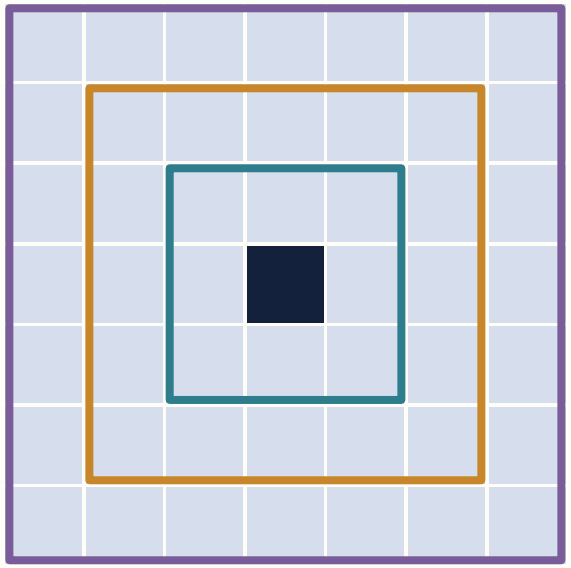
No weights at all, and it works on each channel separately.

```
class MaxPool2:
    def forward(self, X):
        N, C, H, W = X.shape
        Xr = X.reshape(N, C, H // 2, 2, W // 2, 2)  # group 2 by 2 blocks
        out = Xr.max(axis=(3, 5))
        self.mask = (Xr == out[:, :, :, None, :, None])  # remember winners
        self.shape = X.shape
        return out
    def backward(self, G):
        return (self.mask * G[:, :, :, None, :, None]).reshape(self.shape)
```

Backward: only the winning cell in each block receives the gradient; the three losers get zero, because nudging them would not change the output.

Stacked layers see wider: from edges to parts to objects

HOW FAR ONE NEURON CAN SEE



layer 1: 3 by 3 **layer 2: 5 by 5** **layer 3: 7 by 7**

Each 3 by 3 layer widens the view by one pixel on every side. Pooling doubles it.

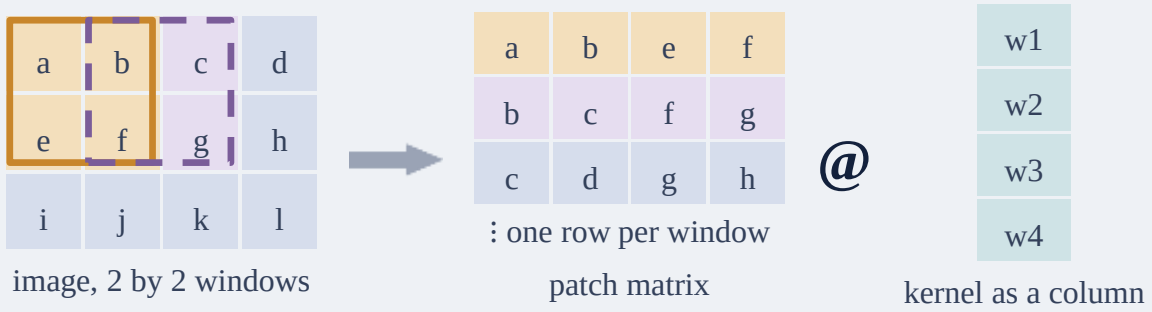
Early layers
edges, colour blobs, simple textures

[icon: Eye] **Middle layers**
corners, circles, repeated patterns, eyes, wheels

[icon: Dog] **Deep layers**
whole objects and scenes: faces, dogs, cars

Two stacked 3 by 3 layers see as far as one 5 by 5, with fewer weights (18 versus 25) and an extra ReLU between.

The patch matrix trick: turn convolution into one matrix multiply



Every window becomes a **row**. One matrix multiply then computes all outputs at once, and with 16 filters the kernel column becomes a 16 column matrix. This is the Dense layer again.

```
imgs = rng.random((200, 32, 32));  kernel = rng.random((3, 3))
t0 = time.time()
slow = np.array([conv2d_naive(im, kernel) for im in imgs])
t1 = time.time()
cols, Ho, Wo = im2col(imgs[:, None], 3, 0)
fast = (cols @ kernel.reshape(9)).reshape(200, Ho, Wo)
t2 = time.time()
print("patch matrix shape:", cols.shape)
print(f"naive loops {np.subtract(t1, t0):.3f} s   patch matrix {np.subtract(t2, t1):.3f} s")
print("same answer:", np.allclose(slow, fast))
```

OUTPUT

```
patch matrix shape: (180000, 9)
naive loops 0.367 s
patch matrix 0.049 s
same answer: True
```

7.5 times faster, identical result. The price: memory, since each pixel is copied into up to 9 rows.

Conv2D: a Dense layer applied to every patch

```
class Conv2D:
    def __init__(self, c_in, c_out, k, rng, pad=1):
        self.k, self.pad, self.c_out = k, pad, c_out
        fan_in = c_in * k * k                  # He start, Module 14
        self.W = rng.normal(0.0, np.sqrt(2 / fan_in), size=(fan_in, c_out))
        self.b = np.zeros(c_out)
    def forward(self, X):
        self.shape = X.shape
        self.cols, self.Ho, self.Wo = im2col(X, self.k, self.pad)
        out = self.cols @ self.W + self.b          # exactly Dense
        N = X.shape[0]
        return out.reshape(N, self.Ho, self.Wo, self.c_out).transpose(0, 3, 1, 2)
    def backward(self, G):
        G2 = G.transpose(0, 2, 3, 1).reshape(len(self.cols), self.c_out)
        self.dW = self.cols.T @ G2                 # same as Dense
        self.db = G2.sum(axis=0)
        dcols = G2 @ self.W.T
        return col2im(dcols, self.shape, self.k, self.pad, self.Ho, self.Wo)
```

Forward: unroll patches into rows, multiply by the filter matrix, fold rows back into maps.

Backward: the three Dense formulas from Module 9, untouched. Then **col2im** adds each patch row's gradient back onto the pixels it came from.

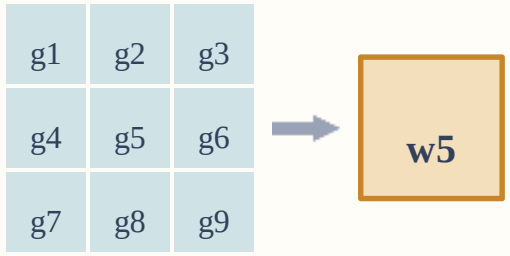
Transposes only reorder axes between (N, C, H, W) for images and one row per position for the matrix maths.

GRADIENT CHECK OUTPUT

```
backprop slope 12.366914
nudging slope  12.366914
```

Shared weights collect blame from every position they touched

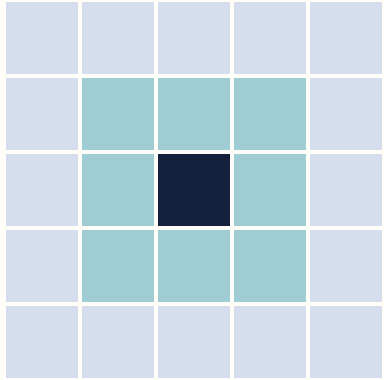
Gradient for a kernel weight



The weight was used at **all 9 output positions**. Its gradient is the **sum** of 9 contributions: each output's blame times the pixel it multiplied there.

Same rule as Module 10: a value used in many places adds up its gradients.

Gradient for an input pixel



The centre pixel (navy) sat inside **9 different windows** (teal neighbourhood). Its gradient adds the blame passed back through each of them.

That adding back is exactly what **col2im** does with its +=.

Big picture: the backward pass of a convolution is itself a convolution, sliding the output gradient over the input or the flipped kernel. No new ideas are needed, only bookkeeping.

Fewer weights, better accuracy: the CNN beats our MLP

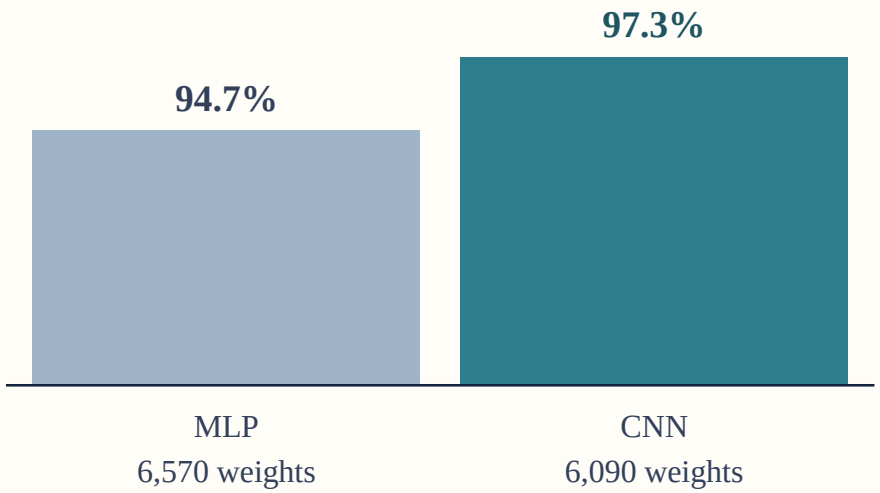
image 1 × 8 × 8 → conv 3×3 16 maps → ReLU, pool 16 × 4 × 4 → conv 3×3 32 maps → ReLU, pool 32 × 2 × 2 → flatten 128 → Dense 10

```
X = (d.images / 16.0)[:, None, :, :]   # keep the grid: (1797, 1, 8, 8)
model = Sequential(Conv2D(1, 16, 3, rng), ReLU(), MaxPool2(),
                   Conv2D(16, 32, 3, rng), ReLU(), MaxPool2(),
                   Flatten(), Dense(32 * 2 * 2, 10, rng))
opt = Adam(model, 0.003)   # same loop, loss and loader as Module 12
```

OUTPUT

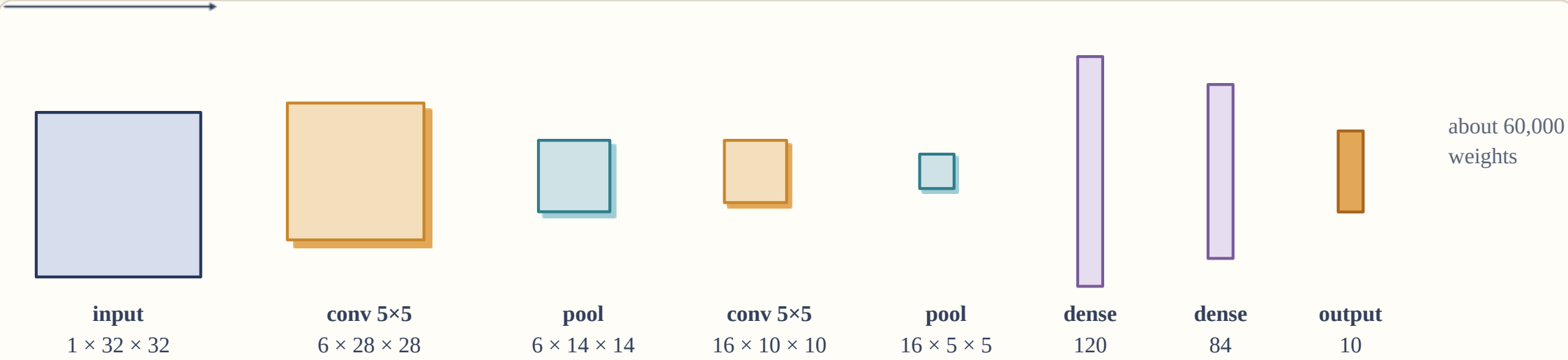
```
parameters: 6090
epoch  1  loss 1.5959  val acc 0.818
epoch  5  loss 0.1018  val acc 0.983
epoch 15  loss 0.0130  val acc 0.997
test accuracy: 0.9733   seconds: 1.9
```

SAME DATA SPLIT, TEST ACCURACY



Errors cut by half with slightly fewer weights, because the CNN knows pixels have neighbours.

The classic blueprint: shrink space, grow channels, then decide



Feature extractor: conv, pool, conv, pool. Spatial size 32 → 5 while channels 1 → 16.

Classifier: flatten 16 × 5 × 5 into 400 numbers, then ordinary Dense layers.

Used commercially in the 1990s to read handwritten cheques. Our digit CNN is a small cousin.

Deeper, then smarter: a decade of CNN milestones

2012
AlexNet
8 layers, 60M weights. ReLU, dropout, two GPUs. Error fell from 26% to 15%.

2014
VGG
16 to 19 layers, only 3×3 convs. Simple, uniform, 138M weights.

2014
GoogLeNet
22 layers, parallel branches of 1×1, 3×3, 5×5. Just 7M weights.

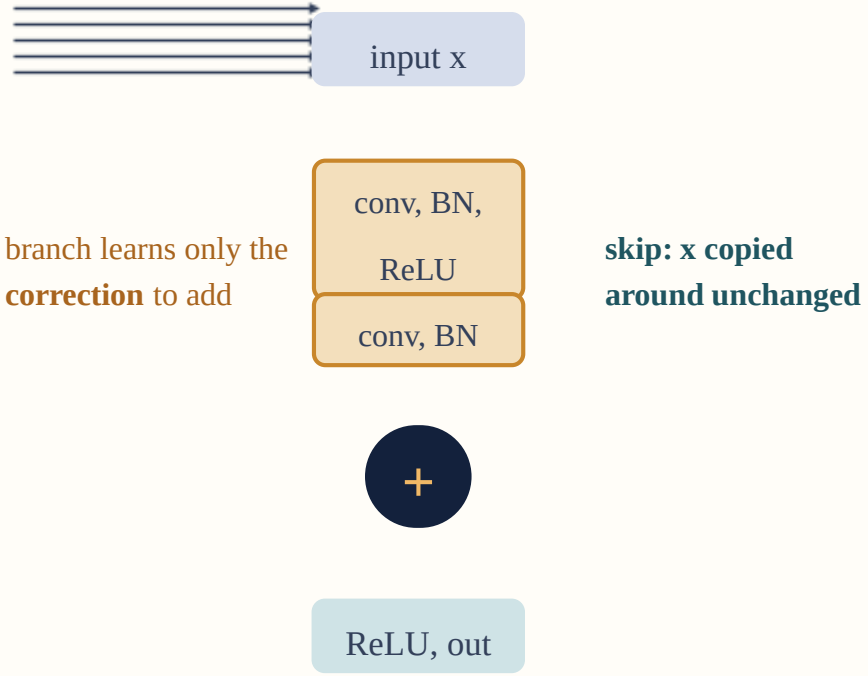
2015
ResNet
152 layers via skip connections. 3.6% error, past human level.

2017 onward
Efficient era
MobileNet, EfficientNet, ConvNeXt; Vision Transformers from 2020.

The pattern: first, depth bought accuracy; then the problem became training very deep networks, and residual connections solved it.

The skip connection: let the signal take the express lane

A RESIDUAL BLOCK



Easy default: if a block has nothing useful to add, its branch learns zero and the block passes x straight through. Extra depth can no longer hurt.

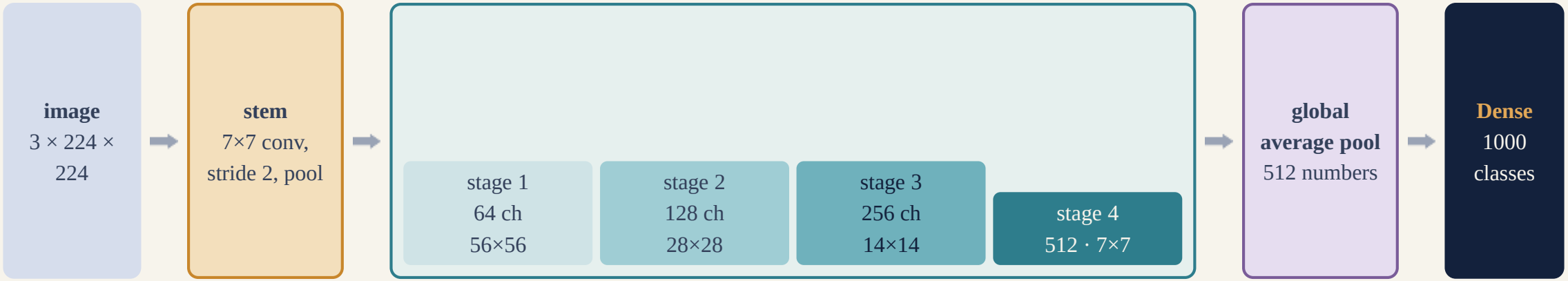
Gradient highway: at every + the backward signal splits, and one copy flows straight down the skip, untouched by any weights.

OUTPUT · GRADIENT REACHING LAYER 1

depth 5 plain 0.00007445 residual 1.0951
depth 10 plain 0.00000000 residual 1.4364
depth 20 plain 0.00000000 residual 3.0610

```
x = x + f if residual else f   # the whole idea
```

Structure of a modern image classifier



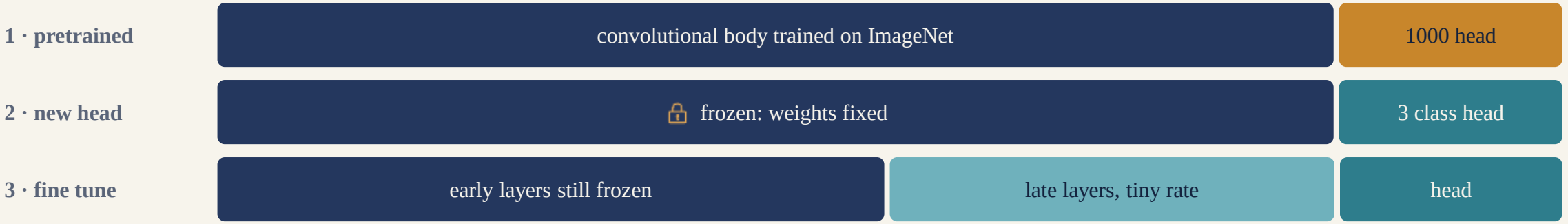
Each stage is a stack of residual blocks; between stages, a stride 2 conv halves the size and doubles the channels.

Conv, BN, ReLU
The standard unit. Batch norm after every convolution keeps each stage's activations well scaled.

Global average pool
Averages each 7×7 map to one number. Replaces giant Dense layers: AlexNet's 37M weight block shrinks to 0.5M.

1×1 convolutions
Mix channels at each pixel without looking at neighbours. Cheaply shrink 256 channels to 64 and back.

Borrow a trained eye: reuse features learned on millions of images



Why it works: edges, textures and shapes are useful for almost any image task. Only the final decision is task specific.

Little data? Stop at step 2. A few hundred images per class can reach strong accuracy.

More data? Step 3: unfreeze later layers with a learning rate about ten times smaller.

```
# PyTorch, for reference (Module 23 covers frameworks)
model = torchvision.models.resnet18(weights="IMAGENET1K_V1")
for p in model.parameters(): p.requires_grad = False   # freeze
model.fc = torch.nn.Linear(512, 3)                     # new, trainable head
```

MODULE 18 · SUMMARY

CNN cheat sheet

Two big ideas

Local connections: look at small patches.
Shared weights: one detector, slid everywhere.

Layer sizes

Count window stops. 3×3 with padding 1 keeps size; stride 2 or 2×2 pooling halves it.

Parameter count

kernel area times input channels, plus one, times filters. Independent of image size.

Typical shape

Spatial size shrinks, channels grow, global average pool, one Dense head.

Go deep safely

Residual skips plus batch norm, He initialisation, and augmentation.

In practice

Start from a pretrained ResNet or EfficientNet and fine tune.

Our results: CNN 97.3% with 6,090 weights versus MLP 94.7% with 6,570, on the same 300 test digits.

Beyond classification: the same blocks power detection, segmentation, and the image side of diffusion models.

19

MODULE NINETEEN

Recurrent Networks and LSTMs

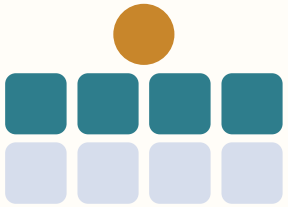
Networks with memory: read a sequence one step at a time, carrying a running summary forward.

IN THIS MODULE

- Sequence problems
- The recurrent cell, rolled and unrolled
- Backpropagation through time
- Why memory fades
- LSTM and GRU gates
- A character level language model

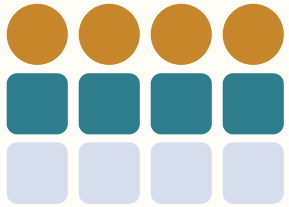
When order matters: four shapes of sequence problem

Many to one



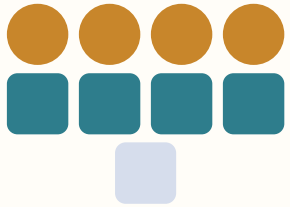
Read a whole review, output one verdict: positive or negative.

Many to many, in step



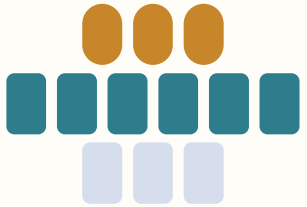
Label every word, or predict the next character at every step.

One to many



From one image, write a caption word by word.

Many to many, delayed

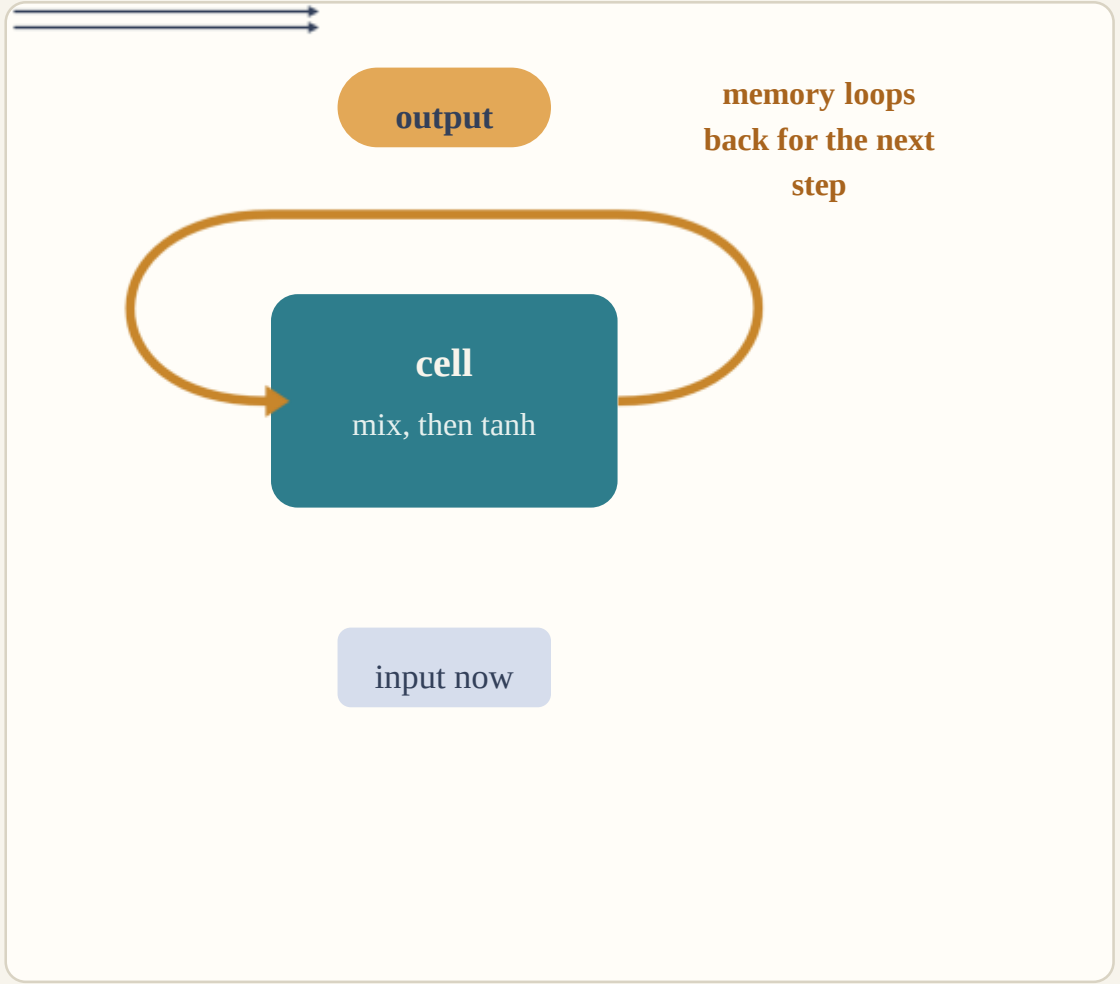


Read a French sentence, then write the English one.

input step · memory (hidden state) · output

Lengths vary: a fixed size MLP cannot cope.

One cell, a loop, and a memory that it rewrites at every step



Each step the cell takes two things: the new input, and its own memory from the previous step. It blends them through weights and a tanh into a **new memory**.

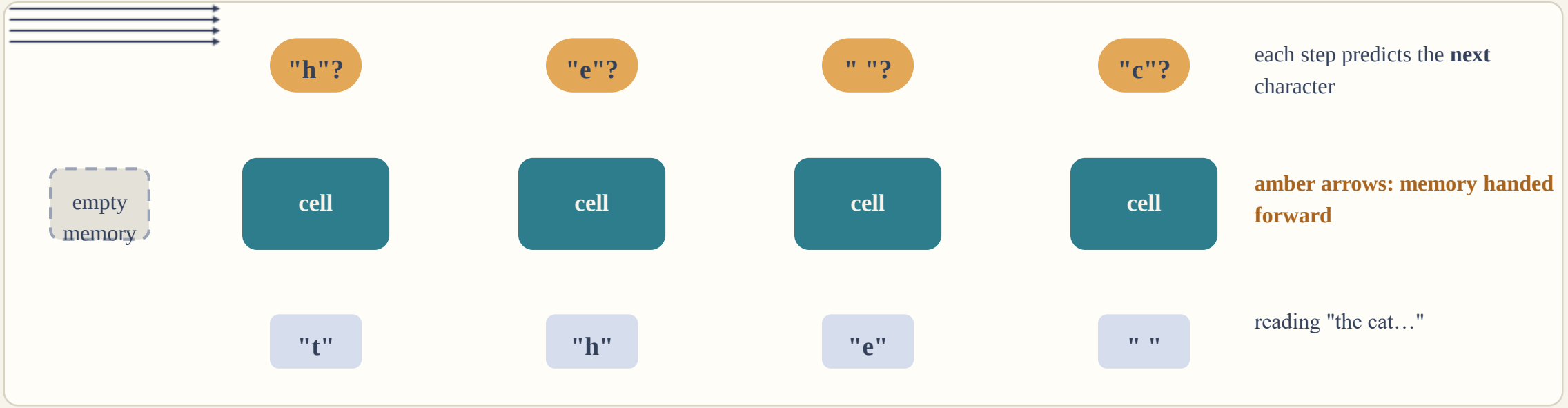
input through input weights + old memory through memory weights

→ tanh → **new memory**

Shared across time: the same weights are used at step 1, step 2, step 100. Like a CNN sharing a kernel across space.

The memory is a vector, often 128 to 1,024 numbers: a compressed summary of everything read so far.

Unroll the loop and an RNN becomes a very deep feedforward network



Four copies, one set of weights. Every teal box is the same cell, so learning at any step improves all steps.

Depth equals length. A 100 character sequence is a 100 layer network in time, with all the deep network problems that brings.

Unrolled, it is an ordinary computation graph, so backpropagation (Module 9) applies unchanged.

A whole RNN layer is a for loop over time

```
N, T, D, H = 2, 5, 3, 4   # batch, time steps, input size, memory size
X = rng.normal(size=(N, T, D))
Wx = rng.normal(0, 0.5, (D, H))   # input to memory
Wh = rng.normal(0, 0.5, (H, H))   # memory to memory
b = np.zeros(H)
h = np.zeros((N, H))              # empty memory at the start
states = []
for t in range(T):
    h = np.tanh(X[:, t] @ Wx + h @ Wh + b)  # same weights every step
    states.append(h)
states = np.stack(states, axis=1)
print("input", X.shape, " all states", states.shape, " final memory", h.shape)
print("weights:", Wx.size + Wh.size + b.size, "whatever the sequence length")
```

OUTPUT

```
input (2, 5, 3)  all states (2, 5, 4)  final memory (2, 4)
weights: 32 whatever the sequence length
```

SHAPES AT ONE STEP

$(2, 3) \; @ \; (3, 4) \rightarrow (2, 4)$

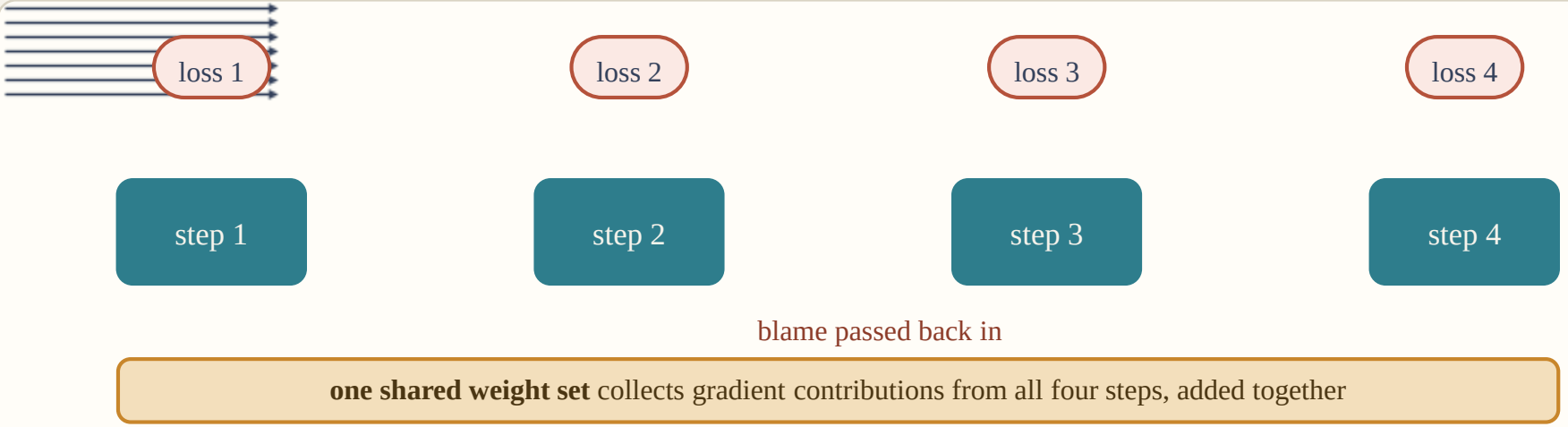
$(2, 4) \; @ \; (4, 4) \rightarrow (2, 4)$

Add, tanh, and the new memory is (2, 4) again, ready for the next step.

Use all states for per step tasks; use only the **final memory** for whole sequence tasks.

Bottleneck: step 5 cannot start until step 4 finishes. RNNs are inherently sequential, which limits speed on GPUs.

Blame flows backwards along the chain of memories



Step 2's memory is blamed for loss 2 directly, **and** for losses 3 and 4 through the steps it fed.

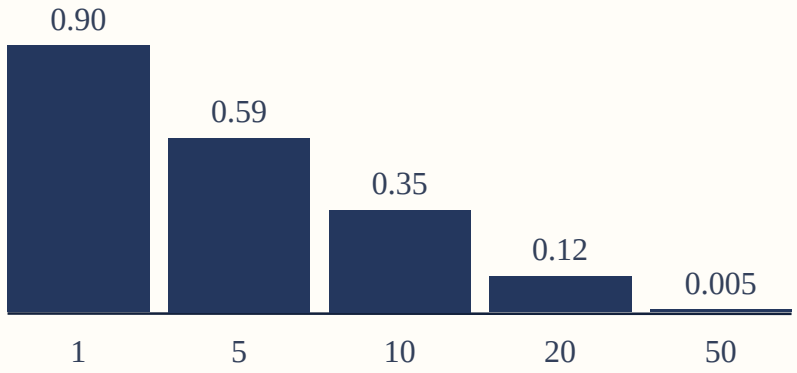
```
dh_next = np.zeros((H, 1))
for t in reversed(range(len(inputs))):      # newest step first
    dh = Why.T @ dy[t] + dh_next            # blame from output and future
    draw = np.subtract(1, hs[t] * hs[t]) * dh # back through tanh
    dWhh += draw @ prev[t].T                # add up, shared weights
    dh_next = Whh.T @ draw                  # hand blame to the past
```

Truncated BPTT: for long texts, backpropagate through chunks of, say, 25 steps, but carry the memory forward between chunks. Our character model does exactly this.

Repeated multiplication: blame fades or explodes over long distances

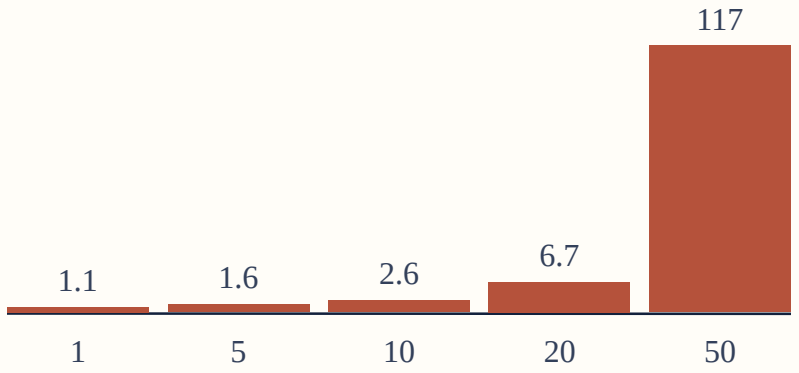
Each step back in time multiplies the blame by roughly the same factor. Watch what a factor slightly below or above one does:

Factor 0.9 per step: vanishing



steps back in time. After 50 steps only half a percent survives: the network cannot learn long range links.

Factor 1.1 per step: exploding

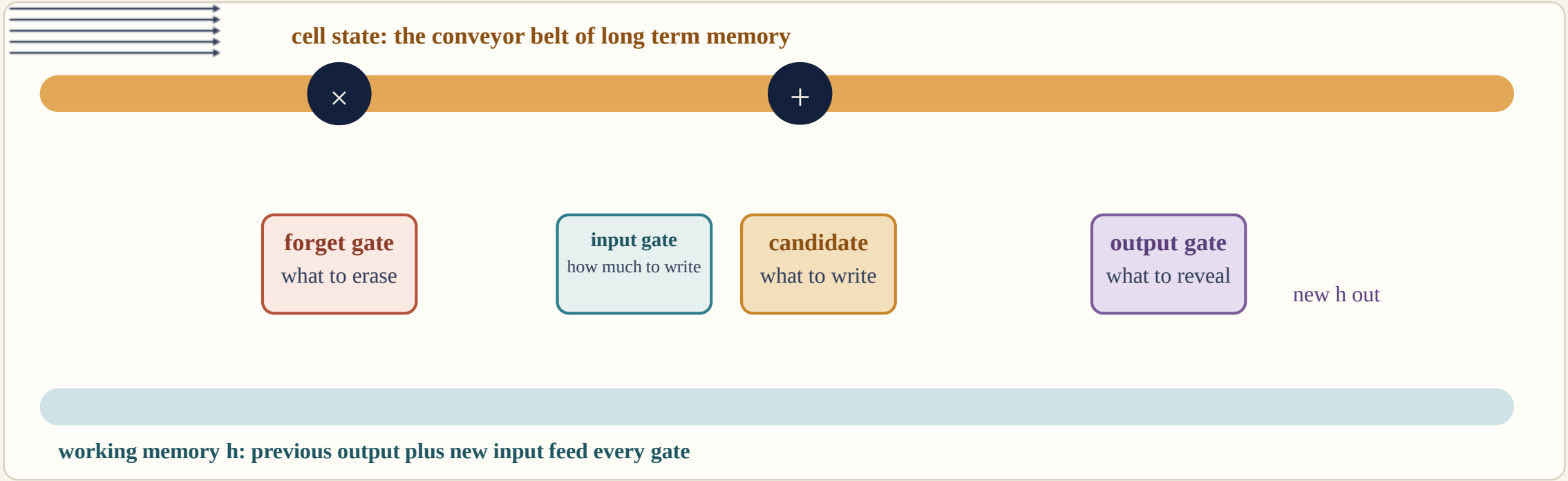


117 times bigger after 50 steps: huge, erratic updates. Fixed by gradient clipping.

Example: "The **keys** that I left on the kitchen table yesterday **were** …" Choosing "were" needs a link 10 words back.

Exploding is easy to clip. Vanishing needs a new design: the **LSTM**.

The LSTM's conveyor belt: a memory lane with gates instead of a blender



Why it fixes vanishing: along the belt, memory is only scaled by the forget gate and added to. If the forget gate stays near 1, information and gradient ride the belt for hundreds of steps.

Each gate is a sigmoid: a dial from 0 (closed) to 1 (open), computed fresh every step from the current input and memory.

Following one memory slot through the four gates

old cell value

2.00

say it remembers "the subject is plural"

forget gate 0.90

keep 90%

1.80

input gate 0.50

times candidate 0.80

adds 0.40

new cell value

2.20

1.80 kept plus 0.40 written

output gate 0.70

times squashed cell 0.976

h 0.683

Forget near 0 wipes the slot: useful when a sentence ends and the old subject no longer matters.

Input near 0 ignores the current word entirely: the memory sails through untouched.

Output decides how much of the memory is shown to the rest of the network right now.

All four dials are **learned**: the network discovers for itself when to remember, overwrite and speak.

The GRU: a lighter LSTM with two gates and one memory

The update gate is a blend slider



New memory: **70% old**, **30% candidate**. One dial does the work of the LSTM's forget and input gates together.

Reset gate: decides how much of the old memory to consult when forming the candidate.

	LSTM	GRU
Gates	forget, input, output	update, reset
Memories	cell state and h	h only
Weights	4 blocks	3 blocks, 25% fewer
Accuracy	similar	similar
Introduced	1997	2014

Rule of thumb: try a GRU when data or compute is limited; neither wins consistently. Both share the key idea: gated, additive memory updates.

An LSTM step in eight lines of NumPy

```
def sigmoid(z): return 1 / (1 + np.exp(np.negative(z)))

def lstm_step(x, h, c, W, b):
    z = np.concatenate([x, h], axis=1) @ W + b   # all four gates at once
    f, i, o, g = np.split(z, 4, axis=1)
    f, i, o, g = sigmoid(f), sigmoid(i), sigmoid(o), np.tanh(g)
    c = f * c + i * g                  # forget some, write some
    h = o * np.tanh(c)                   # reveal some
    return h, c

W = rng.normal(0, 0.3, (D + H, 4 * H))
b = np.zeros(4 * H);  b[:H] = 1.0         # forget gate starts open
h = np.zeros((N, H));  c = np.zeros((N, H))
for t in range(T):
    h, c = lstm_step(X[:, t], h, c, W, b)
```

OUTPUT

```
LSTM weights: 128   h (2, 4)  c (2, 4)
forget gate when its input is zero: 0.731
```

ONE MATRIX, FOUR GATES

f	i	o	g

One multiply makes 4 times H numbers, split into the four gates. Faster than four separate multiplies.

128 weights: 7 inputs (3 data plus 4 memory) times 16, plus 16 biases. Four times the plain RNN.

Forget bias of 1: the gate starts 73% open, so memory survives early training. A widely used trick.

Teaching an RNN to write, one character at a time

THE TRAINING TEXT

"the cat sat on the mat. the dog sat on the log. the cat saw the dog and the dog saw the cat."

repeated 20 times · 15 distinct characters

INPUT AND TARGET: SHIFTED BY ONE

in	t	h	e	·	c	a
out	h	e	·	c	a	t

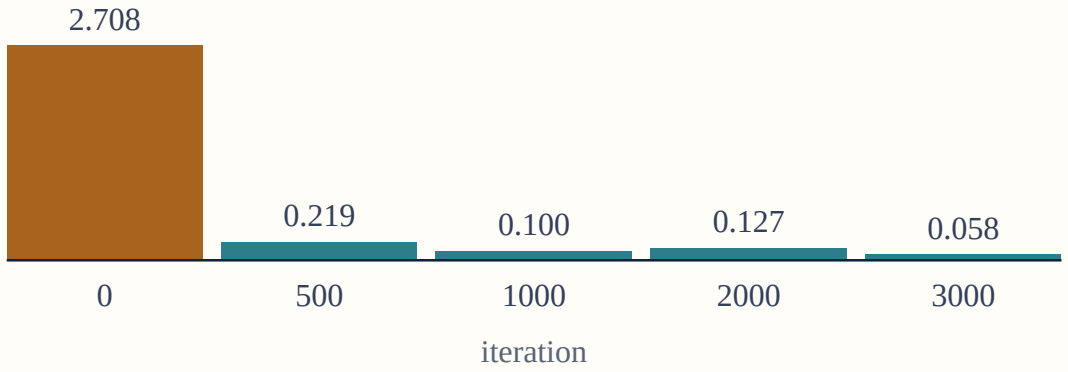
Every position is a 15 way classification: what comes next? (· marks a space)

Recipe: one hot characters, 64 memory slots, chunks of 25 steps, gradient clipping at 5, AdaGrad at rate 0.1, 3,000 iterations.

OUTPUT · BEFORE TRAINING

'gwdslhlntlg..dso ncoahlsmewlecmdtnnda.tlmnegc.nwcchc.ctwdes '

LOSS PER CHARACTER



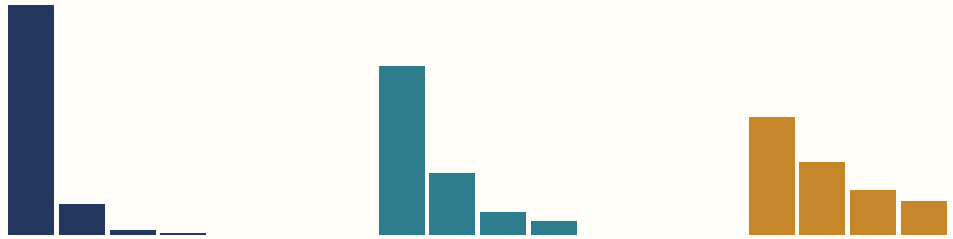
Start at 2.708, close to the no knowledge value for 15 characters (2.71). The sanity check passes.

Generating text: predict, sample, feed back, repeat

feed "t" → RNN step → 15 probabilities → roll a weighted die: "h" → feed "h" next

TEMPERATURE RESHAPES THE DIE

Divide the scores by the temperature before softmax. Four options with scores 3, 2, 1, 0.5:



cool 0.5 **normal 1** **hot 2**

Top choice: **86%** when cool, **63%** normal, **44%** hot.

OUTPUT · AFTER TRAINING

'he dog sat on the log. the cat sat on the mos.the log. the c'

0.5 *'he cat saw the cat. the cat sat on the log. the ca'*

1.0 *'he cat saw the cat. the cat saw the cat. the cat s'*

1.5 *'he cat sat on the awg sat on the log. tcg an the '*

Cool: safe and repetitive. Hot: creative, then broken words like "awg". Chatbots expose the same dial.

What RNNs taught us, and why attention came next

KEEP

- Share weights across time to handle any length
- Next token prediction as a training signal: free labels from raw text
- Gates and additive paths keep gradients alive
- Clip gradients; sample with a temperature

PROBLEMS LEFT OVER

[icon: AlertTriangle] **Sequential:** step 100 waits for step 99, so training cannot use GPU parallelism well

[icon: AlertTriangle] **Bottleneck:** everything read must squeeze through one fixed size memory vector

[icon: AlertTriangle] **Distance:** even LSTMs struggle to link words hundreds of steps apart

The next idea: instead of squeezing the past into one memory, let every word **look directly** at every other word, all at once. That is attention.

20

MODULE TWENTY

Attention and Transformers

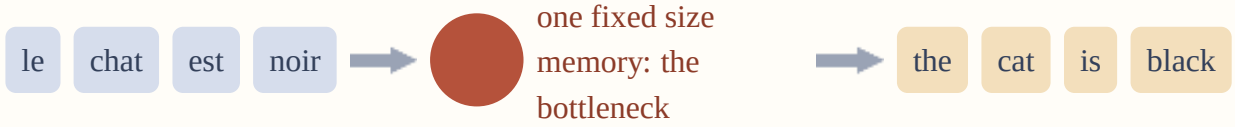
Every token looks at every other token and decides what matters. The architecture behind GPT, Claude and modern AI.

IN THIS MODULE

- Attention as a soft lookup
- Queries, keys and values by hand
- Multiple heads, positions, masks
- The Transformer block
- Tokens, embeddings, next token training
- A tiny GPT trained from scratch

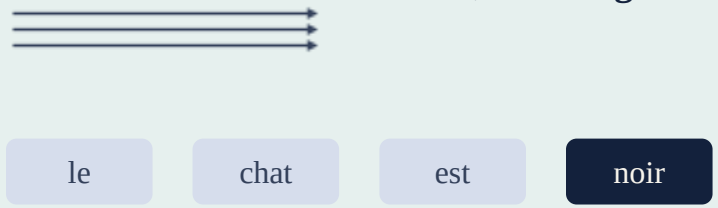
From one squeezed summary to looking back at every word

Before: encoder and decoder joined by one vector



A 50 word sentence must fit through the same small vector as a 5 word one. Translation quality fell sharply for long sentences.

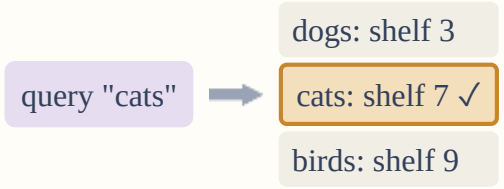
After: the decoder looks back, choosing where to focus



Arrow thickness is the attention weight: when writing "black" the model looks mostly at "noir". Weights are recomputed for every output word.

Attention is a soft library lookup

A normal dictionary: hard lookup



Exactly one key matches; you get exactly one value. All or nothing, and not learnable by gradients.

Attention: soft lookup



Every key matches a little. You get a **blend** of all values, weighted by match: mostly the cat shelf, a bit of the dog shelf. Smooth, so it can be learned.

Query: what this token is looking for

Key: what each token advertises about itself

Value: what each token hands over if chosen

In self attention every token produces all three, from three learned weight matrices, and runs this lookup against every token in the sentence, itself included.

Five steps from token vectors to context aware vectors

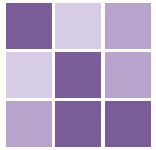
1

Project

Each token vector times three learned matrices gives its **query**, **key** and **value**.

2

Score



Dot product of every query with every key: a T by T grid of match scores.

3

Scale

[icon: Scale]

Divide by the square root of the key length to keep scores moderate.

4

Softmax rows



Each row becomes weights that are positive and add up to 1.

5

Blend



Weighted average of the values: each token's new, context aware vector.

All in parallel: steps 2 to 5 are just two matrix multiplies and a softmax, for every token at once. No loop over time.

After attention, the vector for "bank" in "river bank" has absorbed information from "river".

Attention by hand for three tokens, two numbers each

	query	key	value
the	1, 0	1, 0	1, 0
cat	0, 1	0, 2	0, 1
sat	1, 1	1, 1	0.5, 0.5

FOLLOW "cat" (ROW 2)

Query (0, 1) against each key, multiply pairs and add:

the: 0 · cat: **2** · sat: 1

Scale by the square root of 2, softmax:

0.14 · **0.58** · 0.28

Blend values: **(0.28, 0.72)**, mostly "cat"

ATTENTION WEIGHTS (rows: who is looking)

	the	cat	sat
the	0.40	0.20	0.40
cat	0.14	**0.58**	0.28
sat	0.20	0.40	0.40

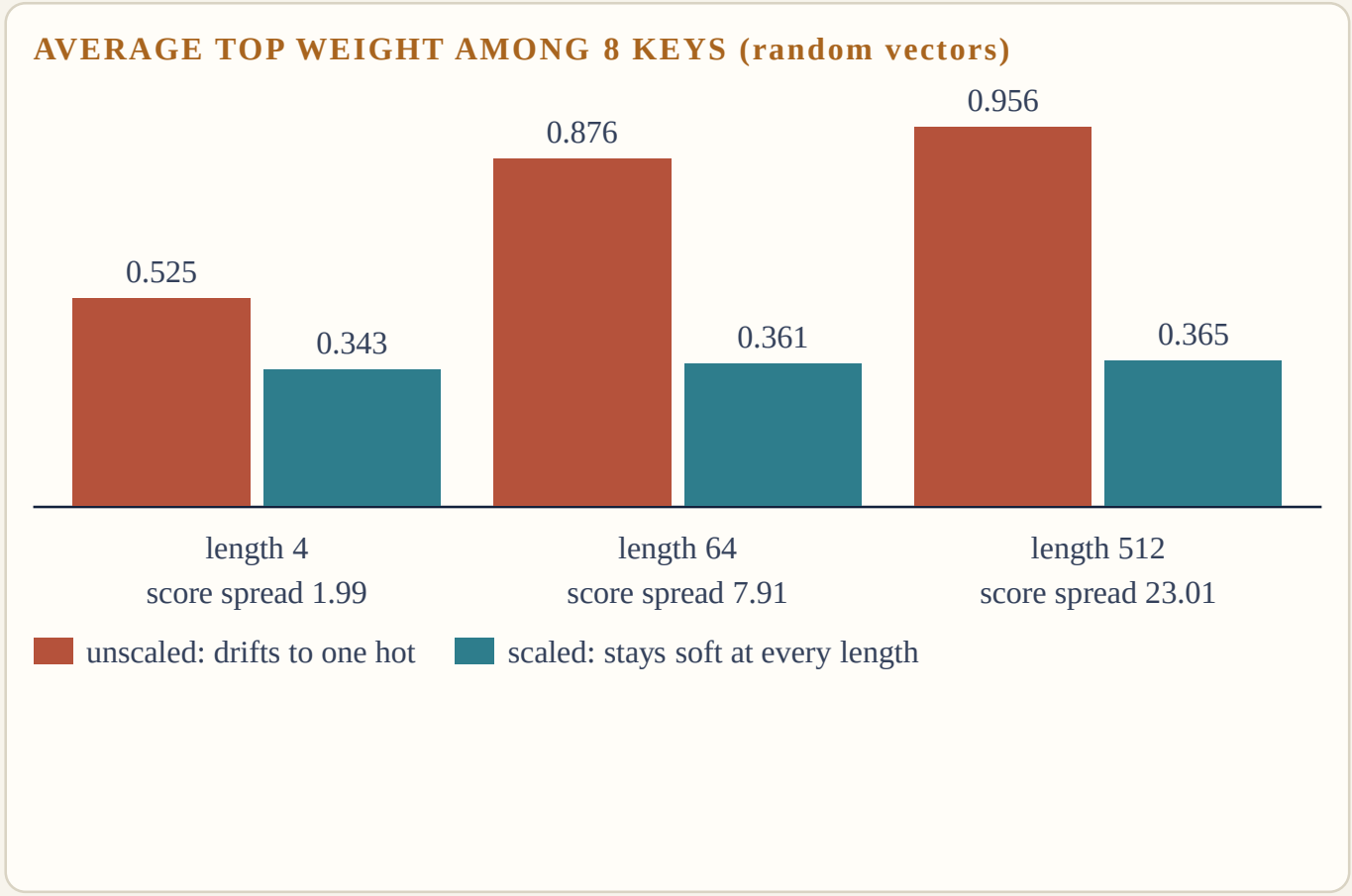
every row adds up to 1

OUTPUT (NumPy check)

```
weights
[[0.401 0.198 0.401]
 [0.14  0.576 0.284]
 [0.198 0.401 0.401]]
outputs
[[0.602 0.398]
 [0.282 0.718]
 [0.398 0.602]]
```

Each new vector is a mixture: "cat" is now 72% cat flavoured, 28% the rest.

Longer vectors give wilder scores, and softmax turns wild into winner takes all



The problem: a dot product adds up one product per entry, so its spread grows with the square root of the length (2, 8, 23 above).

The symptom: one score dominates, softmax gives 0.96 to a single key, and the gradients to every other key nearly vanish.

The cure: divide by the square root of the length. Same trick as Xavier initialisation in Module 14.

Self attention: the whole mechanism in seven lines

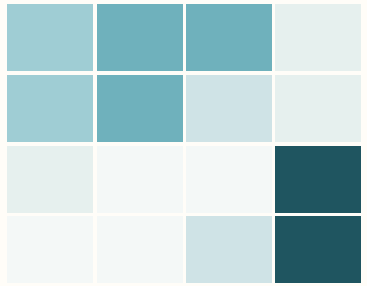
```
def self_attention(X, Wq, Wk, Wv, causal=False):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # three views of every token
    scores = Q @ K.T / np.sqrt(K.shape[1])  # match every query to every key
    if causal:                              # hide the future (slide 244)
        scores = scores + np.triu(np.full(scores.shape, NEG_BIG), k=1)
    weights = softmax(scores)                 # each row sums to 1
    return weights @ V, weights               # blend the values

T, D, Dk = 4, 8, 4                    # 4 tokens, 8 features, 4 per head
X = rng.normal(size=(T, D))
Wq, Wk, Wv = (rng.normal(0, 0.5, (D, Dk)) for _ in range(3))
out, w = self_attention(X, Wq, Wk, Wv)
print("output", out.shape, "  row sums", np.round(w.sum(axis=1), 3))
print(np.round(w, 2))
```

OUTPUT

```
output (4, 4)   row sums [1. 1. 1. 1.]
[[0.24 0.34 0.34 0.09]
 [0.28 0.37 0.21 0.14]
 [0.11 0.02 0.05 0.82]
 [0.05 0.03 0.13 0.8 ]]
```

AS A HEATMAP



Tokens 3 and 4 both focus on token 4; tokens 1 and 2 spread their attention.

Weights here are random, so the pattern means nothing yet. Training shapes it.

Several heads, several questions asked at once

Head 1 · grammar

"The **keys** … were": a verb finds its subject.



Head 2 · references

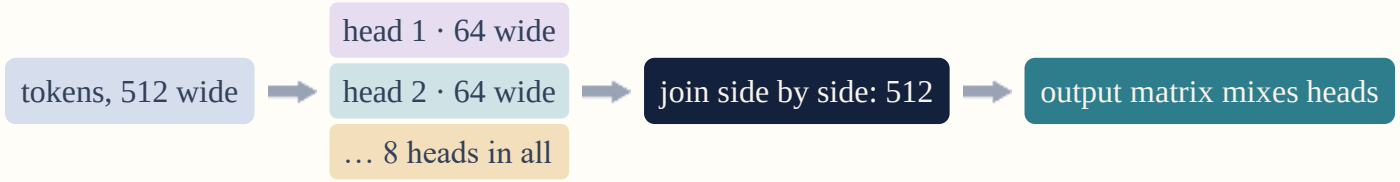
"Anna said she …": a pronoun finds who it means.



Head 3 · neighbours

Look at the word just before: local phrases.





Same cost: 8 heads of 64 do the work of one head of 512, but can attend to 8 different things.

Nobody assigns the roles. Specialisations like these emerge during training.

Attention ignores order, so we stamp each token with its position

The problem

dog bites man

man bites dog

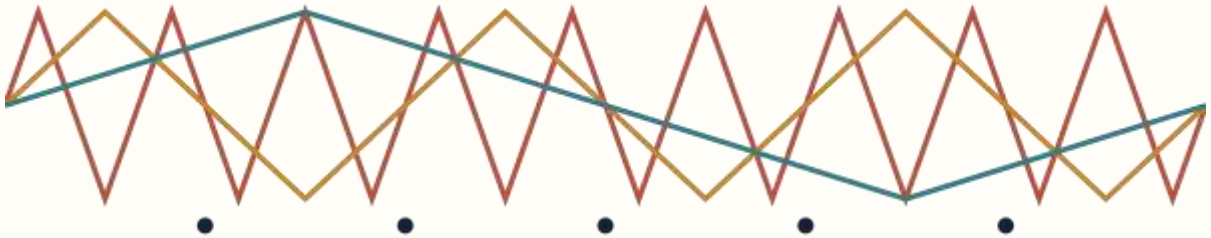
To attention these are the same bag of three words: it would give every word the same output in both sentences.

The fix: add a position signature



word meaning + position stamp → what attention sees

SINUSOIDAL STAMPS: WAVES OF MANY SPEEDS

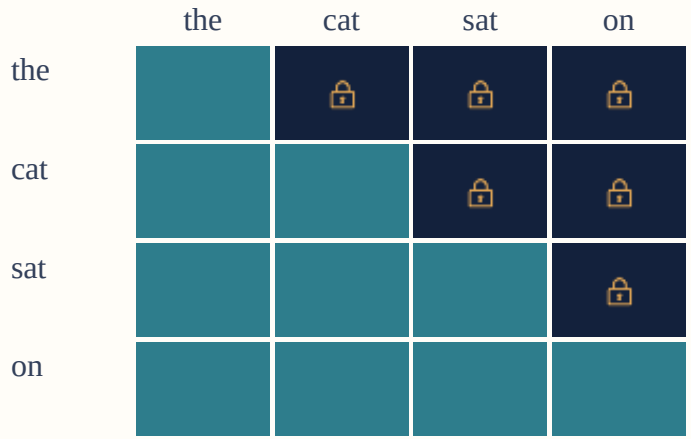


Read each wave's height at a position: fast waves tell nearby positions apart, slow waves tell far ones apart, like the hands of a clock.

Alternatives: GPT models simply **learn** one stamp per position (our tiny GPT does this); many modern models rotate queries and keys by position instead (RoPE).

No peeking: when predicting the next word, the future is hidden

THE MASK (rows look at columns)



Teal: allowed. Locked: a huge negative score is added, so softmax gives it exactly zero weight.

```
out, w = self_attention(X, Wq, Wk, Wv, causal=True)
print(np.round(w, 2))
```

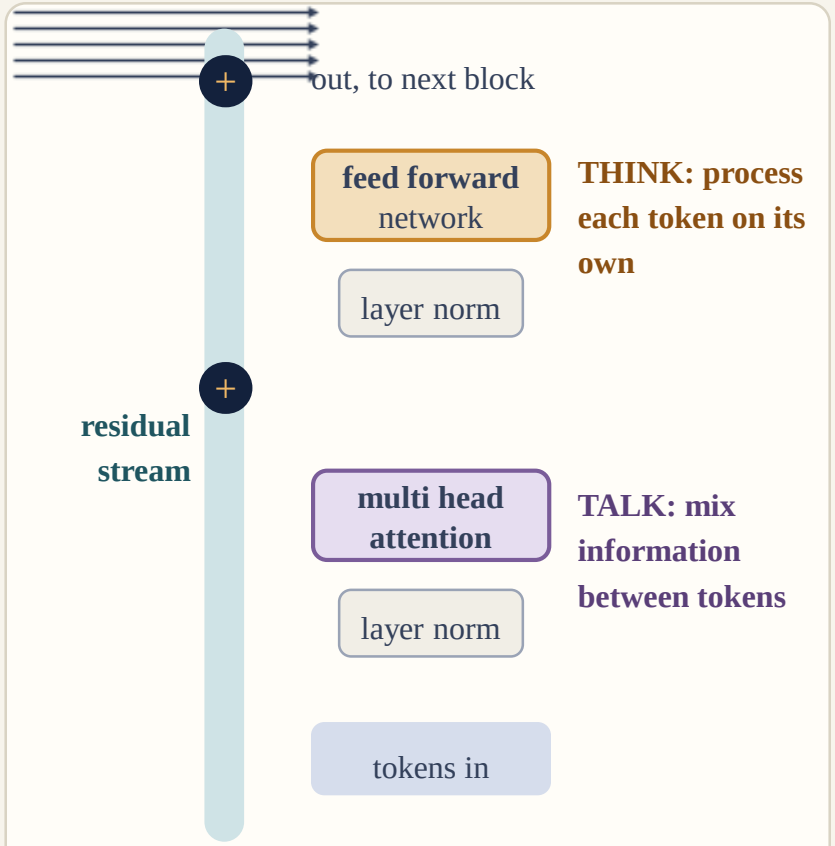
OUTPUT

```
[[1.   0.   0.   0.  ]
 [0.43 0.57 0.   0.  ]
 [0.61 0.09 0.3  0.  ]
 [0.05 0.03 0.13 0.8 ]]
```

Upper right is all zeros. The first token can only look at itself; the last sees everything before it.

Payoff: one pass over a 1,000 token text trains 1,000 next word predictions at once, all honest.

One block: talk to other tokens, then think alone



Attention is the only place tokens exchange information.

Feed forward: a two layer MLP applied to each token separately, widening 4 times then narrowing back. Most of the parameters live here.

Residual adds (Module 18) keep a clean highway for signals and gradients through dozens of blocks.

Layer norm (Module 16) before each sublayer keeps scales steady.

Stack it: GPT 2 small uses 12 identical blocks; large models use around 100.

A full GPT forward pass fits on one screen

```
def layer_norm(x):
    mean = np.mean(x, axis=2, keepdims=True)
    centred = x + np.negative(mean)
    return centred / np.sqrt(np.mean(centred * centred, axis=2, keepdims=True) + 0.00001)

def forward(P, idx):                    # idx: (batch, T) token ids
    T = idx.shape[1]
    mask = onp.triu(onp.full((BLOCK, BLOCK), NEG_BIG), k=1)
    x = P["tok"][idx] + P["pos"][:T]          # meaning plus position
    for l in range(LAYERS):
        x = x + attention(layer_norm(x), P[f"qkv{l}"], P[f"proj{l}"], mask)  # talk
        hidden = np.maximum(0, np.dot(layer_norm(x), P[f"ff1{l}"]))
        x = x + np.dot(hidden, P[f"ff2{l}"])                          # think
    return np.dot(layer_norm(x), P["out"])      # scores for every next char
```

OUR TINY GPT

vocabulary	**18 characters**
width D	**32**
heads	**2 of 16**
blocks	**2**
context	**16 characters**
parameters	**26,240**

Autodiff: written with the small autograd library, which builds the backward pass automatically, just like our Value engine in Module 10 but for whole arrays.

GPT 3 is this exact loop with D of 12,288, 96 blocks and 175 billion parameters.

Tokens: not letters, not words, but common chunks

Characters
Tiny vocabulary, but sequences become very long.

Whole words
Short sequences, but millions of words and no answer for new ones.

Subword pieces
The middle way: frequent words whole, rare words split into pieces.

BYTE PAIR ENCODING: REPEATEDLY GLUE THE COMMONEST PAIR

Corpus: low ×4, lower ×2, newest ×3, widest ×2. Start from single letters (_ marks a word end).

```
for step in range(1, 7):
    pair, count = best_pair(corpus)   # most frequent neighbours
    corpus = merge(corpus, pair)      # glue them everywhere
    print(f"merge {step}: {pair[0]} + {pair[1]}  (seen {count} times)")
```

OUTPUT

```
merge 1: l + o  (seen 6 times)     merge 4: es + t  (seen 5 times)
merge 2: lo + w  (seen 6 times)    merge 5: est + _  (seen 5 times)
merge 3: e + s  (seen 5 times)     merge 6: low + _  (seen 4 times)
['low e r _', 'low_', 'n e w est_', 'w i d est_']
```

RESULT

low_

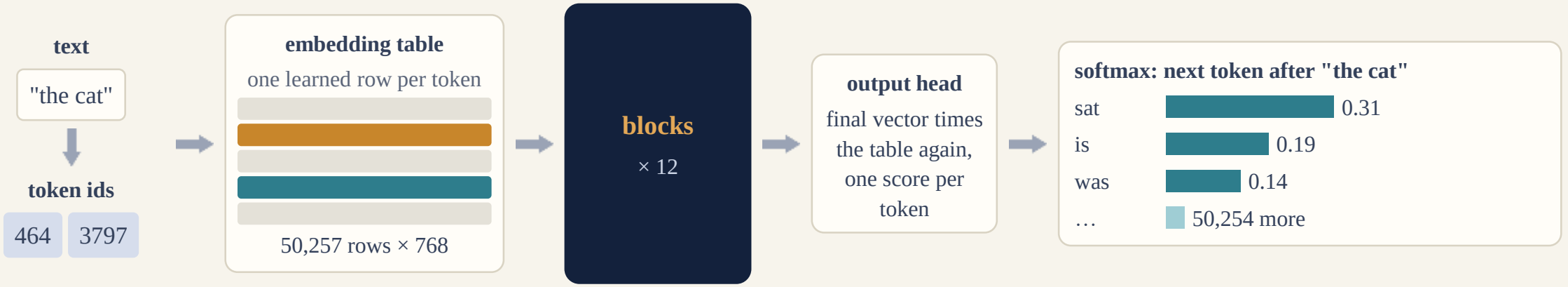
low | e | r | _

n | e | w | est_

The useful suffix **est_** emerged on its own, shared by newest and widest.

Real tokenisers run about 50,000 to 200,000 merges. One token is roughly three quarters of an English word.

Embedding table in, vocabulary scores out



Embedding is just a lookup: token 464 picks row 464. Those rows are learned like any other weight.

Weight tying: many models reuse the same table to score outputs, saving tens of millions of parameters.

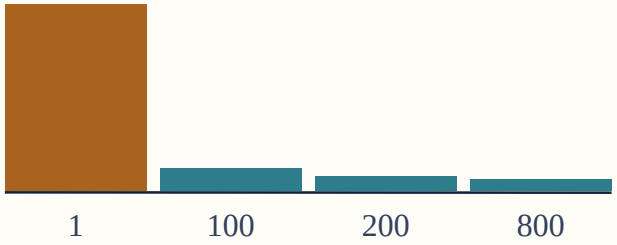
Next token prediction: the only objective a GPT ever sees

```
g = grad(loss_fn)                          # automatic backward pass
for step in range(1, 801):
    ix = rng.integers(0, len(data) + onp.negative(BLOCK + 1), 16)
    xb = onp.stack([data[i:i + BLOCK] for i in ix])        # 16 random windows
    yb = onp.stack([data[i + 1:i + BLOCK + 1] for i in ix]) # shifted by one
    grads = g(P, xb, yb)
    adam_update(P, grads, step, lr=0.003)          # Module 15
```

OUTPUT

```
vocabulary 18  parameters 26240
step   1  loss 2.536
step 100  loss 0.302
step 200  loss 0.198
step 800  loss 0.170
seconds 10.3
```

LOSS



SAMPLE BEFORE TRAINING

'the r rmrrnltettr .wggrbr rgl oargttnrteob'

SAMPLE AFTER 800 STEPS

'the cat sat on the mat. the dog sat on the cat. the bird saw the log. the cat sat on'

"The dog sat on the cat" never appears in the training text. The model **combined** patterns it learned into a new sentence.

Three ways to wire the same block

Encoder only



every token sees every token

Training: hide 15% of words, guess them from both sides.

Good at: understanding: classification, search, tagging.

Example: BERT (2018)

Encoder and decoder



decoder also attends to the encoder

Training: map an input sequence to an output sequence.

Good at: translation, summarisation.

Examples: the original Transformer (2017), T5

Decoder only



causal mask: only the past

Training: predict the next token. Nothing else.

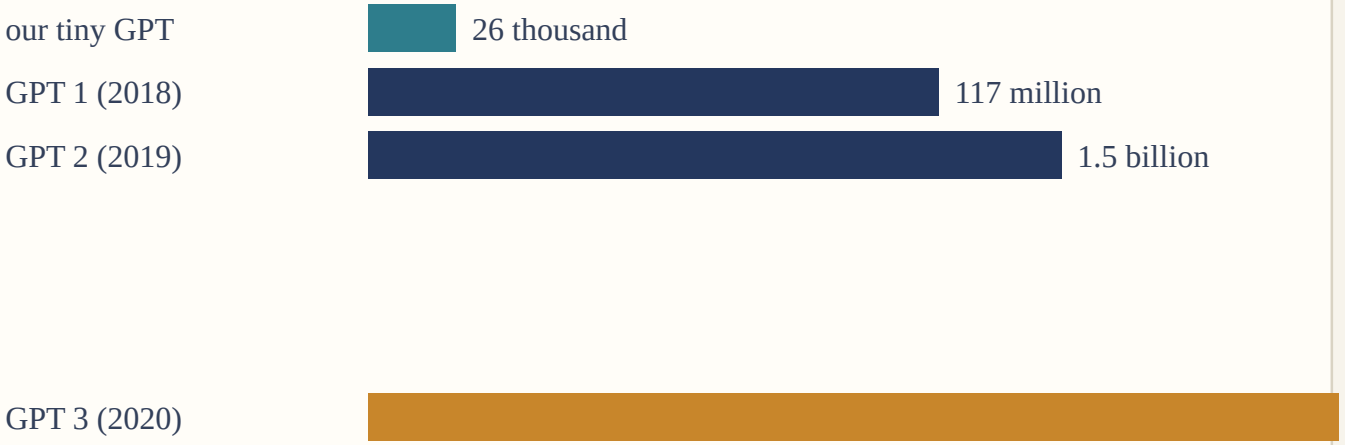
Good at: generating anything; with scale, almost every task.

Examples: GPT series, Claude, Llama: our tiny GPT

The surprise of the last decade: the simplest variant, decoder only with next token prediction, scaled up, turned out to be the most general.

From our 26 thousand parameters to hundreds of billions

PARAMETERS, EACH STEP TEN TIMES BIGGER



Bar length grows by a fixed step for every factor of ten.

Scaling laws: loss falls smoothly and predictably as parameters, data and compute grow together.

Balance: roughly 20 training tokens per parameter is compute optimal (Chinchilla, 2022).

WHAT CHANGES WITH SCALE

Width: 32 → 12,288
Blocks: 2 → 96
Context: 16 → 2,048 tokens and far beyond
Text: 7 KB → hundreds of billions of tokens
Training: 10 CPU seconds → thousands of GPUs for months

What does not change: embeddings, attention, feed forward, residual stream, next token loss, Adam. You have built all of it.

Three training stages turn a next token model into a helpful assistant

1

Pretraining

[icon: Library]

Next token prediction on a vast slice of the internet, books and code. Learns language, facts and reasoning patterns.

Result: a brilliant autocompleter that continues any text, but does not follow instructions reliably.

most of the compute

2

Supervised fine tuning

[icon: MessagesSquare]

Same loss, but on carefully written example conversations: a request, then an ideal answer.

Result: learns the format and habits of a helpful reply.

transfer learning again

3

Learning from feedback



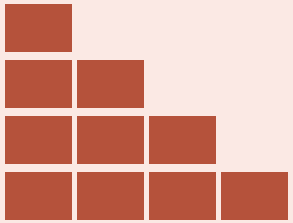
The model writes several answers; preferences (from people or from written principles) score them; reinforcement learning nudges it toward the better ones.

Result: more helpful, honest and harmless behaviour.

Under the hood every stage still updates the same Transformer weights with gradients and an optimiser. Only the data and the training signal change.

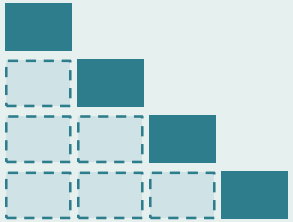
The KV cache: never recompute what the past already told you

Naive generation



Each new token reruns the model over the **whole** text so far. 4 tokens cost 1, 2, 3, 4: ten token computations, growing fast.

With a KV cache



Past keys and values never change (the mask stops them seeing new tokens), so **store** them. Each step computes only the new token: 4 computations.

Price: memory. The cache holds a key and value per token, per layer, per head, which is why long contexts need lots of GPU memory.

Other tricks: fused attention kernels, fewer key and value heads shared across query heads, and lower precision numbers.

Training processes all positions in parallel; generation is still one token at a time.

MODULE 20 · SUMMARY

The Transformer on one page

text → tokens → embedding + position → attention: talk → feed forward: think → × N blocks → next token scores

Attention

Queries match keys; softmax weights blend values. Scaled, multi headed, masked for generation.

Block

Pre norm, attention, add; pre norm, MLP, add. Shape never changes, so stack freely.

Training

Next token loss, AdamW, warm up and cosine, clipping. Then fine tuning and feedback.

Why it won

Parallel training, direct paths between any two tokens, and smooth gains from scale.

Its cost

Attention work grows with the square of the context; generation is sequential.

Our proof

26,240 parameters, ten CPU seconds, and it invented a new sentence.

21

MODULE TWENTY ONE

Embeddings

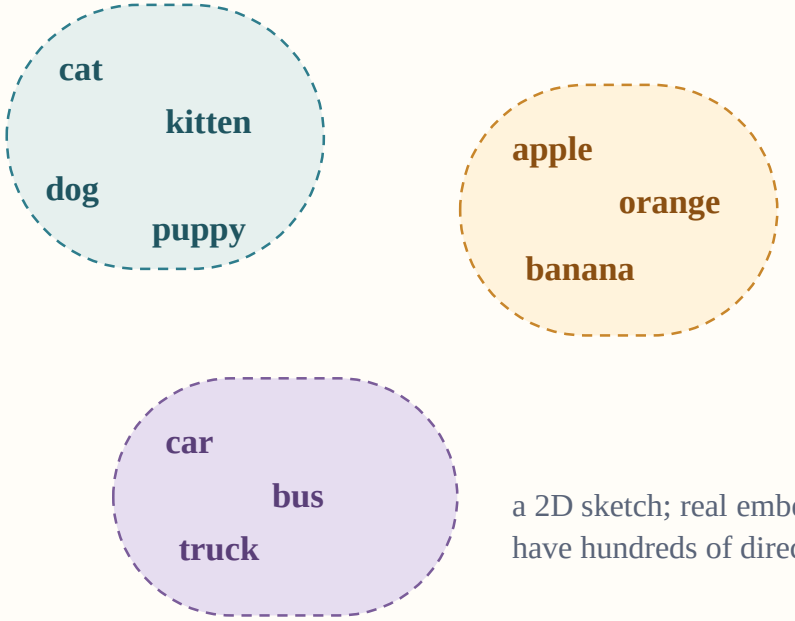
Turning words, images and documents into points in space, where closeness means similarity of meaning.

IN THIS MODULE

- Meaning as geometry
- Word vectors from co occurrence, in code
- Similarity search and retrieval

Similar things land near each other; relationships become directions

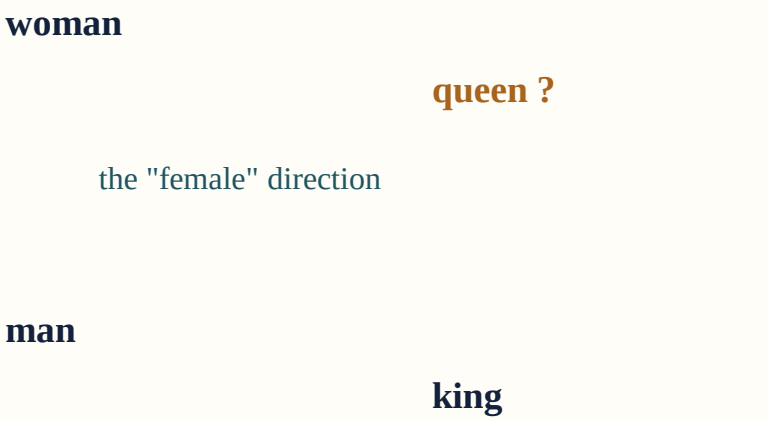
CLUSTERS



a 2D sketch; real embeddings have hundreds of directions

RELATIONSHIPS AS ARROWS





Walk from king along the same arrow that takes man to woman, and the nearest word is queen.

Closeness is measured by the angle between vectors (cosine similarity): 1 means same direction, 0 unrelated.

Nobody tells the model about animals or royalty. The structure emerges from predicting context.

Word vectors from 24 tiny sentences, in a dozen lines

count which words appear within 2 places of each other → keep pairs seen more often than chance → compress each row to 8 numbers (SVD) → compare by angle

```
C = np.zeros((len(vocab), len(vocab)))
for i, w in enumerate(words):                  # neighbours within 2 words
    for j in range(max(0, i + np.negative(2)), min(len(words), i + 3)):
        if j != i: C[idx[w], idx[words[j]]] += 1
total = C.sum();  pw = C.sum(1) / total
pmi = np.log(np.maximum(C / total / np.outer(pw, pw), 0.000001))
ppmi = np.maximum(pmi, 0)          # more often than chance only
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :8] * S[:8]                  # 8 numbers per word
E = E / np.linalg.norm(E, axis=1, keepdims=True)
def nearest(word, k=3):
    sims = E @ E[idx[word]]             # cosine with every word
    ...  # sort high to low, skip the word itself
```

OUTPUT

vocabulary 36 embedding size 8

cat young 0.90, puppy 0.87, water 0.84

kitten puppy 0.96, young 0.93, cat 0.83

apple orange 0.94, ate 0.87, she 0.85

car bus 0.91, drove 0.87, slowly 0.84

truck stopped 0.88, car 0.75, slowly 0.74

With no labels at all: kitten is nearest puppy, apple nearest orange, car nearest bus. Related verbs (ate, drove) also score high: they share contexts.

Semantic search and retrieval: find by meaning, not by keyword



ONCE, AHEAD OF TIME

[icon: Files]
documents

embedding model

vector index

EVERY QUESTION

"how do I stop my model memorising?"

embed

nearest: **"Regularisation and dropout"**, though no words match

Retrieval augmented generation: paste the retrieved passages into a language model's prompt so its answer is grounded in your documents.

Also: recommendations, duplicate detection, clustering, and image search with shared text and image spaces (CLIP).

How sentence embeddings are trained: contrastive learning pulls matching pairs (a question and its answer, an image and its caption) together and pushes mismatched pairs apart.

22

MODULE TWENTY TWO

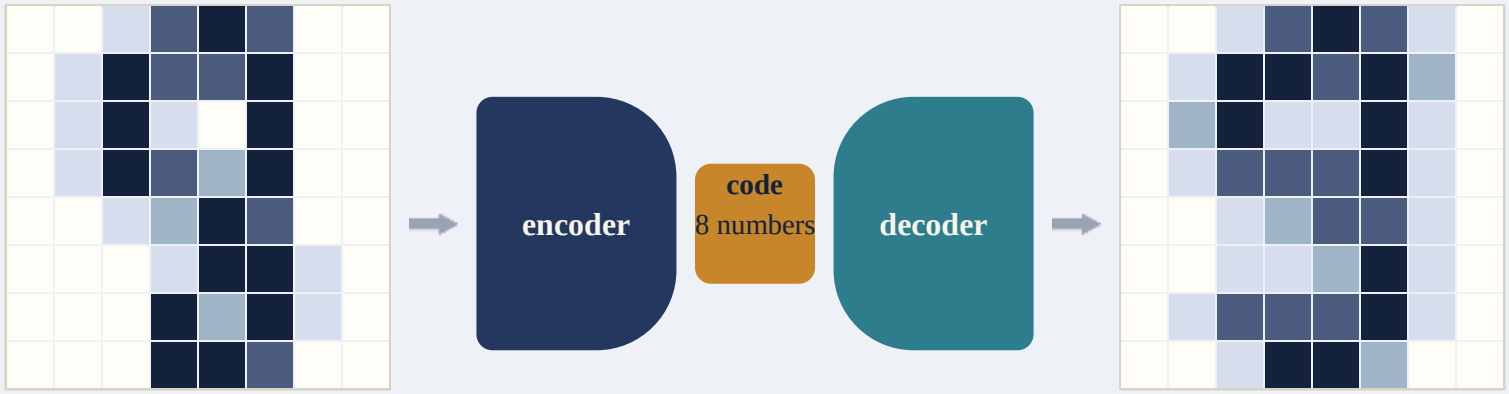
Generative Models

From recognising images to creating them: models that learn the shape of the data well enough to produce new examples.

IN THIS MODULE

- Autoencoders, trained in code
- Variational autoencoders
- Generative adversarial networks
- Diffusion models
- Choosing between them

Squeeze through a narrow code, then rebuild: the auto-encoder



input: an 8

rebuilt from 8 numbers

> Target equals input: no labels needed. The bottleneck forces it to keep only the essence.

```
encoder = Sequential(Dense(64, 32, rng), ReLU(), Dense(32, code, rng))
decoder = Sequential(Dense(code, 32, rng), ReLU(), Dense(32, 64, rng), Sigmoid())
model = Sequential(*encoder.layers, *decoder.layers)
for xb, _ in loader:
    diff = np.subtract(model.forward(xb), xb)     # rebuilt minus original
    model.backward(2 * diff / diff.size)          # squared error slope
    opt.step()
```

OUTPUT · 60 EPOCHS

```
code  2   test error 0.0381
code  8   test error 0.0164
code 16   test error 0.0080
always guess the average image: 0.0726
```

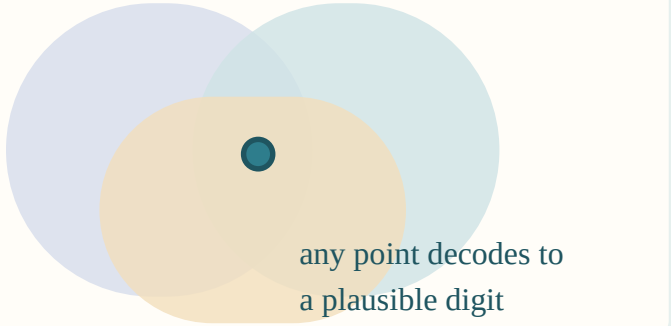
The VAE: make the code space smooth enough to sample from

Plain autoencoder code space



Codes sit in isolated clumps with empty gaps between. Random points fall in the gaps.

VAE code space



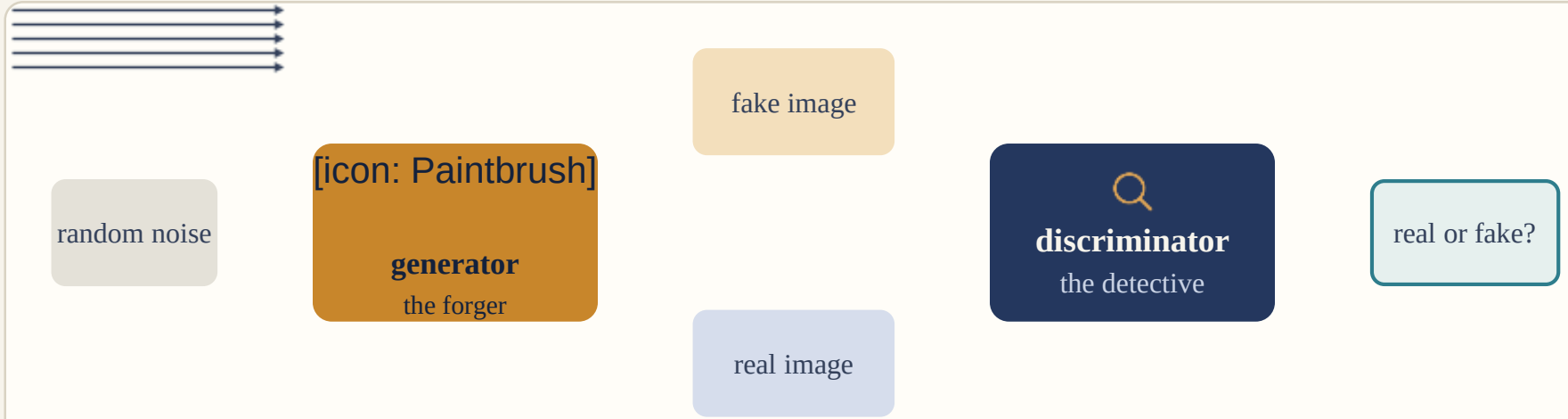
Each input maps to a small **cloud**, not a point, and all clouds are pulled toward one shared region, so they overlap smoothly.

Encoder outputs a centre and a spread; a random point is drawn from that cloud.

Two losses: rebuild well, and keep every cloud close to a standard bell shaped cloud.

To generate: sample a point from the standard cloud and run only the decoder.

The GAN: a forger and a detective who train each other



Detective is trained to tell real from fake.

Forger is trained, through the detective's gradients, to fool it.

Strength: very sharp, realistic images, generated in a single fast pass. Famous for photorealistic faces.

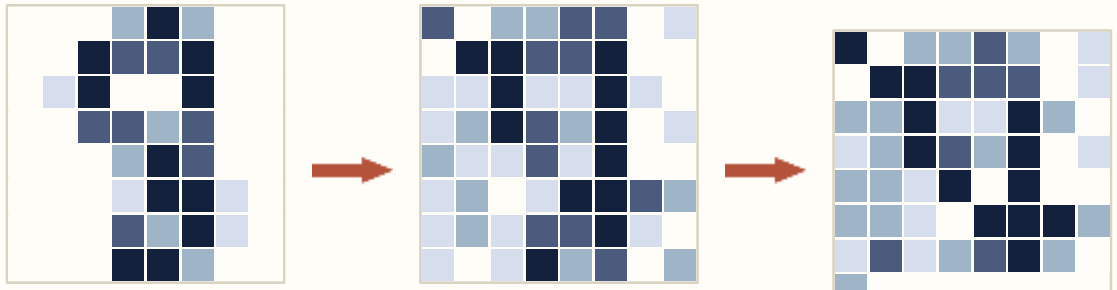
Weakness: unstable training (one side can overpower the other) and mode collapse: the forger finds a few safe outputs and repeats them.

At the ideal balance the detective can only guess: 50% each way.

MODULE 22 · DIFFUSION MODELS

Diffusion: learn to remove noise, then start from pure noise

FORWARD: ADD A LITTLE NOISE, MANY TIMES (fixed, no learning)



clean some noise

Four ways to generate, side by side

Family	Core idea	Quality	Speed	Training
Autoregressive	predict the next piece, repeat (GPT)	excellent	one token at a time	stable
VAE	smooth code space, decode samples	blurry	one pass	stable
GAN	forger versus detective	very sharp	one pass	fragile
Diffusion	learn to denoise, start from noise	state of the art	many steps	stable

Today: autoregressive Transformers lead for text and code; diffusion leads for images, video and audio.

Shared foundation: every one is trained with a loss, backpropagation and Adam on the layers you built.

23

MODULE TWENTY THREE

From Scratch to Frameworks

You built every piece by hand. Frameworks give you the same pieces, plus automatic gradients and GPUs.

IN THIS MODULE

- Our library mapped to PyTorch
- The digit network, rewritten
- Why GPUs, and how to use them
- Precision, saving, deploying

Everything you built has a PyTorch twin

Our NumPy course	PyTorch	Built in module
np.array	torch.tensor (can live on a GPU)	3 · NumPy
Value and backward()	autograd: loss.backward()	10 · Autodiff engine
Dense	nn.Linear	11 · Library
ReLU, Sequential	nn.ReLU, nn.Sequential	11 · Library
SoftmaxCrossEntropy	nn.CrossEntropyLoss	13 · Losses
SGD, Momentum, Adam	torch.optim.SGD, AdamW	15 · Optimisers
DataLoader	torch.utils.data.DataLoader	11 · Library
Dropout, training flag	nn.Dropout, model.train() and model.eval()	16 · Regularisation
Conv2D, MaxPool2	nn.Conv2d, nn.MaxPool2d	18 · CNNs
lstm_step, self_attention	nn.LSTM, nn.MultiheadAttention	19 and 20

Same network, same loop, a framework underneath

```
import torch, torch.nn as nn
device = "cuda" if torch.cuda.is_available() else "cpu"

model = nn.Sequential(nn.Linear(64, 64), nn.ReLU(),
                      nn.Linear(64, 32), nn.ReLU(),
                      nn.Linear(32, 10)).to(device)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(30):
    model.train()
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        loss = loss_fn(model(xb), yb)       # forward
        opt.zero_grad()                     # reset old gradients
        loss.backward()                     # autodiff, Module 10
        opt.step()                          # update, Module 11
```

Line for line it matches our Module 12 loop: forward, loss, backward, step.

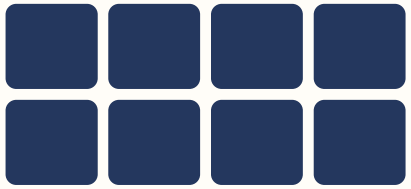
zero_grad is new: PyTorch **adds** new gradients to old ones, so they must be cleared each step (slide 199's warning).

.to(device) moves weights and data to the GPU when there is one. Nothing else changes.

Try it: run this on the digits and compare with our NumPy test accuracy of 0.9467. Expect a similar score, not an identical one.

Why GPUs: deep learning is mostly matrix multiplication

CPU: a few brilliant chefs



Tens of powerful cores, each great at complex, branching, one after another work.

GPU: an army of line cooks



Thousands of simple cores doing the same small multiply and add at once. Each output of a matrix product can be computed independently.

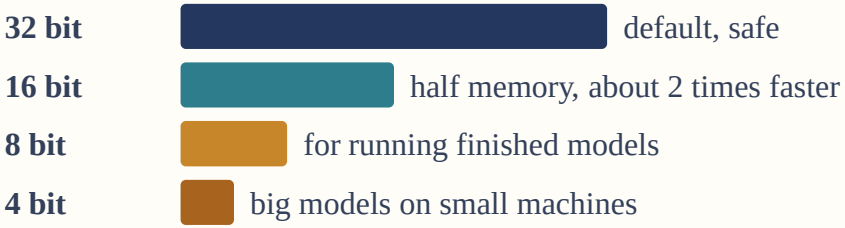
Our 64 to 64 layer on a batch of 32: 131,072 multiply and adds, all independent. Perfect for parallel hardware.

Bottlenecks: moving data between CPU and GPU memory is slow, so keep model and batches on the GPU.

Scale out: big models split data across many GPUs, and even split the model itself.

Fewer bits per number, and never lose a trained model

Number formats



Mixed precision training: do the heavy multiplies in 16 bit, keep a 32 bit master copy of the weights for the updates. Speed with safety.

Quantisation: after training, round weights to 8 or 4 bits. A 7 billion parameter model shrinks from 28 GB to about 3.5 GB.

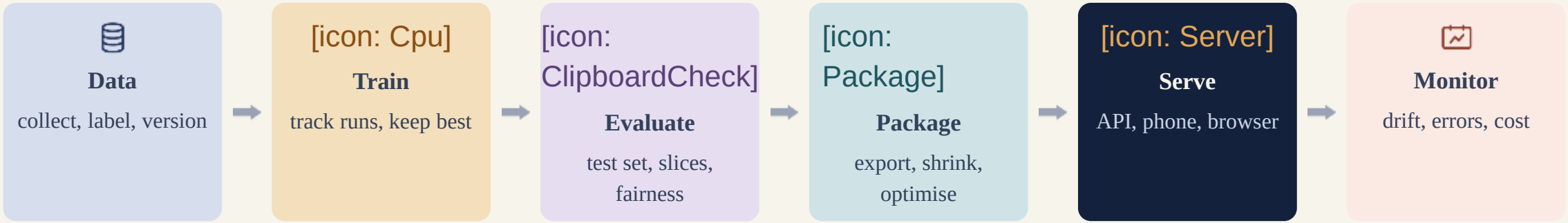
```
# our NumPy library: save every weight array
np.savez("digits.npz", *[p for p, _ in model.params()])

# PyTorch
torch.save({"model": model.state_dict(),
            "optim": opt.state_dict(),
            "epoch": epoch}, "ckpt.pt")
model.load_state_dict(torch.load("ckpt.pt")["model"])
```

Save the optimiser too. Adam's memories (Module 15) are needed to resume training smoothly.

Checkpoint often: long runs crash. Keep the best validation checkpoint (Module 16) and the latest.

From notebook to product: the lifecycle of a model



Export formats
ONNX runs a model outside Python; TorchScript and compiled graphs speed up serving; Core ML and TensorFlow Lite target phones.

Making it smaller
Quantise to fewer bits, prune near zero weights, or distil: train a small student to copy a large teacher's outputs.

Data drift
The world changes; a model trained on last year's data slowly degrades. Monitor live inputs and retrain on schedule.

Reality check: in production systems the model is often a small fraction of the code. Data pipelines, testing and monitoring make up most of it.

24

MODULE TWENTY FOUR

Reference and Next Steps

A glossary, a one page summary of the whole course, and exercises to make it your own.

IN THIS MODULE

Glossary, A to L

Glossary, M to Z

The whole course on one page

Exercises and projects

Where to go next

Glossary, A to L

Activation · the squashing or gating function after a weighted sum, such as ReLU or tanh. *M7*

Adam · optimiser combining momentum with per weight step sizes. *M15*

Attention · a soft lookup where queries match keys to blend values. *M20*

Backpropagation · passing blame backwards through the graph to get every gradient. *M9*

Batch · a group of examples processed together for one update. *M6*

Batch norm · rescaling each feature over the batch to a steady scale. *M16*

Bias · a learned offset in a neuron; also, error from a too simple model. *M4, M16*

Convolution · sliding a small shared kernel across an image. *M18*

Cross entropy · average surprise at the true class; the classification loss. *M13*

Dropout · randomly silencing neurons during training. *M16*

Embedding · a learned vector representing a token or item. *M21*

Epoch · one full pass over the training data. *M6*

Gradient · the direction of steepest uphill for the loss. *M2*

Gradient descent · repeatedly stepping against the gradient. *M6*

He initialisation · random start scaled for ReLU layers. *M14*

Hidden state · an RNN's running memory vector. *M19*

Hyperparameter · a setting chosen by you, not learned, such as learning rate. *M17*

Kernel · the small weight grid of a convolution. *M18*

Layer norm · rescaling each example across its features. *M16, M20*

Learning rate · the size of each optimisation step. *M6*

Logits · raw scores before softmax turns them into probabilities. *M13*

Loss · a single number measuring how wrong the model is. *M13*

LSTM · a gated recurrent cell with a long term memory lane. *M19*

Glossary, M to Z

Momentum · a running velocity that smooths and speeds up descent. *M15*

Overfitting · memorising training quirks instead of the pattern. *M16*

Pooling · shrinking a feature map by keeping the strongest value per block. *M18*

Query, key, value · the three views each token makes for attention. *M20*

ReLU · keep positives, zero out the rest. *M7*

RNN · a network that reads a sequence step by step with memory. *M19*

Softmax · turns scores into probabilities that add up to 1. *M13*

Temperature · the dial that sharpens or flattens sampling. *M19*

Token · a chunk of text the model reads as one unit. *M20*

Transformer · stacked blocks of attention and feed forward layers. *M20*

Validation set · held out data for choosing settings. *M16*

Weight decay · a steady pull of every weight toward zero. *M16*

MLP · a stack of Dense layers with activations between. *M8*

Parameter · a learned number: a weight or a bias. *M4*

Positional encoding · a stamp telling a Transformer where each token sits. *M20*

Regularisation · anything that improves results on unseen data. *M16*

Residual connection · adding a block's input to its output: a gradient highway. *M18*

Sigmoid · S shaped squash into the range 0 to 1. *M7*

Stride · how far a convolution window jumps each move. *M18*

Tensor · an array with any number of axes. *M3*

Transfer learning · starting from weights trained on another task. *M18*

Underfitting · a model too simple to capture the pattern. *M16*

Vanishing gradient · blame shrinking to nothing across many layers or steps. *M14, M19*

Xavier initialisation · random start scaled for tanh and sigmoid. *M14*

Deep learning in one diagram

1 · DATA
arrays of numbers, split three ways, scaled, augmented

→

2 · MODEL
layers of weighted sums and activations

→

3 · LOSS
one number: how wrong

→

4 · BACKPROP
blame for every weight, by the chain rule

→

5 · OPTIMISER
a small step downhill; repeat

Pick the architecture

Tables: MLP
Images: CNN
Sequences, text: Transformer
Generation: GPT or diffusion

Pick the loss

Numbers: squared error
Yes or no: binary cross entropy
One of many: cross entropy
Next token: cross entropy

Sensible defaults

ReLU or GELU, He start
AdamW, rate 0.001
warm up then cosine
batch 32 to 256, clip at 1

When it goes wrong

check start loss
overfit one batch
plot both curves
change one thing at a time

The one sentence version: choose a flexible function, measure its mistakes with a loss, and follow the gradient downhill, carefully, many times.

Exercises: from warm ups to real projects

Warm up

1. Add a Tanh layer to nnlib, with its backward pass, and gradient check it.

2. Retrain the digit MLP with learning rates 0.01, 0.1 and 1.0. Plot all three loss curves.

3. Rerun the Module 14 experiment with 50 layers. Which starts survive?

4. Change the temperature of the character model and describe the samples.

Build

5. Add batch norm to nnlib and measure its effect on the digit CNN.

6. Write the LSTM backward pass and train it on the cat and dog text.

7. Give the tiny GPT four heads and three blocks. Does the loss improve?

8. Turn the autoencoder into a denoiser: add noise to inputs, keep clean targets.

Projects

9. Port nnlib to PyTorch and match your NumPy results on the digits.

10. Fine tune a pretrained ResNet on a small photo set of your own.

11. Train a character GPT on a public domain book and sample from it.

12. Build a semantic search over your notes with a sentence embedding model.

Further study: Goodfellow, Bengio and Courville, *Deep Learning*; Karpathy's Neural Networks: Zero to Hero videos; Stanford CS231n and CS224n lecture notes; fast.ai's Practical Deep Learning course.

DEEP LEARNING FROM SCRATCH · COMPLETE

From a single neuron to a GPT, built by hand

neuron → gradient descent → backprop engine → our own library → CNN 97.3% → RNN writer → **tiny GPT**

Every model you will meet, however large, is made of the parts you have now built and run yourself: weighted sums, activations, a loss, gradients and an optimiser. The rest is scale, data and care.

276	24	0	∞
slides	modules	equations needed	things left to build